Principia Softwarica: The Plan 9 Network Stack `/net/`

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
Dave Presotto and Phil Winterbottom

February 11, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **10**
- 1.1 Motivations . . . 10
- 1.2 The Plan 9 network stack: `/net` . . . 10
- 1.3 Other network stacks . . . 10
- 1.4 Getting started . . . 11
- 1.5 Requirements . . . 11
- 1.6 About this document . . . 11
- 1.7 Copyright . . . 11
- 1.8 Acknowledgments . . . 11

2 Overview **12**
- 2.1 Networking principles . . . 12
  - 2.1.1 Local area networking . . . 12
  - 2.1.2 Packets and frames . . . 12
  - 2.1.3 Internetworking and gateways . . . 12
  - 2.1.4 Reliable communication . . . 12
  - 2.1.5 Network stack . . . 12
  - 2.1.6 Layers . . . 12
- 2.2 `/net/` files interface . . . 12
- 2.3 `helloworlds.c` . . . 12
- 2.4 Code organization . . . 12
- 2.5 Software architecture . . . 12
  - 2.5.1 Trace of a network write . . . 12
  - 2.5.2 Trace of a network read . . . 12
- 2.6 Book structure . . . 12

3 Core Data Structures **13**
- 3.1 IP addresses . . . 13
  - 3.1.1 IPv4 vs IPv6 . . . 13
  - 3.1.2 Network class and network identifier . . . 14
  - 3.1.3 Network mask . . . 15
  - 3.1.4 Network byte order . . . 15
  - 3.1.5 Other address operations . . . 15
- 3.2 IP header . . . 16
- 3.3 IP Interfaces: `Ipifc` . . . 16
  - 3.3.1 User side . . . 16
  - 3.3.2 Kernel side . . . 17
- 3.4 Physical link . . . 18
  - 3.4.1 `Medium` . . . 18
  - 3.4.2 `media` . . . 19

3.4.3 Ethernet medium . . . 19
3.4.4 Ethernet controllers and etherxx . . . 20
3.5 /net filesystem . . . 21
3.5.1 Fs and ipfs . . . 21
3.5.2 IP fragments and IP statistics . . . 22
3.5.3 Protocols . . . 23
3.5.4 Conversations . . . 25
3.5.5 Chan Qid, PROTO(), CONV(), QID() . . . 26
3.6 /net/ether filesystem . . . 27
3.6.1 Netif . . . 27
3.6.2 Netfile . . . 28
3.7 Routes . . . 28
3.7.1 Single Route . . . 28
3.7.2 Routing forest . . . 30
3.8 Blocks . . . 30

4 Initialization **31**
4.1 Kernel side . . . 31
4.1.1 Mounting the ip device . . . 31
4.1.2 Mounting the ethernet device . . . 33
4.2 User side . . . 34
4.2.1 Connect manually . . . 34
4.2.2 dial() . . . 34

5 User/Kernel Bridge **35**
5.1 IP device . . . 35
5.1.1 /net hierarchy and ipwalk() . . . 35
5.1.2 Dispatch functions, ipxxx() . . . 39
5.1.3 /net/x/clone . . . 42
5.1.4 /net/x/y/ctl . . . 44
5.1.5 /net/x/y/data . . . 51
5.1.6 /net/x/y/err . . . 51
5.1.7 /net/x/y/listen . . . 51
5.1.8 Other files . . . 53
5.2 Ethernet device . . . 54
5.2.1 /net/etherx hierarchy and etherwalk() . . . 55

6 Configuration **56**
6.1 /net/ipifc/ protocol . . . 56
6.1.1 Protocol initialisation . . . 56
6.1.2 /net/ipifc/clone . . . 57
6.1.3 Binding medium: /net/ipifc/x/ctl bind . . . 57
6.1.4 /net/ipifc/x/ctl . . . 59
6.1.5 Adding an IP: /net/ipifc/x/ctl add . . . 59
6.2 IP Interface, user side . . . 62
6.2.1 Parsing . . . 62
6.2.2 /net/ipifc/x . . . 64
6.2.3 /net/ipifc/stats . . . 65
6.3 Binding ethernet medium . . . 65
6.3.1 etherbind() . . . 65

6.3.2 `chandial()` . . . 67
6.4 Setting up routes . . . 67
6.5 Advanced configurations . . . 67
6.5.1 IP Routing . . . 67
6.5.2 MTU . . . 68
6.5.3 TOS . . . 68
6.5.4 TTL . . . 69
6.5.5 Dynamic interface, removing, unbinding, etc. . . . 69
6.5.6 Unbind on close . . . 71

7 Physical Transport: Ethernet **73**
7.1 Ethernet addresses . . . 73
7.1.1 Parsing . . . 73
7.1.2 `/net/etherx/addr` . . . 73
7.2 Ethernet header . . . 74
7.3 Ethernet packet . . . 74
7.4 IO . . . 74
7.4.1 Writing . . . 74
7.4.2 Reading . . . 74
7.5 Advanced features . . . 74

8 Inter Network Transport: IP **75**
8.1 IP addresses . . . 75
8.1.1 Parsing . . . 75
8.1.2 Comparisons . . . 77
8.2 IP header . . . 77
8.2.1 Byte ordering . . . 77
8.2.2 Checksum . . . 77
8.3 IP Fragments . . . 78
8.4 IO . . . 79
8.4.1 Writing: `ipoput4()` . . . 79
8.4.2 Reading: `ipiput4()` . . . 83
8.5 Advanced features . . . 89
8.5.1 Gating . . . 89
8.5.2 Manual fragmentation setting . . . 89
8.5.3 Routing . . . 89
8.5.4 Reassembling . . . 90

9 Finding Machines Locally: ARP **91**
9.1 Initialisation . . . 92
9.2 `/net/arp` . . . 92
9.2.1 Reading . . . 93
9.2.2 Writing . . . 93

10 Finding Machines Globally: Routes **96**
10.1 `v4lookup()` . . . 96
10.2 `findipifc()` . . . 97
10.3 Adding routes: `v4addroute()` . . . 98
10.4 Broadcast routes . . . 98
10.5 Route managment . . . 98

10.5.1 Allocation . . . 98
10.5.2 Free . . . 99
10.5.3 Insertion . . . 99
10.5.4 Balancing . . . 102
10.6 /net/iproutes . . . 103
10.6.1 Reading . . . 103
10.6.2 Writing . . . 106
10.7 Self cache . . . 108
10.7.1 Data structures . . . 108
10.7.2 Adding IPs . . . 109
10.7.3 Removing IPs . . . 110
10.7.4 /net/ipselftab . . . 112
10.8 Advanced features . . . 113
10.8.1 Removing routes . . . 113
10.8.2 Flushing routes . . . 114
10.8.3 Router boards . . . 115

11 Basic Communication: UDP **116**
11.1 Protocol initialisation . . . 116
11.2 Protocol data structures . . . 117
11.2.1 Udppriv . . . 117
11.2.2 Addresses hashtable: Ipht . . . 117
11.2.3 Statistics . . . 118
11.3 Protocol header . . . 119
11.4 /net/tcp/clone . . . 120
11.5 /net/tcp/x/ctl . . . 120
11.5.1 Connect . . . 120
11.5.2 Announce . . . 121
11.6 IO . . . 121
11.6.1 Writing: udpkick() . . . 121
11.6.2 Reading: udpiput() . . . 122
11.7 Other files . . . 127
11.7.1 /net/tcp/stats . . . 127
11.7.2 /net/tcp/x/status . . . 128
11.8 Advanced features . . . 128
11.8.1 Ignore advice . . . 128
11.8.2 Not regular forme . . . 128
11.8.3 User level UDP headers . . . 129

12 Reliable Communication: IL **131**
12.1 Protocol initialisation . . . 131
12.2 Protocol data structures . . . 132
12.2.1 IL state . . . 132
12.2.2 IL control packet type . . . 133
12.2.3 Ilcb . . . 133
12.2.4 Ilpriv . . . 134
12.2.5 Statistics . . . 134
12.3 Protocol header . . . 135
12.4 /net/il/clone . . . 136
12.5 /net/il/x/ctl . . . 136

12.5.1 Connect . . . 136
12.5.2 Announce . . . 136
12.6 IO . . . 137
12.6.1 Writing: `ilkick()` . . . 137
12.6.2 Reading: `iliput()` . . . 138
12.7 State machine . . . 140
12.7.1 `ilsendctl()` . . . 140
12.7.2 `ilprocess()` . . . 141
12.7.3 Start . . . 144
12.7.4 Close . . . 146
12.7.5 Syncer . . . 147
12.7.6 Syncee . . . 147
12.7.7 Establised . . . 147
12.8 Features . . . 147
12.8.1 Reliable datagram service . . . 147
12.8.2 In sequence delivery . . . 147
12.8.3 Retransmission of lost messages . . . 147
12.8.4 Out-of-order messages saving . . . 147
12.8.5 Adaptive timeouts . . . 147
12.8.6 Keep-alive . . . 147
12.9 Other files . . . 147
12.9.1 `/net/il/stats` . . . 147
12.9.2 `/net/il/x/status` . . . 147
12.10 Advanced features . . . 148
12.10.1 Fast timeout . . . 148
12.10.2 Special extension . . . 148

13 Standard Reliable Communication: TCP **149**

14 Applications **150**
14.1 Remote shell: Telnet . . . 150
14.2 Remote file access: TFTP . . . 150
14.3 Mail . . . 150

15 Remote Procedure Call: 9P **151**

16 Network File System: `exportfs` **152**

17 Name Resolution: DNS **153**

18 Security **154**
18.1 Denial of service . . . 154

19 Debugging Support **155**

20 Profiling Support **156**

21 Advanced Topics **157**
21.1 Flow Control . . . 157
21.2 IPv6 . . . 157
21.3 Broadcast . . . 157

21.4 Multicast . . . 158
21.5 Network database: ndb . . . 160
21.5.1 /net/ndb . . . 160
21.6 Connection Server . . . 160
21.7 Sniffing . . . 160
21.8 Point to point interface . . . 160
21.9 Proxy . . . 161
21.10 Wifi . . . 161
21.11 VLAN . . . 161
21.12 VPN . . . 161
21.13 Packet filter . . . 161

22 Conclusion **162**

A Debugging **163**
A.1 Dumpers . . . 163
A.1.1 Addresses . . . 163
A.1.2 IP interface . . . 164
A.2 /net/log . . . 165
A.2.1 Netlog . . . 165
A.2.2 /net/log . . . 166
A.2.3 Opening . . . 166
A.2.4 Reading . . . 167
A.2.5 Control . . . 167
A.3 /bin/snoopy . . . 169
A.3.1 /net/ipifc/x/snoop . . . 169
A.3.2 /bin/snoopy . . . 169

B Profiling **170**
B.1 /net/x/stats . . . 170

C Error Management **171**

D Ethernet NE2000 Driver **172**
D.1 Data structures . . . 172
D.1.1 Ethernet Dp8390 controller . . . 172
D.2 Initialisation . . . 172
D.3 Mounting . . . 177
D.4 IO . . . 177
D.4.1 Writing . . . 178
D.4.2 Reading . . . 180
D.5 Advanced features . . . 185
D.5.1 Multicast . . . 185
D.5.2 Promiscuous mode . . . 186
D.5.3 Power managment . . . 186
D.5.4 Plug and play . . . 186

E Utilities **189**
E.1 Globbing . . . 189

F Extra Mediums **190**
F.1 Null medium . . . 190
F.2 Loopback medium . . . 191
F.3 Point to point serial line . . . 193
F.4 Token ring . . . 193

G Extra Protocols **194**
G.1 ICMP . . . 194
G.2 RUDP . . . 194
G.3 GRE . . . 194
G.4 ESP . . . 194
G.5 Datakit and URP . . . 194

H Extra Applications **195**
H.1 Remote login, rlogin . . . 195
H.2 Serving files, FTP . . . 195
H.3 Serving documents, HTTP . . . 195
H.4 Serving mails, SMTP . . . 195
H.5 Serving news, NNTP . . . 195

I Extra Code **196**
I.1 `include/net/` . . . 196
I.1.1 `include/net/ip.h` . . . 196
I.2 `lib_networking/libip/` . . . 198
I.2.1 `lib_networking/libip/parseether.c` . . . 198
I.2.2 `lib_networking/libip/parseip.c` . . . 198
I.2.3 `lib_networking/libip/myetheraddr.c` . . . 199
I.2.4 `lib_networking/libip/myipaddr.c` . . . 199
I.2.5 `lib_networking/libip/eipfmt.c` . . . 200
I.2.6 `lib_networking/libip/equivip.c` . . . 201
I.2.7 `lib_networking/libip/ipaux.c` . . . 201
I.2.8 `lib_networking/libip/bo.c` . . . 202
I.2.9 `lib_networking/libip/classmask.c` . . . 203
I.2.10 `lib_networking/libip/ptclbsum.c` . . . 204
I.2.11 `lib_networking/libip/readipifc.c` . . . 205
I.3 `lib_networking/libip/tests/` . . . 205
I.3.1 `lib_networking/libip/tests/testreadipifc.c` . . . 205
I.4 `kernel/network/ip/` . . . 206
I.4.1 `kernel/network/ip/ip.h` . . . 206
I.4.2 `kernel/network/ip/ip.c` . . . 212
I.5 `kernel/network/` . . . 213
I.5.1 `kernel/network/portfns_network.h` . . . 213
I.5.2 `kernel/network/netif.h` . . . 214
I.5.3 `kernel/network/netif.c` . . . 215
I.6 `kernel/network/386/` . . . 228
I.6.1 `kernel/network/etherif.h` . . . 228
I.6.2 `kernel/network/386/devether.c` . . . 229
I.6.3 `kernel/network/386/ether8390.h` . . . 239
I.6.4 `kernel/network/386/ether8390.c` . . . 239
I.6.5 `kernel/network/386/ether2000.c` . . . 243

I.6.6 kernel/network/386/etherigbe.c . . . 244
I.6.7 kernel/network/386/ethermii.c . . . 282
I.6.8 kernel/network/386/ethermii.h . . . 286
I.7 kernel/network/arm/ . . . 289
I.7.1 network/arm/devether.c . . . 289
I.8 kernel/network/ip/ . . . 299
I.8.1 kernel/network/ip/devip.c . . . 299
I.8.2 kernel/network/ip/arp.c . . . 304
I.8.3 kernel/network/ip/icmp.c . . . 313
I.8.4 kernel/network/ip/chandial.c . . . 323
I.8.5 kernel/network/ip/nullmedium.c . . . 326
I.8.6 kernel/network/ip/loopbackmedium.c . . . 326
I.8.7 kernel/network/ip/ethermedium.c . . . 326
I.8.8 kernel/network/ip/pktmedium.c . . . 338
I.8.9 kernel/network/ip/iproute.c . . . 339
I.8.10 kernel/network/ip/ipaux.c . . . 341
I.8.11 kernel/network/ip/ipifc.c . . . 344
I.8.12 kernel/network/ip/netlog.c . . . 357
I.8.13 kernel/network/ip/ptclbsum.c . . . 359
I.8.14 kernel/network/ip/il.c . . . 361
I.8.15 kernel/network/ip/udp.c . . . 372
I.8.16 kernel/network/ip/tcp.c . . . 375
I.9 kernel/network/ip/ . . . 436

Glossary **485**

Index **486**

References **486**

Chapter 1

Introduction

The goal of this book is to present with full details the source code of a network stack.

1.1 Motivations

Why a network stack? Because I think you are a better programmer if you fully understand how things work under the hood.

Here are a few questions I hope this book will answer:

- What happens when you open a connection to another machine?
- How are opened connections stored in the kernel?
- How can the kernel dispatch incoming packets to the right process?
- How can the network stack offer reliable communications over an unreliable physical network?

1.2 The Plan 9 network stack: `/net`

1.3 Other network stacks

Here are a few network stacks that I considered for this book, but which I ultimately discarded:

- DataKit
- XINU
- Linux
- BSD
- Minix
- Mirage

1.4 Getting started

1.5 Requirements

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course the Plan9's authors who wrote most of this book: Dave Presotto, Phil Winterbottom, and many other people from Bell Labs.

Chapter 2

Overview

2.1 Networking principles

2.1.1 Local area networking

2.1.2 Packets and frames

2.1.3 Internetworking and gateways

2.1.4 Reliable communication

2.1.5 Network stack

2.1.6 Layers

2.2 `/net/` files interface

2.3 `helloworlds.c`

2.4 Code organization

2.5 Software architecture

2.5.1 Trace of a network write

2.5.2 Trace of a network read

2.6 Book structure

Chapter 3

Core Data Structures

3.1 IP addresses

⟨*constant* IPv4addrlen 13a⟩≡ (206b 196a)
```
IPv4addrlen= 4,
```

⟨*typedef ipv4* 13b⟩≡ (208 196c)
```
typedef uchar ipv4[IPv4addrlen];
```

3.1.1 IPv4 vs IPv6

⟨*constant* IPaddrlen 13c⟩≡ (206b 196a)
```
IPaddrlen= 16,
```

⟨*typedef ipaddr* 13d⟩≡ (208 196c)
```
typedef uchar ipaddr[IPaddrlen];
```

⟨*global* v4prefix 13e⟩≡ (201e)
```
/*
 *  prefix of all v4 addresses
 */
ipaddr v4prefix = {
    // first 12
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0xff, 0xff,

    // rest are ipv4 numbers
    0, 0, 0, 0
};
```

⟨*constant* IPv4off 13f⟩≡ (206b 196a)
```
IPv4off= 12,
```

⟨*function* isv4 13g⟩≡ (201e)
```
bool
isv4(ipaddr ip)
{
    return memcmp(ip, v4prefix, IPv4off) == 0;
}
```

⟨*function* v4tov6 14a⟩≡ (201e)

```
/*
 *  the following routines are unrolled with no memset's to speed
 *  up the usual case
 */
void
v4tov6(ipaddr v6, ipv4 v4)
{
    v6[0] = 0; v6[1] = 0; v6[2] = 0; v6[3] = 0;
    v6[4] = 0; v6[5] = 0; v6[6] = 0; v6[7] = 0;
    v6[8] = 0; v6[9] = 0; v6[10] = 0xff; v6[11] = 0xff;

    v6[12] = v4[0];
    v6[13] = v4[1];
    v6[14] = v4[2];
    v6[15] = v4[3];
}
```

⟨*function* v6tov4 14b⟩≡ (201e)

```
errorneg1
v6tov4(ipv4 v4, ipaddr v6)
{
    if(v6[0] == 0 && v6[1] == 0 && v6[2] == 0 && v6[3] == 0
    && v6[4] == 0 && v6[5] == 0 && v6[6] == 0 && v6[7] == 0
    && v6[8] == 0 && v6[9] == 0 && v6[10] == 0xff && v6[11] == 0xff)
    {
        v4[0] = v6[12];
        v4[1] = v6[13];
        v4[2] = v6[14];
        v4[3] = v6[15];
        return OK_0;
    }
    ⟨v6tov4() else if ipv6 address 467a⟩
}
```

⟨*typedef ipv4or6* 14c⟩≡ (208)

```
typedef uchar* ipv4or6;
```

⟨*typedef ipv4p* 14d⟩≡ (208)

```
typedef uchar* ipv4p;
```

⟨*enum* ip_version 14e⟩≡ (208)

```
/* ip versions */
enum ip_version {
  V4=   4,
  V6=   6,
};
```

3.1.2 Network class and network identifier

⟨*macro* CLASS 14f⟩≡ (196c)

```
#define CLASS(p) ((*(uchar*)(p))>>6)
```

3.1.3 Network mask

⟨*function* defmask 15a⟩≡ (203d)

```
uchar*
defmask(ipaddr ip)
{
    if(isv4(ip))
        return classmask[ip[IPv4off]>>6];
    ⟨defmask() if ipv6 467b⟩
}
```

⟨*global* classmask 15b⟩≡ (203d)

```
static ipaddr classmask[4] = {
    // class A
    0xff,0xff,0xff,0xff,  0xff,0xff,0xff,0xff,  0xff,0xff,0xff,0xff,
    0xff,0x00,0x00,0x00, // 255.0.0.0
    // class A
    0xff,0xff,0xff,0xff,  0xff,0xff,0xff,0xff,  0xff,0xff,0xff,0xff,
    0xff,0x00,0x00,0x00, // 255.0.0.0
    // class B
    0xff,0xff,0xff,0xff,  0xff,0xff,0xff,0xff,  0xff,0xff,0xff,0xff,
    0xff,0xff,0x00,0x00, // 255.255.0.0
    // class C
    0xff,0xff,0xff,0xff,  0xff,0xff,0xff,0xff,  0xff,0xff,0xff,0xff,
    0xff,0xff,0xff,0x00, // 255.255.255.0
};
```

⟨*function* maskip 15c⟩≡ (203d)

```
void
maskip(ipaddr from, ipaddr mask, ipaddr to)
{
    int i;

    for(i = 0; i < IPaddrlen; i++)
        to[i] = from[i] & mask[i];
}
```

3.1.4 Network byte order

⟨*typedef iplong* 15d⟩≡ (208 196c)

```
typedef ulong iplong;
```

3.1.5 Other address operations

⟨*global* IPnoaddr 15e⟩≡ (201e)

```
ipaddr IPnoaddr;
```

⟨*macro* ipmove*(*(kernel/network/ip/ip.h)*)* 15f⟩≡ (208)

```
#define ipmove(x, y) memmove(x, y, IPaddrlen)
```

⟨*macro* ipcmp*(*(kernel/network/ip/ip.h)*)* 15g⟩≡ (208)

```
#define ipcmp(x, y) ( (x)[IPaddrlen-1] != (y)[IPaddrlen-1] || memcmp(x, y, IPaddrlen) )
```

3.2 IP header

⟨*struct* Ip4hdr 16a⟩≡ (208)

```
/* on the wire packet header */
struct Ip4hdr
{
  uchar vihl;   /* Version and header length */
  uchar tos;    /* Type of service */
  uchar length[2];  /* packet length */
  uchar id[2];    /* ip->identification */
  uchar frag[2];  /* Fragment information */
  uchar ttl;        /* Time to live */

  // enum<protocol_type>
  uchar proto;    /* Protocol */

  uchar cksum[2]; /* Header checksum */

  ipv4 src;   /* IP source */
  ipv4 dst;   /* IP destination */
};
```

3.3 IP Interfaces: Ipifc

3.3.1 User side

⟨*struct Ipifc (user)* 16b⟩≡ (196c)

```
/* actual interface */
struct Ipifc
{
    /* per ip interface */

    char  dev[64]; // e.g. "/net/ether0"
    int  mtu;
    // list<ref_own<Iplifc> (next = Iplifc.next)
    Iplifc *lifc;

    ⟨Ipifc(user) stat fields 16c⟩
    ⟨Ipifc(user) ipv6 fields 467c⟩

    //Extra
    ⟨Ipifc(user) extra fields 16d⟩
};
```

⟨Ipifc(user) *stat fields* 16c⟩≡ (16b)

```
ulong pktin;
ulong pktout;
ulong errin;
ulong errout;
```

⟨Ipifc(user) *extra fields* 16d⟩≡ (16b) 16e ▷

```
Ipifc *next;
```

⟨Ipifc(user) *extra fields* 16e⟩+≡ (16b) ◁ 16d

```
int index;   /* number of interface in ipifc dir */
```

⟨*struct Iplifc (user)* 17a⟩≡ (196c)

```
/* local address */
struct Iplifc
{
    /* per address on the ip interface */
    ipaddr ip;
    ipaddr mask;
    ipaddr net;  /* ip & mask */

    ⟨Iplifc(user) ipv6 fields 481k⟩

    // Extra
    ⟨Iplifc(user) extra fields 17b⟩
};
```

⟨Iplifc(user) *extra fields* 17b⟩≡ (17a)

```
// list<ref_own<Iplifc>>, head = Ipifc.lifc
Iplifc *next;
```

3.3.2 Kernel side

⟨*struct Ipifc (kernel)* 17c⟩≡ (208)

```
struct Ipifc
{
  char  dev[64];  /* device we're attached to */

  Medium  *m;   /* Media pointer */
  uchar mac[MAClen];  /* MAC address */

  int maxtu;    /* Maximum transfer unit */
  int mintu;    /* Minumum tranfer unit */
  int mbps;   /* megabits per second */

  // list<ref_own<Iplifc>>, next = Iplifc.next
  Iplifc  *lifc;    /* logical interfaces on this physical one */

  ⟨Ipifc(kernel) stat fields 17d⟩
  ⟨Ipifc(kernel) routing fields 30a⟩
  ⟨Ipifc(kernel) priv fields 20b⟩

  ⟨Ipifc(kernel) other fields 17e⟩
  ⟨Ipifc(kernel) ipv6 fields 468c⟩

  //Extra
  RWlock;

};
```

⟨Ipifc(kernel) *stat fields* 17d⟩≡ (17c)

```
/* message statistics */
ulong in;
ulong out;
ulong inerr;
ulong outerr;
```

⟨Ipifc(kernel) *other fields* 17e⟩≡ (17c) 59a ▷

```
Conv  *conv;    /* link to its conversation structure */
```

⟨*struct Iplifc (kernel)* 18a⟩≡ (208)

```
/* logical interface associated with a physical one */
struct Iplifc
{
  ipaddr local;
  ipaddr mask;
  ipaddr net; // local & mask?

  ipaddr remote; // ??

  ⟨Iplifc(kernel) ipv6 fields 482a⟩

  // Extra
  ⟨Iplifc(kernel) extra fields 18b⟩
};
```

⟨Iplifc(kernel) *extra fields* 18b⟩≡ (18a) 108b ▷

```
// list<ref_own<Iplifc>>, head = Ipifc.lifc
Iplifc  *next;
```

3.4 Physical link

3.4.1 Medium

⟨*struct Medium (kernel)* 18c⟩≡ (208)

```
struct Medium
{
  char  *name;

  int hsize;    /* medium header size */
  int maclen;   /* mac address length  */

  int mintu;    /* default min mtu */
  int maxtu;    /* default max mtu */

  ⟨Medium(kernel) methods 18d⟩

  ⟨Medium(kernel) other fields 71c⟩

};
```

⟨Medium(kernel) *methods* 18d⟩≡ (18c)

```
⟨Medium(kernel) binding methods 18e⟩
⟨Medium(kernel) io methods 18f⟩
⟨Medium(kernel) address resolution methods 92a⟩

⟨Medium(kernel) router methods 115a⟩
⟨Medium(kernel) multicast methods 159d⟩
⟨Medium(kernel) ipv6 methods 479a⟩
```

⟨Medium(kernel) *binding methods* 18e⟩≡ (18d)

```
void  (*bind)(Ipifc*, int, char**);
void  (*unbind)(Ipifc*);
```

⟨Medium(kernel) *io methods* 18f⟩≡ (18d) 19a ▷

```
// write packets on the physical network
void  (*bwrite)(Ipifc *ifc, Block *b, int version, ipv4or6 ip);
```

⟨Medium(kernel) *io methods* 19a⟩+≡ (18d) ◁18f

```
/* process packets written to 'data' */
void  (*pktin)(Fs *f, Ipifc *ifc, Block *bp);
```

3.4.2 media

⟨*global* media 19b⟩≡ (354b)

```
Medium *media[Maxmedia] = { 0 };
```

⟨*constant* Maxmedia 19c⟩≡ (344)

```
Maxmedia    = 32,
```

⟨*function* addipmedium 19d⟩≡ (354b)

```
/*
 *  link in a new medium
 */
void
addipmedium(Medium *med)
{
    int i;

    for(i = 0; i < nelem(media)-1; i++)
        if(media[i] == nil){
            media[i] = med;
            break;
        }
}
```

3.4.3 Ethernet medium

⟨*constant* Eaddrlen 19e⟩≡ (214f)

```
Eaddrlen= 6,
```

⟨*typedef eaddr* 19f⟩≡ (215a)

```
typedef uchar eaddr[Eaddrlen];
```

⟨*global ethermedium (kernel)* 19g⟩≡ (336c)

```
Medium ethermedium =
{
    .name=      "ether",

    .hsize=     14,
    .mintu=     60,
    .maxtu=     1514,
    .maclen=    6,

    .bind=      etherbind,
    .unbind=    etherunbind,

    .bwrite=    etherbwrite,

    .ares=      arpenter,
    .areg=      sendgarp,

    .addmulti=  etheraddmulti,
    .remmulti=  etherremmulti,

    .pref2addr= etherpref2addr,
};
```

⟨*global* etherbroadcast 20a⟩≡ (336c)
```
static eaddr etherbroadcast = { 0xff, 0xff, 0xff, 0xff, 0xff, 0xff };
```

⟨Ipifc(kernel) *priv fields* 20b⟩≡ (17c)
```
void  *arg;   /* medium specific */
```

3.4.4 Ethernet controllers and etherxx

⟨*global etherxx (kernel)* 20c⟩≡ (237b)
```
static Ether *etherxx[MaxEther];
```

⟨*constant* MaxEther 20d⟩≡ (228a)
```
MaxEther  = 48, //bcm: 4
```

⟨*struct Ether (kernel)* 20e⟩≡ (229a)
```
struct Ether {

  eaddr ea;

  Queue*  oq;

  ISAConf;      /* hardware info */
  Netif;

  ⟨Ether methods 20i⟩

  ⟨Ether priv fields 20g⟩
  ⟨Ether other fields 20h⟩

  // Extra
  ⟨Ether extra fields 20f⟩

  //bcm: only, but seems dead
  //RWlock;
  //int minmtu;
  //int maxmtu;
  //void* address;
  //int irq;

};
```

⟨Ether *extra fields* 20f⟩≡ (20e)
```
int ctlrno;
```

⟨Ether *priv fields* 20g⟩≡ (20e)
```
void  *ctlr;
```

⟨Ether *other fields* 20h⟩≡ (20e)
```
int tbdf;     /* type+busno+devno+funcno */ //pc: only?
```

⟨Ether *methods* 20i⟩≡ (20e)
```
⟨ether mounting methods 20j⟩
⟨ether io methods 21a⟩

⟨ether other methods 21b⟩
```

⟨ether *mounting methods* 20j⟩≡ (20i)
```
void  (*attach)(Ether*);  /* filled in by reset routine */
void  (*detach)(Ether*);
```

⟨ether *io methods* 21a⟩≡ (20i)
```
void  (*transmit)(Ether*);
void  (*interrupt)(Ureg*, void*);
```

⟨ether *other methods* 21b⟩≡ (20i)
```
long  (*ifstat)(Ether*, void*, long, ulong);
long  (*ctl)(Ether*, void*, long); /* custom ctl messages */
void  (*power)(Ether*, int);  /* power on/off */
void  (*shutdown)(Ether*);  /* shutdown hardware before reboot */
```

3.5 /net filesystem

3.5.1 Fs and ipfs

⟨*struct Fs (kernel)* 21c⟩≡ (208)
```
/*
 *  one per IP protocol stack
 */
struct Fs
{
  IP  *ip;

  // array<option<ref_own<Proto>>>, size is Fs.np
  Proto*  p[Maxproto+1];    /* list of supported protocols */
  int np;

  ⟨Fs(kernel) arp fields 91a⟩
  ⟨Fs(kernel) routing fields 21i⟩
  ⟨Fs(kernel) ndb fields 160a⟩
  ⟨Fs(kernel) logging fields 165c⟩

  ⟨Fs(kernel) other fields 24b⟩
  ⟨Fs(kernel) ipv6 fields 468a⟩

  // Extra
  RWlock;
  ⟨Fs(kernel) extra fields 21g⟩
};
```

⟨*constant* Maxproto 21d⟩≡ (206b)
```
Maxproto= 20,
```

⟨*global* ipfs 21e⟩≡ (302c)
```
Fs  *ipfs[Nfs]; /* attached fs's */
```

⟨*constant* Nfs 21f⟩≡ (26g)
```
Nfs=        128,
```

⟨Fs(kernel) *extra fields* 21g⟩≡ (21c)
```
int dev; // idx in ipfs
```

⟨*global* fslock 21h⟩≡ (302c)
```
QLock   fslock;
```

⟨Fs(kernel) *routing fields* 21i⟩≡ (21c) 30b ▷
```
Proto*  ipifc;      /* kludge for ipifcremroute & ipifcaddroute */
```

3.5.2 IP fragments and IP statistics

⟨*struct IP (kernel)* 22a⟩≡ (208)

```
/* an instance of IP */
struct IP
{
  // list<ref_own<Fragment>>, next = Fragment4.next
  Fragment4*  flisthead4;
  // list<ref_own<Fragment>>, next = Fragment4.next
  Fragment4*  fragfree4;

  Ref   id4;

  ⟨IP(kernel) stat fields 22d⟩
  ⟨IP(kernel) routing fields 90a⟩

  ⟨IP(kernel) ipv6 fields 468b⟩

  // Extra
  QLock   fraglock4;
};
```

⟨*struct* Fragment4 22b⟩≡ (208)

```
struct Fragment4
{
  // list<ref_own<Block>> ? next = Block.???
  Block*  blist;

  iplong   src;
  iplong   dst;

  ushort  id;
  ulong   age;

  // Extra
  ⟨Fragment4 extra fields 22c⟩
};
```

⟨Fragment4 *extra fields* 22c⟩≡ (22b)

```
Fragment4*  next;
```

⟨IP(kernel) *stat fields* 22d⟩≡ (22a)

```
// map<enum<mib_two_counters>, uvlong>
uvlong    stats[Nipstats];
```

⟨*enum* _anon_ (kernel/network/ip/ip.h)3 22e⟩≡ (208)

```
/* MIB II counters */
enum mib_two_counters
{
  Forwarding,
  DefaultTTL,

  InReceives,

  InHdrErrors,
  InAddrErrors,
  ForwDatagrams,
  InUnknownProtos,

  // In stats
  InDiscards,
```

```
    InDelivers,

    // Out stats
    OutRequests,
    OutDiscards,
    OutNoRoutes,

    ReasmTimeout,
    ReasmReqds,
    ReasmOKs,
    ReasmFails,

    // Out fragments
    FragOKs,
    FragFails,
    FragCreates,

    Nipstats,
  };
```

⟨*global* statnames 23a⟩≡ (212b)

```
  static char *statnames[] =
  {
  [Forwarding]    "Forwarding",
  [DefaultTTL]    "DefaultTTL",
  [InReceives]    "InReceives",
  [InHdrErrors]   "InHdrErrors",
  [InAddrErrors]  "InAddrErrors",
  [ForwDatagrams] "ForwDatagrams",
  [InUnknownProtos]   "InUnknownProtos",

  [InDiscards]    "InDiscards",
  [InDelivers]    "InDelivers",

  [OutRequests]   "OutRequests",
  [OutDiscards]   "OutDiscards",
  [OutNoRoutes]   "OutNoRoutes",

  [ReasmTimeout]  "ReasmTimeout",
  [ReasmReqds]    "ReasmReqds",
  [ReasmOKs]  "ReasmOKs",
  [ReasmFails]    "ReasmFails",

  [FragOKs]   "FragOKs",
  [FragFails] "FragFails",
  [FragCreates]   "FragCreates",
  };
```

3.5.3 Protocols

⟨*struct Proto (kernel)* 23b⟩≡ (208)

```
  /*
   *  one per multiplexed protocol
   */
  struct Proto
  {
    char*   name;   /* protocol name */ // e.g. "udp", "tcp", "ipfc", etc

    ⟨Proto(kernel) methods 24e⟩
```

```
    // growing_array<option<ref_own<Proto>>>, size = Proto.nc
    Conv    **conv;   /* array of conversations */
    int   nc;   /* number of conversations */
    int   ac; // number of opened conversations

    ⟨Proto(kernel) priv fields 24d⟩
    ⟨Proto(kernel) other fields 24c⟩

    // Extra
    QLock;
    ⟨Proto(kernel) extra fields 24a⟩
  };
```

⟨Proto(kernel) *extra fields* 24a⟩≡ (23b)

```
  // ref<Fs>, reverse of Fs.p[this.x]
  Fs    *f;   /* file system this proto is part of */
  // index in Fs.p[]
  int   x;    /* protocol index */
```

⟨Fs(kernel) *other fields* 24b⟩≡ (21c) 108a ▷

```
  // map<enum<protocol_type>, ref<Proto>>
  Proto*  t2p[256];   /* vector of all protocols */
```

⟨Proto(kernel) *other fields* 24c⟩≡ (23b) 48c ▷

```
  // enum<protocol_type>
  int   ipproto;  /* ip protocol type */
```

⟨Proto(kernel) *priv fields* 24d⟩≡ (23b) 26d ▷

```
  void    *priv;
```

⟨Proto(kernel) *methods* 24e⟩≡ (23b)

```
  ⟨Proto(kernel) protocol methods 24f⟩
  ⟨Proto(kernel) conversation ctl methods 24h⟩
  ⟨Proto(kernel) conversation inspection methods 53g⟩
  ⟨Proto(kernel) conversation methods 24g⟩
```

⟨Proto(kernel) *protocol methods* 24f⟩≡ (24e) 44d ▷

```
  void    (*create)(Conv*);
  void    (*close)(Conv*);
```

⟨Proto(kernel) *conversation methods* 24g⟩≡ (24e) 44b ▷

```
  void    (*rcv)(Proto*, Ipifc*, Block*);
```

⟨Proto(kernel) *conversation ctl methods* 24h⟩≡ (24e) 24i ▷

```
  char*   (*connect)(Conv*, char**, int);
  char*   (*announce)(Conv*, char**, int);
```

⟨Proto(kernel) *conversation ctl methods* 24i⟩+≡ (24e) ◁ 24h 24j ▷

```
  char*   (*ctl)(Conv*, char**, int);
```

⟨Proto(kernel) *conversation ctl methods* 24j⟩+≡ (24e) ◁ 24i

```
  char*   (*bind)(Conv*, char**, int);
```

3.5.4 Conversations

⟨*struct Conv (kernel)* 25a⟩≡ (208)

```
/*
 *  one per conversation directory
 */
struct Conv
{

  ipaddr laddr; /* local IP address */
  ipaddr raddr; /* remote IP address */

  ushort  lport;      /* local port number */
  ushort  rport;      /* remote port number */

  char  *owner;     /* protections */
  int perm;

  // enum<conversation_state>
  int state;

  ⟨Conv(kernel) queue fields 26a⟩
  ⟨Conv(kernel) listen fields 52b⟩
  ⟨Conv(kernel) routing fields 28d⟩

  ⟨Conv(kernel) synchronisation fields 25d⟩
  ⟨Conv(kernel) priv fields 26c⟩
  ⟨Conv(kernel) snoop fields 169b⟩
  ⟨Conv(kernel) error fields 26b⟩

  ⟨Conv(kernel) udp fields 128b⟩
  ⟨Conv(kernel) multicast fields 159e⟩

  ⟨Conv(kernel) other fields 26e⟩
  ⟨Conv(kernel) ipv6 fields 481h⟩

  // Extra
  QLock;
  ⟨Conv(kernel) extra fields 25b⟩
};
```

⟨Conv(kernel) *extra fields* 25b⟩≡ (25a)

```
// ref<Proto> reverse of Proto.conv[this.x]
Proto*  p;
// index in Proto.conv[]
int x;      /* conversation index */
```

⟨*enum* conversation_state 25c⟩≡ (208)

```
enum conversation_state
{
  Idle=   0,

  Announcing= 1,
  Announced=  2,

  Connecting= 3,
  Connected=  4,
};
```

⟨Conv(kernel) *synchronisation fields* 25d⟩≡ (25a)

```
Rendez  cr;
```

⟨Conv(kernel) *queue fields* 26a⟩≡ (25a) 51c ▷

```
Queue*  rq;     /* queued data waiting to be read */
Queue*  wq;     /* queued data waiting to be written */
```

⟨Conv(kernel) *error fields* 26b⟩≡ (25a)

```
char  cerr[ERRMAX];
```

⟨Conv(kernel) *priv fields* 26c⟩≡ (25a)

```
void* ptcl;     /* protocol specific stuff */
```

⟨Proto(kernel) *priv fields* 26d⟩+≡ (23b) ◁ 24d

```
int   ptclsize; /* size of per protocol ctl block */
```

⟨Conv(kernel) *other fields* 26e⟩≡ (25a) 48b ▷

```
int inuse;      /* opens of listen/data/ctl */
```

3.5.5 Chan Qid, PROTO(), CONV(), QID()

⟨*enum* qid*(*(kernel/network/ip/devip.c)*)* 26f⟩≡ (302c)

```
enum
{
    Qtopdir=    1,      /* top level directory */

    Qtopbase,
    Qarp=       Qtopbase,
    Qiproute,
    ⟨Qid toplevel extra cases 112a⟩

    Qprotodir,          /* directory for a protocol */
    Qprotobase,
    Qclone=     Qprotobase,
    ⟨Qid protocol extra cases 53a⟩

    Qconvdir,           /* directory for a conversation */
    Qconvbase,
    Qctl=       Qconvbase,
    Qdata,
    ⟨Qid conversation extra cases 51d⟩
    ⟨Qid conversation extra cases, last entry 169a⟩
};
```

⟨*enum* misc*(*(kernel/network/ip/devip.c)*)* 26g⟩≡ (302c)

```
enum
{
    Logtype=    5,
    Masktype=   (1<<Logtype)-1,

    Logconv=    12,
    Maskconv=   (1<<Logconv)-1,
    Shiftconv=  Logtype,

    Logproto=   8,
    Maskproto=  (1<<Logproto)-1,
    Shiftproto= Logtype + Logconv,

    ⟨constant Nfs 21f⟩
};
```

⟨*macro* QID 26h⟩≡ (302c)

```
#define QID(p, cv, y)    ( ((p)<<(Shiftproto)) | ((cv)<<Shiftconv) | (y) )
```

⟨*macro* PROTO 27a⟩≡ (302c)

```
#define PROTO(x)    ( (((ulong)(x).path) >> Shiftproto) & Maskproto )
```

⟨*macro* CONV 27b⟩≡ (302c)

```
#define CONV(x)     ( (((ulong)(x).path) >> Shiftconv) & Maskconv )
```

⟨*macro* TYPE 27c⟩≡ (302c)

```
#define TYPE(x)     ( ((ulong)(x).path) & Masktype )
```

3.6 /net/ether filesystem

3.6.1 Netif

⟨*struct Netif (kernel)* 27d⟩≡ (215a)

```
/*
 *  a network interface
 */
struct Netif
{
  /* multiplexing */
  char  name[KNAMELEN];   /* for top level directory */

  // growing_array?<option<ref_own<Netfile>>>, size = nfile?
  Netfile **f;
  int nfile;      /* max number of Netfiles */

  /* about net */
  int alen;     /* address length */
  int link;     /* link status */

  int limit;      /* flow control */

  int minmtu;
  int maxmtu;
  int mtu;
  int mbps;     /* megabits per sec */

  uchar addr[Nmaxaddr];
  uchar bcast[Nmaxaddr];

  int prom;     /* number of promiscuous opens */
  int scan;     /* number of base station scanners */
  int all;      /* number of -1 multiplexors */

  ⟨Netif(kernel) stat fields 28b⟩

  /* routines for touching the hardware */
  void  *arg;

  ⟨Netif(kernel) methods 28a⟩

  // Extra
  QLock;

};
```

⟨Netif(kernel) *methods* 28a⟩≡ (27d)

```
void  (*promiscuous)(void*, int);
int (*hwmtu)(void*, int); /* get/set mtu */
void  (*scanbs)(void*, uint); /* scan for base stations */

⟨Netif(kernel) multicast fields 159h⟩
⟨Netif(kernel) multicast methods 159i⟩
```

⟨Netif(kernel) *stat fields* 28b⟩≡ (27d)

```
/* statistics */
int misses;
uvlong  inpackets;
uvlong  outpackets;
int crcs;   /* input crc errors */
int oerrs;    /* output errors */
int frames;   /* framing errors */
int overflows;  /* packet overflows */
int buffs;    /* buffering errors */
int soverflows; /* software overflow */
```

3.6.2 Netfile

⟨*struct* Netfile 28c⟩≡ (215a)

```
/*
 *  one per multiplexed connection
 */
struct Netfile
{
  int inuse;
  ulong mode;
  char  owner[KNAMELEN];

  Queue *in;      /* input buffer */

  int type;     /* multiplexor type */
  int prom;     /* promiscuous mode */
  int scan;     /* base station scanning interval */
  int bridge;     /* bridge mode */
  int headersonly;    /* headers only - no data */

  uchar maddr[8];   /* bitmask of multicast addresses requested */
  int nmaddr;     /* number of multicast addresses */

  // Extra
  QLock;
};
```

3.7 Routes

3.7.1 Single Route

⟨Conv(kernel) *routing fields* 28d⟩≡ (25a) 28e ▷

```
Route *r;     /* last route used */
```

⟨Conv(kernel) *routing fields* 28e⟩+≡ (25a) ◁ 28d

```
ulong rgen;     /* routetable generation for *r */
```

⟨*global* v4routegeneration 29a⟩≡ (339c)

```
static ulong v4routegeneration;
```

⟨*struct Route (kernel)* 29b⟩≡ (208)

```
struct Route
{
  RouteTree;

  union {
    V4route v4;
    ⟨Route ipv6 route union case 481g⟩
  };

};
```

⟨*struct* V4route 29c⟩≡ (208)

```
struct V4route
{
  iplong address;
  iplong endaddress;

  ipv4 gate;
};
```

⟨*struct RouteTree (kernel)* 29d⟩≡ (208)

```
struct  RouteTree
{
  // bitset<enum<route_type> >
  uchar type;

  Ipifc *ifc; // !!! the missing link!

  ⟨Routetree other fields 29e⟩

  // Extra
  Route*  right;
  Route*  left;
  Route*  mid;

};
```

⟨Routetree *other fields* 29e⟩≡ (29d) 29f ▷

```
char  tag[4];
```

⟨Routetree *other fields* 29f⟩+≡ (29d) ◁ 29e 59b ▷

```
int ref;
```

⟨*enum* _anon_ (kernel/network/ip/ip.h)6 29g⟩≡ (208)

```
enum route_type
{
  /* type bits */
  Rv4=    (1<<0),   /* this is a version 4 route */

  Rifc=   (1<<1),   /* this route is a directly connected interface */
  Rptpt=    (1<<2),   /* this route is a pt to pt interface */

  Runi=   (1<<3),   /* a unicast self address */

  Rbcast=   (1<<4),   /* a broadcast self address */
  Rmulti=   (1<<5),   /* a multicast self address */

  Rproxy=   (1<<6),   /* this route should be proxied */
};
```

⟨Ipifc(kernel) *routing fields* 30a⟩≡ (17c)

```
Routerparams rp;  /* router parameters as in RFC 2461, pp.40|43.
        used only if node is router */
```

3.7.2 Routing forest

⟨Fs(kernel) *routing fields* 30b⟩+≡ (21c) ◁21i 101c▷

```
// hash<ip, ref<Route>> where hash function is V4H
Route *v4root[1<<Lroot];  /* v4 routing forest */
```

⟨*constant* Lroot 30c⟩≡ (206b)

```
/* 2^Lroot trees in the root table */
Lroot=    10,
```

3.8 Blocks

⟨*macro* BLKIPVER 30d⟩≡ (212b)

```
#define BLKIPVER(xp)    (((Ip4hdr*)((xp)->rp))->vihl&0xF0)
```

⟨*macro* BLKIP 30e⟩≡ (212b)

```
#define BLKIP(xp)   ((Ip4hdr*)((xp)->rp))
```

Chapter 4

Initialization

4.1 Kernel side

4.1.1 Mounting the ip device

⟨*function* ipattach 31a⟩≡ (302c)

```
static Chan*
ipattach(char* spec)
{
    Chan *c;
    int dev;

    dev = atoi(spec);
    if(dev >= Nfs)
        error("bad specification");

    // initialize ip stack
    ipgetfs(dev);

    c = devattach('I', spec);
    mkqid(&c->qid, QID(0, 0, Qtopdir), 0, QTDIR);
    c->dev = dev;
    c->aux = newipaux(up->user, "none");

    return c;
}
```

⟨*function* ipgetfs 31b⟩≡ (302c)

```
static Fs*
ipgetfs(int dev)
{
    extern void (*ipprotoinit[])(Fs*);
    Fs *f;
    int i;

    if(dev >= Nfs)
        return nil;

    qlock(&fslock);
    if(ipfs[dev] == nil){
        f = smalloc(sizeof(Fs));

        ip_init(f);
        arpinit(f);
        netloginit(f);
```

```
        for(i = 0; ipprotoinit[i]; i++)
            ipprotoinit[i](f);

        f->dev = dev;
        ipfs[dev] = f;
    }
    qunlock(&fslock);

    return ipfs[dev];
}
```

⟨*struct* IPaux 32a⟩≡ (208)

```
/*
 *  Hanging off every ip channel's ->aux is the following structure.
 *  It maintains the state used by devip and iproute.
 */
struct IPaux
{
  char  *owner;   /* the user that did the attach */
  char  tag[4];
};
```

⟨*macro* ATTACHER 32b⟩≡ (302c)

```
#define ATTACHER(c) (((IPaux*)((c)->aux))->owner)
```

⟨*function* newipaux 32c⟩≡ (302c)

```
IPaux*
newipaux(char *owner, char *tag)
{
    IPaux *a;
    int n;

    a = smalloc(sizeof(IPaux));

    kstrdup(&a->owner, owner);
    memset(a->tag, ' ', sizeof(a->tag));
    n = strlen(tag);
    if(n > sizeof(a->tag))
        n = sizeof(a->tag);
    memmove(a->tag, tag, n);

    return a;
}
```

ip_init()

⟨*function* ip_init 32d⟩≡ (212b)

```
void
ip_init(Fs *f)
{
    IP *ip;

    ip = smalloc(sizeof(IP));
    initfrag(ip, 100);
    f->ip = ip;

    ⟨ip_init() ipv6 init 471e⟩
}
```

⟨*function* initfrag 33a⟩≡ (212b)

```
void
initfrag(IP *ip, int size)
{
    Fragment4 *fq4, *eq4;
    ⟨initfrag() locals 471f⟩

    ip->fragfree4 = (Fragment4*)malloc(sizeof(Fragment4) * size);
    if(ip->fragfree4 == nil)
        panic("initfrag");

    eq4 = &ip->fragfree4[size];
    for(fq4 = ip->fragfree4; fq4 < eq4; fq4++)
        fq4->next = fq4+1;
    ip->fragfree4[size-1].next = nil;

    ⟨initfrag() ipv6 init fragfree6 471g⟩
}
```

arpinit()

netloginit()

ipprotoinit()

4.1.2 Mounting the ethernet device

⟨*function* etherattach 33b⟩≡ (237b)

```
Chan*
etherattach(char* spec)
{
    ulong ctlrno;
    char *p;
    Chan *chan;

    ctlrno = 0;
    if(spec && *spec){
        ctlrno = strtoul(spec, &p, 0);
        if((ctlrno == 0 && p == spec) || *p || (ctlrno >= MaxEther))
            error(Ebadarg);
    }
    if(etherxx[ctlrno] == nil)
        error(Enodev);

    chan = devattach('l', spec);
    if(waserror()){
        chanfree(chan);
        nexterror();
    }
    chan->dev = ctlrno;
    if(etherxx[ctlrno]->attach)
        // Ethernet controller dispatch
        etherxx[ctlrno]->attach(etherxx[ctlrno]);

    poperror();
    return chan;
}
```

4.2 User side

4.2.1 Connect manually

4.2.2 `dial()`

Chapter 5

User/Kernel Bridge

5.1 IP device

⟨*global* ipdevtab 35a⟩≡ (302c)

```
Dev ipdevtab = {
    .dc       =    'I',
    .name     =    "ip",

    .attach   =    ipattach,
    .walk     =    ipwalk,
    .open     =    ipopen,
    .close    =    ipclose,
    .read     =    ipread,
    .write    =    ipwrite,
    .stat     =    ipstat,
    .wstat    =    ipwstat,

    .create   =    ipcreate,
    .remove   =    ipremove,
    .bread    =    ipbread,
    .bwrite   =    ipbwrite,
    .reset    =    ipreset,
    .init     =    devinit,
    .shutdown =    devshutdown,
};
```

⟨*function* ipcreate 35b⟩≡ (302c)

```
static void
ipcreate(Chan*, char*, int, ulong)
{
    error(Eperm);
}
```

⟨*function* ipremove 35c⟩≡ (302c)

```
static void
ipremove(Chan*)
{
    error(Eperm);
}
```

5.1.1 /net hierarchy and ipwalk()

⟨*function* ipwalk 35d⟩≡ (302c)

```
static Walkqid*
```

```
ipwalk(Chan* c, Chan *nc, char **name, int nname)
{
    IPaux *a = c->aux;
    Walkqid* w;

    w = devwalk(c, nc, name, nname, nil, 0, ipgen);

    if(w != nil && w->clone != nil)
        w->clone->aux = newipaux(a->owner, a->tag);

    return w;
}
```

⟨*function* ipgen 36a⟩≡ (302c)

```
static int
ipgen(Chan *c, char*, Dirtab*, int, int s, Dir *dp)
{
    Fs *f;
    Qid q;
    Conv *cv;

    f = ipfs[c->dev];
    switch(TYPE(c->qid)) {
    ⟨ipgen() switch TYPE qid cases 36b⟩
    }
    return -1;
}
```

/net/

⟨ipgen() *switch TYPE qid cases* 36b⟩≡ (36a) 36d ▷

```
case Qtopdir:
    if(s == DEVDOTDOT){
        mkqid(&q, QID(0, 0, Qtopdir), 0, QTDIR);
        snprint(up->genbuf, sizeof up->genbuf, "#I%lud", c->dev);
        devdir(c, q, up->genbuf, 0, network, 0555, dp);
        return 1;
    }
    if(s < f->np) {
        if(f->p[s]->connect == nil)
            return 0;   /* protocol with no user interface */
        mkqid(&q, QID(s, 0, Qprotodir), 0, QTDIR);
        devdir(c, q, f->p[s]->name, 0, network, 0555, dp);
        return 1;
    }
    s -= f->np;
    return ip1gen(c, Qtopbase+s, dp);
```

⟨*global* network 36c⟩≡ (302c)

```
static char network[] = "network";
```

/net/xxx

⟨ipgen() *switch TYPE qid cases* 36d⟩+≡ (36a) ◁36b 37b ▷

```
case Qarp:
case Qiproute:
case Qipselftab:
case Qndb:
case Qlog:
    return ip1gen(c, TYPE(c->qid), dp);
```

⟨*function* ip1gen 37a⟩≡ (302c)

```
static int
ip1gen(Chan *c, int i, Dir *dp)
{
    Fs *f;
    Qid q;
    int prot;
    int len = 0;
    char *p;
    ⟨ip1gen() locals 160d⟩

    f = ipfs[c->dev];
    prot = 0666;
    mkqid(&q, QID(0, 0, i), 0, QTFILE);
    switch(i) {
    ⟨ip1gen() switch TYPE qid cases 92d⟩
    default:
        return -1;
    }
    devdir(c, q, p, len, network, prot, dp);
    ⟨ipgen() if Qndb, adjust mtime 160e⟩
    return 1;
}
```

/net/proto/

⟨ipgen() *switch TYPE qid cases* 37b⟩+≡ (36a) ◁36d 37c▷

```
case Qprotodir:
    if(s == DEVDOTDOT){
        mkqid(&q, QID(0, 0, Qtopdir), 0, QTDIR);
        snprint(up->genbuf, sizeof up->genbuf, "#I%lud", c->dev);
        devdir(c, q, up->genbuf, 0, network, 0555, dp);
        return 1;
    }
    if(s < f->p[PROTO(c->qid)]->ac) {
        cv = f->p[PROTO(c->qid)]->conv[s];
        snprint(up->genbuf, sizeof up->genbuf, "%d", s);
        mkqid(&q, QID(PROTO(c->qid), s, Qconvdir), 0, QTDIR);
        devdir(c, q, up->genbuf, 0, cv->owner, 0555, dp);
        return 1;
    }
    s -= f->p[PROTO(c->qid)]->ac;
    return ip2gen(c, s+Qprotobase, dp);
```

/net/proto/xxx

⟨ipgen() *switch TYPE qid cases* 37c⟩+≡ (36a) ◁37b 38a▷

```
case Qclone:
case Qstats:
    return ip2gen(c, TYPE(c->qid), dp);
```

⟨*function* ip2gen 37d⟩≡ (302c)

```
static int
ip2gen(Chan *c, int i, Dir *dp)
{
    Qid q;

    switch(i) {
```

```
    case Qclone:
        mkqid(&q, QID(PROTO(c->qid), 0, Qclone), 0, QTFILE);
        devdir(c, q, "clone", 0, network, 0666, dp);
        return 1;
    case Qstats:
        mkqid(&q, QID(PROTO(c->qid), 0, Qstats), 0, QTFILE);
        devdir(c, q, "stats", 0, network, 0444, dp);
        return 1;
    }
    return -1;
}
```

/net/proto/conv/

⟨ipgen() *switch TYPE qid cases* 38a⟩+≡ (36a) ◁37c 38b▷

```
case Qconvdir:
    if(s == DEVDOTDOT){
        s = PROTO(c->qid);
        mkqid(&q, QID(s, 0, Qprotodir), 0, QTDIR);
        devdir(c, q, f->p[s]->name, 0, network, 0555, dp);
        return 1;
    }
    return ip3gen(c, s+Qconvbase, dp);
```

/net/proto/conv/xxx

⟨ipgen() *switch TYPE qid cases* 38b⟩+≡ (36a) ◁38a

```
case Qctl:
case Qdata:
case Qerr:
case Qlisten:
case Qlocal:
case Qremote:
case Qstatus:
case Qsnoop:
    return ip3gen(c, TYPE(c->qid), dp);
```

⟨*function* ip3gen 38c⟩≡ (302c)

```
static int
ip3gen(Chan *c, int i, Dir *dp)
{
    Qid q;
    Conv *cv;
    char *p;

    cv = ipfs[c->dev]->p[PROTO(c->qid)]->conv[CONV(c->qid)];
    if(cv->owner == nil)
        kstrdup(&cv->owner, eve);
    mkqid(&q, QID(PROTO(c->qid), CONV(c->qid), i), 0, QTFILE);
    switch(i) {
    ⟨ip3gen() switch TYPE qid cases 39a⟩
    default:
        return -1;
    }
    devdir(c, q, p, 0, cv->owner, 0444, dp);
    return 1;
}
```

⟨ip3gen() *switch TYPE qid cases* 39a⟩≡ (38c) 51e ▷

```
case Qctl:
    devdir(c, q, "ctl", 0, cv->owner, cv->perm, dp);
    return 1;
case Qdata:
    devdir(c, q, "data", qlen(cv->rq), cv->owner, cv->perm, dp);
    return 1;
```

5.1.2 Dispatch functions, ipxxx()

ipopen()

⟨*global* m2p 39b⟩≡ (302c)

```
static int m2p[] = {
    [OREAD]     4,
    [OWRITE]    2,
    [ORDWR]     6
};
```

⟨*function* ipopen 39c⟩≡ (302c)

```
static Chan*
ipopen(Chan* c, int omode)
{
    Fs *f;
    int perm;
    Proto *p;
    Conv *cv, *nc;

    f = ipfs[c->dev];
    perm = m2p[omode&3];
    switch(TYPE(c->qid)) {
    ⟨ipopen() switch TYPE qid cases 39d⟩
    default:
        break;
    }
    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    return c;
}
```

⟨ipopen() *switch TYPE qid cases* 39d⟩≡ (39c) 40a ▷

```
case Qtopdir:
case Qprotodir:
case Qconvdir:

case Qipselftab:

case Qstatus:
case Qlocal:
case Qremote:
case Qstats:
    if(omode != OREAD)
        error(Eperm);
    break;
```

⟨ipopen() *switch TYPE qid cases* 40a⟩+≡ (39c) ◁39d 42a▷

```
case Qctl:
case Qdata:
case Qerr:
    p = f->p[PROTO(c->qid)];
    qlock(p);
    cv = p->conv[CONV(c->qid)];
    qlock(cv);
    if(waserror()) {
        qunlock(cv);
        qunlock(p);
        nexterror();
    }
    if((perm & (cv->perm>>6)) != perm) {
        if(strcmp(ATTACHER(c), cv->owner) != 0)
            error(Eperm);
        if((perm & cv->perm) != perm)
            error(Eperm);

    }
    cv->inuse++;
    if(cv->inuse == 1){
        kstrdup(&cv->owner, ATTACHER(c));
        cv->perm = 0660;
    }
    qunlock(cv);
    qunlock(p);
    poperror();
    break;
```

ipclose()

⟨*function* ipclose 40b⟩≡ (302c)

```
static void
ipclose(Chan* c)
{
    Fs *f;

    f = ipfs[c->dev];
    switch(TYPE(c->qid)) {
    ⟨ipclose() switch TYPE qid cases 40c⟩
    default:
        break;
    }
    free(((IPaux*)c->aux)->owner);
    free(c->aux);
}
```

⟨ipclose() *switch TYPE qid cases* 40c⟩≡ (40b) 166d▷

```
case Qctl:
case Qdata:
case Qerr:
    if(c->flag & COPEN)
        closeconv(f->p[PROTO(c->qid)]->conv[CONV(c->qid)]);
    break;
```

⟨*function* closeconv 40d⟩≡ (302c)

```
void
closeconv(Conv *cv)
```

```
{
    Conv *nc;
    ⟨closeconv() locals 159f⟩

    qlock(cv);

    if(--cv->inuse > 0) {
        qunlock(cv);
        return;
    }
    ⟨closeconv() close incoming calls 52c⟩

    kstrdup(&cv->owner, network);
    cv->perm = 0660;
    ⟨closeconv() if multi, call ipifcremmulti 159g⟩
    cv->r = nil;
    cv->rgen = 0;
    // Protocol dispatch
    cv->p->close(cv);
    cv->state = Idle;

    qunlock(cv);
}
```

ipread()

⟨*function* ipread 41a⟩≡ (302c)

```
static long
ipread(Chan *ch, void *a, long n, vlong off)
{
    Fs *f;
    Proto *x;
    Conv *cv;
    char *buf, *p;
    long rv;
    ulong offset = off;

    f = ipfs[ch->dev];
    p = a;
    switch(TYPE(ch->qid)) {
    ⟨ipread() switch TYPE qid cases 41b⟩
    default:
        error(Eperm);
        return -1; // unreachable

    }
}
```

⟨ipread() *switch TYPE qid cases* 41b⟩≡ (41a) 44e ▷

```
case Qtopdir:
case Qprotodir:
case Qconvdir:
    return devdirread(ch, a, n, 0, 0, ipgen);
```

ipwrite()

⟨*function* ipwrite 41c⟩≡ (302c)

```
static long
```

```
ipwrite(Chan* ch, void *v, long n, vlong off)
{
    Fs *f;
    Proto *x;
    Conv *cv;
    char *p; // err?
    Cmdbuf *cb;
    char *a;
    ulong offset = off;
    ⟨ipwrite() locals 158d⟩

    f = ipfs[ch->dev];
    a = v;
    switch(TYPE(ch->qid)){
    ⟨ipwrite() switch TYPE qid cases 44f⟩
    default:
        error(Eperm);
    }
    return n;
}
```

5.1.3 /net/x/clone

⟨ipopen() *switch TYPE qid cases* 42a⟩+≡ (39c) ◁40a 52a▷

```
case Qclone:
    p = f->p[PROTO(c->qid)];

    qlock(p);
    if(waserror()){
        qunlock(p);
        nexterror();
    }

    cv = Fsprotoclone(p, ATTACHER(c));

    qunlock(p);
    poperror();
    if(cv == nil) {
        error(Enodev);
        break;
    }
    mkqid(&c->qid, QID(p->x, cv->x, Qctl), 0, QTFILE);
    break;
```

⟨*function* Fsprotoclone 42b⟩≡ (302c)

```
/*
 *  called with protocol locked
 */
Conv*
Fsprotoclone(Proto *p, char *user)
{
    Conv *cv, **pp, **ep;

retry:
    cv = nil;
    ⟨Fsprotoclone() finding an available conversation in the protocol 43⟩
    ⟨Fsprotoclone() if no more available conv, garbage collect and retry 44c⟩

    cv->inuse = 1;
```

```
    kstrdup(&cv->owner, user);
    cv->perm = 0660;

    cv->state = Idle;

    ipmove(cv->laddr, IPnoaddr);
    ipmove(cv->raddr, IPnoaddr);
    cv->lport = 0;
    cv->rport = 0;

    cv->r = nil;
    cv->rgen = 0;

    cv->restricted = false;
    cv->maxfragsize = 0;
    cv->ttl = MAXTTL;

    qreopen(cv->rq);
    qreopen(cv->wq);
    qreopen(cv->eq);

    qunlock(cv);
    return cv;
}
```

Finding a free entry

⟨Fsprotoclone() *finding an available conversation in the protocol* 43⟩≡ (42b)

```
ep = &p->conv[p->nc];
for(pp = p->conv; pp < ep; pp++) {
    cv = *pp;
    // found an unallocated entry in the array
    if(cv == nil){
        cv = malloc(sizeof(Conv));
        if(cv == nil)
            error(Enomem);
        qlock(cv);

        cv->p = p;
        cv->x = pp - p->conv;
        if(p->ptclsize != 0){
            cv->ptcl = malloc(p->ptclsize);
            if(cv->ptcl == nil) {
                free(cv);
                error(Enomem);
            }
        }
        *pp = cv;
        p->ac++;
        cv->eq = qopen(1024, Qmsg, 0, 0);

        // !! Protocol dispatch !!! will create extra queues
        (*p->create)(cv);

        break;
    }
    ⟨Fsprotoclone() if found an unused entry 44a⟩
}
```

Reuse an unused entry

⟨Fsprotoclone() *if found an unused entry* 44a⟩≡ (43)

```
if(canqlock(cv)){
    /*
     *  make sure both processes and protocol
     *  are done with this Conv
     */
    if(cv->inuse == 0 && (p->inuse == nil ||
        // Protocol dispatch
        (*p->inuse)(cv) == false)
      )
        break;

    qunlock(cv);
}
```

⟨Proto(kernel) *conversation methods* 44b⟩+≡ (24e) ◁24g 194▷

```
bool   (*inuse)(Conv*);
```

Garbage collecting

⟨Fsprotoclone() *if no more available conv, garbage collect and retry* 44c⟩≡ (42b)

```
if(pp >= ep) {
    if(p->gc)
        print("Fsprotoclone: garbage collecting Convs\n");
    if(p->gc != nil &&
        // Protocol dispatch
        (*p->gc)(p)
       )
        goto retry;

    /* debugging: do we ever get here? */
    if (cpuserver)
        panic("Fsprotoclone: all conversations in use");
    return nil;
}
```

⟨Proto(kernel) *protocol methods* 44d⟩+≡ (24e) ◁24f 53b▷

```
int   (*gc)(Proto*);  /* returns true if any conversations are freed */
```

5.1.4 /net/x/y/ctl

⟨ipread() *switch TYPE qid cases* 44e⟩+≡ (41a) ◁41b 51a▷

```
case Qctl:
    buf = smalloc(16);
    snprint(buf, 16, "%lud", CONV(ch->qid));
    rv = readstr(offset, p, n, buf);
    free(buf);
    return rv;
```

⟨ipwrite() *switch TYPE qid cases* 44f⟩≡ (41c) 51b▷

```
case Qctl:
    x = f->p[PROTO(ch->qid)];
    cv = x->conv[CONV(ch->qid)];
    cb = parsecmd(a, n);

    qlock(cv);
    if(waserror()) {
```

```
        qunlock(cv);
        free(cb);
        nexterror();
    }
    if(cb->nf < 1)
        error("short control request");

    ⟨ipwrite() Qctl case, if connect string 45a⟩
    ⟨ipwrite() Qctl case, else if announce string 47a⟩
    ⟨ipwrite() Qctl case, else if other string 50b⟩
    else if(x->ctl != nil) {
        // Protocol dispatch
        p = x->ctl(cv, cb->f, cb->nf);
        if(p != nil)
            error(p);
    } else
        error("unknown control request");
    qunlock(cv);
    free(cb);
    poperror();
    break;
```

Connect

⟨ipwrite() *Qctl case, if connect string* 45a⟩≡ (44f)

```
if(strcmp(cb->f[0], "connect") == 0)
    connectctlmsg(x, cv, cb);
```

⟨*function* connectctlmsg 45b⟩≡ (302c)

```
static void
connectctlmsg(Proto *p, Conv *c, Cmdbuf *cb)
{
    char *err;

    if(c->state != Idle)
        error(Econinuse);

    c->state = Connecting;
    c->cerr[0] = '\0';

    if(p->connect == nil)
        error("connect not supported");
    // Protocol dispatch
    err = p->connect(c, cb->f, cb->nf);
    if(err != nil)
        error(err);

    qunlock(c);
    if(waserror()){
        qlock(c);
        nexterror();
    }
    sleep(&c->cr, connected, c);
    qlock(c);
    poperror();

    if(c->cerr[0] != '\0')
        error(c->cerr);
}
```

⟨*function* connected 46a⟩≡ (302c)

```
/*
 *  initiate connection and sleep till its set up
 */
static bool
connected(void* a)
{
    return ((Conv*)a)->state == Connected;
}
```

⟨*function* Fsstdconnect 46b⟩≡ (302c)

```
/*
 *  called by protocol connect routine to set addresses
 */
char*
Fsstdconnect(Conv *c, char *argv[], int argc)
{
    char *err;

    switch(argc) {
    case 2:
        err = setraddrport(c, argv[1]);
        if(err != nil)
            return err;
        setladdr(c);
        err = setlport(c);
        if (err != nil)
            return err;
        break;
    case 3:
        err = setraddrport(c, argv[1]);
        if(err != nil)
            return err;
        err = setladdrport(c, argv[2], 0);
        if(err != nil)
            return err;
    default:
        return "bad args to connect";
    }

    ⟨Fsstdconnect() set ipversion field to V4 or V6 481i⟩

    return nil;
}
```

⟨*function* Fsconnected 46c⟩≡ (302c)

```
int
Fsconnected(Conv* c, char* msg)
{
    if(msg != nil && *msg != '\0')
        strncpy(c->cerr, msg, ERRMAX-1);

    switch(c->state){
    case Connecting:
        c->state = Connected;
        break;
    ⟨Fsconnected() switch state cases 48a⟩
    }
    wakeup(&c->cr);
    return 0;
}
```

Announce

⟨ipwrite() *Qctl case, else if announce string* 47a⟩≡ (44f)

```
else if(strcmp(cb->f[0], "announce") == 0)
    announcectlmsg(x, cv, cb);
```

⟨*function* announcectlmsg 47b⟩≡ (302c)

```
static void
announcectlmsg(Proto *p, Conv *c, Cmdbuf *cb)
{
    char *err;

    if(c->state != Idle)
        error(Econinuse);

    c->state = Announcing;
    c->cerr[0] = '\0';

    if(p->announce == nil)
        error("announce not supported");
    // Protocol dispatch
    err = p->announce(c, cb->f, cb->nf);

    if(err != nil)
        error(err);

    qunlock(c);
    if(waserror()){
        qlock(c);
        nexterror();
    }
    sleep(&c->cr, announced, c);
    qlock(c);
    poperror();

    if(c->cerr[0] != '\0')
        error(c->cerr);
}
```

⟨*function* announced 47c⟩≡ (302c)

```
/*
 *  initiate announcement and sleep till its set up
 */
static bool
announced(void* a)
{
    return ((Conv*)a)->state == Announced;
}
```

⟨*function* Fsstdannounce 47d⟩≡ (302c)

```
/*
 *  called by protocol announce routine to set addresses
 */
char*
Fsstdannounce(Conv* c, char* argv[], int argc)
{
    memset(c->raddr, 0, sizeof(c->raddr));
    c->rport = 0;
    switch(argc){
    default:
```

```
        break;
    case 2:
        return setladdrport(c, argv[1], 1);
    }
    return "bad args to announce";
}
```

⟨Fsconnected() *switch state cases* 48a⟩≡ (46c)

```
case Announcing:
    c->state = Announced;
    break;
```

Port settings

⟨Conv(kernel) *other fields* 48b⟩+≡ (25a) ◁26e 68e▷

```
bool restricted;   /* remote port is restricted */
```

⟨Proto(kernel) *other fields* 48c⟩+≡ (23b) ◁24c

```
ushort    nextrport;
```

⟨*function* setlport 48d⟩≡ (302c)

```
/*
 *  pick a local port and set it
 */
char *
setlport(Conv* c)
{
    Proto *p;
    int i, port;

    p = c->p;
    qlock(p);
    if(c->restricted){
        /* Restricted ports cycle between 600 and 1024. */
        for(i=0; i<1024-600; i++){
            if(p->nextrport >= 1024 || p->nextrport < 600)
                p->nextrport = 600;
            port = p->nextrport++;
            if(!lportinuse(p, port))
                goto chosen;
        }
    }else{
        /*
         * Unrestricted ports are chosen randomly
         * between 2^15 and 2^16.  There are at most
         * 4*Nchan = 4096 ports in use at any given time,
         * so even in the worst case, a random probe has a
         * 1 - 4096/2^15 = 87% chance of success.
         * If 64 successive probes fail, there is a bug somewhere
         * (or a once in 10^58 event has happened, but that's
         * less likely than a venti collision).
         */
        for(i=0; i<64; i++){
            port = (1<<15) + nrand(1<<15);
            if(!lportinuse(p, port))
                goto chosen;
        }
    }
    qunlock(p);
```

```
    /*
     * debugging: let's see if we ever get this.
     * if we do (and we're a cpu server), we might as well restart
     * since we're now unable to service new connections.
     */
    panic("setlport: out of ports");
    return "no ports available";

chosen:
    c->lport = port;
    qunlock(p);
    return nil;
}
```

⟨*function* setladdrport 49⟩≡ (302c)

```
/*
 *  set a local address and port from a string of the form
 *  [address!]port[!r]
 */
char*
setladdrport(Conv* c, char* str, int announcing)
{
    char *p;
    char *rv;
    ushort lport;
    ipaddr addr;

    /*
     *  ignore restricted part if it exists.  it's
     *  meaningless on local ports.
     */
    p = strchr(str, '!');
    if(p != nil){
        *p++ = 0;
        if(strcmp(p, "r") == 0)
            p = nil;
    }

    c->lport = 0;
    if(p == nil){
        if(announcing)
            ipmove(c->laddr, IPnoaddr);
        else
            setladdr(c);
        p = str;
    } else {
        if(strcmp(str, "*") == 0)
            ipmove(c->laddr, IPnoaddr);
        else {
            if(parseip(addr, str) == -1)
                return Ebadip;
            if(ipforme(c->p->f, addr))
                ipmove(c->laddr, addr);
            else
                return "not a local IP address";
        }
    }

    /* one process can get all connections */
    if(announcing && strcmp(p, "*") == 0){
```

```
        if(!iseve())
            error(Eperm);
        return setluniqueport(c, 0);
    }

    lport = atoi(p);
    if(lport <= 0)
        rv = setlport(c);
    else
        rv = setluniqueport(c, lport);
    return rv;
}
```

⟨*function* setraddrport 50a⟩≡ (302c)

```
static char*
setraddrport(Conv* c, char* str)
{
    char *p;

    p = strchr(str, '!');
    if(p == nil)
        return "malformed address";
    *p++ = 0;
    if (parseip(c->raddr, str) == -1)
        return Ebadip;
    c->rport = atoi(p);
    p = strchr(p, '!');
    if(p){
        if(strstr(p, "!r") != nil)
            c->restricted = true;
    }
    return nil;
}
```

Bind

⟨ipwrite() *Qctl case, else if other string* 50b⟩≡ (44f) 68f ▷

```
else if(strcmp(cb->f[0], "bind") == 0)
    bindctlmsg(x, cv, cb);
```

⟨*function* bindctlmsg 50c⟩≡ (302c)

```
static void
bindctlmsg(Proto *x, Conv *cv, Cmdbuf *cb)
{
    char *p;

    if(x->bind == nil)
        p = Fsstdbind(cv, cb->f, cb->nf);
    else
        // Protocol dispatch
        p = x->bind(cv, cb->f, cb->nf);
    if(p != nil)
        error(p);
}
```

⟨*function* Fsstdbind 50d⟩≡ (302c)

```
/*
 *  called by protocol bind routine to set addresses
 */
```

```
char*
Fsstdbind(Conv* cv, char* argv[], int argc)
{
    switch(argc){
    default:
        break;
    case 2:
        return setladdrport(cv, argv[1], 0);
    }
    return "bad args to bind";
}
```

5.1.5 /net/x/y/data

⟨ipread() *switch TYPE qid cases* 51a⟩+≡ (41a) ◁44e 51f▷

```
case Qdata:
    cv = f->p[PROTO(ch->qid)]->conv[CONV(ch->qid)];
    return qread(cv->rq, a, n);
```

⟨ipwrite() *switch TYPE qid cases* 51b⟩+≡ (41c) ◁44f 93d▷

```
case Qdata:
    cv = f->p[PROTO(ch->qid)]->conv[CONV(ch->qid)];
    if(cv->wq == nil)
        error(Eperm);
    qwrite(cv->wq, a, n);
    break;
```

5.1.6 /net/x/y/err

⟨Conv(kernel) *queue fields* 51c⟩+≡ (25a) ◁26a

```
Queue*  eq;     /* returned error packets */
```

⟨Qid *conversation extra cases* 51d⟩≡ (26f) 51g▷

```
Qerr,
```

⟨ip3gen() *switch TYPE qid cases* 51e⟩+≡ (38c) ◁39a 51h▷

```
case Qerr:
    devdir(c, q, "err", qlen(cv->eq), cv->owner, cv->perm, dp);
    return 1;
```

⟨ipread() *switch TYPE qid cases* 51f⟩+≡ (41a) ◁51a 53c▷

```
case Qerr:
    cv = f->p[PROTO(ch->qid)]->conv[CONV(ch->qid)];
    return qread(cv->eq, a, n);
```

5.1.7 /net/x/y/listen

⟨Qid *conversation extra cases* 51g⟩+≡ (26f) ◁51d 53e▷

```
Qlisten,
```

⟨ip3gen() *switch TYPE qid cases* 51h⟩+≡ (38c) ◁51e 53f▷

```
case Qlisten:
    devdir(c, q, "listen", 0, cv->owner, cv->perm, dp);
    return 1;
```

⟨ipopen() *switch TYPE qid cases* 52a⟩+≡ (39c) ◁42a 93a▷

```
  case Qlisten:
      cv = f->p[PROTO(c->qid)]->conv[CONV(c->qid)];
      if((perm & (cv->perm>>6)) != perm) {
          if(strcmp(ATTACHER(c), cv->owner) != 0)
              error(Eperm);
          if((perm & cv->perm) != perm)
              error(Eperm);

      }

      if(cv->state != Announced)
          error("not announced");

      if(waserror()){
          closeconv(cv);
          nexterror();
      }
      qlock(cv);
      cv->inuse++;
      qunlock(cv);

      nc = nil;
      while(nc == nil) {
          /* give up if we got a hangup */
          if(qisclosed(cv->rq))
              error("listen hungup");

          qlock(&cv->listenq);
          if(waserror()) {
              qunlock(&cv->listenq);
              nexterror();
          }

          /* wait for a connect */
          sleep(&cv->listenr, incoming, cv);

          qlock(cv);
          nc = cv->incall;
          if(nc != nil){
              cv->incall = nc->next;
              mkqid(&c->qid, QID(PROTO(c->qid), nc->x, Qctl), 0, QTFILE);
              kstrdup(&cv->owner, ATTACHER(c));
          }
          qunlock(cv);

          qunlock(&cv->listenq);
          poperror();
      }
      closeconv(cv);
      poperror();
      break;
```

⟨Conv(kernel) *listen fields* 52b⟩≡ (25a)

```
  Conv* incall;     /* calls waiting to be listened for */
  Conv* next;
  QLock listenq;
  Rendez  listenr;
```

⟨closeconv() *close incoming calls* 52c⟩≡ (40d)

```
/* close all incoming calls since no listen will ever happen */
for(nc = cv->incall; nc; nc = cv->incall){
    cv->incall = nc->next;
    closeconv(nc);
}
cv->incall = nil;
```

5.1.8 Other files

/net/x/stats

⟨Qid *protocol extra cases* 53a⟩≡ (26f)

```
Qstats,
```

⟨Proto(kernel) *protocol methods* 53b⟩+≡ (24e) ◁44d

```
int   (*stats)(Proto*, char*, int);
```

⟨ipread() *switch TYPE qid cases* 53c⟩+≡ (41a) ◁51f 53h▷

```
case Qstats:
    x = f->p[PROTO(ch->qid)];
    if(x->stats == nil)
        error("stats not implemented");
    buf = smalloc(Statelen);

    // Protocol dispatch
    (*x->stats)(x, buf, Statelen);

    rv = readstr(offset, p, n, buf);
    free(buf);
    return rv;
```

⟨*constant* Statelen 53d⟩≡ (300a)

```
    Statelen=   32*1024,
```

/net/x/y/status

⟨Qid *conversation extra cases* 53e⟩+≡ (26f) ◁51g 54a▷

```
Qstatus,
```

⟨ip3gen() *switch TYPE qid cases* 53f⟩+≡ (38c) ◁51h 54b▷

```
case Qstatus:
    p = "status";
    break;
```

⟨Proto(kernel) *conversation inspection methods* 53g⟩≡ (24e) 54c▷

```
int   (*state)(Conv*, char*, int);
```

⟨ipread() *switch TYPE qid cases* 53h⟩+≡ (41a) ◁53c 54d▷

```
case Qstatus:
    buf = smalloc(Statelen);
    x = f->p[PROTO(ch->qid)];
    cv = x->conv[CONV(ch->qid)];

    // Protocol dispatch
    (*x->state)(cv, buf, Statelen-2);

    rv = readstr(offset, p, n, buf);
    free(buf);
    return rv;
```

/net/x/y/local, /net/x/y/remote

⟨Qid *conversation extra cases* 54a⟩+≡ (26f) ◁53e
```
  Qlocal,
  Qremote,
```

⟨ip3gen() *switch TYPE qid cases* 54b⟩+≡ (38c) ◁53f 169c▷
```
  case Qlocal:
      p = "local";
      break;
  case Qremote:
      p = "remote";
      break;
```

⟨Proto(kernel) *conversation inspection methods* 54c⟩+≡ (24e) ◁53g 54e▷
```
  int   (*local)(Conv*, char*, int);
```

⟨ipread() *switch TYPE qid cases* 54d⟩+≡ (41a) ◁53h 54f▷
```
  case Qlocal:
      buf = smalloc(Statelen);
      x = f->p[PROTO(ch->qid)];
      cv = x->conv[CONV(ch->qid)];
      if(x->local == nil) {
          snprint(buf, Statelen, "%I!%d\n", cv->laddr, cv->lport);
      } else {
          // Protocol dispatch
          (*x->local)(cv, buf, Statelen-2);
      }
      rv = readstr(offset, p, n, buf);
      free(buf);
      return rv;
```

⟨Proto(kernel) *conversation inspection methods* 54e⟩+≡ (24e) ◁54c
```
  int   (*remote)(Conv*, char*, int);
```

⟨ipread() *switch TYPE qid cases* 54f⟩+≡ (41a) ◁54d 93b▷
```
  case Qremote:
      buf = smalloc(Statelen);
      x = f->p[PROTO(ch->qid)];
      cv = x->conv[CONV(ch->qid)];
      if(x->remote == nil) {
          snprint(buf, Statelen, "%I!%d\n", cv->raddr, cv->rport);
      } else {
          // Protocol dispatch
          (*x->remote)(cv, buf, Statelen-2);
      }
      rv = readstr(offset, p, n, buf);
      free(buf);
      return rv;
```

5.2 Ethernet device

⟨*global* etherdevtab 54g⟩≡ (237b)
```
  Dev etherdevtab = {
      .dc       =    'l',
      .name     =    "ether",

      .attach   =    etherattach,
      .walk     =    etherwalk,
```

```
    .open     =    etheropen,
    .close    =    etherclose,
    .read     =    etherread,
    .write    =    etherwrite,
    .stat     =    etherstat,
    .wstat    =    etherwstat,

    .reset    =    etherreset,
    .init     =    devinit,
    .shutdown =    ethershutdown,
    .create   =    ethercreate,
    .bread    =    etherbread,
    .bwrite   =    etherbwrite,
    .remove   =    devremove,
};
```

5.2.1 /net/etherx hierarchy and etherwalk()

Chapter 6

Configuration

6.1 /net/ipifc/ protocol

6.1.1 Protocol initialisation

⟨*function* ipifcinit 56a⟩≡ (354b)

```
void
ipifcinit(Fs *f)
{
    Proto *ipifc;

    ipifc = smalloc(sizeof(Proto));

    ipifc->name = "ipifc";
    ipifc->create = ipifccreate;
    ipifc->close = ipifcclose;

    ipifc->bind = ipifcbind;
    ipifc->connect = ipifcconnect;
    ipifc->announce = nil;
    ipifc->ctl = ipifcctl;

    ipifc->rcv = nil;
    ipifc->advise = nil;
    ipifc->inuse = ipifcinuse;

    ipifc->local = ipifclocal;
    ipifc->state = ipifcstate;
    ipifc->stats = ipifcstats;

    ipifc->ipproto = -1;

    ipifc->nc = Maxmedia;
    ipifc->ptclsize = sizeof(Ipifc);

    ⟨ipifcinit() modify f 57b⟩

    Fsproto(f, ipifc);
}
```

⟨*function* Fsproto 56b⟩≡ (302c)

```
int
Fsproto(Fs *f, Proto *p)
{
    if(f->np >= Maxproto)
```

```
        return -1;

    p->f = f;

    ⟨Fsproto() adjust f-¿t2p 57a⟩

    p->conv = malloc(sizeof(Conv*) * (p->nc+1));
    if(p->conv == nil)
        panic("Fsproto");

    p->nextrport = 600;

    p->x = f->np;
    f->p[f->np++] = p;

    return 0;
}
```

⟨Fsproto() *adjust f-¿t2p* 57a⟩≡ (56b)

```
if(p->ipproto > 0){
    if(f->t2p[p->ipproto] != nil)
        return -1;
    f->t2p[p->ipproto] = p;
}
```

⟨ipifcinit() *modify f* 57b⟩≡ (56a) 108e ▷

```
f->ipifc = ipifc;   /* hack for ipifcremroute, findipifc, ... */
```

6.1.2 /net/ipifc/clone

⟨*function* ipifccreate 57c⟩≡ (354b)

```
/*
 *  called when a new ipifc structure is created
 */
static void
ipifccreate(Conv *cv)
{
    Ipifc *ifc;

    cv->rq = qopen(QMAX, 0, 0, 0);
    cv->wq = qopen(QMAX, Qkick, ipifckick, cv);

    cv->sq = qopen(QMAX, 0, 0, 0);

    ifc = (Ipifc*)cv->ptcl;
    ifc->m = nil;

    ifc->conv = cv;
    ifc->reassemble = false;
}
```

6.1.3 Binding medium: /net/ipifc/x/ctl bind

⟨*function* ipifcbind 57d⟩≡ (354b)

```
/*
 *  attach a device (or pkt driver) to the interface.
 *  called with cv locked
 */
```

```
static char*
ipifcbind(Conv *cv, char **argv, int argc)
{
    Ipifc *ifc;
    Medium *m;

    if(argc < 2)
        return Ebadarg;

    ifc = (Ipifc*)cv->ptcl;

    /* bind the device to the interface */
    m = ipfindmedium(argv[1]);
    if(m == nil)
        return "unknown interface type";

    wlock(ifc);
    if(ifc->m != nil){
        wunlock(ifc);
        return "interface already bound";
    }
    if(waserror()){
        wunlock(ifc);
        nexterror();
    }

    // This time Medium dispatch
    /* do medium specific binding */
    (*m->bind)(ifc, argc, argv);

    /* set the bound device name */
    if(argc > 2)
        strncpy(ifc->dev, argv[2], sizeof(ifc->dev));
    else
        snprint(ifc->dev, sizeof ifc->dev, "%s%d", m->name, cv->x);
    ifc->dev[sizeof(ifc->dev)-1] = 0;

    /* set up parameters */
    ifc->m = m;

    ifc->mintu = ifc->m->mintu;
    ifc->maxtu = ifc->m->maxtu;
    if(ifc->m->unbindonclose == false)
        ifc->conv->inuse++;

    ifc->rp.mflag = 0;      /* default not managed */
    ifc->rp.oflag = 0;
    ifc->rp.maxraint = 600000;  /* millisecs */
    ifc->rp.minraint = 200000;
    ifc->rp.linkmtu = 0;        /* no mtu sent */
    ifc->rp.reachtime = 0;
    ifc->rp.rxmitra = 0;
    ifc->rp.ttl = MAXTTL;
    ifc->rp.routerlt = 3 * ifc->rp.maxraint;

    /* any ancillary structures (like routes) no longer pertain */
    ifc->ifcid++;

    /* reopen all the queues closed by a previous unbind */
    qreopen(cv->rq);
```

```
    qreopen(cv->eq);
    qreopen(cv->sq);

    wunlock(ifc);
    poperror();

    return nil;
}
```

⟨Ipifc(kernel) *other fields* 59a⟩+≡ (17c) ◁17e 90b▷

```
uchar ifcid;    /* incremented each 'bind/unbind/add/remove' */
```

⟨Routetree *other fields* 59b⟩+≡ (29d) ◁29f 102a▷

```
uchar ifcid;    /* must match ifc->id */
```

⟨*function* ipfindmedium 59c⟩≡ (354b)

```
/*
 *  find the medium with this name
 */
Medium*
ipfindmedium(char *name)
{
    Medium **mp;

    for(mp = media; *mp != nil; mp++)
        if(strcmp((*mp)->name, name) == 0)
            break;
    return *mp;
}
```

6.1.4 /net/ipifc/x/ctl

⟨*function* ipifcctl 59d⟩≡ (354b)

```
/*
 *  non-standard control messages.
 *  called with cv->car locked.
 */
static char*
ipifcctl(Conv* cv, char** argv, int argc)
{
    Ipifc *ifc;
    int i;

    ifc = (Ipifc*)cv->ptcl;
    ⟨ipifcctl() if add string 59e⟩
    ⟨ipifcctl() else if other string 67⟩

    return "unsupported ctl";
}
```

6.1.5 Adding an IP: /net/ipifc/x/ctl add

⟨ipifcctl() *if add string* 59e⟩≡ (59d)

```
if(strcmp(argv[0], "add") == 0)
    return ipifcadd(ifc, argv, argc, false, nil);
```

⟨*function* ipifcadd 60⟩≡ (354b)

```
/*
 *  add an address to an interface.
 */
char*
ipifcadd(Ipifc *ifc, char **argv, int argc, bool tentative, Iplifc *lifcp)
{
    int i;
    int mtu;
    // enum<route_type>
    int type;
    ipaddr ip;
    ipaddr mask;
    ipaddr net; // ip & mask
    ipaddr rem;
    Iplifc *lifc, **l;
    Fs *f;

    ⟨ipifcadd() locals 157c⟩

    if(ifc->m == nil)
        return "ipifc not yet bound to device";

    f = ifc->conv->p->f;

    type = Rifc;
    memset(ip, 0, IPaddrlen);
    memset(mask, 0, IPaddrlen);
    memset(rem, 0, IPaddrlen);

    switch(argc){
    ⟨ipifcadd() switch argc, proxy case, and fall through 161b⟩
    ⟨ipifcadd() switch argc, mtu setting case, and fall through 68b⟩
    ⟨ipifcadd() switch argc cases, setting ip, mask, net, rem 61a⟩
    default:
        return Ebadarg;
    }

    ⟨ipifcadd() set tentative for ipv6 482c⟩
    wlock(ifc);
    ⟨ipifcadd() check if already a local address for this ifc 61c⟩

    /* add the address to the list of logical ifc's for this ifc */
    lifc = smalloc(sizeof(Iplifc));
    ipmove(lifc->local, ip);
    ipmove(lifc->mask, mask);
    ipmove(lifc->remote, rem);
    ipmove(lifc->net, net);
    ⟨ipifcadd() set ipv6 fields for lifc 483a⟩
    // add_tail(lifc, ifc->lifc)
    lifc->next = nil;
    for(l = &ifc->lifc; *l; l = &(*l)->next)
        ;
    *l = lifc;

    ⟨ipifcadd() check for point to point interface 160i⟩

    /* add local routes */
    if(isv4(ip))
        v4addroute(f, tifc, rem+IPv4off, mask+IPv4off, rem+IPv4off, type);
```

```
    ⟨ipifcadd() add route if ipv6 case 481j⟩

    addselfcache(f, ifc, lifc, ip, Runi);

    ⟨ipifcadd() register proxy if point to point interface or proxy 161a⟩

    if(isv4(ip) || ipcmp(ip, IPnoaddr) == 0) {
        ⟨ipifcadd() add broadcast addresses to self cache 157d⟩
    }
    ⟨ipifcadd() if ipv6 add multicast addresses to self cache 482i⟩

    /* register the address on this network for address resolution */
    if(isv4(ip) && ifc->m->areg != nil)
        // Medium dispatch
        (*ifc->m->areg)(ifc, ip);

out:
    wunlock(ifc);
    ⟨ipifcadd() if ipv6 tentative and broacast 482g⟩
    return nil;
}
```

⟨ipifcadd() *switch argc cases, setting ip, mask, net, rem* 61a⟩≡ (60)

```
// add <ip> <mask> <rem>
case 4:
    if (parseip(ip, argv[1]) == -1 || parseip(rem, argv[3]) == -1)
        return Ebadip;
    parseipmask(mask, argv[2]);
    maskip(rem, mask, net);
    break;
// add <ip> <mask>
case 3:
    if (parseip(ip, argv[1]) == -1)
        return Ebadip;
    parseipmask(mask, argv[2]);
    maskip(ip, mask, rem);
    maskip(rem, mask, net);
    break;
// simplest case, add <ip>
case 2:
    if (parseip(ip, argv[1]) == -1)
        return Ebadip;
    memmove(mask, defmask(ip), IPaddrlen);
    maskip(ip, mask, rem);
    maskip(rem, mask, net);
    break;
```

⟨*global* tifc 61b⟩≡ (354b)

```
static char tifc[] = "ifc ";
```

⟨ipifcadd() *check if already a local address for this ifc* 61c⟩≡ (60)

```
/* ignore if this is already a local address for this ifc */
for(lifc = ifc->lifc; lifc; lifc = lifc->next) {
    if(ipcmp(lifc->local, ip) == 0) {
        ⟨ipifcadd() when already local address for ifc, copy ipv6 fields 483b⟩
        goto out;
    }
}
```

6.2 IP Interface, user side

6.2.1 Parsing

⟨*function* readipifc 62a⟩≡ (205b)

```
Ipifc*
readipifc(char *net, Ipifc *ifc, int index)
{
    int fd, i, n;
    Dir *dir;
    char directory[128];
    char buf[128];
    Ipifc **l;

    _freeifc(ifc);

    l = &ifc;
    ifc = nil;

    if(net == nil)
        net = "/net";
    snprint(directory, sizeof(directory), "%s/ipifc", net);

    if(index >= 0){
        snprint(buf, sizeof(buf), "%s/%d/status", directory, index);
        _readipifc(buf, l, index);
    } else {
        fd = open(directory, OREAD);
        if(fd < 0)
            return nil;
        n = dirreadall(fd, &dir);
        close(fd);

        for(i = 0; i < n; i++){
            if(strcmp(dir[i].name, "clone") == 0)
                continue;
            if(strcmp(dir[i].name, "stats") == 0)
                continue;
            snprint(buf, sizeof(buf), "%s/%s/status", directory, dir[i].name);
            l = _readipifc(buf, l, atoi(dir[i].name));
        }
        free(dir);
    }

    return ifc;
}
```

⟨*function* _readipifc 62b⟩≡ (205b)

```
static Ipifc**
_readipifc(char *file, Ipifc **l, int index)
{
    int i, n, fd, lines;
    char buf[4*1024];
    char *line[32];
    char *f[64];
    Ipifc *ifc, **l0;
    Iplifc *lifc, **ll;

    /* read the file */
    fd = open(file, OREAD);
```

```
    if(fd < 0)
        return l;
    n = 0;
    while((i = read(fd, buf+n, sizeof(buf)-1-n)) > 0 && n < sizeof(buf) - 1)
        n += i;
    buf[n] = 0;
    close(fd);

    //if(strncmp(buf, "device", 6) != 0)
    //    return _readoldipifc(buf, l, index);

    /* ignore ifcs with no associated device */
    if(strncmp(buf+6, "  ", 2) == 0)
        return l;
    /* allocate new interface */
    *l = ifc = mallocz(sizeof(Ipifc), 1);
    if(ifc == nil)
        return l;
    l0 = l;
    l = &ifc->next;
    ifc->index = index;

    lines = getfields(buf, line, nelem(line), 1, "\n");

    /* pick off device specific info(first line) */
    n = tokenize(line[0], f, nelem(f));
    if(n%2 != 0)
        goto lose;
    strncpy(ifc->dev, findfield("device", f, n), sizeof(ifc->dev));
    ifc->dev[sizeof(ifc->dev)-1] = 0;
    if(ifc->dev[0] == 0){
lose:
        free(ifc);
        *l0 = nil;
        return l;
    }
    ifc->mtu          = strtoul(findfield("maxtu", f, n), nil, 10);
    ifc->sendra6      = atoi(findfield("sendra", f, n));
    ifc->recvra6      = atoi(findfield("recvra", f, n));
    ifc->rp.mflag     = atoi(findfield("mflag", f, n));
    ifc->rp.oflag     = atoi(findfield("oflag", f, n));
    ifc->rp.maxraint  = atoi(findfield("maxraint", f, n));
    ifc->rp.minraint  = atoi(findfield("minraint", f, n));
    ifc->rp.linkmtu   = atoi(findfield("linkmtu", f, n));
    ifc->rp.reachtime = atoi(findfield("reachtime", f, n));
    ifc->rp.rxmitra   = atoi(findfield("rxmitra", f, n));
    ifc->rp.ttl       = atoi(findfield("ttl", f, n));
    ifc->rp.routerlt  = atoi(findfield("routerlt", f, n));
    ifc->pktin        = strtoul(findfield("pktin", f, n), nil, 10);
    ifc->pktout       = strtoul(findfield("pktout", f, n), nil, 10);
    ifc->errin        = strtoul(findfield("errin", f, n), nil, 10);
    ifc->errout       = strtoul(findfield("errout", f, n), nil, 10);

    /* now read the addresses */
    ll = &ifc->lifc;
    for(i = 1; i < lines; i++){
        n = tokenize(line[i], f, nelem(f));
        if(n < 5)
            break;
```

```
        /* allocate new local address */
        *ll = lifc = mallocz(sizeof(Iplifc), 1);
        ll = &lifc->next;

        parseip(lifc->ip, f[0]);
        parseipmask(lifc->mask, f[1]);
        parseip(lifc->net, f[2]);

        lifc->validlt = strtoul(f[3], nil, 10);
        lifc->preflt = strtoul(f[4], nil, 10);
    }

    return l;
}
```

⟨*function* _freeifc 64a⟩≡ (205b)

```
static void
_freeifc(Ipifc *ifc)
{
    Ipifc *next;
    Iplifc *lnext, *lifc;

    if(ifc == nil)
        return;
    for(; ifc; ifc = next){
        next = ifc->next;
        for(lifc = ifc->lifc; lifc; lifc = lnext){
            lnext = lifc->next;
            free(lifc);
        }
        free(ifc);
    }
}
```

⟨*function* findfield 64b⟩≡ (205b)

```
static char*
findfield(char *name, char **f, int n)
{
    int i;

    for(i = 0; i < n-1; i++)
        if(strcmp(f[i], name) == 0)
            return f[i+1];
    return "";
}
```

6.2.2 /net/ipifc/x

⟨*function* myipaddr 64c⟩≡ (200b)

```
/* find first ip addr that isn't the friggin loopback address
 * unless there are no others
 */
errorneg1
myipaddr(ipaddr ip, char *net)
{
    Ipifc *nifc;
    Iplifc *lifc;
    static Ipifc *ifc;
    ipaddr mynet;
```

```
    ifc = readipifc(net, ifc, -1);
    for(nifc = ifc; nifc; nifc = nifc->next)
        for(lifc = nifc->lifc; lifc; lifc = lifc->next){
            maskip(lifc->ip, loopbackmask, mynet);
            if(ipcmp(mynet, loopbacknet) == 0){
                continue;
            }
            if(ipcmp(lifc->ip, IPnoaddr) != 0){
                ipmove(ip, lifc->ip);
                return OK_0;
            }
        }
    ipmove(ip, IPnoaddr);
    return ERROR_NEG1;
}
```

6.2.3 /net/ipifc/stats

⟨*function* ipifcstats 65a⟩≡ (354b)

```
int
ipifcstats(Proto *ipifc, char *buf, int len)
{
    return ipstats(ipifc->f, buf, len);
}
```

⟨*function* ipstats 65b⟩≡ (212b)

```
int
ipstats(Fs *f, char *buf, int len)
{
    IP *ip;
    char *p, *e;
    int i;

    ip = f->ip;
    ip->stats[DefaultTTL] = MAXTTL;

    p = buf;
    e = p+len;
    for(i = 0; i < Nipstats; i++)
        p = seprint(p, e, "%s: %llud\n", statnames[i], ip->stats[i]);
    return p - buf;
}
```

6.3 Binding ethernet medium

6.3.1 etherbind()

⟨*function* etherbind 65c⟩≡ (336c)

```
/*
 *  called to bind an IP ifc to an ethernet device
 *  called with ifc wlock'd
 */
static void
etherbind(Ipifc *ifc, int argc, char **argv)
{
    Chan *mchan4, *cchan4, *achan, *mchan6, *cchan6, *schan;
```

```
char addr[Maxpath]; //char addr[2*KNAMELEN];
char dir[Maxpath];  //char dir[2*KNAMELEN];
char *buf;
int n;
char *ptr;
Etherrock *er;

if(argc < 2)
    error(Ebadarg);

mchan4 = cchan4 = achan = mchan6 = cchan6 = nil;
buf = nil;
if(waserror()){
    if(mchan4 != nil)
        cclose(mchan4);
    if(cchan4 != nil)
        cclose(cchan4);
    if(achan != nil)
        cclose(achan);
    if(mchan6 != nil)
        cclose(mchan6);
    if(cchan6 != nil)
        cclose(cchan6);
    if(buf != nil)
        free(buf);
    nexterror();
}

/*
 *  open ipv4 conversation
 *
 *  the dial will fail if the type is already open on
 *  this device.
 */
snprint(addr, sizeof(addr), "%s!0x800", argv[2]);   /* ETIP4 */
mchan4 = chandial(addr, nil, dir, &cchan4);

/*
 *  make it non-blocking
 */
devtab[cchan4->type]->write(cchan4, nbmsg, strlen(nbmsg), 0);

/*
 *  get mac address and speed
 */
snprint(addr, sizeof(addr), "%s/stats", argv[2]);
buf = smalloc(512);
schan = namec(addr, Aopen, OREAD, 0);
if(waserror()){
    cclose(schan);
    nexterror();
}
n = devtab[schan->type]->read(schan, buf, 511, 0);
cclose(schan);
poperror();
buf[n] = 0;

ptr = strstr(buf, "addr: ");
if(!ptr)
    error(Eio);
```

```
    ptr += 6;
    parsemac(ifc->mac, ptr, 6);

    ptr = strstr(buf, "mbps: ");
    if(ptr){
        ptr += 6;
        ifc->mbps = atoi(ptr);
    } else
        ifc->mbps = 100;

    /*
     *  open arp conversation
     */
    snprint(addr, sizeof(addr), "%s!0x806", argv[2]);   /* ETARP */
    achan = chandial(addr, nil, nil, nil);

    /*
     *  open ipv6 conversation
     *
     *  the dial will fail if the type is already open on
     *  this device.
     */
    snprint(addr, sizeof(addr), "%s!0x86DD", argv[2]);  /* ETIP6 */
    mchan6 = chandial(addr, nil, dir, &cchan6);

    /*
     *  make it non-blocking
     */
    devtab[cchan6->type]->write(cchan6, nbmsg, strlen(nbmsg), 0);

    er = smalloc(sizeof(*er));
    er->mchan4 = mchan4;
    er->cchan4 = cchan4;
    er->achan = achan;
    er->mchan6 = mchan6;
    er->cchan6 = cchan6;
    er->f = ifc->conv->p->f;
    ifc->arg = er;

    free(buf);
    poperror();

    kproc("etherread4", etherread4, ifc);
    kproc("recvarpproc", recvarpproc, ifc);
    kproc("etherread6", etherread6, ifc);
}
```

6.3.2 chandial()

6.4 Setting up routes

6.5 Advanced configurations

6.5.1 IP Routing

⟨ipifcctl() *else if other string* 67⟩≡ (59d) 68c ▷

```
else if(strcmp(argv[0], "iprouting") == 0){
```

```
    i = 1;
    if(argc > 1)
        i = atoi(argv[1]);
    iprouting(cv->p->f, i);
    return nil;
}
```

⟨*function* iprouting 68a⟩≡ (212b)

```
void
iprouting(Fs *f, bool on)
{
    f->ip->iprouting = on;
    if(f->ip->iprouting == false)
        f->ip->stats[Forwarding] = 2;
    else
        f->ip->stats[Forwarding] = 1;
}
```

6.5.2 MTU

⟨ipifcadd() *switch argc, mtu setting case, and fall through* 68b⟩≡ (60)

```
// add <ip> <mask> <rem> <mtu>
case 5:
    mtu = strtoul(argv[4], 0, 0);
    if(mtu >= ifc->m->mintu && mtu <= ifc->m->maxtu)
        ifc->maxtu = mtu;
    /* fall through */
```

⟨ipifcctl() *else if other string* 68c⟩+≡ (59d) ◁67 69f▷

```
else if(strcmp(argv[0], "mtu") == 0)
    return ipifcsetmtu(ifc, argv, argc);
```

⟨*function* ipifcsetmtu 68d⟩≡ (354b)

```
/*
 *  change an interface's mtu
 */
char*
ipifcsetmtu(Ipifc *ifc, char **argv, int argc)
{
    int mtu;

    if(argc < 2 || ifc->m == nil)
        return Ebadarg;
    mtu = strtoul(argv[1], 0, 0);
    if(mtu < ifc->m->mintu || mtu > ifc->m->maxtu)
        return Ebadarg;
    ifc->maxtu = mtu;
    return nil;
}
```

6.5.3 TOS

⟨Conv(kernel) *other fields* 68e⟩+≡ (25a) ◁48b 69b▷

```
uint  tos;      /* type of service */
```

⟨ipwrite() *Qctl case, else if other string* 68f⟩+≡ (44f) ◁50b 69c▷

```
else if(strcmp(cb->f[0], "tos") == 0)
    tosctlmsg(cv, cb);
```

⟨*function* tosctlmsg 69a⟩≡ (302c)

```
static void
tosctlmsg(Conv *c, Cmdbuf *cb)
{
    if(cb->nf < 2)
        c->tos = 0;
    else
        c->tos = atoi(cb->f[1]);
}
```

6.5.4 TTL

⟨Conv(kernel) *other fields* 69b⟩+≡ (25a) ◁68e 89c▷

```
uint  ttl;      /* max time to live */
```

⟨ipwrite() *Qctl case, else if other string* 69c⟩+≡ (44f) ◁68f 89d▷

```
else if(strcmp(cb->f[0], "ttl") == 0)
    ttlctlmsg(cv, cb);
```

⟨*function* ttlctlmsg 69d⟩≡ (302c)

```
static void
ttlctlmsg(Conv *c, Cmdbuf *cb)
{
    if(cb->nf < 2)
        c->ttl = MAXTTL;
    else
        c->ttl = atoi(cb->f[1]);
}
```

⟨*constant* MAXTTL 69e⟩≡ (206b)

```
MAXTTL=   255,
```

6.5.5 Dynamic interface, removing, unbinding, etc.

⟨ipifcctl() *else if other string* 69f⟩+≡ (59d) ◁68c 71a▷

```
else if(strcmp(argv[0], "remove") == 0)
    return ipifcrem(ifc, argv, argc);
```

⟨*function* ipifcrem 69g⟩≡ (354b)

```
/*
 *  remove an address from an interface.
 *  called with c->car locked
 */
char*
ipifcrem(Ipifc *ifc, char **argv, int argc)
{
    char *rv;
    ipaddr ip, mask, rem;
    Iplifc *lifc;

    if(argc < 3)
        return Ebadarg;

    if (parseip(ip, argv[1]) == -1)
        return Ebadip;
    parseipmask(mask, argv[2]);
    if(argc < 4)
        maskip(ip, mask, rem);
```

```
    else
        if (parseip(rem, argv[3]) == -1)
            return Ebadip;

    wlock(ifc);

    /*
     *  find address on this interface and remove from chain.
     *  for pt to pt we actually specify the remote address as the
     *  addresss to remove.
     */
    for(lifc = ifc->lifc; lifc != nil; lifc = lifc->next) {
        if (memcmp(ip, lifc->local, IPaddrlen) == 0
        && memcmp(mask, lifc->mask, IPaddrlen) == 0
        && memcmp(rem, lifc->remote, IPaddrlen) == 0)
            break;
    }

    rv = ipifcremlifc(ifc, lifc);
    wunlock(ifc);
    return rv;
}
```

⟨*function* ipifcremlifc 70⟩≡ (354b)

```
/*
 *  remove a logical interface from an ifc
 *  always called with ifc wlock'd
 */
static char*
ipifcremlifc(Ipifc *ifc, Iplifc *lifc)
{
    Iplifc **l;
    Fs *f;

    f = ifc->conv->p->f;

    /*
     *  find address on this interface and remove from chain.
     *  for pt to pt we actually specify the remote address as the
     *  addresss to remove.
     */
    for(l = &ifc->lifc; *l != nil && *l != lifc; l = &(*l)->next)
        ;
    if(*l == nil)
        return "address not on this interface";
    *l = lifc->next;

    /* disassociate any addresses */
    while(lifc->link)
        remselfcache(f, ifc, lifc, lifc->link->self->a);

    /* remove the route for this logical interface */
    if(isv4(lifc->local))
        v4delroute(f, lifc->remote+IPv4off, lifc->mask+IPv4off, 1);
    ⟨ipifcremlifc() if ipv6 local 483c⟩

    free(lifc);
    return nil;
}
```

⟨ipifcctl() *else if other string* 71a⟩+≡ (59d) ◁69f 90c▷

```
else if(strcmp(argv[0], "unbind") == 0)
    return ipifcunbind(ifc);
```

⟨*function* ipifcunbind 71b⟩≡ (354b)

```
/*
 *  detach a device from an interface, close the interface
 *  called with ifc->conv closed
 */
static char*
ipifcunbind(Ipifc *ifc)
{
    char *err;

    if(waserror()){
        wunlock(ifc);
        nexterror();
    }
    wlock(ifc);

    /* dissociate routes */
    if(ifc->m != nil && ifc->m->unbindonclose == false)
        ifc->conv->inuse--;
    ifc->ifcid++;

    /* disassociate logical interfaces (before zeroing ifc->arg) */
    while(ifc->lifc){
        err = ipifcremlifc(ifc, ifc->lifc);
        /*
         * note: err non-zero means lifc not found,
         * which can't happen in this case.
         */
        if(err)
            error(err);
    }

    /* disassociate device */
    if(ifc->m && ifc->m->unbind)
        (*ifc->m->unbind)(ifc);
    memset(ifc->dev, 0, sizeof(ifc->dev));
    ifc->arg = nil;
    ifc->reassemble = false;

    /* close queues to stop queuing of packets */
    qclose(ifc->conv->rq);
    qclose(ifc->conv->wq);
    qclose(ifc->conv->sq);

    ifc->m = nil;
    wunlock(ifc);
    poperror();
    return nil;
}
```

6.5.6 Unbind on close

⟨Medium(kernel) *other fields* 71c⟩≡ (18c)

```
bool unbindonclose;  /* if non-zero, unbind on last close */
```

⟨*function* ipifcclose 72⟩≡ (354b)

```
/*
 *  called after last close of ipifc data or ctl
 *  called with c locked, we must unlock
 */
static void
ipifcclose(Conv *c)
{
    Ipifc *ifc;
    Medium *m;

    ifc = (Ipifc*)c->ptcl;
    m = ifc->m;
    if(m && m->unbindonclose)
        ipifcunbind(ifc);
}
```

Chapter 7

Physical Transport: Ethernet

7.1 Ethernet addresses

7.1.1 Parsing

⟨*function* parseether 73a⟩≡ (198a)

```
errorneg1
parseether(uchar* to, char *from)
{
    char nip[4];
    char *p;
    int i;

    p = from;
    for(i = 0; i < 6; i++){
        if(*p == '\0')
            return ERROR_NEG1;
        nip[0] = *p++;
        if(*p == '\0')
            return ERROR_NEG1;
        nip[1] = *p++;
        nip[2] = '\0';
        to[i] = strtoul(nip, 0, 16);
        if(*p == ':')
            p++;
    }
    return OK_0;
}
```

7.1.2 /net/etherx/addr

⟨*function* myetheraddr 73b⟩≡ (199b)

```
errorneg1
myetheraddr(uchar *to, char *dev)
{
    int n, fd;
    char buf[256];

    if(*dev == '/')
        sprint(buf, "%s/addr", dev);
    else
        sprint(buf, "/net/%s/addr", dev);

    fd = open(buf, OREAD);
```

```
    if(fd < 0)
        return ERROR_NEG1;

    n = read(fd, buf, sizeof buf -1 );
    close(fd);
    if(n <= 0)
        return ERROR_NEG1;
    buf[n] = '\0';

    parseether(to, buf);
    return OK_0;
}
```

7.2 Ethernet header

⟨*struct* Etherhdr 74a⟩≡ (336c)

```
struct Etherhdr
{
    eaddr   d;
    eaddr   s;
    uchar   t[2];
};
```

7.3 Ethernet packet

⟨*struct* Etherpkt 74b⟩≡ (215a)

```
struct Etherpkt
{
  eaddr d;
  eaddr s;
  uchar type[2];

  uchar data[1500];
};
```

7.4 IO

7.4.1 Writing

7.4.2 Reading

7.5 Advanced features

Chapter 8

Inter Network Transport: IP

8.1 IP addresses

8.1.1 Parsing

⟨*function* v4parseip 75a⟩≡ (199a)

```
char*
v4parseip(ipv4 to, char *from)
{
    int i;
    char *p;

    p = from;
    for(i = 0; i < IPv4addrlen && *p; i++){
        to[i] = strtoul(p, &p, 0);
        if(*p == '.')
            p++;
    }
    switch(CLASS(to)){
    case 0: /* class A - 1 uchar net */
    case 1:
        if(i == 3){
            to[3] = to[2];
            to[2] = to[1];
            to[1] = 0;
        } else if (i == 2){
            to[3] = to[1];
            to[1] = 0;
        }
        break;
    case 2: /* class B - 2 uchar net */
        if(i == 3){
            to[3] = to[2];
            to[2] = 0;
        }
        break;
    }
    return p;
}
```

⟨*function* parseip 75b⟩≡ (199a)

```
/*
 * 'from' may contain an address followed by other characters,
 * at least in /boot, so we permit whitespace (and more) after the address.
 * we do ensure that "delete" cannot be parsed as "de::".
```

```
 *
 * some callers don't check the return value for errors, so
 * set `to' to something distinctive in the case of a parse error.
 */
vlong
parseip(ipaddr to, char *from)
{
    int i, elipsis = 0;
    bool v4 = true;
    ulong x;
    char *p, *op;

    memset(to, 0, IPaddrlen);
    p = from;
    for(i = 0; i < IPaddrlen && ipcharok(*p); i+=2){
        op = p;
        x = strtoul(p, &p, 16);
        if((*p == '.' && i <= IPaddrlen-4) || (*p == '\0' && i == 0)){
            /* ends with v4 */
            p = v4parseip(to+i, op);
            i += 4;
            break;
        }

        /* v6: at most 4 hex digits, followed by colon or delim */
        if(x != (ushort)x || *p != ':' && !delimchar(*p)) {
            memset(to, 0, IPaddrlen);
            return -1;   /* parse error */
        }
        to[i] = x>>8;
        to[i+1] = x;
        if(*p == ':'){
            v4 = false;
            if(*++p == ':'){ /* :: is elided zero short(s) */
                if (elipsis) {
                    memset(to, 0, IPaddrlen);
                    return -1; /* second :: */
                }
                elipsis = i+2;
                p++;
            }
        } else if (p == op)  /* strtoul made no progress? */
            break;
    }
    if (p == from || !delimchar(*p)) {
        memset(to, 0, IPaddrlen);
        return -1;    /* parse error */
    }
    if(i < IPaddrlen){
        memmove(&to[elipsis+IPaddrlen-i], &to[elipsis], i-elipsis);
        memset(&to[elipsis], 0, IPaddrlen-i);
    }

    if(v4){
        to[10] = to[11] = 0xff;
        return nhgetl(to + IPv4off);
    } else
        return 6;
}
```

8.1.2 Comparisons

⟨*macro* ipcmp 77a⟩≡ (196c)

```
#define ipcmp(x, y) memcmp(x, y, IPaddrlen)
```

⟨*macro* ipmove 77b⟩≡ (196c)

```
#define ipmove(x, y) memmove(x, y, IPaddrlen)
```

⟨*function* equivip4 77c⟩≡ (201a)

```
bool
equivip4(ipv4 a, ipv4 b)
{
    int i;

    for(i = 0; i < 4; i++)
        if(a[i] != b[i])
            return false;
    return true;
}
```

8.2 IP header

8.2.1 Byte ordering

8.2.2 Checksum

⟨*function* ptclcsum 77d⟩≡ (343)

```
ushort
ptclcsum(Block *bp, int offset, int len)
{
    uchar *addr;
    ulong losum, hisum;
    ushort csum;
    int odd, blocklen, x;

    /* Correct to front of data area */
    while(bp != nil && offset && offset >= BLEN(bp)) {
        offset -= BLEN(bp);
        bp = bp->next;
    }
    if(bp == nil)
        return 0;

    addr = bp->rp + offset;
    blocklen = BLEN(bp) - offset;

    if(bp->next == nil) {
        if(blocklen < len)
            len = blocklen;
        return ~ptclbsum(addr, len) & 0xffff;
    }

    losum = 0;
    hisum = 0;

    odd = 0;
    while(len) {
        x = blocklen;
```

```
        if(len < x)
            x = len;

        csum = ptclbsum(addr, x);
        if(odd)
            hisum += csum;
        else
            losum += csum;
        odd = (odd+x) & 1;
        len -= x;

        bp = bp->next;
        if(bp == nil)
            break;
        blocklen = BLEN(bp);
        addr = bp->rp;
    }

    losum += hisum>>8;
    losum += (hisum&0xff)<<8;
    while((csum = losum>>16) != 0)
        losum = csum + (losum & 0xffff);

    return ~losum & 0xffff;
}
```

⟨*function* ipcsum 78a⟩≡ (212b)

```
ushort
ipcsum(uchar *addr)
{
    int len;
    ulong sum;

    sum = 0;
    len = (addr[0]&0xf)<<2;

    while(len > 0) {
        sum += addr[0]<<8 | addr[1] ;
        len -= 2;
        addr += 2;
    }

    sum = (sum & 0xffff) + (sum >> 16);
    sum = (sum & 0xffff) + (sum >> 16);

    return (sum^0xffff);
}
```

8.3 IP Fragments

⟨*function* ipfragfree4 78b⟩≡ (212b)

```
/*
 * ipfragfree4 - Free a list of fragments - assume hold fraglock4
 */
void
ipfragfree4(IP *ip, Fragment4 *frag)
{
    Fragment4 *fl, **l;
```

```
    if(frag->blist)
        freeblist(frag->blist);

    frag->src = 0;
    frag->id = 0;
    frag->blist = nil;

    l = &ip->flisthead4;
    for(fl = *l; fl; fl = fl->next) {
        if(fl == frag) {
            *l = frag->next;
            break;
        }
        l = &fl->next;
    }

    frag->next = ip->fragfree4;
    ip->fragfree4 = frag;

}
```

⟨*function* ipfragallo4 79a⟩≡ (212b)

```
/*
 * ipfragallo4 - allocate a reassembly queue - assume hold fraglock4
 */
Fragment4 *
ipfragallo4(IP *ip)
{
    Fragment4 *f;

    while(ip->fragfree4 == nil) {
        /* free last entry on fraglist */
        for(f = ip->flisthead4; f->next; f = f->next)
            ;
        ipfragfree4(ip, f);
    }
    f = ip->fragfree4;
    ip->fragfree4 = f->next;
    f->next = ip->flisthead4;
    ip->flisthead4 = f;
    f->age = NOW + 30000;

    return f;
}
```

8.4 IO

8.4.1 Writing: ipoput4()

⟨*function* ipoput4 79b⟩≡ (212b)

```
int
ipoput4(Fs *f, Block *bp, bool gating, int ttl, int tos, Conv *c)
{
    IP *ip;
    Ip4hdr *eh;
    Ipifc *ifc;
    Route *r, *sr;
```

```
    ipv4p gate;
    int len, medialen;
    int rv = OK_0;

    ⟨ipoput4() locals 81c⟩

    ip = f->ip;

    /* Fill out the ip header */
    eh = (Ip4hdr*)(bp->rp);

    ip->stats[OutRequests]++;

    /* Number of uchars in data and ip header to write */
    len = blocklen(bp);

    ⟨ipoput4() if gating 89a⟩

    ⟨ipoput4() error if too big packet of length len 83b⟩

    // Finding the route for the destination!
    r = v4lookup(f, eh->dst, c);
    ⟨ipoput4() error if no route r 83c⟩

    ifc = r->ifc;
    ⟨ipoput4() set gate according to type of route 81a⟩

    if(!gating) {
        eh->vihl = IP_VER4|IP_HLEN4;
        eh->tos = tos;
    }
    eh->ttl = ttl;

    ⟨ipoput4() rlock ifc, goto free if cant 83e⟩
    ⟨ipoput4() error if no medium attached to interface ifc 83d⟩

    /* If we dont need to fragment just send it */
    ⟨ipoput4() if manual fragmentation setting 89b⟩
    else
        medialen = ifc->maxtu - ifc->m->hsize;

    ⟨ipoput4() if no need to fragment, write simply to medium and return 81b⟩
    ⟨ipoput4() else, need to fragment 81d⟩

raise:
    runlock(ifc);
    poperror();
free:
    freeblist(bp);
    return rv;
}
```

Header constants

⟨*constant* IP_VER4 80a⟩≡ (206b)

```
IP_VER4=  0x40,
```

⟨*constant* IP_HLEN4 80b⟩≡ (206b)

```
IP_HLEN4= 5,    /* v4: Header length in words */
```

Finding the gateway

⟨ipoput4() *set gate according to type of route* 81a⟩≡ (79b)

```
if(r->type & (Rifc|Runi))
    gate = eh->dst;
else
⟨ipoput4() adjust gate and ifc if broadcast or multicast case 157a⟩
else
    gate = r->v4.gate;
```

Small send

⟨ipoput4() *if no need to fragment, write simply to medium and return* 81b⟩≡ (79b)

```
if(len <= medialen) {
    if(!gating)
        hnputs(eh->id, incref(&ip->id4));
    hnputs(eh->length, len);
    // no fragment
    if(!gating){
        eh->frag[0] = 0;
        eh->frag[1] = 0;
    }
    eh->cksum[0] = 0;
    eh->cksum[1] = 0;
    hnputs(eh->cksum, ipcsum(&eh->vihl));
    assert(bp->next == nil);

    // Medium dispatch
    ifc->m->bwrite(ifc, bp, V4, gate);

    runlock(ifc);
    poperror();
    return OK_0;
}
```

Fragmentation

⟨ipoput4() *locals* 81c⟩≡ (79b)

```
Ip4hdr *feh;
ulong fragoff;
Block *xp, *nb;
int lid, seglen, chunk, dlen, blklen, offset;
```

⟨ipoput4() *else, need to fragment* 81d⟩≡ (79b)

```
if(eh->frag[0] & (IP_DF>>8)){
    if (!gating)
      print("%V: DF set\n", eh->dst);
    ip->stats[FragFails]++;
    ip->stats[OutDiscards]++;
    icmpcantfrag(f, bp, medialen);
    netlog(f, Logip, "%V: eh->frag[0] & (IP_DF>>8)\n", eh->dst);
    goto raise;
}

seglen = (medialen - IP4HDR) & ~7;
if(seglen < 8){
    ip->stats[FragFails]++;
    ip->stats[OutDiscards]++;
```

```
    netlog(f, Logip, "%V seglen < 8\n", eh->dst);
    goto raise;
}

dlen = len - IP4HDR;
xp = bp;
if(gating)
    lid = nhgets(eh->id);
else
    lid = incref(&ip->id4);

offset = IP4HDR;
while(xp != nil && offset && offset >= BLEN(xp)) {
    offset -= BLEN(xp);
    xp = xp->next;
}
xp->rp += offset;

if(gating)
    fragoff = nhgets(eh->frag)<<3;
else
    fragoff = 0;
dlen += fragoff;

for(; fragoff < dlen; fragoff += seglen) {
    nb = allocb(IP4HDR+seglen);
    feh = (Ip4hdr*)(nb->rp);

    memmove(nb->wp, eh, IP4HDR);
    nb->wp += IP4HDR;

    if((fragoff + seglen) >= dlen) {
        seglen = dlen - fragoff;
        hnputs(feh->frag, fragoff>>3);
    }
    else
        hnputs(feh->frag, (fragoff>>3)|IP_MF);

    hnputs(feh->length, seglen + IP4HDR);
    hnputs(feh->id, lid);

    /* Copy up the data area */
    chunk = seglen;
    while(chunk) {
        if(!xp) {
            ip->stats[OutDiscards]++;
            ip->stats[FragFails]++;
            freeblist(nb);
            netlog(f, Logip, "!xp: chunk %d\n", chunk);
            goto raise;
        }
        blklen = chunk;
        if(BLEN(xp) < chunk)
            blklen = BLEN(xp);
        memmove(nb->wp, xp->rp, blklen);
        nb->wp += blklen;
        xp->rp += blklen;
        chunk -= blklen;
        if(xp->rp == xp->wp)
            xp = xp->next;
```

```
    }

    feh->cksum[0] = 0;
    feh->cksum[1] = 0;
    hnputs(feh->cksum, ipcsum(&feh->vihl));

    // Medium dispatch, send this fragment
    ifc->m->bwrite(ifc, nb, V4, gate);

    ip->stats[FragCreates]++;
}
ip->stats[FragOKs]++;
```

Error managment

⟨*constant* IP_MAX 83a⟩≡ (206b)

```
IP_MAX=   64*1024,  /* Max. Internet packet size, v4 & v6 */
```

⟨ipoput4() *error if too big packet of length len* 83b⟩≡ (79b)

```
if(len >= IP_MAX){
    ip->stats[OutDiscards]++;
    netlog(f, Logip, "exceeded ip max size %V\n", eh->dst);
    goto free;
}
```

⟨ipoput4() *error if no route r* 83c⟩≡ (79b)

```
if(r == nil){
    ip->stats[OutNoRoutes]++;
    netlog(f, Logip, "no interface %V\n", eh->dst);
    rv = -1;
    goto free;
}
```

⟨ipoput4() *error if no medium attached to interface ifc* 83d⟩≡ (79b)

```
if(ifc->m == nil)
    goto raise;
```

⟨ipoput4() *rlock ifc, goto free if cant* 83e⟩≡ (79b)

```
if(!canrlock(ifc))
    goto free;
if(waserror()){
    runlock(ifc);
    nexterror();
}
```

8.4.2 Reading: ipiput4()

⟨*function* ipiput4 83f⟩≡ (212b)

```
void
ipiput4(Fs *f, Ipifc *ifc, Block *bp)
{
    IP *ip;
    Ip4hdr *h;
    ipaddr v6dst;
    // enum<protocol_type>
    int proto;
    Proto *p;
    bool notforme;
```

```
    ⟨ipiput4() locals 84b⟩

    ⟨ipiput4() call ipiput6 if block is not ipv4 480c⟩

    ip = f->ip;
    ip->stats[InReceives]++;

    ⟨ipiput4() ensure we have all the header in the first block 88⟩

    h = (Ip4hdr*)(bp->rp);

    ⟨ipiput4() check header checksum 84a⟩

    v4tov6(v6dst, h->dst);
    notforme = ipforme(f, v6dst) == 0;

    ⟨ipiput4() check header length and version 85a⟩

    ⟨ipiput4() if notforme 89f⟩
    // else have a ipforme

    ⟨ipiput4() possibly defragment and reassemble 85d⟩

    /* don't let any frag info go up the stack */
    h->frag[0] = 0;
    h->frag[1] = 0;

    proto = h->proto;
    p = Fsrcvpcol(f, proto);

    if(p != nil && p->rcv != nil) {
        ip->stats[InDelivers]++;

        // Protocol dispatch
        (*p->rcv)(p, ifc, bp);
        return;
    }

    ip->stats[InDiscards]++;
    ip->stats[InUnknownProtos]++;
    freeblist(bp);
}
```

Checks

⟨ipiput4() *check header checksum* 84a⟩≡ (83f)

```
/* dump anything that whose header doesn't checksum */
if((bp->flag & Bipck) == 0 && ipcsum(&h->vihl)) {
    ip->stats[InHdrErrors]++;
    netlog(f, Logip, "ip: checksum error %V\n", h->src);
    freeblist(bp);
    return;
}
```

⟨ipiput4() *locals* 84b⟩≡ (83f) 85c ▷

```
int hl;
int olen;
uchar *dp;
```

⟨ipiput4() *check header length and version* 85a⟩≡ (83f)

```
/* Check header length and version */
if((h->vihl&0x0F) != IP_HLEN4) {
    hl = (h->vihl&0xF)<<2;
    if(hl < (IP_HLEN4<<2)) {
        ip->stats[InHdrErrors]++;
        netlog(f, Logip, "ip: %V bad hivl %ux\n", h->src, h->vihl);
        freeblist(bp);
        return;
    }
    /* If this is not routed strip off the options */
    if(notforme == false) {
        olen = nhgets(h->length);
        dp = bp->rp + (hl - (IP_HLEN4<<2));
        memmove(dp, h, IP_HLEN4<<2);
        bp->rp = dp;
        h = (Ip4hdr*)(bp->rp);
        h->vihl = (IP_VER4|IP_HLEN4);
        hnputs(h->length, olen-hl+(IP_HLEN4<<2));
    }
}
```

Protocol dispatching

⟨*function* Fsrcvpcol 85b⟩≡ (302c)

```
Proto*
Fsrcvpcol(Fs* f, uchar proto)
{
   return f->t2p[proto];
}
```

Small receive

Reassembling

⟨ipiput4() *locals* 85c⟩+≡ (83f) ◁84b 89e▷

```
ushort frag;
```

⟨ipiput4() *possibly defragment and reassemble* 85d⟩≡ (83f)

```
frag = nhgets(h->frag);
if(frag) {
    h->tos = 0;
    if(frag & IP_MF)
        h->tos = 1;
    bp = ip4reassemble(ip, frag, bp, h);
    if(bp == nil)
        return;
    h = (Ip4hdr*)(bp->rp);
}
```

⟨*function* ip4reassemble 85e⟩≡ (212b)

```
Block*
ip4reassemble(IP *ip, int offset, Block *bp, Ip4hdr *ih)
{
    int fend;
    ushort id;
    Fragment4 *f, *fnext;
    iplong src, dst;
    Block *bl, **l, *last, *prev;
```

```
int ovlap, len, fragsize, pktposn;

src = nhgetl(ih->src);
dst = nhgetl(ih->dst);
id = nhgets(ih->id);

/*
 *  block lists are too hard, pullupblock into a single block
 */
if(bp->next){
    bp = pullupblock(bp, blocklen(bp));
    ih = (Ip4hdr*)(bp->rp);
}

qlock(&ip->fraglock4);

/*
 *  find a reassembly queue for this fragment
 */
for(f = ip->flisthead4; f; f = fnext){
    fnext = f->next;    /* because ipfragfree4 changes the list */
    if(f->src == src && f->dst == dst && f->id == id)
        break;
    if(f->age < NOW){
        ip->stats[ReasmTimeout]++;
        ipfragfree4(ip, f);
    }
}

/*
 *  if this isn't a fragmented packet, accept it
 *  and get rid of any fragments that might go
 *  with it.
 */
if(!ih->tos && (offset & ~(IP_MF|IP_DF)) == 0) {
    if(f != nil) {
        ipfragfree4(ip, f);
        ip->stats[ReasmFails]++;
    }
    qunlock(&ip->fraglock4);
    return bp;
}

if(bp->base+IPFRAGSZ >= bp->rp){
    bp = padblock(bp, IPFRAGSZ);
    bp->rp += IPFRAGSZ;
}

BKFG(bp)->foff = offset<<3;
BKFG(bp)->flen = nhgets(ih->length)-IP4HDR;

/* First fragment allocates a reassembly queue */
if(f == nil) {
    f = ipfragallo4(ip);
    f->id = id;
    f->src = src;
    f->dst = dst;

    f->blist = bp;
```

```
    qunlock(&ip->fraglock4);
    ip->stats[ReasmReqds]++;
    return nil;
}

/*
 *  find the new fragment's position in the queue
 */
prev = nil;
l = &f->blist;
bl = f->blist;
while(bl != nil && BKFG(bp)->foff > BKFG(bl)->foff) {
    prev = bl;
    l = &bl->next;
    bl = bl->next;
}

/* Check overlap of a previous fragment - trim away as necessary */
if(prev) {
    ovlap = BKFG(prev)->foff + BKFG(prev)->flen - BKFG(bp)->foff;
    if(ovlap > 0) {
        if(ovlap >= BKFG(bp)->flen) {
            freeblist(bp);
            qunlock(&ip->fraglock4);
            return nil;
        }
        BKFG(prev)->flen -= ovlap;
    }
}

/* Link onto assembly queue */
bp->next = *l;
*l = bp;

/* Check to see if succeeding segments overlap */
if(bp->next) {
    l = &bp->next;
    fend = BKFG(bp)->foff + BKFG(bp)->flen;
    /* Take completely covered segments out */
    while(*l) {
        ovlap = fend - BKFG(*l)->foff;
        if(ovlap <= 0)
            break;
        if(ovlap < BKFG(*l)->flen) {
            BKFG(*l)->flen -= ovlap;
            BKFG(*l)->foff += ovlap;
            /* move up ih hdrs */
            memmove((*l)->rp + ovlap, (*l)->rp, IP4HDR);
            (*l)->rp += ovlap;
            break;
        }
        last = (*l)->next;
        (*l)->next = nil;
        freeblist(*l);
        *l = last;
    }
}

/*
 *  look for a complete packet.  if we get to a fragment
```

```
     *  without IP_MF set, we're done.
     */
    pktposn = 0;
    for(bl = f->blist; bl; bl = bl->next) {
        if(BKFG(bl)->foff != pktposn)
            break;
        if((BLKIP(bl)->frag[0]&(IP_MF>>8)) == 0) {
            bl = f->blist;
            len = nhgets(BLKIP(bl)->length);
            bl->wp = bl->rp + len;

            /* Pullup all the fragment headers and
             * return a complete packet
             */
            for(bl = bl->next; bl; bl = bl->next) {
                fragsize = BKFG(bl)->flen;
                len += fragsize;
                bl->rp += IP4HDR;
                bl->wp = bl->rp + fragsize;
            }

            bl = f->blist;
            f->blist = nil;
            ipfragfree4(ip, f);
            ih = BLKIP(bl);
            hnputs(ih->length, len);
            qunlock(&ip->fraglock4);
            ip->stats[ReasmOKs]++;
            return bl;
        }
        pktposn += BKFG(bl)->flen;
    }
    qunlock(&ip->fraglock4);
    return nil;
}
```

Error managment

Misc

⟨ipiput4() *ensure we have all the header in the first block* 88⟩≡ (83f)

```
/*
 *  Ensure we have all the header info in the first
 *  block.  Make life easier for other protocols by
 *  collecting up to the first 64 bytes in the first block.
 */
if(BLEN(bp) < 64) {
    hl = blocklen(bp);
    if(hl < IP4HDR)
        hl = IP4HDR;
    if(hl > 64)
        hl = 64;
    bp = pullupblock(bp, hl);
    if(bp == nil)
        return;
}
```

8.5 Advanced features

8.5.1 Gating

⟨ipoput4() *if gating* 89a⟩≡ (79b)

```
  if(gating){
      chunk = nhgets(eh->length);
      if(chunk > len){
          ip->stats[OutDiscards]++;
          netlog(f, Logip, "short gated packet\n");
          goto free;
      }
      if(chunk < len)
          len = chunk;
  }
```

8.5.2 Manual fragmentation setting

⟨ipoput4() *if manual fragmentation setting* 89b⟩≡ (79b)

```
  if(c && c->maxfragsize && c->maxfragsize < ifc->maxtu)
      medialen = c->maxfragsize - ifc->m->hsize;
```

⟨Conv(kernel) *other fields* 89c⟩+≡ (25a) ◁69b

```
  // option<int>, None = 0
  int maxfragsize;    /* If set, used for fragmentation */
```

⟨ipwrite() *Qctl case, else if other string* 89d⟩+≡ (44f) ◁69c 128c▷

```
  else if(strcmp(cb->f[0], "maxfragsize") == 0){
      if(cb->nf < 2)
          error("maxfragsize needs size");

      cv->maxfragsize = (int)strtol(cb->f[1], nil, 0);

  }
```

8.5.3 Routing

⟨ipiput4() *locals* 89e⟩+≡ (83f) ◁85c

```
  int hop, tos;
  Route *r;
  Conv conv;
```

⟨ipiput4() *if notforme* 89f⟩≡ (83f)

```
      /* route */
      if(notforme) {
          if(!ip->iprouting){
              freeblist(bp);
              return;
          }

          /* don't forward to source's network */
          memset(&conv, 0, sizeof conv);
          conv.r = nil;
          r = v4lookup(f, h->dst, &conv);
          if(r == nil || r->ifc == ifc){
              ip->stats[OutDiscards]++;
              freeblist(bp);
              return;
```

```
        }

        /* don't forward if packet has timed out */
        hop = h->ttl;
        if(hop < 1) {
            ip->stats[InHdrErrors]++;
            icmpttlexceeded(f, ifc->lifc->local, bp);
            freeblist(bp);
            return;
        }

        /* reassemble if the interface expects it */
        if(r->ifc == nil) panic("nil route rfc");
        if(r->ifc->reassemble){
            frag = nhgets(h->frag);
            if(frag) {
                h->tos = 0;
                if(frag & IP_MF)
                    h->tos = 1;
                bp = ip4reassemble(ip, frag, bp, h);
                if(bp == nil)
                    return;
                h = (Ip4hdr*)(bp->rp);
            }
        }

        ip->stats[ForwDatagrams]++;
        tos = h->tos;
        hop = h->ttl;

        ipoput4(f, bp, true, hop - 1, tos, &conv);

        return;
    }
```

⟨IP(kernel) *routing fields* 90a⟩≡ (22a)

```
  bool iprouting;  /* true if we route like a gateway */
```

8.5.4 Reassembling

⟨Ipifc(kernel) *other fields* 90b⟩+≡ (17c) ◁59a

```
  bool reassemble; /* reassemble IP packets before forwarding */
```

⟨ipifcctl() *else if other string* 90c⟩+≡ (59d) ◁71a 159b▷

```
  else if(strcmp(argv[0], "reassemble") == 0){
      ifc->reassemble = true;
      return nil;
  }
```

Chapter 9

Finding Machines Locally: ARP

⟨Fs(kernel) *arp fields* 91a⟩≡ (21c)

```
Arp *arp;
```

⟨*struct* Arp 91b⟩≡ (313a)

```
/*
 *  one per Fs
 */
struct Arp
{
    Arpent  *rxmt;
    Proc    *rxmitp;    /* neib sol re-transmit proc */

    Rendez  rxmtq;
    Block   *dropf, *dropl;

    Arpent  *hash[NHASH];
    Arpent  cache[NCACHE];

    // Extra
    QLock;
    ⟨Arp extra fields 91c⟩

};
```

⟨Arp *extra fields* 91c⟩≡ (91b)

```
// ref<Fs>, reverse of Fs.arp
Fs  *f;
```

⟨*struct* Arpent 91d⟩≡ (208)

```
struct Arpent
{
  ipaddr ip;
  uchar mac[MAClen];

  Medium  *type;      /* media type */

  Block*  hold;
  Block*  last;
  uint  ctime;      /* time entry was created or refreshed */
  uint  utime;      /* time entry was last used */
  uchar state;
  Arpent  *nextrxt;   /* re-transmit chain */
  uint  rtime;      /* time for next retransmission */
  uchar rxtsrem;
```

```
    Ipifc *ifc;
    uchar ifcid;      /* must match ifc->id */

    // Extra
    Arpent* hash;
  };
```

⟨Medium(kernel) *address resolution methods* 92a⟩≡ (18d)

```
  /* address resolution */
  void  (*ares)(Fs*, int, uchar*, uchar*, int, int);  /* resolve */
  void  (*areg)(Ipifc*, uchar*);      /* register */
```

9.1 Initialisation

⟨*function* arpinit 92b⟩≡ (313a)

```
  void
  arpinit(Fs *f)
  {
      f->arp = smalloc(sizeof(Arp));
      f->arp->f = f;
      f->arp->rxmt = nil;
      f->arp->dropf = f->arp->dropl = nil;
      kproc("rxmitproc", rxmitproc, f->arp);
  }
```

⟨*function* rxmitproc 92c⟩≡ (313a)

```
  static void
  rxmitproc(void *v)
  {
      Arp *arp = v;
      long wakeupat;

      arp->rxmitp = up;
      //print("arp rxmitproc started\n");
      if(waserror()){
          arp->rxmitp = 0;
          pexit("hangup", 1);
      }
      for(;;){
          wakeupat = rxmitsols(arp);
          if(wakeupat == 0)
              sleep(&arp->rxmtq, rxready, v);
          else if(wakeupat > ReTransTimer/4)
              tsleep(&arp->rxmtq, returnfalse, 0, wakeupat);
      }
  }
```

9.2 /net/arp

⟨ip1gen() *switch TYPE qid cases* 92d⟩≡ (37a) 103a ▷

```
  case Qarp:
      p = "arp";
      prot = 0664;
      break;
```

⟨ipopen() *switch TYPE qid cases* 93a⟩+≡ (39c) ◁52a 160f▷

```
case Qarp:
case Qiproute:
    if(omode != OREAD && !iseve())
        error(Eperm);
    break;
```

9.2.1 Reading

⟨ipread() *switch TYPE qid cases* 93b⟩+≡ (41a) ◁54f 103b▷

```
case Qarp:
    return arpread(f->arp, a, offset, n);
```

⟨*function* arpread 93c⟩≡ (313a)

```
int
arpread(Arp *arp, char *p, ulong offset, int len)
{
    Arpent *a;
    int n;
    char mac[2*MAClen+1];

    if(offset % Alinelen)
        return 0;

    offset = offset/Alinelen;
    len = len/Alinelen;

    n = 0;
    for(a = arp->cache; len > 0 && a < &arp->cache[NCACHE]; a++){
        if(a->state == 0)
            continue;
        if(offset > 0){
            offset--;
            continue;
        }
        len--;
        qlock(arp);
        convmac(mac, &mac[sizeof mac], a->mac, a->type->maclen);
        n += snprint(p+n, Alinelen+1, aformat, a->type->name,
            arpstate[a->state], a->ip, mac);    /* +1 for NUL */
        qunlock(arp);
    }

    return n;
}
```

9.2.2 Writing

⟨ipwrite() *switch TYPE qid cases* 93d⟩+≡ (41c) ◁51b 103c▷

```
case Qarp:
    return arpwrite(f, a, n);
```

⟨*function* arpwrite 93e⟩≡ (313a)

```
int
arpwrite(Fs *fs, char *s, int len)
{
    int n;
    Route *r;
```

```
Arp *arp;
Block *bp;
Arpent *a, *fl, **l;
Medium *m;
char *f[4], buf[256];
ipaddr ip;
uchar mac[MAClen];

arp = fs->arp;

if(len == 0)
    error(Ebadarp);
if(len >= sizeof(buf))
    len = sizeof(buf)-1;
strncpy(buf, s, len);
buf[len] = 0;
if(len > 0 && buf[len-1] == '\n')
    buf[len-1] = 0;

n = getfields(buf, f, 4, 1, " ");
if(strcmp(f[0], "flush") == 0){
    qlock(arp);
    for(a = arp->cache; a < &arp->cache[NCACHE]; a++){
        memset(a->ip, 0, sizeof(a->ip));
        memset(a->mac, 0, sizeof(a->mac));
        a->hash = nil;
        a->state = 0;
        a->utime = 0;
        while(a->hold != nil){
            bp = a->hold->list;
            freeblist(a->hold);
            a->hold = bp;
        }
    }
    memset(arp->hash, 0, sizeof(arp->hash));
    /* clear all pkts on these lists (rxmt, dropf/l) */
    arp->rxmt = nil;
    arp->dropf = nil;
    arp->dropl = nil;
    qunlock(arp);
} else if(strcmp(f[0], "add") == 0){
    switch(n){
    default:
        error(Ebadarg);
    case 3:
        if (parseip(ip, f[1]) == -1)
            error(Ebadip);
        if(isv4(ip))
            r = v4lookup(fs, ip+IPv4off, nil);
        else
            r = v6lookup(fs, ip, nil);
        if(r == nil)
            error("Destination unreachable");
        m = r->ifc->m;
        n = parsemac(mac, f[2], m->maclen);
        break;
    case 4:
        m = ipfindmedium(f[1]);
        if(m == nil)
            error(Ebadarp);
```

```
            if (parseip(ip, f[2]) == -1)
                error(Ebadip);
            n = parsemac(mac, f[3], m->maclen);
            break;
        }

        if(m->ares == nil)
            error(Ebadarp);

        m->ares(fs, V6, ip, mac, n, 0);
    } else if(strcmp(f[0], "del") == 0){
        if(n != 2)
            error(Ebadarg);

        if (parseip(ip, f[1]) == -1)
            error(Ebadip);
        qlock(arp);

        l = &arp->hash[haship(ip)];
        for(a = *l; a; a = a->hash){
            if(memcmp(ip, a->ip, sizeof(a->ip)) == 0){
                *l = a->hash;
                break;
            }
            l = &a->hash;
        }

        if(a){
            /* take out of re-transmit chain */
            l = &arp->rxmt;
            for(fl = *l; fl; fl = fl->nextrxt){
                if(fl == a){
                    *l = a->nextrxt;
                    break;
                }
                l = &fl->nextrxt;
            }

            a->nextrxt = nil;
            a->hash = nil;
            a->hold = nil;
            a->last = nil;
            a->ifc = nil;
            memset(a->ip, 0, sizeof(a->ip));
            memset(a->mac, 0, sizeof(a->mac));
        }
        qunlock(arp);
    } else
        error(Ebadarp);

    return len;
}
```

Chapter 10

Finding Machines Globally: Routes

10.1 v4lookup()

⟨*function* v4lookup 96a⟩≡ (339c)

```
Route*
v4lookup(Fs *f, ipv4 a, Conv *c)
{
    Route *p, *q;
    ulong la;
    ⟨v4lookup() locals 97a⟩

    ⟨v4lookup() return cached route if still valid route 96d⟩

    la = nhgetl(a);
    q = nil;
    ⟨v4lookup() ternary search for route q for la in route forest 96b⟩
    ⟨v4lookup() make sure route q has an up to date ifc 97b⟩

    if(c != nil){
        c->r = q;
        c->rgen = v4routegeneration;
    }

    return q;
}
```

⟨v4lookup() *ternary search for route q for la in route forest* 96b⟩≡ (96a)

```
for(p=f->v4root[V4H(la)]; p;)
    if(la >= p->v4.address) {
        if(la <= p->v4.endaddress) {
            q = p;
            p = p->mid;
        } else
            p = p->right;
    } else
        p = p->left;
```

⟨*macro* V4H 96c⟩≡ (339c)

```
#define V4H(a)  ((a&0x07ffffff)>>(32-Lroot-5))
```

⟨v4lookup() *return cached route if still valid route* 96d⟩≡ (96a)

```
if(c != nil && c->r != nil && c->r->ifc != nil && c->rgen == v4routegeneration)
    return c->r;
```

⟨v4lookup() *locals* 97a⟩≡ (96a)

```
ipaddr gate;
Ipifc *ifc;
```

⟨v4lookup() *make sure route q has an up to date ifc* 97b⟩≡ (96a)

```
if(q && (q->ifc == nil || q->ifcid != q->ifc->ifcid)){
    if(q->type & Rifc) {
        hnputl(gate+IPv4off, q->v4.address);
        memmove(gate, v4prefix, IPv4off);
    } else
        v4tov6(gate, q->v4.gate);

    ifc = findipifc(f, gate, q->type);
    if(ifc == nil)
        return nil;
    q->ifc = ifc;
    q->ifcid = ifc->ifcid;
}
```

10.2 findipifc()

⟨*function* findipifc 97c⟩≡ (354b)

```
/*
 *  find the ifc on same net as the remote system.  If none,
 *  return nil.
 */
Ipifc*
findipifc(Fs *f, ipaddr remote, int type)
{
    Ipifc *ifc, *x;
    Iplifc *lifc;
    Conv **cp, **e;
    ipaddr gnet, xmask;

    x = nil;
    memset(xmask, 0, IPaddrlen);

    /* find most specific match */
    e = &f->ipifc->conv[f->ipifc->nc];
    for(cp = f->ipifc->conv; cp < e; cp++){
        if(*cp == nil)
            continue;
        ifc = (Ipifc*)(*cp)->ptcl;
        for(lifc = ifc->lifc; lifc; lifc = lifc->next){
            maskip(remote, lifc->mask, gnet);
            if(ipcmp(gnet, lifc->net) == 0){
                if(x == nil || ipcmp(lifc->mask, xmask) > 0){
                    x = ifc;
                    ipmove(xmask, lifc->mask);
                }
            }
        }
    }
    if(x != nil)
        return x;

    ⟨findipifc() if broadcast or multicast route 157b⟩
    return nil;
}
```

10.3 Adding routes: v4addroute()

⟨*function* v4addroute 98a⟩≡ (339c)

```
void
v4addroute(Fs *f, char *tag, ipv4 a, ipv4 mask, ipv4 gate, int type)
{
    Route *p;
    ulong m;
    ulong sa; // start address
    ulong ea; // end address
    int h, eh;

    m = nhgetl(mask);
    sa = nhgetl(a) & m;
    ea = sa | ~m;

    eh = V4H(ea);
    for(h=V4H(sa); h<=eh; h++) {
        p = allocroute(Rv4 | type);
        p->v4.address = sa;
        p->v4.endaddress = ea;
        memmove(p->v4.gate, gate, sizeof(p->v4.gate));
        memmove(p->tag, tag, sizeof(p->tag));

        wlock(&routelock);
        addnode(f, &f->v4root[h], p);
        ⟨v4addroute() if f has a route queue 101b⟩
        wunlock(&routelock);
    }
    v4routegeneration++;

    ipifcaddroute(f, Rv4, a, mask, gate, type);
}
```

⟨*global* routelock 98b⟩≡ (339c)

```
static RWlock   routelock;
```

10.4 Broadcast routes

10.5 Route managment

⟨*global* v4freelist 98c⟩≡ (339c)

```
/* these are used for all instances of IP */
static Route*   v4freelist;
```

10.5.1 Allocation

⟨*function* allocroute 98d⟩≡ (339c)

```
static Route*
allocroute(int type)
{
    Route *r;
    int n;
    Route **l;

    if(type & Rv4){
```

```
        n = sizeof(RouteTree) + sizeof(V4route);
        l = &v4freelist;
    }
    ⟨allocroute() if ipv6 route 483d⟩

    r = *l;
    if(r != nil){
        *l = r->mid;
    } else {
        r = malloc(n);
        if(r == nil)
            panic("out of routing nodes");
    }
    memset(r, 0, n);

    r->type = type;
    r->ifc = nil;
    r->ref = 1;

    return r;
}
```

10.5.2 Free

⟨*function* freeroute 99a⟩≡ (339c)

```
static void
freeroute(Route *r)
{
    Route **l;

    r->left = nil;
    r->right = nil;
    if(r->type & Rv4)
        l = &v4freelist;
    ⟨freeroute() if ipv6 route 483e⟩
    r->mid = *l;
    *l = r;
}
```

10.5.3 Insertion

⟨*enum* _anon_ (kernel/network/ip/iproute.c) 99b⟩≡ (339c)

```
/*
 *  compare 2 v4 or v6 ranges
 */
enum
{
    Rpreceeds,
    Rfollows,
    Requals,
    Rcontains,
    Rcontained,
};
```

⟨*function* rangecompare 99c⟩≡ (339c)

```
static int
rangecompare(Route *a, Route *b)
{
```

```
    if(a->type & Rv4){
        if(a->v4.endaddress < b->v4.address)
            return Rpreceeds;

        if(a->v4.address > b->v4.endaddress)
            return Rfollows;

        if(a->v4.address <= b->v4.address
        && a->v4.endaddress >= b->v4.endaddress){

            if(a->v4.address == b->v4.address
            && a->v4.endaddress == b->v4.endaddress)
                return Requals;
            return Rcontains;
        }
        return Rcontained;
    }
    ⟨rangecompare() if ipv6 routes 483f⟩
}
```

⟨*function* addnode 100⟩≡ (339c)

```
/*
 *  add a new node to the tree
 */
static void
addnode(Fs *f, Route **cur, Route *new)
{
    Route *p;

    p = *cur;
    if(p == 0) {
        *cur = new;
        new->depth = 1;
        return;
    }

    switch(rangecompare(new, p)){
    case Rpreceeds:
        addnode(f, &p->left, new);
        break;
    case Rfollows:
        addnode(f, &p->right, new);
        break;
    case Rcontains:
        /*
         *  if new node is superset
         *  of tree node,
         *  replace tree node and
         *  queue tree node to be
         *  merged into root.
         */
        *cur = new;
        new->depth = 1;
        addqueue(&f->queue, p);
        break;
    case Requals:
        /*
         *  supercede the old entry if the old one isn't
         *  a local interface.
         */
```

```
        if((p->type & Rifc) == 0){
            p->type = new->type;
            p->ifcid = -1;
            copygate(p, new);
        } else if(new->type & Rifc)
            p->ref++;
        freeroute(new);
        break;
    case Rcontained:
        addnode(f, &p->mid, new);
        break;
    }

    balancetree(cur);
}
```

⟨*function* copygate 101a⟩≡ (339c)

```
static void
copygate(Route *old, Route *new)
{
    if(new->type & Rv4)
        memmove(old->v4.gate, new->v4.gate, IPv4addrlen);
    else
        memmove(old->v6.gate, new->v6.gate, IPaddrlen);
}
```

⟨v4addroute() *if f has a route queue* 101b⟩≡ (98a)

```
while(p = f->queue) {
    f->queue = p->mid;
    walkadd(f, &f->v4root[h], p->left);
    freeroute(p);
}
```

⟨Fs(kernel) *routing fields* 101c⟩+≡ (21c) ◁30b

```
Route *queue;     /* used as temp when reinjecting routes */
```

⟨*function* walkadd 101d⟩≡ (339c)

```
/*
 *  walk down a tree adding nodes back in
 */
static void
walkadd(Fs *f, Route **root, Route *p)
{
    Route *l, *r;

    l = p->left;
    r = p->right;
    p->left = 0;
    p->right = 0;
    addnode(f, root, p);
    if(l)
        walkadd(f, root, l);
    if(r)
        walkadd(f, root, r);
}
```

⟨*function* addqueue 101e⟩≡ (339c)

```
static void
addqueue(Route **q, Route *r)
{
```

```
    Route *l;

    if(r == nil)
        return;

    l = allocroute(r->type);
    l->mid = *q;
    *q = l;
    l->left = r;
}
```

10.5.4 Balancing

⟨Routetree *other fields* 102a⟩+≡ (29d) ◁59b

```
uchar depth;
```

⟨*function* balancetree 102b⟩≡ (339c)

```
/*
 *  balance the tree at the current node
 */
static void
balancetree(Route **cur)
{
    Route *p, *l, *r;
    int dl, dr;

    /*
     * if left and right are
     * too out of balance,
     * rotate tree node
     */
    p = *cur;
    dl = 0; if(l = p->left) dl = l->depth;
    dr = 0; if(r = p->right) dr = r->depth;

    if(dl > dr+1) {
        p->left = l->right;
        l->right = p;
        *cur = l;
        calcd(p);
        calcd(l);
    } else
    if(dr > dl+1) {
        p->right = r->left;
        r->left = p;
        *cur = r;
        calcd(p);
        calcd(r);
    } else
        calcd(p);
}
```

⟨*function* calcd 102c⟩≡ (339c)

```
/*
 *  calculate depth
 */
static void
calcd(Route *p)
{
```

```
    Route *q;
    int d;

    if(p) {
        d = 0;
        q = p->left;
        if(q)
            d = q->depth;
        q = p->right;
        if(q && q->depth > d)
            d = q->depth;
        q = p->mid;
        if(q && q->depth > d)
            d = q->depth;
        p->depth = d+1;
    }
}
```

10.6 /net/iproutes

⟨ip1gen() *switch TYPE qid cases* 103a⟩+≡ (37a) ◁92d 112b▷

```
case Qiproute:
    p = "iproute";
    prot = 0664;
    break;
```

⟨ipread() *switch TYPE qid cases* 103b⟩+≡ (41a) ◁93b 112c▷

```
case Qiproute:
    return routeread(f, a, offset, n);
```

⟨ipwrite() *switch TYPE qid cases* 103c⟩+≡ (41c) ◁93d 160h▷

```
case Qiproute:
    return routewrite(f, ch, a, n);
```

10.6.1 Reading

⟨*function* routeread 103d⟩≡ (339c)

```
long
routeread(Fs *f, char *p, ulong offset, int n)
{
    Routewalk rw;

    rw.p = p;
    rw.e = p+n;
    rw.o = -offset;
    rw.walk = sprintroute;

    ipwalkroutes(f, &rw);

    return rw.p - p;
}
```

⟨*global* rformat 103e⟩≡ (339c)

```
static char *rformat = "%-15I %-4M %-15I %4.4s %4.4s %3s\n";
```

⟨*function* sprintroute 104a⟩≡ (339c)

```
/*
 *  this code is not in rr to reduce stack size
 */
static void
sprintroute(Route *r, Routewalk *rw)
{
    int nifc, n;
    char t[5], *iname, ifbuf[5];
    ipaddr addr, mask, gate;
    char *p;

    convroute(r, addr, mask, gate, t, &nifc);
    iname = "-";
    if(nifc != -1) {
        iname = ifbuf;
        snprint(ifbuf, sizeof ifbuf, "%d", nifc);
    }
    p = seprint(rw->p, rw->e, rformat, addr, mask, gate, t, r->tag, iname);
    if(rw->o < 0){
        n = p - rw->p;
        if(n > -rw->o){
            memmove(rw->p, rw->p-rw->o, n+rw->o);
            rw->p = p + rw->o;
        }
        rw->o += n;
    } else
        rw->p = p;
}
```

⟨*function* convroute 104b⟩≡ (339c)

```
void
convroute(Route *r, uchar *addr, uchar *mask, uchar *gate, char *t, int *nifc)
{
    int i;

    if(r->type & Rv4){
        memmove(addr, v4prefix, IPv4off);
        hnputl(addr+IPv4off, r->v4.address);
        memset(mask, 0xff, IPv4off);
        hnputl(mask+IPv4off, ~(r->v4.endaddress ^ r->v4.address));
        memmove(gate, v4prefix, IPv4off);
        memmove(gate+IPv4off, r->v4.gate, IPv4addrlen);
    } else {
        for(i = 0; i < IPllen; i++){
            hnputl(addr + 4*i, r->v6.address[i]);
            hnputl(mask + 4*i, ~(r->v6.endaddress[i] ^ r->v6.address[i]));
        }
        memmove(gate, r->v6.gate, IPaddrlen);
    }

    routetype(r->type, t);

    if(r->ifc)
        *nifc = r->ifc->conv->x;
    else
        *nifc = -1;
}
```

⟨*function* routetype 104c⟩≡ (339c)

```
void
routetype(int type, char *p)
{
    memset(p, ' ', 4);
    p[4] = 0;
    if(type & Rv4)
        *p++ = '4';
    else
        *p++ = '6';
    if(type & Rifc)
        *p++ = 'i';
    if(type & Runi)
        *p++ = 'u';
    else if(type & Rbcast)
        *p++ = 'b';
    else if(type & Rmulti)
        *p++ = 'm';
    if(type & Rptpt)
        *p = 'p';
}
```

⟨*function* ipwalkroutes 105a⟩≡ (339c)

```
void
ipwalkroutes(Fs *f, Routewalk *rw)
{
    rlock(&routelock);
    if(rw->e > rw->p) {
        for(rw->h = 0; rw->h < nelem(f->v4root); rw->h++)
            if(rr(f->v4root[rw->h], rw) == 0)
                break;
    }
    if(rw->e > rw->p) {
        for(rw->h = 0; rw->h < nelem(f->v6root); rw->h++)
            if(rr(f->v6root[rw->h], rw) == 0)
                break;
    }
    runlock(&routelock);
}
```

⟨*function* rr 105b⟩≡ (339c)

```
/*
 *  recurse descending tree, applying the function in Routewalk
 */
static int
rr(Route *r, Routewalk *rw)
{
    int h;

    if(rw->e <= rw->p)
        return 0;
    if(r == nil)
        return 1;

    if(rr(r->left, rw) == 0)
        return 0;

    if(r->type & Rv4)
        h = V4H(r->v4.address);
    ⟨rr() else if ipv6 address 481c⟩
```

```
    if(h == rw->h)
        rw->walk(r, rw);

    if(rr(r->mid, rw) == 0)
        return 0;

    return rr(r->right, rw);
}
```

10.6.2 Writing

⟨*function* routewrite 106⟩≡ (339c)

```
long
routewrite(Fs *f, Chan *c, char *p, int n)
{
    int h, changed;
    char *tag;
    Cmdbuf *cb;
    ipaddr addr, mask, gate;
    IPaux *a, *na;
    Route *q;

    cb = parsecmd(p, n);
    if(waserror()){
        free(cb);
        nexterror();
    }

    if(strcmp(cb->f[0], "flush") == 0){
        tag = cb->f[1];
        for(h = 0; h < nelem(f->v4root); h++)
            for(changed = 1; changed;){
                wlock(&routelock);
                changed = routeflush(f, f->v4root[h], tag);
                wunlock(&routelock);
            }
        for(h = 0; h < nelem(f->v6root); h++)
            for(changed = 1; changed;){
                wlock(&routelock);
                changed = routeflush(f, f->v6root[h], tag);
                wunlock(&routelock);
            }
    } else if(strcmp(cb->f[0], "remove") == 0){
        if(cb->nf < 3)
            error(Ebadarg);
        if (parseip(addr, cb->f[1]) == -1)
            error(Ebadip);
        parseipmask(mask, cb->f[2]);
        if(memcmp(addr, v4prefix, IPv4off) == 0)
            v4delroute(f, addr+IPv4off, mask+IPv4off, 1);
        else
            v6delroute(f, addr, mask, 1);
    } else if(strcmp(cb->f[0], "add") == 0){
        if(cb->nf < 4)
            error(Ebadarg);
        if(parseip(addr, cb->f[1]) == -1 ||
            parseip(gate, cb->f[3]) == -1)
            error(Ebadip);
        parseipmask(mask, cb->f[2]);
```

```
        tag = "none";
        if(c != nil){
            a = c->aux;
            tag = a->tag;
        }
        if(memcmp(addr, v4prefix, IPv4off) == 0)
            v4addroute(f, tag, addr+IPv4off, mask+IPv4off, gate+IPv4off, 0);
        else
            v6addroute(f, tag, addr, mask, gate, 0);
    } else if(strcmp(cb->f[0], "tag") == 0) {
        if(cb->nf < 2)
            error(Ebadarg);

        a = c->aux;
        na = newipaux(a->owner, cb->f[1]);
        c->aux = na;
        free(a);
    } else if(strcmp(cb->f[0], "route") == 0) {
        if(cb->nf < 2)
            error(Ebadarg);
        if (parseip(addr, cb->f[1]) == -1)
            error(Ebadip);

        q = iproute(f, addr);
        print("%I: ", addr);
        if(q == nil)
            print("no route\n");
        else
            printroute(q);
    }

    poperror();
    free(cb);
    return n;
}
```

⟨*function* iproute 107a⟩≡ (339c)

```
Route *
iproute(Fs *fs, ipaddr ip)
{
    if(isv4(ip))
        return v4lookup(fs, ip+IPv4off, nil);
    else
        return v6lookup(fs, ip, nil);
}
```

⟨*function* printroute 107b⟩≡ (339c)

```
static void
printroute(Route *r)
{
    int nifc;
    char t[5], *iname, ifbuf[5];
    ipaddr addr, mask, gate;

    convroute(r, addr, mask, gate, t, &nifc);
    iname = "-";
    if(nifc != -1) {
        iname = ifbuf;
        snprint(ifbuf, sizeof ifbuf, "%d", nifc);
    }
```

```
    print(rformat, addr, mask, gate, t, r->tag, iname);
}
```

10.7 Self cache

⟨Fs(kernel) *other fields* 108a⟩+≡ (21c) ◁24b

```
Ipselftab *self;
```

⟨Iplifc(kernel) *extra fields* 108b⟩+≡ (18a) ◁18b

```
Iplink  *link;    /* addresses linked to this lifc */
```

⟨*function* ipforme 108c⟩≡ (354b)

```
/*
 *  returns
 *  0       - no match
 *  Runi
 *  Rbcast
 *  Rmcast
 */
int
ipforme(Fs *f, ipaddr addr)
{
    Ipself *p;

    p = f->self->hash[hashipa(addr)];
    for(; p; p = p->next){
        if(ipcmp(addr, p->a) == 0)
            return p->type;
    }

    /* hack to say accept anything */
    if(f->self->acceptall)
        return Runi;
    return 0;
}
```

10.7.1 Data structures

⟨*struct* Ipselftab 108d⟩≡ (354b)

```
struct Ipselftab
{
    int inited;
    bool acceptall;  /* true if an interface has the null address */

    Ipself  *hash[NHASH];   /* hash chains */

    // Extra
    QLock;

};
```

⟨ipifcinit() *modify f* 108e⟩+≡ (56a) ◁57b

```
f->self = smalloc(sizeof(Ipselftab));   /* hack for ipforme */
```

⟨*struct* Ipself 109a⟩≡ (354b)

```
/*
 *  cache of local addresses (addresses we answer to)
 */
struct Ipself
{
    uchar   type;       /* type of address */
    ipaddr   a;

    ulong   expire;

    //Extra
    int ref;
    Ipself  *hnext;     /* next address in the hash table */
    Iplink  *link;      /* binding twixt Ipself and Ipifc */
    Ipself  *next;      /* free list */
};
```

⟨*struct* Iplink 109b⟩≡ (208)

```
/* binding twixt Ipself and Iplifc */
struct Iplink
{
  ulong expire;

  Ipself  *self;
  Iplifc  *lifc;

  // Extra
  int ref;
  Iplink  *next;    /* free list */
  Iplink  *selflink;  /* next link for this local address */
  Iplink  *lifclink;  /* next link for this ifc */
};
```

10.7.2 Adding IPs

⟨*function* addselfcache 109c⟩≡ (354b)

```
/*
 *  add to self routing cache
 *  called with c->car locked
 */
static void
addselfcache(Fs *f, Ipifc *ifc, Iplifc *lifc, ipaddr a, int type)
{
    Ipself *p;
    Iplink *lp;
    int h;

    qlock(f->self);

    /* see if the address already exists */
    h = hashipa(a);
    for(p = f->self->hash[h]; p; p = p->next)
        if(memcmp(a, p->a, IPaddrlen) == 0)
            break;

    /* allocate a local address and add to hash chain */
    if(p == nil){
        p = smalloc(sizeof(*p));
```

```
        ipmove(p->a, a);
        p->type = type;
        p->next = f->self->hash[h];
        f->self->hash[h] = p;

        /* if the null address, accept all packets */
        if(ipcmp(a, v4prefix) == 0 || ipcmp(a, IPnoaddr) == 0)
            f->self->acceptall = true;
    }

    /* look for a link for this lifc */
    for(lp = p->link; lp; lp = lp->selflink)
        if(lp->lifc == lifc)
            break;

    /* allocate a lifc-to-local link and link to both */
    if(lp == nil){
        lp = smalloc(sizeof(*lp));
        lp->ref = 1;
        lp->lifc = lifc;
        lp->self = p;
        lp->selflink = p->link;
        p->link = lp;
        lp->lifclink = lifc->link;
        lifc->link = lp;

        /* add to routing table */
        if(isv4(a))
            v4addroute(f, tifc, a+IPv4off, IPallbits+IPv4off,
                a+IPv4off, type);
        else
            v6addroute(f, tifc, a, IPallbits, a, type);

        if((type & Rmulti) && ifc->m->addmulti != nil)
            (*ifc->m->addmulti)(ifc, a, lifc->local);
    } else
        lp->ref++;

    qunlock(f->self);
}
```

⟨*macro* hashipa 110a⟩≡ (354b)

```
/* quick hash for ip addresses */
#define hashipa(a) ( ( ((a)[IPaddrlen-2]<<8) | (a)[IPaddrlen-1] )%NHASH )
```

10.7.3 Removing IPs

⟨*function* remselfcache 110b⟩≡ (354b)

```
/*
 *  Decrement reference for this address on this link.
 *  Unlink from selftab if this is the last ref.
 *  called with c->car locked
 */
static void
remselfcache(Fs *f, Ipifc *ifc, Iplifc *lifc, ipaddr a)
{
    Ipself *p, **l;
    Iplink *link, **l_self, **l_lifc;
```

```
qlock(f->self);

/* find the unique selftab entry */
l = &f->self->hash[hashipa(a)];
for(p = *l; p; p = *l){
    if(ipcmp(p->a, a) == 0)
        break;
    l = &p->next;
}

if(p == nil)
    goto out;

/*
 *  walk down links from an ifc looking for one
 *  that matches the selftab entry
 */
l_lifc = &lifc->link;
for(link = *l_lifc; link; link = *l_lifc){
    if(link->self == p)
        break;
    l_lifc = &link->lifclink;
}

if(link == nil)
    goto out;

/*
 *  walk down the links from the selftab looking for
 *  the one we just found
 */
l_self = &p->link;
for(link = *l_self; link; link = *l_self){
    if(link == *l_lifc)
        break;
    l_self = &link->selflink;
}

if(link == nil)
    panic("remselfcache");

if(--(link->ref) != 0)
    goto out;

if((p->type & Rmulti) && ifc->m->remmulti != nil)
    (*ifc->m->remmulti)(ifc, a, lifc->local);

/* ref == 0, remove from both chains and free the link */
*l_lifc = link->lifclink;
*l_self = link->selflink;
iplinkfree(link);

if(p->link != nil)
    goto out;

/* remove from routing table */
if(isv4(a))
    v4delroute(f, a+IPv4off, IPallbits+IPv4off, 1);
else
    v6delroute(f, a, IPallbits, 1);
```

```
    /* no more links, remove from hash and free */
    *l = p->next;
    ipselffree(p);

    /* if IPnoaddr, forget */
    if(ipcmp(a, v4prefix) == 0 || ipcmp(a, IPnoaddr) == 0)
        f->self->acceptall = false;

out:
    qunlock(f->self);
}
```

10.7.4 /net/ipselftab

⟨Qid *toplevel extra cases* 112a⟩≡ (26f) 160b ▷

```
Qipselftab,
```

⟨ip1gen() *switch TYPE qid cases* 112b⟩+≡ (37a) ◁ 103a 160c ▷

```
case Qipselftab:
    p = "ipselftab";
    prot = 0444;
    break;
```

⟨ipread() *switch TYPE qid cases* 112c⟩+≡ (41a) ◁ 103b 160g ▷

```
case Qipselftab:
    return ipselftabread(f, a, offset, n);
```

⟨*function* ipselftabread 112d⟩≡ (354b)

```
long
ipselftabread(Fs *f, char *cp, ulong offset, int n)
{
    int i, m, nifc, off;
    Ipself *p;
    Iplink *link;
    char state[8];

    m = 0;
    off = offset;
    qlock(f->self);
    for(i = 0; i < NHASH && m < n; i++){
        for(p = f->self->hash[i]; p != nil && m < n; p = p->next){
            nifc = 0;
            for(link = p->link; link; link = link->selflink)
                nifc++;
            routetype(p->type, state);
            m += snprint(cp + m, n - m, stformat, p->a, nifc, state);
            if(off > 0){
                off -= m;
                m = 0;
            }
        }
    }
    qunlock(f->self);
    return m;
}
```

10.8 Advanced features

10.8.1 Removing routes

⟨*function* v4delroute 113a⟩≡ (339c)

```
void
v4delroute(Fs *f, ipv4 a, ipv4 mask, bool dolock)
{
    Route **r, *p;
    Route rt;
    int h, eh;
    ulong m;

    m = nhgetl(mask);
    rt.v4.address = nhgetl(a) & m;
    rt.v4.endaddress = rt.v4.address | ~m;
    rt.type = Rv4;

    eh = V4H(rt.v4.endaddress);
    for(h=V4H(rt.v4.address); h<=eh; h++) {
        if(dolock)
            wlock(&routelock);
        r = looknode(&f->v4root[h], &rt);
        if(r) {
            p = *r;
            if(--(p->ref) == 0){
                *r = 0;
                addqueue(&f->queue, p->left);
                addqueue(&f->queue, p->mid);
                addqueue(&f->queue, p->right);
                freeroute(p);
                while(p = f->queue) {
                    f->queue = p->mid;
                    walkadd(f, &f->v4root[h], p->left);
                    freeroute(p);
                }
            }
        }
        if(dolock)
            wunlock(&routelock);
    }
    v4routegeneration++;

    ipifcremroute(f, Rv4, a, mask);
}
```

⟨*function* looknode 113b⟩≡ (339c)

```
Route**
looknode(Route **cur, Route *r)
{
    Route *p;

    for(;;){
        p = *cur;
        if(p == 0)
            return 0;

        switch(rangecompare(r, p)){
        case Rcontains:
            return 0;
```

```
        case Rpreceeds:
            cur = &p->left;
            break;
        case Rfollows:
            cur = &p->right;
            break;
        case Rcontained:
            cur = &p->mid;
            break;
        case Requals:
            return cur;
        }
    }
}
```

10.8.2 Flushing routes

⟨*function* routeflush 114a⟩≡ (339c)

```
/*
 *  recurse until one route is deleted
 *    returns 0 if nothing is deleted, 1 otherwise
 */
int
routeflush(Fs *f, Route *r, char *tag)
{
    if(r == nil)
        return 0;
    if(routeflush(f, r->mid, tag))
        return 1;
    if(routeflush(f, r->left, tag))
        return 1;
    if(routeflush(f, r->right, tag))
        return 1;
    if((r->type & Rifc) == 0){
        if(tag == nil || strncmp(tag, r->tag, sizeof(r->tag)) == 0){
            delroute(f, r, 0);
            return 1;
        }
    }
    return 0;
}
```

⟨*function* delroute 114b⟩≡ (339c)

```
/*
 *  this code is not in routeflush to reduce stack size
 */
void
delroute(Fs *f, Route *r, int dolock)
{
    ipaddr addr, mask, gate;
    char t[5];
    int nifc;

    convroute(r, addr, mask, gate, t, &nifc);
    if(r->type & Rv4)
        v4delroute(f, addr+IPv4off, mask+IPv4off, dolock);
    else
        v6delroute(f, addr, mask, dolock);
}
```

10.8.3 Router boards

⟨Medium(kernel) *router methods* 115a⟩≡ (18d)

```
/* routes for router boards */
void  (*addroute)(Ipifc *ifc, int, uchar*, uchar*, uchar*, int);
void  (*remroute)(Ipifc *ifc, int, uchar*, uchar*);
```

⟨*function* ipifcaddroute 115b⟩≡ (354b)

```
/*
 * distribute routes to active interfaces like the
 * TRIP linecards
 */
void
ipifcaddroute(Fs *f, int vers, uchar *addr, uchar *mask, uchar *gate, int type)
{
    Medium *m;
    Conv **cp, **e;
    Ipifc *ifc;

    e = &f->ipifc->conv[f->ipifc->nc];
    for(cp = f->ipifc->conv; cp < e; cp++){
        if(*cp != nil) {
            ifc = (Ipifc*)(*cp)->ptcl;
            m = ifc->m;
            if(m && m->addroute)
                m->addroute(ifc, vers, addr, mask, gate, type);
        }
    }
}
```

⟨*function* ipifcremroute 115c⟩≡ (354b)

```
void
ipifcremroute(Fs *f, int vers, uchar *addr, uchar *mask)
{
    Medium *m;
    Conv **cp, **e;
    Ipifc *ifc;

    e = &f->ipifc->conv[f->ipifc->nc];
    for(cp = f->ipifc->conv; cp < e; cp++){
        if(*cp != nil) {
            ifc = (Ipifc*)(*cp)->ptcl;
            m = ifc->m;
            if(m && m->remroute)
                m->remroute(ifc, vers, addr, mask);
        }
    }
}
```

Chapter 11

Basic Communication: UDP

11.1 Protocol initialisation

⟨*function* udpinit 116a⟩≡ (374)

```
void
udpinit(Fs *fs)
{
    Proto *udp;

    udp = smalloc(sizeof(Proto));
    udp->priv = smalloc(sizeof(Udppriv));

    udp->name = "udp";
    udp->create = udpcreate;
    udp->close = udpclose;

    udp->connect = udpconnect;
    udp->announce = udpannounce;
    udp->ctl = udpctl;

    udp->rcv = udpiput;

    udp->state = udpstate;
    udp->stats = udpstats;

    udp->advise = udpadvise;

    udp->ipproto = IP_UDPPROTO;

    udp->nc = Nchans;
    udp->ptclsize = sizeof(Udpcb);

    Fsproto(fs, udp);
}
```

⟨*constant* IP_UDPPROTO 116b⟩≡ (372b)

```
IP_UDPPROTO = 17,
```

⟨*constant* Nchans 116c⟩≡ (206b)

```
Nchans=   1024,
```

11.2 Protocol data structures

11.2.1 Udppriv

⟨*struct* Udppriv 117a⟩≡ (374)

```
struct Udppriv
{
    // hash<(ipaddr * port) , ref<Conv>>
    Ipht        ht;

    ⟨Udppriv stat fields 118c⟩
};
```

11.2.2 Addresses hashtable: Ipht

⟨*struct* Ipht 117b⟩≡ (208)

```
/*
 *  hash table for 2 ip addresses + 2 ports
 */
struct Ipht
{
  // hash<ipconvhash, ref<Conv>>, next = Iphash.next
  Iphash  *tab[Nipht];

  // Extra
  Lock;
};
```

⟨*constant* Nipht 117c⟩≡ (207b)

```
Nipht=    521,  /* convenient prime */
```

⟨*struct* Iphash 117d⟩≡ (208)

```
struct Iphash
{
  Conv  *c;
  // enum<matchtype>
  int match;

  // Extra
  Iphash  *next;
};
```

⟨*enum* matchtype 117e⟩≡ (208)

```
enum matchtype {
  IPmatchexact= 0,  /* match on 4 tuple */

  IPmatchany,   /* *!* */
  IPmatchport,    /* *!port */
  IPmatchaddr,    /* addr!* */
  IPmatchpa,    /* addr!port */
};
```

⟨*function* iphash 117f⟩≡ (343)

```
/*
 *  hashing tcp, udp, ... connections
 */
ulong
iphash(ipaddr sa, ushort sp, ipaddr da, ushort dp)
{
```

```
    return ((sa[IPaddrlen-1]<<24) ^ (sp << 16) ^ (da[IPaddrlen-1]<<8) ^ dp )
      % Nipht; // pad's first network bugfix :)
}
```

⟨*function* iphtadd 118a⟩≡ (343)

```
void
iphtadd(Ipht *ht, Conv *c)
{
    ulong hv;
    Iphash *h;

    hv = iphash(c->raddr, c->rport, c->laddr, c->lport);
    h = smalloc(sizeof(Iphash));
    if(ipcmp(c->raddr, IPnoaddr) != 0)
        h->match = IPmatchexact;
    else {
        if(ipcmp(c->laddr, IPnoaddr) != 0){
            if(c->lport == 0)
                h->match = IPmatchaddr;
            else
                h->match = IPmatchpa;
        } else {
            if(c->lport == 0)
                h->match = IPmatchany;
            else
                h->match = IPmatchport;
        }
    }
    h->c = c;

    lock(ht);
    // add_hash(h, ht)
    h->next = ht->tab[hv];
    ht->tab[hv] = h;
    unlock(ht);
}
```

⟨*function* iphtrem 118b⟩≡ (343)

```
void
iphtrem(Ipht *ht, Conv *c)
{
    ulong hv;
    Iphash **l, *h;

    hv = iphash(c->raddr, c->rport, c->laddr, c->lport);
    lock(ht);
    // del_hash(hv, ht)
    for(l = &ht->tab[hv]; (*l) != nil; l = &(*l)->next)
        if((*l)->c == c){
            h = *l;
            (*l) = h->next;
            free(h);
            break;
        }
    unlock(ht);
}
```

11.2.3 Statistics

⟨Udppriv *stat fields* 118c⟩≡ (117a)

```
/* MIB counters */
Udpstats    ustats;
/* non-MIB stats */
ulong       csumerr;        /* checksum errors */
ulong       lenerr;         /* short packet */
```

⟨*struct* Udpstats 119a⟩≡ (374)

```
/* MIB II counters */
struct Udpstats
{
    uvlong  udpInDatagrams;
    ulong   udpNoPorts;
    ulong   udpInErrors;
    uvlong  udpOutDatagrams;
};
```

11.3 Protocol header

⟨*struct* Udp4hdr 119b⟩≡ (374)

```
struct Udp4hdr
{
    /* ip header */
    uchar   vihl;       /* Version and header length */
    uchar   tos;        /* Type of service */
    uchar   length[2];  /* packet length */
    uchar   id[2];      /* Identification */
    uchar   frag[2];    /* Fragment information */
    uchar   Unused; // ttl

    uchar   udpproto;   /* Protocol */
    uchar   udpplen[2]; /* Header plus data length */
    ipv4    udpsrc;    /* Ip source */
    ipv4    udpdst;    /* Ip destination */

    /* udp header */
    uchar   udpsport[2];    /* Source port */
    uchar   udpdport[2];    /* Destination port */
    uchar   udplen[2];  /* data length */
    uchar   udpcksum[2];    /* Checksum */
};
```

⟨*constant* UDP4_IPHDR_SZ 119c⟩≡ (372b)

```
UDP4_IPHDR_SZ = 20,
```

⟨*constant* UDP_UDPHDR_SZ 119d⟩≡ (372b)

```
UDP_UDPHDR_SZ   = 8,
```

⟨*constant* UDP4_PHDR_OFF 119e⟩≡ (372b)

```
UDP4_PHDR_OFF = 8,
```

⟨*constant* UDP4_PHDR_SZ 119f⟩≡ (372b)

```
UDP4_PHDR_SZ = 12,
```

⟨*struct Udphdr (user)* 120a⟩≡ (196c)

```
struct Udphdr
{
    ipaddr raddr; /* V6 remote address */
    ipaddr laddr; /* V6 local address */
    ipaddr ifcaddr; /* V6 ifc addr msg was received on */

    uchar rport[2];  /* remote port */
    uchar lport[2];  /* local port */
};
```

11.4 /net/tcp/clone

⟨*function* udpcreate 120b⟩≡ (374)

```
static void
udpcreate(Conv *cv)
{
    cv->rq = qopen(128*1024, Qmsg, 0, 0);
    cv->wq = qbypass(udpkick, cv);
}
```

⟨*function* udpclose 120c⟩≡ (374)

```
static void
udpclose(Conv *c)
{
    Udpcb *ucb;
    Udppriv *upriv;

    upriv = c->p->priv;
    iphtrem(&upriv->ht, c);

    c->state = Idle;
    qclose(c->rq);
    qclose(c->wq);
    qclose(c->eq);
    ipmove(c->laddr, IPnoaddr);
    ipmove(c->raddr, IPnoaddr);
    c->lport = 0;
    c->rport = 0;

    ucb = (Udpcb*)c->ptcl;
    ucb->headers = 0;
}
```

11.5 /net/tcp/x/ctl

11.5.1 Connect

⟨*function* udpconnect 120d⟩≡ (374)

```
static char*
udpconnect(Conv *c, char **argv, int argc)
{
    char *err;
    Udppriv *upriv;

    upriv = c->p->priv;
```

```
    err = Fsstdconnect(c, argv, argc);
    Fsconnected(c, err);
    if(err != nil)
        return err;

    iphtadd(&upriv->ht, c);
    return nil;
}
```

11.5.2 Announce

⟨*function* udpannounce 121a⟩≡ (374)

```
static char*
udpannounce(Conv *c, char** argv, int argc)
{
    char *err;
    Udppriv *upriv;

    upriv = c->p->priv;

    err = Fsstdannounce(c, argv, argc);
    if(err != nil)
        return err;
    Fsconnected(c, nil);

    iphtadd(&upriv->ht, c);

    return nil;
}
```

11.6 IO

11.6.1 Writing: udpkick()

⟨*function* udpkick 121b⟩≡ (374)

```
void
udpkick(void *x, Block *bp)
{
    Conv *c = x;
    Conv *rc;
    Udp4hdr *uh4;
    Udppriv *upriv;
    int dlen, ptcllen;
    Fs *f;
    int version;
    ⟨udpkick() locals 129e⟩

    upriv = c->p->priv;
    f = c->p->f;

//  netlog(c->p->f, Logudp, "udp: kick\n"); /* frequent and uninteresting */
    if(bp == nil)
        return;

    ⟨udpkick() special headers processing 129f⟩
    ⟨udpkick() set version to V4 or V6 480a⟩
```

```
    dlen = blocklen(bp);

    /* fill in pseudo header and compute checksum */
    switch(version){
    case V4:
        bp = padblock(bp, UDP4_IPHDR_SZ+UDP_UDPHDR_SZ);
        if(bp == nil)
            return;

        uh4 = (Udp4hdr *)(bp->rp);
        ptcllen = dlen + UDP_UDPHDR_SZ;
        uh4->Unused = 0;
        uh4->udpproto = IP_UDPPROTO;
        uh4->frag[0] = 0;
        uh4->frag[1] = 0;
        hnputs(uh4->udpplen, ptcllen);
        ⟨udpkick() if special headers 130a⟩
        else {
            v6tov4(uh4->udpdst, c->raddr);
            hnputs(uh4->udpdport, c->rport);
            if(ipcmp(c->laddr, IPnoaddr) == 0)
                findlocalip(f, c->laddr, c->raddr);
            v6tov4(uh4->udpsrc, c->laddr);
            rc = c;
        }
        hnputs(uh4->udpsport, c->lport);
        hnputs(uh4->udplen, ptcllen);

        uh4->udpcksum[0] = 0;
        uh4->udpcksum[1] = 0;
        hnputs(uh4->udpcksum,
               ptclcsum(bp, UDP4_PHDR_OFF, dlen+UDP_UDPHDR_SZ+UDP4_PHDR_SZ));
        uh4->vihl = IP_VER4;

        // Let's go, let's send the data
        ipoput4(f, bp, false, c->ttl, c->tos, rc);

        break;

    ⟨udpkick() switch version ipv6 case 479d⟩

    default:
        panic("udpkick: version %d", version);
    }
    upriv->ustats.udpOutDatagrams++;
}
```

11.6.2 Reading: udpiput()

⟨*function* udpiput 122⟩≡ (374)

```
void
udpiput(Proto *udp, Ipifc *ifc, Block *bp)
{
    Udppriv *upriv;
    Fs *f;
    Udp4hdr *uh4;
    int version;
    ipaddr raddr, laddr;
```

```
    ushort rport, lport;
    Conv *c;
    Udpcb *ucb;
    int len;
    ⟨udpiput() locals 126a⟩
    uchar *p;

    upriv = udp->priv;
    upriv->ustats.udpInDatagrams++;
    f = udp->f;
    uh4 = (Udp4hdr*)(bp->rp);
    ⟨udpiput() set version to V4 or V6 481f⟩

    ⟨udpiput() checking checksum and setting rxxx, lxxx 126b⟩

    qlock(udp);

    // find the corresponding conversation
    c = iphtlook(&upriv->ht, raddr, rport, laddr, lport);
    ⟨udpiput() if no conversation found 127a⟩

    ucb = (Udpcb*)c->ptcl;

    if(c->state == Announced){
        if(ucb->headers == 0){
            /* create a new conversation */
            ⟨udpiput() new conv to create, adjust laddr if not Runi 128d⟩
            c = Fsnewcall(c, raddr, rport, laddr, lport, version);
            if(c == nil){
                qunlock(udp);
                freeblist(bp);
                return;
            }
            // port may have changed?
            iphtadd(&upriv->ht, c);
            ucb = (Udpcb*)c->ptcl;
        }
    }

    qlock(c);
    qunlock(udp);


    ⟨udpiput() trim the packet, adjust bp removing header 126c⟩

    netlog(f, Logudpmsg, "udp: %I.%d -> %I.%d l %d\n", raddr, rport,
           laddr, lport, len);

    ⟨udpiput() if special headers 130b⟩

    if(bp->next)
        bp = concatblock(bp);

    ⟨udpiput() if reading queue is full 127b⟩

    qpass(c->rq, bp);
    qunlock(c);

}
```

Conversation dispatch

⟨*function* iphtlook 124⟩≡ (343)

```
/* look for a matching conversation with the following precedence
 *  connected && raddr,rport,laddr,lport
 *  announced && laddr,lport
 *  announced && *,lport
 *  announced && laddr,*
 *  announced && *,*
 */
Conv*
iphtlook(Ipht *ht, ipaddr sa, ushort sp, ipaddr da, ushort dp)
{
    ulong hv;
    Iphash *h;
    Conv *c;

    /* exact 4 pair match (connection) */
    hv = iphash(sa, sp, da, dp);
    lock(ht);
    for(h = ht->tab[hv]; h != nil; h = h->next){
        if(h->match == IPmatchexact) {
          c = h->c;
          if(sp == c->rport && dp == c->lport
             && ipcmp(sa, c->raddr) == 0 && ipcmp(da, c->laddr) == 0){
              unlock(ht);
              return c;
          }
        }
    }

    /* match local address and port */
    hv = iphash(IPnoaddr, 0, da, dp);
    for(h = ht->tab[hv]; h != nil; h = h->next){
        if(h->match == IPmatchpa) {
          c = h->c;
          if(dp == c->lport && ipcmp(da, c->laddr) == 0){
              unlock(ht);
              return c;
          }
       }
    }

    /* match just port */
    hv = iphash(IPnoaddr, 0, IPnoaddr, dp);
    for(h = ht->tab[hv]; h != nil; h = h->next){
        if(h->match == IPmatchport) {
          c = h->c;
          if(dp == c->lport){
              unlock(ht);
              return c;
          }
        }
    }

    /* match local address */
    hv = iphash(IPnoaddr, 0, da, 0);
    for(h = ht->tab[hv]; h != nil; h = h->next){
        if(h->match == IPmatchaddr) {
          c = h->c;
```

```
            if(ipcmp(da, c->laddr) == 0){
                unlock(ht);
                return c;
            }
        }
    }

    /* look for something that matches anything */
    hv = iphash(IPnoaddr, 0, IPnoaddr, 0);
    for(h = ht->tab[hv]; h != nil; h = h->next){
        if(h->match == IPmatchany) {
            c = h->c;
            unlock(ht);
            return c;
        }
    }
    unlock(ht);
    return nil;
}
```

New conversation

⟨*function* Fsnewcall 125⟩≡ (302c)

```
/*
 *  called with protocol locked
 */
Conv*
Fsnewcall(Conv *c, ipaddr raddr, ushort rport, ipaddr laddr, ushort lport, uchar version)
{
    Conv *nc;
    Conv **l;
    int i;

    qlock(c);
    i = 0;
    for(l = &c->incall; *l; l = &(*l)->next)
        i++;
    if(i >= Maxincall) {
        static bool beenhere;

        qunlock(c);
        if (!beenhere) {
            beenhere = true;
            print("Fsnewcall: incall queue full (%d) on port %d\n",
                i, c->lport);
        }
        return nil;
    }

    /* find a free conversation */
    nc = Fsprotoclone(c->p, network);
    if(nc == nil) {
        qunlock(c);
        return nil;
    }
    ipmove(nc->raddr, raddr);
    nc->rport = rport;
    ipmove(nc->laddr, laddr);
    nc->lport = lport;
```

```
    nc->next = nil;
    *l = nc;

    nc->state = Connected;
    nc->ipversion = version;

    qunlock(c);

    wakeup(&c->listenr);

    return nc;
}
```

Checks

⟨udpiput() *locals* 126a⟩≡ (122) 480d ▷

```
int ottl, olen;
```

⟨udpiput() *checking checksum and setting rxxx, lxxx* 126b⟩≡ (122)

```
/* Put back pseudo header for checksum
 * (remember old values for icmpnoconv()) */
switch(version) {
case V4:
    ottl = uh4->Unused;
    uh4->Unused = 0;
    len = nhgets(uh4->udplen);
    olen = nhgets(uh4->udpplen);
    hnputs(uh4->udpplen, len);

    v4tov6(raddr, uh4->udpsrc);
    v4tov6(laddr, uh4->udpdst);
    lport = nhgets(uh4->udpdport);
    rport = nhgets(uh4->udpsport);

    if(nhgets(uh4->udpcksum)) {
        if(ptclcsum(bp, UDP4_PHDR_OFF, len+UDP4_PHDR_SZ)) {
            upriv->ustats.udpInErrors++;
            netlog(f, Logudp, "udp: checksum error %I\n", raddr);
            DPRINT("udp: checksum error %I\n", raddr);
            freeblist(bp);
            return;
        }
    }
    uh4->Unused = ottl;
    hnputs(uh4->udpplen, olen);
    break;

⟨udpiput() checking checksum ipv6 case 480e⟩

default:
    panic("udpiput: version %d", version);
    return; /* to avoid a warning */
}
```

Trim the packet

⟨udpiput() *trim the packet, adjust bp removing header* 126c⟩≡ (122)

```
/*
 * Trim the packet down to data size
 */
len -= UDP_UDPHDR_SZ;
switch(version){
case V4:
    bp = trimblock(bp, UDP4_IPHDR_SZ+UDP_UDPHDR_SZ, len);
    break;
⟨udpiput() trim the packet, ipv6 case 481b⟩
default:
    bp = nil;
    panic("udpiput4: version %d", version);
}
if(bp == nil){
    qunlock(c);
    netlog(f, Logudp, "udp: len err %I.%d -> %I.%d\n", raddr, rport,
           laddr, lport);
    upriv->lenerr++;
    return;
}
```

Error managment

⟨udpiput() *if no conversation found* 127a⟩≡ (122)

```
if(c == nil){
    /* no conversation found */
    upriv->ustats.udpNoPorts++;
    qunlock(udp);
    netlog(f, Logudp, "udp: no conv %I!%d -> %I!%d\n", raddr, rport,
           laddr, lport);

    switch(version){
    case V4:
        icmpnoconv(f, bp);
        break;
     ⟨udpiput() no conversation found, ipv6 case 481a⟩
    default:
        panic("udpiput2: version %d", version);
    }

    freeblist(bp);
    return;
}
```

⟨udpiput() *if reading queue is full* 127b⟩≡ (122)

```
if(qfull(c->rq)){
    qunlock(c);
    netlog(f, Logudp, "udp: qfull %I.%d -> %I.%d\n", raddr, rport,
           laddr, lport);
    freeblist(bp);
    return;
}
```

11.7 Other files

11.7.1 /net/tcp/stats

⟨*function* udpstats 127c⟩≡ (374)

```
int
udpstats(Proto *udp, char *buf, int len)
{
    Udppriv *upriv;

    upriv = udp->priv;
    return snprint(buf, len, "InDatagrams: %llud\nNoPorts: %lud\n"
        "InErrors: %lud\nOutDatagrams: %llud\n",
        upriv->ustats.udpInDatagrams,
        upriv->ustats.udpNoPorts,
        upriv->ustats.udpInErrors,
        upriv->ustats.udpOutDatagrams);
}
```

11.7.2 /net/tcp/x/status

⟨*function* udpstate 128a⟩≡ (374)

```
static int
udpstate(Conv *cv, char *state, int n)
{
    return snprint(state, n, "%s qin %d qout %d\n",
        cv->inuse ? "Open" : "Closed",
        cv->rq ? qlen(cv->rq) : 0,
        cv->wq ? qlen(cv->wq) : 0
    );
}
```

11.8 Advanced features

11.8.1 Ignore advice

⟨Conv(kernel) *udp fields* 128b⟩≡ (25a)

```
bool ignoreadvice;   /* don't terminate connection on icmp errors */
```

⟨ipwrite() *Qctl case, else if other string* 128c⟩+≡ (44f) ◁89d 158e▷

```
else if(strcmp(cb->f[0], "ignoreadvice") == 0)
    cv->ignoreadvice = true;
```

11.8.2 Not regular forme

⟨udpiput() *new conv to create, adjust laddr if not Runi* 128d⟩≡ (122)

```
if(ipforme(f, laddr) != Runi) {
    switch(version){
    case V4:
        v4tov6(laddr, ifc->lifc->local);
        break;
    ⟨udpiput() new conv to create, ipv6 case 481e⟩
    default:
        panic("udpiput3: version %d", version);
    }
}
```

11.8.3 User level UDP headers

⟨*struct* Udpcb 129a⟩≡ (374)

```
struct Udpcb
{
    QLock;
    ⟨Idpcb other fields 129b⟩
};
```

⟨Idpcb *other fields* 129b⟩≡ (129a)

```
uchar   headers;
```

⟨*function* udpctl 129c⟩≡ (374)

```
char*
udpctl(Conv *c, char **f, int n)
{
    Udpcb *ucb;

    ucb = (Udpcb*)c->ptcl;
    if(n == 1){
        if(strcmp(f[0], "headers") == 0){
            ucb->headers = 7;   /* new headers format */
            return nil;
        }
    }
    return "unknown control request";
}
```

⟨*constant* UDP_USEAD7 129d⟩≡ (372b)

```
UDP_USEAD7  = 52,
```

⟨udpkick() *locals* 129e⟩≡ (121b) 479c ▷

```
Udpcb *ucb;
ipaddr laddr, raddr;
ushort rport;
```

⟨udpkick() *special headers processing* 129f⟩≡ (121b)

```
ucb = (Udpcb*)c->ptcl;
switch(ucb->headers) {
case 7:
    /* get user specified addresses */
    bp = pullupblock(bp, UDP_USEAD7);
    if(bp == nil)
        return;
    ipmove(raddr, bp->rp);
    bp->rp += IPaddrlen;
    ipmove(laddr, bp->rp);
    bp->rp += IPaddrlen;

    /* pick interface closest to dest */
    if(ipforme(f, laddr) != Runi)
        findlocalip(f, laddr, raddr);
    bp->rp += IPaddrlen;        /* Ignore ifc address */
    rport = nhgets(bp->rp);
    bp->rp += 2+2;          /* Ignore local port */
    break;
default:
    rport = 0;
    break;
}
```

⟨udpkick() *if special headers* 130a⟩≡ (121b)

```
if(ucb->headers) {
    v6tov4(uh4->udpdst, raddr);
    hnputs(uh4->udpdport, rport);
    v6tov4(uh4->udpsrc, laddr);
    rc = nil;
}
```

⟨udpiput() *if special headers* 130b⟩≡ (122)

```
switch(ucb->headers){
case 7:
    /* pass the src address */
    bp = padblock(bp, UDP_USEAD7);
    p = bp->rp;
    ipmove(p, raddr); p += IPaddrlen;
    ipmove(p, laddr); p += IPaddrlen;
    ipmove(p, ifc->lifc->local); p += IPaddrlen;
    hnputs(p, rport); p += 2;
    hnputs(p, lport);
    break;
}
```

Chapter 12

Reliable Communication: IL

12.1 Protocol initialisation

⟨*function* ilinit 131a⟩≡ (370b)

```
void
ilinit(Fs *f)
{
    Proto *il;

    inittimescale();

    il = smalloc(sizeof(Proto));
    il->priv = smalloc(sizeof(Ilpriv));

    il->name = "il";
    il->create = ilcreate;
    il->close = ilclose;

    il->connect = ilconnect;
    il->announce = ilannounce;
    il->ctl = nil;

    il->rcv = iliput;

    il->state = ilstate;
    il->stats = ilxstats;

    il->inuse = ilinuse;
    il->advise = iladvise;

    il->gc = nil;
    il->ipproto = IP_ILPROTO;

    il->nc = scalednconv();
    il->ptclsize = sizeof(Ilcb);

    Fsproto(f, il);
}
```

⟨*constant* IP_ILPROTO 131b⟩≡ (361c)

```
IP_ILPROTO = 40,
```

⟨*function* inittimescale 131c⟩≡ (370b)

```
/* calculate scale constants that converts fast ticks to ms (more or less) */
static void
```

```
inittimescale(void)
{
    uvlong hz;

    arch_fastticks(&hz);
    if(hz > 1000){
        scalediv = hz/1000;
        scalemul = 1;
    } else {
        scalediv = 1;
        scalemul = 1000/hz;
    }
}
```

⟨*global* scalexxx 132a⟩≡ (370b)

```
static ulong scalediv, scalemul;
```

⟨*function* scalednconv 132b⟩≡ (302c)

```
ulong
scalednconv(void)
{
    if(cpuserver && conf.npage*BY2PG >= 128*MB)
        return Nchans*4;
    return Nchans;
}
```

12.2 Protocol data structures

12.2.1 IL state

⟨*enum* _anon_ (kernel/network/ip/il.c) 132c⟩≡ (370b)

```
enum il_state    /* Connection state */
{
    Ilclosed,

    Ilsyncer,
    Ilsyncee,
    Ilestablished,
    Illistening,
    Ilclosing,

    Ilopening,  /* only for file server */
};
```

⟨*global* ilstates 132d⟩≡ (370b)

```
char *ilstates[] =
{
    "Closed",
    "Syncer",
    "Syncee",
    "Established",
    "Listen",
    "Closing",
    "Opening",  /* only for file server */
};
```

⟨*function* ilinuse 133a⟩≡ (370b)

```
static int
ilinuse(Conv *c)
{
    Ilcb *ic;

    ic = (Ilcb*)(c->ptcl);
    return ic->state != Ilclosed;

}
```

12.2.2 IL control packet type

⟨*enum* _anon_ (kernel/network/ip/il.c)2 133b⟩≡ (370b)

```
enum    /* Packet types */
{
    Ilsync,
    Ildata,
    Ildataquery,
    Ilack,
    Ilquery,
    Ilstate,
    Ilclose,
};
```

⟨*global* iltype 133c⟩≡ (370b)

```
char *iltype[] =
{
    "sync",
    "data",
    "dataquery",
    "ack",
    "query",
    "state",
    "close"
};
```

12.2.3 Ilcb

⟨*struct* Ilcb 133d⟩≡ (370b)

```
struct Ilcb   /* Control block */
{
    // enum<il_state>
    int state;  /* Connection state */

    Conv *conv;

    QLock ackq;  /* Unacknowledged queue */
    Block *unacked;
    Block *unackedtail;
    ulong unackedbytes;
    QLock outo;  /* Out of order packet queue */
    Block *outoforder;
    ulong next;  /* Id of next to send */
    ulong recvd;  /* Last packet received */
    ulong acksent; /* Last packet acked */
    ulong start;  /* Local start id */
    ulong rstart;  /* Remote start id */
```

```
    int window;  /* Maximum receive window */
    int rxquery; /* number of queries on this connection */
    int rxtot;  /* number of retransmits on this connection */
    int rexmit;  /* number of retransmits of *unacked */
    ulong qt[Nqt+1]; /* state table for query messages */
    int qtx;  /* ... index into qt */

    /* if set, fasttimeout causes a connection request to terminate after 4*Iltickms */
    int fasttimeout;

    /* timers */
    ulong lastxmit; /* time of last xmit */
    ulong lastrecv; /* time of last recv */
    ulong timeout; /* retransmission time for *unacked */
    ulong acktime; /* time to send next ack */
    ulong querytime; /* time to send next query */

    /* adaptive measurements */
    int delay;  /* Average of the fixed rtt delay */
    int rate;  /* Average uchar rate */
    int mdev;  /* Mean deviation of rtt */
    int maxrtt;  /* largest rtt seen */
    ulong rttack;  /* The ack we are waiting for */
    int rttlen;  /* Length of rttack packet */
    uvlong rttstart; /* Time we issued rttack packet */
};
```

12.2.4 Ilpriv

⟨*struct* Ilpriv 134a⟩≡ (370b)

```
struct Ilpriv
{
    Ipht ht;

    ulong dup;
    ulong dupb;

    ⟨Ilpriv stat fields 134b⟩

    /* keeping track of the ack kproc */
    int ackprocstarted;
    QLock apl;
};
```

12.2.5 Statistics

⟨Ilpriv *stat fields* 134b⟩≡ (134a) 134c ▷

```
ulong csumerr;  /* checksum errors */
ulong hlenerr;  /* header length error */
ulong lenerr;   /* short packet */
ulong order;   /* out of order */
ulong rexmit;   /* retransmissions */
```

⟨Ilpriv *stat fields* 134c⟩+≡ (134a) ◁ 134b

```
// map<enum<il_stat>, uvlong>
uvlong stats[Nstats];
```

⟨*enum* _anon_ (kernel/network/ip/il.c)6 135a⟩≡ (370b)

```
enum il_stat
{
    InMsgs,
    OutMsgs,
    CsumErrs,  /* checksum errors */
    HlenErrs,  /* header length error */
    LenErrs,  /* short packet */
    OutOfOrder,  /* out of order */
    Retrans,  /* retransmissions */
    DupMsg,
    DupBytes,
    DroppedMsgs,

    Nstats,
};
```

⟨*global* statnames((kernel/network/ip/il.c)) 135b⟩≡ (370b)

```
static char *statnames[] =
{
[InMsgs] "InMsgs",
[OutMsgs] "OutMsgs",
[CsumErrs] "CsumErrs",
[HlenErrs] "HlenErr",
[LenErrs] "LenErrs",
[OutOfOrder] "OutOfOrder",
[Retrans] "Retrans",
[DupMsg] "DupMsg",
[DupBytes] "DupBytes",
[DroppedMsgs] "DroppedMsgs",
};
```

12.3 Protocol header

⟨*struct* Ilhdr 135c⟩≡ (370b)

```
struct Ilhdr
{
    /* ip header */
    uchar vihl;  /* Version and header length */
    uchar tos;  /* Type of service */
    uchar length[2]; /* packet length */
    uchar id[2];  /* Identification */
    uchar frag[2]; /* Fragment information */
    uchar ttl;  /* Time to live */

    uchar proto;  /* Protocol */
    uchar cksum[2]; /* Header checksum */
    ipv4 src;  /* Ip source */
    ipv4 dst;  /* Ip destination */

    /* IL header */
    uchar ilsum[2]; /* Checksum including header */
    uchar illen[2]; /* Packet length */
    uchar iltype;  /* Packet type */
    uchar ilspec;  /* Special */
    uchar ilsrc[2]; /* Src port */
    uchar ildst[2]; /* Dst port */
    uchar ilid[4]; /* Sequence id */
```

```
    uchar ilack[4]; /* Acked sequence */
};
```

⟨*constant* IL_xxxSIZE 136a⟩≡ (361c)

```
IL_IPSIZE  = 20,
IL_HDRSIZE = 18,
```

12.4 /net/il/clone

⟨*function* ilcreate 136b⟩≡ (370b)

```
static void
ilcreate(Conv *c)
{
    c->rq = qopen(Maxrq, 0, 0, c);
    c->wq = qbypass(ilkick, c);
}
```

⟨*constant Maxrq(IL)* 136c⟩≡ (361a)

```
Maxrq  = 64*1024,
```

12.5 /net/il/x/ctl

⟨*enum* mode*(*(kernel/network/ip/il.c)*)* 136d⟩≡ (370b)

```
enum mode {
    IL_LISTEN = 0,
    IL_CONNECT = 1,
};
```

12.5.1 Connect

⟨*function* ilconnect 136e⟩≡ (370b)

```
static char*
ilconnect(Conv *c, char **argv, int argc)
{
    char *err;
    char *p;
    bool fast = false;

    ⟨ilconnect() set fast 148⟩

    err = Fsstdconnect(c, argv, argc);
    if(err != nil)
        return err;
    return ilstart(c, IL_CONNECT, fast);
}
```

12.5.2 Announce

⟨*function* ilannounce 136f⟩≡ (370b)

```
/* called with c locked */
static char*
ilannounce(Conv *c, char **argv, int argc)
{
    char *err;
```

```
    err = Fsstdannounce(c, argv, argc);
    if(err != nil)
        return err;
    err = ilstart(c, IL_LISTEN, false);
    if(err != nil)
        return err;
    Fsconnected(c, nil);

    return nil;
}
```

12.6 IO

12.6.1 Writing: ilkick()

⟨*function* ilkick 137⟩≡ (370b)

```
void
ilkick(void *x, Block *bp)
{
    Conv *c = x;
    Ilhdr *ih;
    Ilcb *ic;
    int dlen;
    ulong id, ack;
    Fs *f;
    Ilpriv *priv;

    f = c->p->f;
    priv = c->p->priv;
    ic = (Ilcb*)c->ptcl;

    if(bp == nil)
        return;

    switch(ic->state) {
    case Ilclosed:
    case Illistening:
    case Ilclosing:
        freeblist(bp);
        qhangup(c->rq, nil);
        return;
    }

    dlen = blocklen(bp);

    /* Make space to fit il & ip */
    bp = padblock(bp, IL_IPSIZE+IL_HDRSIZE);
    ih = (Ilhdr *)(bp->rp);
    ih->vihl = IP_VER4;

    /* Ip fields */
    ih->frag[0] = 0;
    ih->frag[1] = 0;
    v6tov4(ih->dst, c->raddr);
    v6tov4(ih->src, c->laddr);
    ih->proto = IP_ILPROTO;
```

```
    /* Il fields */
    hnputs(ih->illen, dlen+IL_HDRSIZE);
    hnputs(ih->ilsrc, c->lport);
    hnputs(ih->ildst, c->rport);

    qlock(&ic->ackq);
    id = ic->next++;
    hnputl(ih->ilid, id);
    ack = ic->recvd;
    hnputl(ih->ilack, ack);
    ic->acksent = ack;
    ic->acktime = NOW + AckDelay;
    ih->iltype = Ildata;
    ih->ilspec = 0;
    ih->ilsum[0] = 0;
    ih->ilsum[1] = 0;

    /* Checksum of ilheader plus data (not ip & no pseudo header) */
    if(ilcksum)
        hnputs(ih->ilsum, ptclcsum(bp, IL_IPSIZE, dlen+IL_HDRSIZE));

    ilackq(ic, bp);

    qunlock(&ic->ackq);

    /* Start the round trip timer for this packet if the timer is free */
    if(ic->rttack == 0) {
        ic->rttack = id;
        ic->rttstart = arch_fastticks(nil);
        ic->rttlen = dlen + IL_IPSIZE + IL_HDRSIZE;
    }

    if(later(NOW, ic->timeout, nil))
        ilsettimeout(ic);

    // Let's go, let's send the data
    ipoput4(f, bp, false, c->ttl, c->tos, c);

    priv->stats[OutMsgs]++;
}
```

12.6.2 Reading: `iliput()`

⟨*function* `iliput` 138⟩≡ (370b)

```
void
iliput(Proto *il, Ipifc*, Block *bp)
{
    char *st;
    Ilcb *ic;
    Ilhdr *ih;
    ipaddr raddr, laddr;
    ushort sp, dp, csum;
    int plen, illen;
    Conv *new, *s;
    Ilpriv *ipriv;

    ipriv = il->priv;

    ih = (Ilhdr *)bp->rp;
```

```
    plen = blocklen(bp);
    if(plen < IL_IPSIZE+IL_HDRSIZE){
        //netlog(il->f, Logil, "il: hlenerr\n");
        ipriv->stats[HlenErrs]++;
        goto raise;
    }

    illen = nhgets(ih->illen);
    if(illen+IL_IPSIZE > plen){
        //netlog(il->f, Logil, "il: lenerr\n");
        ipriv->stats[LenErrs]++;
        goto raise;
    }

    sp = nhgets(ih->ildst);
    dp = nhgets(ih->ilsrc);
    v4tov6(raddr, ih->src);
    v4tov6(laddr, ih->dst);

    if((csum = ptclcsum(bp, IL_IPSIZE, illen)) != 0) {
        if(ih->iltype > Ilclose)
            st = "?";
        else
            st = iltype[ih->iltype];
        ipriv->stats[CsumErrs]++;
        //netlog(il->f, Logil, "il: cksum %ux %s, pkt(%ux id %ud ack %I/%d->%d)\n",
//   csum, st, nhgetl(ih->ilid), nhgetl(ih->ilack), raddr, sp, dp);
        USED(csum); USED(st);
        goto raise;
    }

    qlock(il);
    s = iphtlook(&ipriv->ht, raddr, dp, laddr, sp);
    if(s == nil){
        if(ih->iltype == Ilsync)
            ilreject(il->f, ih);  /* no listener */
        qunlock(il);
        goto raise;
    }

    ic = (Ilcb*)s->ptcl;
    if(ic->state == Illistening){
        if(ih->iltype != Ilsync){
            qunlock(il);
            if(ih->iltype > Ilclose)
                st = "?";
            else
                st = iltype[ih->iltype];
            ilreject(il->f, ih);  /* no channel and not sync */
            //netlog(il->f, Logil, "il: no channel, pkt(%s id %ud ack %ud %I/%ud->%ud)\n",
//    st, nhgetl(ih->ilid), nhgetl(ih->ilack), raddr, sp, dp);
            USED(st);
            goto raise;
        }

        new = Fsnewcall(s, raddr, dp, laddr, sp, V4);
        if(new == nil){
            qunlock(il);
            //netlog(il->f, Logil, "il: bad newcall %I/%ud->%ud\n", raddr, sp, dp);
            ilsendctl(s, ih, Ilclose, 0, nhgetl(ih->ilid), 0);
```

```
            goto raise;
        }
        s = new;

        ic = (Ilcb*)s->ptcl;

        ic->conv = s;
        ic->state = Ilsyncee;
        ilcbinit(ic);
        ic->rstart = nhgetl(ih->ilid);
        iphtadd(&ipriv->ht, s);
    }

    qlock(s);
    qunlock(il);
    if(waserror()){
        qunlock(s);
        nexterror();
    }

    ilprocess(s, ih, bp);

    qunlock(s);
    poperror();
    return;
raise:
    freeblist(bp);
}
```

12.7 State machine

12.7.1 ilsendctl()

⟨*function* ilsendctl 140⟩≡ (370b)

```
void
ilsendctl(Conv *ipc, Ilhdr *inih, int type, ulong id, ulong ack, int ilspec)
{
    Ilhdr *ih;
    Ilcb *ic;
    Block *bp;
    int ttl, tos;

    bp = allocb(IL_IPSIZE+IL_HDRSIZE);
    bp->wp += IL_IPSIZE+IL_HDRSIZE;

    ih = (Ilhdr *)(bp->rp);
    ih->vihl = IP_VER4;

    /* Ip fields */
    ih->proto = IP_ILPROTO;
    hnputs(ih->illen, IL_HDRSIZE);
    ih->frag[0] = 0;
    ih->frag[1] = 0;
    if(inih) {
        hnputl(ih->dst, nhgetl(inih->src));
        hnputl(ih->src, nhgetl(inih->dst));
        hnputs(ih->ilsrc, nhgets(inih->ildst));
        hnputs(ih->ildst, nhgets(inih->ilsrc));
```

```
        hnputl(ih->ilid, nhgetl(inih->ilack));
        hnputl(ih->ilack, nhgetl(inih->ilid));
        ttl = MAXTTL;
        tos = DFLTTOS;
    }
    else {
        v6tov4(ih->dst, ipc->raddr);
        v6tov4(ih->src, ipc->laddr);
        hnputs(ih->ilsrc, ipc->lport);
        hnputs(ih->ildst, ipc->rport);
        hnputl(ih->ilid, id);
        hnputl(ih->ilack, ack);
        ic = (Ilcb*)ipc->ptcl;
        ic->acksent = ack;
        ic->acktime = NOW;
        ttl = ipc->ttl;
        tos = ipc->tos;
    }
    ih->iltype = type;
    ih->ilspec = ilspec;
    ih->ilsum[0] = 0;
    ih->ilsum[1] = 0;

    if(ilcksum)
        hnputs(ih->ilsum, ptclcsum(bp, IL_IPSIZE, IL_HDRSIZE));

if(ipc==nil)
    panic("ipc is nil caller is %#p", getcallerpc(&ipc));
if(ipc->p==nil)
    panic("ipc->p is nil");

    //netlog(ipc->p->f, Logilmsg, "ctl(%s id %d ack %d %d->%d)\n",
    //  iltype[ih->iltype], nhgetl(ih->ilid), nhgetl(ih->ilack),
    //  nhgets(ih->ilsrc), nhgets(ih->ildst));

    ipoput4(ipc->p->f, bp, false, ttl, tos, ipc);
}
```

12.7.2 ilprocess()

⟨*function* ilprocess 141⟩≡ (370b)

```
/* DEBUG */
void
ilprocess(Conv *s, Ilhdr *h, Block *bp)
{
    Ilcb *ic;

    ic = (Ilcb*)s->ptcl;

    USED(ic);
    //netlog(s->p->f, Logilmsg, "%11s rcv %lud/%lud snt %lud/%lud pkt(%s id %d ack %ud %ud->%ud) ",
//  ilstates[ic->state],  ic->rstart, ic->recvd, ic->start,
//  ic->next, iltype[h->iltype], nhgetl(h->ilid),
//  nhgetl(h->ilack), nhgets(h->ilsrc), nhgets(h->ildst));

    _ilprocess(s, h, bp);

    //netlog(s->p->f, Logilmsg, "%11s rcv %lud snt %lud\n", ilstates[ic->state], ic->recvd, ic->next);
}
```

⟨*function* _ilprocess 142⟩≡ (370b)

```
void
_ilprocess(Conv *s, Ilhdr *h, Block *bp)
{
    Ilcb *ic;
    ulong id, ack;
    Ilpriv *priv;

    id = nhgetl(h->ilid);
    ack = nhgetl(h->ilack);

    ic = (Ilcb*)s->ptcl;

    ic->lastrecv = NOW;
    ic->querytime = NOW + QueryTime;
    priv = s->p->priv;
    priv->stats[InMsgs]++;

    switch(ic->state) {
    default:
        //netlog(s->p->f, Logil, "il: unknown state %d\n", ic->state);
    case Ilclosed:
        freeblist(bp);
        break;
    case Ilsyncer:
        switch(h->iltype) {
        default:
            break;
        case Ilsync:
            if(ack != ic->start)
                ilhangup(s, "connection rejected");
            else {
                ic->recvd = id;
                ic->rstart = id;
                ilsendctl(s, nil, Ilack, ic->next, ic->recvd, 0);
                ic->state = Ilestablished;
                ic->fasttimeout = 0;
                ic->rexmit = 0;
                Fsconnected(s, nil);
                ilpullup(s);
            }
            break;
        case Ilclose:
            if(ack == ic->start)
                ilhangup(s, "connection rejected");
            break;
        }
        freeblist(bp);
        break;
    case Ilsyncee:
        switch(h->iltype) {
        default:
            break;
        case Ilsync:
            if(id != ic->rstart || ack != 0){
                illocalclose(s);
            } else {
                ic->recvd = id;
                ilsendctl(s, nil, Ilsync, ic->start, ic->recvd, 0);
            }
```

```
        break;
    case Ilack:
        if(ack == ic->start) {
            ic->state = Ilestablished;
            ic->fasttimeout = 0;
            ic->rexmit = 0;
            ilpullup(s);
        }
        break;
    case Ildata:
        if(ack == ic->start) {
            ic->state = Ilestablished;
            ic->fasttimeout = 0;
            ic->rexmit = 0;
            goto established;
        }
        break;
    case Ilclose:
        if(ack == ic->start)
            ilhangup(s, "remote close");
        break;
    }
    freeblist(bp);
    break;
case Ilestablished:
established:
    switch(h->iltype) {
    case Ilsync:
        if(id != ic->rstart)
            ilhangup(s, "remote close");
        else
            ilsendctl(s, nil, Ilack, ic->next, ic->rstart, 0);
        freeblist(bp);
        break;
    case Ildata:
        /*
         * avoid consuming all the mount rpc buffers in the
         * system.  if the input queue is too long, drop this
         * packet.
         */
        if (s->rq && qlen(s->rq) >= Maxrq) {
            priv->stats[DroppedMsgs]++;
            freeblist(bp);
            break;
        }

        ilackto(ic, ack, bp);
        iloutoforder(s, h, bp);
        ilpullup(s);
        break;
    case Ildataquery:
        ilackto(ic, ack, bp);
        iloutoforder(s, h, bp);
        ilpullup(s);
        ilsendctl(s, nil, Ilstate, ic->next, ic->recvd, h->ilspec);
        break;
    case Ilack:
        ilackto(ic, ack, bp);
        freeblist(bp);
        break;
```

```
        case Ilquery:
            ilackto(ic, ack, bp);
            ilsendctl(s, nil, Ilstate, ic->next, ic->recvd, h->ilspec);
            freeblist(bp);
            break;
        case Ilstate:
            if(ack >= ic->rttack)
                ic->rttack = 0;
            ilackto(ic, ack, bp);
            if(h->ilspec > Nqt)
                h->ilspec = 0;
            if(ic->qt[h->ilspec] > ack){
                ilrexmit(ic);
                ilsettimeout(ic);
            }
            freeblist(bp);
            break;
        case Ilclose:
            freeblist(bp);
            if(ack < ic->start || ack > ic->next)
                break;
            ic->recvd = id;
            ilsendctl(s, nil, Ilclose, ic->next, ic->recvd, 0);
            ic->state = Ilclosing;
            ilsettimeout(ic);
            ilfreeq(ic);
            break;
        }
        break;
    case Illistening:
        freeblist(bp);
        break;
    case Ilclosing:
        switch(h->iltype) {
        case Ilclose:
            ic->recvd = id;
            ilsendctl(s, nil, Ilclose, ic->next, ic->recvd, 0);
            if(ack == ic->next)
                ilhangup(s, nil);
            break;
        default:
            break;
        }
        freeblist(bp);
        break;
    }
}
```

12.7.3 Start

⟨*function* ilstart 144⟩≡ (370b)

```
char*
ilstart(Conv *c, int type, bool fasttimeout)
{
    Ilcb *ic;
    Ilpriv *ipriv;
    char kpname[KNAMELEN];

    ipriv = c->p->priv;
```

```
    if(ipriv->ackprocstarted == 0){
        qlock(&ipriv->apl);
        if(ipriv->ackprocstarted == 0){
            sprint(kpname, "#I%dilack", c->p->f->dev);
            kproc(kpname, ilackproc, c->p);
            ipriv->ackprocstarted = 1;
        }
        qunlock(&ipriv->apl);
    }

    ic = (Ilcb*)c->ptcl;
    ic->conv = c;

    if(ic->state != Ilclosed)
        return nil;

    ilcbinit(ic);

    if(fasttimeout){
        /* timeout if we can't connect quickly */
        ic->fasttimeout = 1;
        ic->timeout = NOW+Iltickms;
        ic->rexmit = MaxRexmit - 4;
    };

    switch(type) {
    case IL_CONNECT:
        ic->state = Ilsyncer;
        iphtadd(&ipriv->ht, c);
        ilsendctl(c, nil, Ilsync, ic->start, ic->recvd, 0);
        break;
    case IL_LISTEN:
        ic->state = Illistening;
        iphtadd(&ipriv->ht, c);
        break;
    default:
        //netlog(c->p->f, Logil, "il: start: type %d\n", type);
        break;
    }

    return nil;
}
```

⟨*function* ilcbinit 145⟩≡ (370b)

```
void
ilcbinit(Ilcb *ic)
{
    ic->start = nrand(0x1000000);
    ic->next = ic->start+1;
    ic->recvd = 0;
    ic->window = Defaultwin;
    ic->unackedbytes = 0;
    ic->unacked = nil;
    ic->outoforder = nil;
    ic->rexmit = 0;
    ic->rxtot = 0;
    ic->rxquery = 0;
    ic->qtx = 1;
    ic->fasttimeout = 0;
```

```
    /* timers */
    ic->delay = DefRtt<<LogAGain;
    ic->mdev = DefRtt<<LogDGain;
    ic->rate = DefByteRate<<LogAGain;
    ic->querytime = NOW + QueryTime;
    ic->lastrecv = NOW; /* or we'll timeout right away */
    ilsettimeout(ic);
}
```

12.7.4 Close

⟨*function* `ilclose` 146a⟩≡ (370b)

```
static void
ilclose(Conv *c)
{
    Ilcb *ic;

    ic = (Ilcb*)c->ptcl;

    qclose(c->rq);
    qclose(c->wq);
    qclose(c->eq);

    switch(ic->state) {
    case Ilclosing:
    case Ilclosed:
        break;
    case Ilsyncer:
    case Ilsyncee:
    case Ilestablished:
        ic->state = Ilclosing;
        ilsettimeout(ic);
        ilsendctl(c, nil, Ilclose, ic->next, ic->recvd, 0);
        break;
    case Illistening:
        illocalclose(c);
        break;
    }
    ilfreeq(ic);
}
```

⟨*function* `illocalclose` 146b⟩≡ (370b)

```
void
illocalclose(Conv *c)
{
    Ilcb *ic;
    Ilpriv *ipriv;

    ipriv = c->p->priv;
    ic = (Ilcb*)c->ptcl;
    ic->state = Ilclosed;
    iphtrem(&ipriv->ht, c);
    ipmove(c->laddr, IPnoaddr);
    c->lport = 0;
}
```

12.7.5 Syncer

12.7.6 Syncee

12.7.7 Establised

12.8 Features

12.8.1 Reliable datagram service

⟨*global* `ilcksum` 147a⟩≡ (370b)

```
bool  ilcksum = true;
```

12.8.2 In sequence delivery

12.8.3 Retransmission of lost messages

12.8.4 Out-of-order messages saving

12.8.5 Adaptive timeouts

12.8.6 Keep-alive

12.9 Other files

12.9.1 /net/il/stats

⟨*function* `ilxstats` 147b⟩≡ (370b)

```
int
ilxstats(Proto *il, char *buf, int len)
{
    Ilpriv *priv;
    char *p, *e;
    int i;

    priv = il->priv;
    p = buf;
    e = p+len;
    for(i = 0; i < Nstats; i++)
        p = seprint(p, e, "%s: %llud\n", statnames[i], priv->stats[i]);
    return p - buf;
}
```

12.9.2 /net/il/x/status

⟨*function* `ilstate` 147c⟩≡ (370b)

```
static int
ilstate(Conv *c, char *state, int n)
{
    Ilcb *ic;

    ic = (Ilcb*)(c->ptcl);
    return snprint(state, n, "%s qin %d qout %d del %5.5d Br %5.5d md %5.5d una %5.5lud rex %5.5d rxq %5.5d max
        ilstates[ic->state],
        c->rq ? qlen(c->rq) : 0,
```

```
        c->wq ? qlen(c->wq) : 0,
        ic->delay>>LogAGain, ic->rate>>LogAGain, ic->mdev>>LogDGain,
        ic->unackedbytes, ic->rxtot, ic->rxquery, ic->maxrtt);
}
```

12.10 Advanced features

12.10.1 Fast timeout

⟨ilconnect() *set fast* 148⟩≡ (136e)

```
/* huge hack to quickly try an il connection */
if(argc > 1){
    p = strstr(argv[1], "!fasttimeout");
    if(p != nil){
        *p = '\0';
        fast = true;
    }
}
```

12.10.2 Special extension

Chapter 13

Standard Reliable Communication: TCP

⟨*function* tcpinit 149⟩≡ (431b)

```
void
tcpinit(Fs *fs)
{
    Proto *tcp;
    Tcppriv *tpriv;

    tcp = smalloc(sizeof(Proto));
    tpriv = tcp->priv = smalloc(sizeof(Tcppriv));

    tcp->name = "tcp";
    tcp->connect = tcpconnect;
    tcp->announce = tcpannounce;
    tcp->ctl = tcpctl;
    tcp->state = tcpstate;
    tcp->create = tcpcreate;
    tcp->close = tcpclose;
    tcp->rcv = tcpiput;
    tcp->advise = tcpadvise;
    tcp->stats = tcpstats;
    tcp->inuse = tcpinuse;
    tcp->gc = tcpgc;

    tcp->ipproto = IP_TCPPROTO;
    tcp->nc = scalednconv();
    tcp->ptclsize = sizeof(Tcpctl);
    tpriv->stats[MaxConn] = tcp->nc;

    Fsproto(fs, tcp);
}
```

Chapter 14

Applications

14.1 Remote shell: Telnet

14.2 Remote file access: TFTP

14.3 Mail

Chapter 15

Remote Procedure Call: 9P

Chapter 16

Network File System: `exportfs`

Chapter 17

Name Resolution: DNS

Chapter 18

Security

18.1 Denial of service

Chapter 19

Debugging Support

Chapter 20

Profiling Support

Chapter 21

Advanced Topics

21.1 Flow Control

21.2 IPv6

21.3 Broadcast

⟨ipoput4() *adjust gate and ifc if broadcast or multicast case* 157a⟩≡ (81a)

```
if(r->type & (Rbcast|Rmulti)) {
    gate = eh->dst;
    sr = v4lookup(f, eh->src, nil);
    if(sr != nil && (sr->type & Runi))
        ifc = sr->ifc;
}
```

⟨findipifc() *if broadcast or multicast route* 157b⟩≡ (97c)

```
/* for now for broadcast and multicast, just use first interface */
if(type & (Rbcast|Rmulti)){
    for(cp = f->ipifc->conv; cp < e; cp++){
        if(*cp == 0)
            continue;
        ifc = (Ipifc*)(*cp)->ptcl;
        if(ifc->lifc != nil)
            return ifc;
    }
}
```

⟨ipifcadd() *locals* 157c⟩≡ (60) 482h ▷

```
ipaddr bcast;
```

⟨ipifcadd() *add broadcast addresses to self cache* 157d⟩≡ (60)

```
/* add subnet directed broadcast address to the self cache */
for(i = 0; i < IPaddrlen; i++)
    bcast[i] = (ip[i] & mask[i]) | ~mask[i];
addselfcache(f, ifc, lifc, bcast, Rbcast);

/* add subnet directed network address to the self cache */
for(i = 0; i < IPaddrlen; i++)
    bcast[i] = (ip[i] & mask[i]) & mask[i];
addselfcache(f, ifc, lifc, bcast, Rbcast);

/* add network directed broadcast address to the self cache */
memmove(mask, defmask(ip), IPaddrlen);
```

```
for(i = 0; i < IPaddrlen; i++)
    bcast[i] = (ip[i] & mask[i]) | ~mask[i];
addselfcache(f, ifc, lifc, bcast, Rbcast);

/* add network directed network address to the self cache */
memmove(mask, defmask(ip), IPaddrlen);
for(i = 0; i < IPaddrlen; i++)
    bcast[i] = (ip[i] & mask[i]) & mask[i];
addselfcache(f, ifc, lifc, bcast, Rbcast);

addselfcache(f, ifc, lifc, IPv4bcast, Rbcast);
```

⟨*global* IPv4bcast 158a⟩≡ (201e)

```
/*
 *  well known IP addresses
 */
ipaddr IPv4bcast = {
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0xff, 0xff,

    0xff, 0xff, 0xff, 0xff
};
```

21.4 Multicast

⟨*struct* Ipmulti 158b⟩≡ (208)

```
/*
 *  one per multicast-lifc pair used by a Conv
 */
struct Ipmulti
{
  ipaddr ma;
  ipaddr ia;
  Ipmulti *next;
};
```

⟨*struct* Ipmcast 158c⟩≡ (354b)

```
struct Ipmcast
{
    Ipmcast *next;
    ipaddr   ma;  /* multicast address */
    ipaddr   ia;  /* interface address */
};
```

⟨ipwrite() *locals* 158d⟩≡ (41c)

```
ipaddr ia, ma;
```

⟨ipwrite() *Qctl case, else if other string* 158e⟩+≡ (44f) ◁128c 159a▷

```
else if(strcmp(cb->f[0], "addmulti") == 0){
    if(cb->nf < 2)
        error("addmulti needs interface address");
    if(cb->nf == 2){
        if(!ipismulticast(cv->raddr))
            error("addmulti for a non multicast address");
        if (parseip(ia, cb->f[1]) == -1)
            error(Ebadip);
        ipifcaddmulti(cv, cv->raddr, ia);
```

```
    } else {
        if (parseip(ia, cb->f[1]) == -1 ||
            parseip(ma, cb->f[2]) == -1)
            error(Ebadip);
        if(!ipismulticast(ma))
            error("addmulti for a non multicast address");
        ipifcaddmulti(cv, ma, ia);
    }
}
```

⟨ipwrite() *Qctl case, else if other string* 159a⟩+≡ (44f) ◁158e

```
else if(strcmp(cb->f[0], "remmulti") == 0){
    if(cb->nf < 2)
        error("remmulti needs interface address");
    if(!ipismulticast(cv->raddr))
        error("remmulti for a non multicast address");
    if (parseip(ia, cb->f[1]) == -1)
        error(Ebadip);
    ipifcremmulti(cv, cv->raddr, ia);
}
```

⟨ipifcctl() *else if other string* 159b⟩+≡ (59d) ◁90c 159c▷

```
else if(strcmp(argv[0], "joinmulti") == 0)
    return ipifcjoinmulti(ifc, argv, argc);
```

⟨ipifcctl() *else if other string* 159c⟩+≡ (59d) ◁159b 482d▷

```
else if(strcmp(argv[0], "leavemulti") == 0)
    return ipifcleavemulti(ifc, argv, argc);
```

⟨Medium(kernel) *multicast methods* 159d⟩≡ (18d)

```
/* for arming interfaces to receive multicast */
void  (*addmulti)(Ipifc *ifc, uchar *a, uchar *ia);
void  (*remmulti)(Ipifc *ifc, uchar *a, uchar *ia);

/* for routing multicast groups */
void  (*joinmulti)(Ipifc *ifc, uchar *a, uchar *ia);
void  (*leavemulti)(Ipifc *ifc, uchar *a, uchar *ia);
```

⟨Conv(kernel) *multicast fields* 159e⟩≡ (25a)

```
Ipmulti *multi;     /* multicast bindings for this interface */
```

⟨closeconv() *locals* 159f⟩≡ (40d)

```
Ipmulti *mp;
```

⟨closeconv() *if multi, call ipifcremmulti* 159g⟩≡ (40d)

```
while((mp = cv->multi) != nil)
    ipifcremmulti(cv, mp->ma, mp->ia);
```

⟨Netif(kernel) *multicast fields* 159h⟩≡ (28a)

```
Netaddr *maddr;     /* known multicast addresses */
int nmaddr;     /* number of known multicast addresses */
Netaddr *mhash[Nmhash];   /* hash table of multicast addresses */
```

⟨Netif(kernel) *multicast methods* 159i⟩≡ (28a)

```
void  (*multicast)(void*, uchar*, int);
```

21.5 Network database: ndb

⟨Fs(kernel) *ndb fields* 160a⟩≡ (21c)
```
char  ndb[1024];    /* an ndb entry for this interface */
int ndbvers;
long  ndbmtime;
```

21.5.1 /net/ndb

⟨Qid *toplevel extra cases* 160b⟩+≡ (26f) ◁112a 166a▷
```
Qndb,
```

⟨ip1gen() *switch TYPE qid cases* 160c⟩+≡ (37a) ◁112b 166b▷
```
case Qndb:
    p = "ndb";
    len = strlen(f->ndb);
    q.vers = f->ndbvers;
    break;
```

⟨ip1gen() *locals* 160d⟩≡ (37a)
```
extern ulong    kerndate;
```

⟨ipgen() *if Qndb, adjust mtime* 160e⟩≡ (37a)
```
if(i == Qndb && f->ndbmtime > kerndate)
    dp->mtime = f->ndbmtime;
```

⟨ipopen() *switch TYPE qid cases* 160f⟩+≡ (39c) ◁93a 166c▷
```
case Qndb:
    if(omode & (OWRITE|OTRUNC) && !iseve())
        error(Eperm);
    if((omode & (OWRITE|OTRUNC)) == (OWRITE|OTRUNC))
        f->ndb[0] = 0;
    break;
```

⟨ipread() *switch TYPE qid cases* 160g⟩+≡ (41a) ◁112c 166e▷
```
case Qndb:
    return readstr(offset, a, n, f->ndb);
```

⟨ipwrite() *switch TYPE qid cases* 160h⟩+≡ (41c) ◁103c 166f▷
```
case Qndb:
    return ndbwrite(f, a, offset, n);
    break;
```

21.6 Connection Server

21.7 Sniffing

21.8 Point to point interface

⟨ipifcadd() *check for point to point interface* 160i⟩≡ (60)
```
/* check for point-to-point interface */
if(ipcmp(ip, v6loopback)) /* skip v6 loopback, it's a special address */
if(ipcmp(mask, IPallbits) == 0)
    type |= Rptpt;
```

⟨ipifcadd() *register proxy if point to point interface or proxy* 161a⟩≡ (60)

```
if((type & (Rproxy|Rptpt)) == (Rproxy|Rptpt)){
    ipifcregisterproxy(f, ifc, rem);
    goto out;
}
```

21.9 Proxy

⟨ipifcadd() *switch argc, proxy case, and fall through* 161b⟩≡ (60)

```
// add <ip> <mask> <rem> <mtu> proxy
case 6:
    if(strcmp(argv[5], "proxy") == 0)
        type |= Rproxy;
    /* fall through */
```

21.10 Wifi

21.11 VLAN

21.12 VPN

21.13 Packet filter

Chapter 22

Conclusion

Appendix A

Debugging

A.1 Dumpers

A.1.1 Addresses

⟨*function* eipfmt 163⟩≡ (200e)

```
int
eipfmt(Fmt *f)
{
    char buf[5*8];
    static char *efmt = "%.2ux%.2ux%.2ux%.2ux%.2ux%.2ux"; // ethernet
    static char *ifmt = "%d.%d.%d.%d"; // internet, v4
    uchar *p;
    ipaddr ip;
    ulong *lp;
    ushort s;
    int i, j, n, eln, eli;

    switch(f->r) {
    case 'E':  /* Ethernet address */
        p = va_arg(f->args, uchar*);
        snprint(buf, sizeof buf, efmt, p[0], p[1], p[2], p[3], p[4], p[5]);
        return fmtstrcpy(f, buf);

    case 'I':  /* Ip address */
        p = va_arg(f->args, uchar*);
common:
        if(memcmp(p, v4prefix, 12) == 0){
            snprint(buf, sizeof buf, ifmt, p[12], p[13], p[14], p[15]);
            return fmtstrcpy(f, buf);
        }

        /* find longest elision */
        eln = eli = -1;
        for(i = 0; i < 16; i += 2){
            for(j = i; j < 16; j += 2)
                if(p[j] != 0 || p[j+1] != 0)
                    break;
            if(j > i && j - i > eln){
                eli = i;
                eln = j - i;
            }
        }

        /* print with possible elision */
```

```
        n = 0;
        for(i = 0; i < 16; i += 2){
            if(i == eli){
                n += sprint(buf+n, "::");
                i += eln;
                if(i >= 16)
                    break;
            } else if(i != 0)
                n += sprint(buf+n, ":");
            s = (p[i]<<8) + p[i+1];
            n += sprint(buf+n, "%ux", s);
        }
        return fmtstrcpy(f, buf);

    case 'i':  /* v6 address as 4 longs */
        lp = va_arg(f->args, ulong*);
        for(i = 0; i < 4; i++)
            hnputl(ip+4*i, *lp++);
        p = ip;
        goto common;

    case 'V':  /* v4 ip address */
        p = va_arg(f->args, uchar*);
        snprint(buf, sizeof buf, ifmt, p[0], p[1], p[2], p[3]);
        return fmtstrcpy(f, buf);

    case 'M':  /* ip mask */
        p = va_arg(f->args, uchar*);

        /* look for a prefix mask */
        for(i = 0; i < 16; i++)
            if(p[i] != 0xff)
                break;
        if(i < 16){
            if((prefixvals[p[i]] & Isprefix) == 0)
                goto common;
            for(j = i+1; j < 16; j++)
                if(p[j] != 0)
                    goto common;
            n = 8*i + (prefixvals[p[i]] & ~Isprefix);
        } else
            n = 8*16;

        /* got one, use /xx format */
        snprint(buf, sizeof buf, "/%d", n);
        return fmtstrcpy(f, buf);
    }
    return fmtstrcpy(f, "(eipfmt)");
}
```

A.1.2 IP interface

⟨*function* main 164⟩≡ (205c)

```
void
main(void)
{
    Ipifc *ifc, *list;
    Iplifc *lifc;
    int i;
```

```
    fmtinstall('I', eipfmt);
    fmtinstall('M', eipfmt);

    list = readipifc("/net", nil, -1);
    for(ifc = list; ifc; ifc = ifc->next){
        print("ipifc %s %d\n", ifc->dev, ifc->mtu);
        for(lifc = ifc->lifc; lifc; lifc = lifc->next)
            print("\t%I %M %I\n", lifc->ip, lifc->mask, lifc->net);
    }
}
```

A.2 /net/log

A.2.1 Netlog

⟨*struct* Netlog 165a⟩≡ (359a)

```
/*
 *  action log
 */
struct Netlog {
    int opens;

    // array<char> of size Nlog
    char*   buf;
    int len;

    char    *rptr;
    char    *end;


    int logmask;            /* mask of things to debug */
    ipaddr   iponly;      /* ip address to print debugging for */
    int iponlyset;

    // Extra
    Lock;
    QLock;
    Rendez;
};
```

⟨*enum* _anon_ (kernel/network/ip/netlog.c) 165b⟩≡ (359a)

```
enum {
    Nlog        = 16*1024,
};
```

⟨Fs(kernel) *logging fields* 165c⟩≡ (21c)

```
Netlog  *alog;
```

⟨*function* netloginit 165d⟩≡ (359a)

```
void
netloginit(Fs *f)
{
    f->alog = smalloc(sizeof(Netlog));
}
```

A.2.2 /net/log

⟨Qid *toplevel extra cases* 166a⟩+≡ (26f) ◁160b

```
Qlog,
```

⟨ip1gen() *switch TYPE qid cases* 166b⟩+≡ (37a) ◁160c

```
case Qlog:
    p = "log";
    break;
```

⟨ipopen() *switch TYPE qid cases* 166c⟩+≡ (39c) ◁160f 169d▷

```
case Qlog:
    netlogopen(f);
    break;
```

⟨ipclose() *switch TYPE qid cases* 166d⟩+≡ (40b) ◁40c 169e▷

```
case Qlog:
    if(c->flag & COPEN)
        netlogclose(f);
    break;
```

⟨ipread() *switch TYPE qid cases* 166e⟩+≡ (41a) ◁160g 169f▷

```
case Qlog:
    return netlogread(f, a, offset, n);
```

⟨ipwrite() *switch TYPE qid cases* 166f⟩+≡ (41c) ◁160h

```
case Qlog:
    netlogctl(f, a, n);
    return n;
```

A.2.3 Opening

⟨*function* netlogopen 166g⟩≡ (359a)

```
void
netlogopen(Fs *f)
{
    lock(f->alog);
    if(waserror()){
        unlock(f->alog);
        nexterror();
    }
    if(f->alog->opens == 0){
        if(f->alog->buf == nil)
            f->alog->buf = malloc(Nlog);
        if(f->alog->buf == nil)
            error(Enomem);
        f->alog->rptr = f->alog->buf;
        f->alog->end = f->alog->buf + Nlog;
    }
    f->alog->opens++;
    unlock(f->alog);
    poperror();
}
```

A.2.4 Reading

⟨*function* netlogread 167a⟩≡ (359a)

```
long
netlogread(Fs *f, void *a, ulong, long n)
{
    int i, d;
    char *p, *rptr;

    qlock(f->alog);
    if(waserror()){
        qunlock(f->alog);
        nexterror();
    }

    for(;;){
        lock(f->alog);
        if(f->alog->len){
            if(n > f->alog->len)
                n = f->alog->len;
            d = 0;
            rptr = f->alog->rptr;
            f->alog->rptr += n;
            if(f->alog->rptr >= f->alog->end){
                d = f->alog->rptr - f->alog->end;
                f->alog->rptr = f->alog->buf + d;
            }
            f->alog->len -= n;
            unlock(f->alog);

            i = n-d;
            p = a;
            memmove(p, rptr, i);
            memmove(p+i, f->alog->buf, d);
            break;
        }
        else
            unlock(f->alog);

        sleep(f->alog, netlogready, f);
    }

    qunlock(f->alog);
    poperror();

    return n;
}
```

⟨*function* netlogready 167b⟩≡ (359a)

```
static bool
netlogready(void *a)
{
    Fs *f = a;

    return f->alog->len;
}
```

A.2.5 Control

⟨*enum* _anon_ (kernel/network/ip/netlog.c)2 167c⟩≡ (359a)

```
enum
{
    CMset,
    CMclear,
    CMonly,
};
```

⟨*global* routecmd 168a⟩≡ (359a)

```
static
Cmdtab routecmd[] = {
    CMset,      "set",      0,
    CMclear,    "clear",    0,
    CMonly,     "only",     0,
};
```

⟨*function* netlogctl 168b⟩≡ (359a)

```
void
netlogctl(Fs *f, char* s, int n)
{
    int i, set;
    Netlogflag *fp;
    Cmdbuf *cb;
    Cmdtab *ct;

    cb = parsecmd(s, n);
    if(waserror()){
        free(cb);
        nexterror();
    }

    if(cb->nf < 2)
        error(Ebadnetctl);

    ct = lookupcmd(cb, routecmd, nelem(routecmd));

    SET(set);

    switch(ct->index){
    case CMset:
        set = 1;
        break;

    case CMclear:
        set = 0;
        break;

    case CMonly:
        parseip(f->alog->iponly, cb->f[1]);
        if(ipcmp(f->alog->iponly, IPnoaddr) == 0)
            f->alog->iponlyset = 0;
        else
            f->alog->iponlyset = 1;
        free(cb);
        poperror();
        return;

    default:
        cmderror(cb, "unknown netlog control message");
    }
```

```
    for(i = 1; i < cb->nf; i++){
        for(fp = flags; fp->name; fp++)
            if(strcmp(fp->name, cb->f[i]) == 0)
                break;
        if(fp->name == nil)
            continue;
        if(set)
            f->alog->logmask |= fp->mask;
        else
            f->alog->logmask &= ~fp->mask;
    }

    free(cb);
    poperror();
}
```

A.3 /bin/snoopy

A.3.1 /net/ipifc/x/snoop

⟨Qid *conversation extra cases, last entry* 169a⟩≡ (26f)

```
Qsnoop,
```

⟨Conv(kernel) *snoop fields* 169b⟩≡ (25a)

```
Ref snoopers;   /* number of processes with snoop open */
Queue*  sq;     /* snooping queue */
```

⟨ip3gen() *switch TYPE qid cases* 169c⟩+≡ (38c) ◁54b

```
case Qsnoop:
    if(strcmp(cv->p->name, "ipifc") != 0)
        return -1;
    devdir(c, q, "snoop", qlen(cv->sq), cv->owner, 0400, dp);
    return 1;
```

⟨ipopen() *switch TYPE qid cases* 169d⟩+≡ (39c) ◁166c

```
case Qsnoop:
    if(omode != OREAD)
        error(Eperm);
    p = f->p[PROTO(c->qid)];
    cv = p->conv[CONV(c->qid)];
    if(strcmp(ATTACHER(c), cv->owner) != 0 && !iseve())
        error(Eperm);
    incref(&cv->snoopers);
    break;
```

⟨ipclose() *switch TYPE qid cases* 169e⟩+≡ (40b) ◁166d

```
case Qsnoop:
    if(c->flag & COPEN)
        decref(&f->p[PROTO(c->qid)]->conv[CONV(c->qid)]->snoopers);
    break;
```

⟨ipread() *switch TYPE qid cases* 169f⟩+≡ (41a) ◁166e

```
case Qsnoop:
    cv = f->p[PROTO(ch->qid)]->conv[CONV(ch->qid)];
    return qread(cv->sq, a, n);
```

A.3.2 /bin/snoopy

Appendix B

Profiling

B.1 `/net/x/stats`

Appendix C

Error Management

Appendix D

Ethernet NE2000 Driver

D.1 Data structures

D.1.1 Ethernet Dp8390 controller

⟨*struct* Dp8390 172a⟩≡ (239c)

```
/*
 * Ctlr for the boards using the National Semiconductor DP8390
 * and SMC 83C90 Network Interface Controller.
 * Common code is in ether8390.c.
 */
struct Dp8390 {
    ulong port;   /* I/O address of 8390 */
    ulong data;   /* I/O data port if no shared memory */

    uchar width;   /* data transfer width in bytes */
    bool ram;   /* true if card has shared memory */
    uchar dummyrr;  /* do dummy remote read */

    uchar nxtpkt;   /* receive: software bndry */
    uchar pstart;
    uchar pstop;

    int txbusy;   /* transmit */
    uchar tstart;   /* 8390 ring addresses */

    ⟨Dp8390 multicast fields 185a⟩

    // Extra
    Lock;
};
```

D.2 Initialisation

⟨*function* ether2000link 172b⟩≡ (243b)

```
void
ether2000link(void)
{
    addethercard("NE2000", ne2000reset);
}
```

⟨*function* ne2000reset 172c⟩≡ (243b)

```
static errorneg1
```

```
ne2000reset(Ether* edev)
{
    ushort buf[16];
    ulong port;
    Dp8390 *dp8390;
    int i;
    eaddr ea;

    if(edev->port == 0)
        ne2000pnp(edev);
    if(edev->port == 0)
        return ERROR_NEG1;

    /*
     * Set up the software configuration.
     * Use defaults for irq, mem and size
     * if not specified.
     * Must have a port, no more default.
     */
    if(edev->irq == 0)
        edev->irq = 2;
    if(edev->mem == 0)
        edev->mem = 0x4000;
    if(edev->size == 0)
        edev->size = 16*1024;
    port = edev->port;

    if(ioalloc(edev->port, 0x20, 0, "ne2000") < 0)
        return ERROR_NEG1;

    edev->ctlr = malloc(sizeof(Dp8390));
    dp8390 = edev->ctlr;
    if(dp8390 == nil)
        error(Enomem);

    dp8390->width = 2;
    dp8390->ram = false;

    dp8390->port = port;
    dp8390->data = port+Data;

    dp8390->tstart = HOWMANY(edev->mem, Dp8390BufSz);
    dp8390->pstart = dp8390->tstart + HOWMANY(sizeof(Etherpkt), Dp8390BufSz);
    dp8390->pstop = dp8390->tstart + HOWMANY(edev->size, Dp8390BufSz);

    dp8390->dummyrr = 1;
    for(i = 0; i < edev->nopt; i++){
        if(strcmp(edev->opt[i], "nodummyrr"))
            continue;
        dp8390->dummyrr = 0;
        break;
    }

    /*
     * Reset the board. This is done by doing a read
     * followed by a write to the Reset address.
     */
    buf[0] = inb(port+Reset);
    arch_delay(2);
    outb(port+Reset, buf[0]);
```

```
    arch_delay(2);

    /*
     * Init the (possible) chip, then use the (possible)
     * chip to read the (possible) PROM for ethernet address
     * and a marker byte.
     * Could just look at the DP8390 command register after
     * initialisation has been tried, but that wouldn't be
     * enough, there are other ethernet boards which could
     * match.
     * Parallels has buf[0x0E] == 0x00 whereas real hardware
     * usually has 0x57.
     */

    // will fill in the Ether callbacks
    dp8390reset(edev);

    memset(buf, 0, sizeof(buf));
    dp8390read(dp8390, buf, 0, sizeof(buf));
    i = buf[0x0E] & 0xFF;
    if((i != 0x00 && i != 0x57) || (buf[0x0F] & 0xFF) != 0x57){
        iofree(edev->port);
        free(edev->ctlr);
        return ERROR_NEG1;
    }

    /*
     * Stupid machine. Shorts were asked for,
     * shorts were delivered, although the PROM is a byte array.
     * Set the ethernet address.
     */
    memset(ea, 0, Eaddrlen);
    if(memcmp(ea, edev->ea, Eaddrlen) == 0){
        for(i = 0; i < sizeof(edev->ea); i++)
            edev->ea[i] = buf[i];
    }
    dp8390setea(edev);

    return OK_0;
}
```

⟨*function* dp8390read 174a⟩≡ (242b)

```
void*
dp8390read(Dp8390* ctlr, void* to, ulong from, ulong len)
{
    void *v;

    ilock(ctlr);
    v = _dp8390read(ctlr, to, from, len);
    iunlock(ctlr);

    return v;
}
```

⟨*function* dp8390reset 174b⟩≡ (242b)

```
int
dp8390reset(Ether* ether)
{
    Dp8390 *ctlr;
```

```
    ctlr = ether->ctlr;

    /*
     * This is the initialisation procedure described
     * as 'mandatory' in the datasheet, with references
     * to the 3C503 technical reference manual.
     */
    disable(ctlr);
    if(ctlr->width != 1)
        regw(ctlr, Dcr, Ft4WORD|Ls|Wts);
    else
        regw(ctlr, Dcr, Ft4WORD|Ls);

    regw(ctlr, Rbcr0, 0);
    regw(ctlr, Rbcr1, 0);

    regw(ctlr, Tcr, LpbkNIC);
    regw(ctlr, Rcr, Mon);

    /*
     * Init the ring hardware and software ring pointers.
     * Can't initialise ethernet address as it may not be
     * known yet.
     */
    ringinit(ctlr);
    regw(ctlr, Tpsr, ctlr->tstart);

    /*
     * Clear any pending interrupts and mask then all off.
     */
    regw(ctlr, Isr, 0xFF);
    regw(ctlr, Imr, 0);

    /*
     * Leave the chip initialised,
     * but in monitor mode.
     */
    regw(ctlr, Cr, Page0|RdABORT|Sta);



    /*
     * Set up the software configuration.
     */
    ether->attach = attach;
    ether->shutdown = shutdown;

    ether->transmit = transmit;
    ether->interrupt = interrupt;
    ether->ifstat = 0;

    ether->promiscuous = promiscuous;
    ether->multicast = multicast;
    ether->arg = ether;

    return 0;
}
```

⟨*function* ringinit 175⟩≡ (242b)

```
static void
```

```
ringinit(Dp8390* ctlr)
{
    regw(ctlr, Pstart, ctlr->pstart);
    regw(ctlr, Pstop, ctlr->pstop);
    regw(ctlr, Bnry, ctlr->pstop-1);

    regw(ctlr, Cr, Page1|RdABORT|Stp);
    regw(ctlr, Curr, ctlr->pstart);
    regw(ctlr, Cr, Page0|RdABORT|Stp);

    ctlr->nxtpkt = ctlr->pstart;
}
```

⟨*function* dp8390setea 176a⟩≡ (242b)

```
void
dp8390setea(Ether* ether)
{
    int i;
    uchar cr;
    Dp8390 *ctlr;

    ctlr = ether->ctlr;

    /*
     * Set the ethernet address into the chip.
     * Take care to restore the command register
     * afterwards. Don't care about multicast
     * addresses as multicast is never enabled
     * (currently).
     */
    ilock(ctlr);
    cr = regr(ctlr, Cr) & ~Txp;
    regw(ctlr, Cr, Page1|(~(Ps1|Ps0) & cr));
    for(i = 0; i < Eaddrlen; i++)
        regw(ctlr, Par0+i, ether->ea[i]);
    regw(ctlr, Cr, cr);
    iunlock(ctlr);
}
```

⟨*function* disable 176b⟩≡ (242b)

```
static void
disable(Dp8390* ctlr)
{
    int timo;

    /*
     * Stop the chip. Set the Stp bit and wait for the chip
     * to finish whatever was on its tiny mind before it sets
     * the Rst bit.
     * The timeout is needed because there may not be a real
     * chip there if this is called when probing for a device
     * at boot.
     */
    regw(ctlr, Cr, Page0|RdABORT|Stp);
    regw(ctlr, Rbcr0, 0);
    regw(ctlr, Rbcr1, 0);
    for(timo = 10000; (regr(ctlr, Isr) & Rst) == 0 && timo; timo--)
            ;
}
```

⟨*function* shutdown 177a⟩≡ (242b)

```
static void
shutdown(Ether *ether)
{
    Dp8390 *ctlr;

    ctlr = ether->ctlr;
    disable(ctlr);
}
```

D.3 Mounting

⟨*function* attach 177b⟩≡ (242b)

```
static void
attach(Ether* ether)
{
    Dp8390 *ctlr;
    uchar r;

    ctlr = ether->ctlr;

    /*
     * Enable the chip for transmit/receive.
     * The init routine leaves the chip in monitor
     * mode. Clear the missed-packet counter, it
     * increments while in monitor mode.
     * Sometimes there's an interrupt pending at this
     * point but there's nothing in the Isr, so
     * any pending interrupts are cleared and the
     * mask of acceptable interrupts is enabled here.
     */
    r = Ab;
    if(ether->prom)
        r |= Pro;
    if(ether->nmaddr)
        r |= Am;
    ilock(ctlr);
    regw(ctlr, Isr, 0xFF);
    regw(ctlr, Imr, Cnt|Ovw|Txe|Rxe|Ptx|Prx);
    regw(ctlr, Rcr, r);
    r = regr(ctlr, Ref2);
    regw(ctlr, Tcr, LpbkNORMAL);
    iunlock(ctlr);
    USED(r);
}
```

D.4 IO

⟨*macro* regw 177c⟩≡ (239c)

```
#define regw(c, r, v) outb((c)->port+(r), (v))
```

⟨*macro* regr 177d⟩≡ (239c)

```
/*
 * x86-specific code.
 */
#define regr(c, r) inb((c)->port+(r))
```

D.4.1 Writing

⟨*function* transmit*(*(kernel/network/386/ether8390.c)*)* 178a⟩≡ (242b)

```
static void
transmit(Ether* ether)
{
    Dp8390 *ctlr;

    ctlr = ether->ctlr;

    ilock(ctlr);
    txstart(ether);
    iunlock(ctlr);
}
```

⟨*function* txstart 178b⟩≡ (242b)

```
static void
txstart(Ether* ether)
{
    int len;
    Dp8390 *ctlr;
    Block *bp;
    uchar minpkt[ETHERMINTU], *rp;

    ctlr = ether->ctlr;

    /*
     * This routine is called both from the top level and from interrupt
     * level and expects to be called with ctlr already locked.
     */
    if(ctlr->txbusy)
        return;
    bp = qget(ether->oq);
    if(bp == nil)
        return;

    /*
     * Make sure the packet is of minimum length;
     * copy it to the card's memory by the appropriate means;
     * start the transmission.
     */
    len = BLEN(bp);
    rp = bp->rp;
    if(len < ETHERMINTU){
        rp = minpkt;
        memmove(rp, bp->rp, len);
        memset(rp+len, 0, ETHERMINTU-len);
        len = ETHERMINTU;
    }

    if(ctlr->ram)
        memmove((void*)(ether->mem+ctlr->tstart*Dp8390BufSz), rp, len);
    else
        dp8390write(ctlr, ctlr->tstart*Dp8390BufSz, rp, len);
    freeb(bp);

    regw(ctlr, Tbcr0, len & 0xFF);
    regw(ctlr, Tbcr1, (len>>8) & 0xFF);
    regw(ctlr, Cr, Page0|RdABORT|Txp|Sta);
```

```
    ether->outpackets++;
    ctlr->txbusy = 1;
}
```

⟨*function* dp8390write 179⟩≡ (242b)

```
static void*
dp8390write(Dp8390* ctlr, ulong to, void* from, ulong len)
{
    ulong crda;
    uchar cr;
    int timo, width;

top:
    /*
     * Write some data to offset 'to' in the card's memory
     * using the DP8390 remote DMA facility, reading it at
     * 'from' in main memory, via the I/O data port.
     */
    cr = regr(ctlr, Cr) & ~Txp;
    regw(ctlr, Cr, Page0|RdABORT|Sta);
    regw(ctlr, Isr, Rdc);

    len = ROUNDUP(len, ctlr->width);

    /*
     * Set up the remote DMA address and count.
     * This is straight from the DP8390[12D] datasheet,
     * hence the initial set up for read.
     * Assumption here that the A7000 EtherV card will
     * never need a dummyrr.
     */
    if(ctlr->dummyrr && (ctlr->width == 1 || ctlr->width == 2)){
        if(ctlr->width == 2)
            width = 1;
        else
            width = 0;
        crda = to-1-width;
        regw(ctlr, Rbcr0, (len+1+width) & 0xFF);
        regw(ctlr, Rbcr1, ((len+1+width)>>8) & 0xFF);
        regw(ctlr, Rsar0, crda & 0xFF);
        regw(ctlr, Rsar1, (crda>>8) & 0xFF);
        regw(ctlr, Cr, Page0|RdREAD|Sta);

        for(timo=0;; timo++){
            if(timo > 10000){
                print("ether8390: dummyrr timeout; assuming nodummyrr\n");
                ctlr->dummyrr = 0;
                goto top;
            }
            crda = regr(ctlr, Crda0);
            crda |= regr(ctlr, Crda1)<<8;
            if(crda == to){
                /*
                 * Start the remote DMA write and make sure
                 * the registers are correct.
                 */
                regw(ctlr, Cr, Page0|RdWRITE|Sta);

                crda = regr(ctlr, Crda0);
                crda |= regr(ctlr, Crda1)<<8;
```

```
                if(crda != to)
                    panic("crda write %lud to %lud\n", crda, to);

                break;
            }
        }
    }
    else{
        regw(ctlr, Rsar0, to & 0xFF);
        regw(ctlr, Rsar1, (to>>8) & 0xFF);
        regw(ctlr, Rbcr0, len & 0xFF);
        regw(ctlr, Rbcr1, (len>>8) & 0xFF);
        regw(ctlr, Cr, Page0|RdWRITE|Sta);
    }

    /*
     * Pump the data into the I/O port
     * then wait for the remote DMA to finish.
     */
    rdwrite(ctlr, from, len);
    for(timo = 10000; (regr(ctlr, Isr) & Rdc) == 0 && timo; timo--)
            ;

    regw(ctlr, Isr, Rdc);
    regw(ctlr, Cr, cr);

    return (void*)to;
}
```

⟨*function* rdwrite 180a⟩≡ (239c)

```
static void
rdwrite(Dp8390* ctlr, void* from, int len)
{
    switch(ctlr->width){
    default:
        panic("dp8390 rdwrite: width %d\n", ctlr->width);
        break;

    case 2:
        outss(ctlr->data, from, len/2);
        break;

    case 1:
        outsb(ctlr->data, from, len);
        break;
    }
}
```

D.4.2 Reading

⟨*function* interrupt*(*(kernel/network/386/ether8390.c)*)* 180b⟩≡ (242b)

```
static void
interrupt(Ureg*, void* arg)
{
    Ether *ether;
    Dp8390 *ctlr;
    uchar isr, r;

    ether = arg;
```

```
    ctlr = ether->ctlr;

    /*
     * While there is something of interest,
     * clear all the interrupts and process.
     */
    ilock(ctlr);
    regw(ctlr, Imr, 0x00);
    while(isr = (regr(ctlr, Isr) & (Cnt|Ovw|Txe|Rxe|Ptx|Prx))){
        if(isr & Ovw){
            overflow(ether);
            regw(ctlr, Isr, Ovw);
            ether->overflows++;
        }

        /*
         * Packets have been received.
         * Take a spin round the ring.
         */
        if(isr & (Rxe|Prx)){
            receive(ether);
            regw(ctlr, Isr, Rxe|Prx);
        }

        /*
         * A packet completed transmission, successfully or
         * not. Start transmission on the next buffered packet,
         * and wake the output routine.
         */
        if(isr & (Txe|Ptx)){
            r = regr(ctlr, Tsr);
            if((isr & Txe) && (r & (Cdh|Fu|Crs|Abt))){
                print("dp8390: Tsr %#2.2ux", r);
                ether->oerrs++;
            }

            regw(ctlr, Isr, Txe|Ptx);

            if(isr & Ptx)
                ether->outpackets++;
            ctlr->txbusy = 0;
            txstart(ether);
        }

        if(isr & Cnt){
            ether->frames += regr(ctlr, Ref0);
            ether->crcs += regr(ctlr, Ref1);
            ether->buffs += regr(ctlr, Ref2);
            regw(ctlr, Isr, Cnt);
        }
    }
    regw(ctlr, Imr, Cnt|Ovw|Txe|Rxe|Ptx|Prx);
    iunlock(ctlr);
}
```

⟨*function* receive*(*(kernel/network/386/ether8390.c)*)* 181⟩≡ (242b)

```
static void
receive(Ether* ether)
{
    Dp8390 *ctlr;
```

```
uchar curr, *p;
Hdr hdr;
ulong count, data, len;
Block *bp;

ctlr = ether->ctlr;
for(curr = getcurr(ctlr); ctlr->nxtpkt != curr; curr = getcurr(ctlr)){
    data = ctlr->nxtpkt*Dp8390BufSz;
    if(ctlr->ram)
        memmove(&hdr, (void*)(ether->mem+data), sizeof(Hdr));
    else
        _dp8390read(ctlr, &hdr, data, sizeof(Hdr));

    /*
     * Don't believe the upper byte count, work it
     * out from the software next-page pointer and
     * the current next-page pointer.
     */
    if(hdr.next > ctlr->nxtpkt)
        len = hdr.next - ctlr->nxtpkt - 1;
    else
        len = (ctlr->pstop-ctlr->nxtpkt) + (hdr.next-ctlr->pstart) - 1;
    if(hdr.len0 > (Dp8390BufSz-sizeof(Hdr)))
        len--;

    len = ((len<<8)|hdr.len0)-4;

    /*
     * Chip is badly scrogged, reinitialise the ring.
     */
    if(hdr.next < ctlr->pstart || hdr.next >= ctlr->pstop
      || len < 60 || len > sizeof(Etherpkt)){
        print("dp8390: H%2.2ux+%2.2ux+%2.2ux+%2.2ux,%lud\n",
            hdr.status, hdr.next, hdr.len0, hdr.len1, len);
        regw(ctlr, Cr, Page0|RdABORT|Stp);
        ringinit(ctlr);
        regw(ctlr, Cr, Page0|RdABORT|Sta);

        return;
    }

    /*
     * If it's a good packet read it in to the software buffer.
     * If the packet wraps round the hardware ring, read it in
     * two pieces.
     */
    if((hdr.status & (Fo|Fae|Crce|Prxok)) == Prxok && (bp = iallocb(len))){
        p = bp->rp;
        bp->wp = p+len;
        data += sizeof(Hdr);

        if((data+len) >= ctlr->pstop*Dp8390BufSz){
            count = ctlr->pstop*Dp8390BufSz - data;
            if(ctlr->ram)
                memmove(p, (void*)(ether->mem+data), count);
            else
                _dp8390read(ctlr, p, data, count);
            p += count;
            data = ctlr->pstart*Dp8390BufSz;
            len -= count;
```

```
            }
            if(len){
                if(ctlr->ram)
                    memmove(p, (void*)(ether->mem+data), len);
                else
                    _dp8390read(ctlr, p, data, len);
            }

            /*
             * Copy the packet to whoever wants it.
             */
            etheriq(ether, bp, 1);
        }

        /*
         * Finished with this packet, update the
         * hardware and software ring pointers.
         */
        ctlr->nxtpkt = hdr.next;

        hdr.next--;
        if(hdr.next < ctlr->pstart)
            hdr.next = ctlr->pstop-1;
        regw(ctlr, Bnry, hdr.next);
    }
}
```

⟨*function* getcurr 183a⟩≡ (242b)

```
static uchar
getcurr(Dp8390* ctlr)
{
    uchar cr, curr;

    cr = regr(ctlr, Cr) & ~Txp;
    regw(ctlr, Cr, Page1|(~(Ps1|Ps0) & cr));
    curr = regr(ctlr, Curr);
    regw(ctlr, Cr, cr);

    return curr;
}
```

⟨*function* _dp8390read 183b⟩≡ (242b)

```
static void*
_dp8390read(Dp8390* ctlr, void* to, ulong from, ulong len)
{
    uchar cr;
    int timo;

    /*
     * Read some data at offset 'from' in the card's memory
     * using the DP8390 remote DMA facility, and place it at
     * 'to' in main memory, via the I/O data port.
     */
    cr = regr(ctlr, Cr) & ~Txp;
    regw(ctlr, Cr, Page0|RdABORT|Sta);
    regw(ctlr, Isr, Rdc);

    /*
     * Set up the remote DMA address and count.
     */
```

```
    len = ROUNDUP(len, ctlr->width);
    regw(ctlr, Rbcr0, len & 0xFF);
    regw(ctlr, Rbcr1, (len>>8) & 0xFF);
    regw(ctlr, Rsar0, from & 0xFF);
    regw(ctlr, Rsar1, (from>>8) & 0xFF);

    /*
     * Start the remote DMA read and suck the data
     * out of the I/O port.
     */
    regw(ctlr, Cr, Page0|RdREAD|Sta);
    rdread(ctlr, to, len);

    /*
     * Wait for the remote DMA to complete. The timeout
     * is necessary because this routine may be called on
     * a non-existent chip during initialisation and, due
     * to the miracles of the bus, it's possible to get this
     * far and still be talking to a slot full of nothing.
     */
    for(timo = 10000; (regr(ctlr, Isr) & Rdc) == 0 && timo; timo--)
            ;

    regw(ctlr, Isr, Rdc);
    regw(ctlr, Cr, cr);

    return to;
}
```

⟨*function* overflow 184⟩≡ (242b)

```
static void
overflow(Ether *ether)
{
    Dp8390 *ctlr;
    uchar txp;
    int resend;

    ctlr = ether->ctlr;

    /*
     * The following procedure is taken from the DP8390[12D] datasheet,
     * it seems pretty adamant that this is what has to be done.
     */
    txp = regr(ctlr, Cr) & Txp;
    regw(ctlr, Cr, Page0|RdABORT|Stp);
    arch_delay(2);
    regw(ctlr, Rbcr0, 0);
    regw(ctlr, Rbcr1, 0);

    resend = 0;
    if(txp && (regr(ctlr, Isr) & (Txe|Ptx)) == 0)
        resend = 1;

    regw(ctlr, Tcr, LpbkNIC);
    regw(ctlr, Cr, Page0|RdABORT|Sta);
    receive(ether);
    regw(ctlr, Isr, Ovw);
    regw(ctlr, Tcr, LpbkNORMAL);

    if(resend)
```

```
        regw(ctlr, Cr, Page0|RdABORT|Txp|Sta);
}
```

D.5 Advanced features

D.5.1 Multicast

⟨Dp8390 *multicast fields* 185a⟩≡ (172a)

```
uchar mar[8];   /* shadow multicast address registers */
int mref[64];  /* reference counts for multicast groups */
```

⟨*global* reverse 185b⟩≡ (242b)

```
static uchar reverse[64];
```

⟨*function* multicast 185c⟩≡ (242b)

```
static void
multicast(void* arg, uchar *addr, int on)
{
    Ether *ether;
    Dp8390 *ctlr;
    int i;
    ulong h;

    ether = arg;
    ctlr = ether->ctlr;
    if(reverse[1] == 0){
        for(i = 0; i < 64; i++)
            reverse[i] = ((i&1)<<5) | ((i&2)<<3) | ((i&4)<<1)
                    | ((i&8)>>1) | ((i&16)>>3) | ((i&32)>>5);
    }

    /*
     *  change filter bits
     */
    h = ethercrc(addr, 6);
    ilock(ctlr);
    setbit(ctlr, reverse[h&0x3f], on);
    setfilter(ether, ctlr);
    iunlock(ctlr);
}
```

⟨*global* allmar 185d⟩≡ (242b)

```
static uchar allmar[8] = { 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff };
```

⟨*function* setbit 185e⟩≡ (242b)

```
static void
setbit(Dp8390 *ctlr, int bit, int on)
{
    int i, h;

    i = bit/8;
    h = bit%8;
    if(on){
        if(++(ctlr->mref[bit]) == 1)
            ctlr->mar[i] |= 1<<h;
    } else {
        if(--(ctlr->mref[bit]) <= 0){
            ctlr->mref[bit] = 0;
```

```
            ctlr->mar[i] &= ~(1<<h);
        }
    }
}
```

⟨*function* setfilter 186a⟩≡ (242b)

```
static void
setfilter(Ether *ether, Dp8390 *ctlr)
{
    uchar r, cr;
    int i;
    uchar *mar;

    r = Ab;
    mar = 0;
    if(ether->prom){
        r |= Pro|Am;
        mar = allmar;
    } else if(ether->nmaddr){
        r |= Am;
        mar = ctlr->mar;
    }
    if(mar){
        cr = regr(ctlr, Cr) & ~Txp;
        regw(ctlr, Cr, Page1|(~(Ps1|Ps0) & cr));
        for(i = 0; i < 8; i++)
            regw(ctlr, Mar0+i, *(mar++));
        regw(ctlr, Cr, cr);
    }
    regw(ctlr, Rcr, r);
}
```

D.5.2 Promiscuous mode

⟨*function* promiscuous 186b⟩≡ (242b)

```
static void
promiscuous(void *arg, int )
{
    Ether *ether;
    Dp8390 *ctlr;

    ether = arg;
    ctlr = ether->ctlr;

    ilock(ctlr);
    setfilter(ether, ctlr);
    iunlock(ctlr);
}
```

D.5.3 Power managment

D.5.4 Plug and play

⟨*struct* Ctlr*(*(kernel/network/386/ether2000.c)*)* 186c⟩≡ (243b)

```
struct Ctlr {
    Pcidev* pcidev;
    Ctlr* next;
    int active;
};
```

⟨*global* ctlrhead((kernel/network/386/ether2000.c)) 187a⟩≡ (243b)

```
static Ctlr* ctlrhead;
```

⟨*global* ctlrtail((kernel/network/386/ether2000.c)) 187b⟩≡ (243b)

```
static Ctlr* ctlrtail;
```

⟨*function* ne2000pnp 187c⟩≡ (243b)

```
static void
ne2000pnp(Ether* edev)
{
    int i, id;
    Pcidev *p;
    Ctlr *ctlr;

    /*
     * Make a list of all ethernet controllers
     * if not already done.
     */
    if(ctlrhead == nil){
        p = nil;
        while(p = pcimatch(p, 0, 0)){
            if(p->ccrb != 0x02 || p->ccru != 0)
                continue;
            ctlr = malloc(sizeof(Ctlr));
            if(ctlr == nil)
                error(Enomem);
            ctlr->pcidev = p;

            if(ctlrhead != nil)
                ctlrtail->next = ctlr;
            else
                ctlrhead = ctlr;
            ctlrtail = ctlr;
        }
    }

    /*
     * Is it a card with an unrecognised vid+did?
     * Normally a search is made through all the found controllers
     * for one which matches any of the known vid+did pairs.
     * If a vid+did pair is specified a search is made for that
     * specific controller only.
     */
    id = 0;
    for(i = 0; i < edev->nopt; i++){
        if(cistrncmp(edev->opt[i], "id=", 3) == 0)
            id = strtol(&edev->opt[i][3], nil, 0);
    }

    if(id != 0)
        ne2000match(edev, id);
    else for(i = 0; ne2000pci[i].name; i++){
        if(ne2000match(edev, ne2000pci[i].id) != nil)
            break;
    }
}
```

⟨*global* ne2000pci 187d⟩≡ (243b)

```
static struct {
    char* name;
```

```
    int id;
} ne2000pci[] = {
    { "Realtek 8029", (0x8029<<16)|0x10EC, },
    { "Winbond 89C940", (0x0940<<16)|0x1050, },
    { nil },
};
```

⟨*function* ne2000match 188⟩≡ (243b)

```
static Ctlr*
ne2000match(Ether* edev, int id)
{
    int port;
    Pcidev *p;
    Ctlr *ctlr;

    /*
     * Any adapter matches if no edev->port is supplied,
     * otherwise the ports must match.
     */
    for(ctlr = ctlrhead; ctlr != nil; ctlr = ctlr->next){
        if(ctlr->active)
            continue;
        p = ctlr->pcidev;
        if(((p->did<<16)|p->vid) != id)
            continue;
        port = p->mem[0].bar & ~0x01;
        if(edev->port != 0 && edev->port != port)
            continue;

        /*
         * It suffices to fill these in,
         * the rest is gleaned from the card.
         */
        edev->port = port;
        edev->irq = p->intl;

        ctlr->active = 1;

        return ctlr;
    }

    return nil;
}
```

Appendix E

Utilities

E.1 Globbing

Appendix F

Extra Mediums

F.1 Null medium

⟨*global* nullmedium 190a⟩≡ (326a)

```
Medium nullmedium =
{
    .name=      "null",

    .bind=      nullbind,
    .unbind=    nullunbind,

    .bwrite=    nullbwrite,
};
```

⟨*function* nullbind 190b⟩≡ (326a)

```
static void
nullbind(Ipifc*, int, char**)
{
    error("cannot bind null device");
}
```

⟨*function* nullunbind 190c⟩≡ (326a)

```
static void
nullunbind(Ipifc*)
{
}
```

⟨*function* nullbwrite 190d⟩≡ (326a)

```
static void
nullbwrite(Ipifc*, Block*, int, uchar*)
{
    error("nullbwrite");
}
```

⟨*function* nullmediumlink 190e⟩≡ (326a)

```
void
nullmediumlink(void)
{
    addipmedium(&nullmedium);
}
```

F.2 Loopback medium

⟨*global* loopbackmedium 191a⟩≡ (326b)

```
Medium loopbackmedium =
{
    .name=      "loopback",

    .hsize=     0,
    .mintu=     0,
    .maxtu=     Maxtu,
    .maclen=    0,

    .bind=      loopbackbind,
    .unbind=    loopbackunbind,
    .bwrite=    loopbackbwrite,
};
```

⟨*enum* _anon_ (kernel/network/ip/loopbackmedium.c) 191b⟩≡ (326b)

```
enum
{
    Maxtu=  16*1024,
};
```

⟨*function* loopbackmediumlink 191c⟩≡ (326b)

```
void
loopbackmediumlink(void)
{
    addipmedium(&loopbackmedium);
}
```

⟨*struct* LB 191d⟩≡ (326b)

```
struct LB
{
    Queue   *q;
    Fs  *f;
    Proc    *readp;
};
```

⟨*function* loopbackbind 191e⟩≡ (326b)

```
static void
loopbackbind(Ipifc *ifc, int, char**)
{
    LB *lb;

    lb = smalloc(sizeof(LB));
    lb->f = ifc->conv->p->f;
    lb->q = qopen(1024*1024, Qmsg, nil, nil);
    ifc->arg = lb;
    ifc->mbps = 1000;

    kproc("loopbackread", loopbackread, ifc);

}
```

⟨*function* loopbackunbind 191f⟩≡ (326b)

```
static void
loopbackunbind(Ipifc *ifc)
{
    LB *lb = ifc->arg;
```

```
    if(lb->readp)
        postnote(lb->readp, 1, "unbind", 0);

    /* wait for reader to die */
    while(lb->readp != nil)
        tsleep(&up->sleepr, returnfalse, 0, 300);

    /* clean up */
    qfree(lb->q);
    free(lb);
}
```

⟨*function* loopbackread 192a⟩≡ (326b)

```
static void
loopbackread(void *a)
{
    Ipifc *ifc;
    Block *bp;
    LB *lb;

    ifc = a;
    lb = ifc->arg;
    lb->readp = up; /* hide identity under a rock for unbind */
    if(waserror()){
        lb->readp = 0;
        pexit("hangup", 1);
    }
    for(;;){
        bp = qbread(lb->q, Maxtu);
        if(bp == nil)
            continue;
        ifc->in++;
        if(!canrlock(ifc)){
            freeb(bp);
            continue;
        }
        if(waserror()){
            runlock(ifc);
            nexterror();
        }
        if(ifc->lifc == nil)
            freeb(bp);
        else
            ipiput4(lb->f, ifc, bp);
        runlock(ifc);
        poperror();
    }
}
```

⟨*function* loopbackbwrite 192b⟩≡ (326b)

```
static void
loopbackbwrite(Ipifc *ifc, Block *bp, int, uchar*)
{
    LB *lb;

    lb = ifc->arg;
    if(qpass(lb->q, bp) < 0)
        ifc->outerr++;
    ifc->out++;
}
```

F.3 Point to point serial line

F.4 Token ring

Appendix G

Extra Protocols

G.1 ICMP

⟨Proto(kernel) *conversation methods* 194⟩+≡ (24e) ◁44b

```
void    (*advise)(Proto*, Block*, char*);
```

G.2 RUDP

G.3 GRE

G.4 ESP

G.5 Datakit and URP

Appendix H

Extra Applications

H.1 Remote login, rlogin

H.2 Serving files, FTP

H.3 Serving documents, HTTP

H.4 Serving mails, SMTP

H.5 Serving news, NNTP

Appendix I

Extra Code

I.1 include/net/

I.1.1 include/net/ip.h

⟨*enum* _anon_ 196a⟩≡ (196c)

```
enum
{
    ⟨constant IPaddrlen 13c⟩
    ⟨constant IPv4addrlen 13a⟩
    ⟨constant IPv4off 13f⟩

    IPllen=  4,
    IPV4HDR_LEN= 20,

    /* vihl & vcf[0] values */
    IP_VER4=  0x40,
    IP_VER6= 0x60,
};
```

⟨*constant* Udphdrsize 196b⟩≡ (196c)

```
/*
 *  user level udp headers with control message "headers"
 */
#define Udphdrsize 52 /* size of a Udphdr */
```

⟨include/net/ip.h 196c⟩≡

```
#pragma src "/sys/src/libip"
#pragma lib "libip.a"

⟨enum _anon_ 196a⟩

⟨typedef ipv4 13b⟩
⟨typedef ipaddr 13d⟩
⟨typedef iplong 15d⟩


// forward decl
typedef struct Ipifc Ipifc;
typedef struct Iplifc Iplifc;
typedef struct Ipv6rp Ipv6rp;
typedef struct Ip6hdr Ip6hdr;
typedef struct Icmp6hdr Icmp6hdr;
typedef struct Udphdr Udphdr;
```

```
/*
 *  for reading /net/ipifc
 */
```

⟨*struct Iplifc (user)* 17a⟩

⟨*struct* Ipv6rp 468e⟩

⟨*struct Ipifc (user)* 16b⟩

⟨*macro* ISIPV6MCAST 468f⟩
⟨*macro* ISIPV6LINKLOCAL 468g⟩

⟨*enum* _anon_ (include/net/ip.h) 469a⟩

⟨*struct* Ip6hdr 469b⟩

⟨*struct* Icmp6hdr 469c⟩

⟨*constant* Udphdrsize 196b⟩

⟨*struct Udphdr (user)* 120a⟩

```
uchar* defmask(ipaddr);
void maskip(ipaddr, ipaddr, ipaddr);
int  eipfmt(Fmt*);
bool isv4(ipaddr);
vlong parseip(uchar*, char*);
vlong parseipmask(uchar*, char*);
char* v4parseip(uchar*, char*);
//char* v4parsecidr(uchar*, uchar*, char*);
int parseether(uchar*, char*);
int myipaddr(uchar*, char*);
int myetheraddr(uchar*, char*);
int equivip4(uchar*, uchar*);
int equivip6(uchar*, uchar*);

Ipifc* readipifc(char*, Ipifc*, int);

void hnputs(void*, ushort);
void hnputl(void*, uint);
void hnputv(void*, uvlong);
ushort nhgets(void*);
uint nhgetl(void*);
uvlong nhgetv(void*);

ushort ptclbsum(uchar*, int);

int  v6tov4(uchar*, uchar*);
void v4tov6(uchar*, uchar*);
```

⟨*macro* ipcmp 77a⟩
⟨*macro* ipmove 77b⟩

```
extern ipaddr IPv4bcast;
extern ipaddr IPv4bcastobs;
extern ipaddr IPv4allsys;
extern ipaddr IPv4allrouter;
extern ipaddr IPnoaddr;
```

```
extern ipaddr v4prefix;
extern ipaddr IPallbits;

⟨macro CLASS 14f⟩

#pragma varargck type "I" uchar*
#pragma varargck type "V" uchar*
#pragma varargck type "E" uchar*
#pragma varargck type "M" uchar*
```

I.2 lib_networking/libip/

I.2.1 lib_networking/libip/parseether.c

⟨lib_networking/libip/parseether.c 198a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function parseether 73a⟩
```

I.2.2 lib_networking/libip/parseip.c

⟨function ipcharok 198b⟩≡ (199a)

```
static int
ipcharok(int c)
{
    return c == '.' || c == ':' || isascii(c) && isxdigit(c);
}
```

⟨function delimchar 198c⟩≡ (199a)

```
static bool
delimchar(int c)
{
    if(c == '\0')
        return true;
    if(c == '.' || c == ':' || isascii(c) && isalnum(c))
        return false;
    return true;
}
```

⟨function parseipmask 198d⟩≡ (199a)

```
/*
 *  hack to allow ip v4 masks to be entered in the old
 *  style
 */
vlong
parseipmask(uchar *to, char *from)
{
    int i, w;
    vlong x;
    uchar *p;

    if(*from == '/'){
        /* as a number of prefix bits */
        i = atoi(from+1);
        if(i < 0)
            i = 0;
```

```
        if(i > 128)
            i = 128;
        w = i;
        memset(to, 0, IPaddrlen);
        for(p = to; i >= 8; i -= 8)
            *p++ = 0xff;
        if(i > 0)
            *p = ~((1<<(8-i))-1);
        x = nhgetl(to+IPv4off);
        /*
         * identify as ipv6 if the mask is inexpressible as a v4 mask
         * (because it has too few mask bits).  Arguably, we could
         * always return 6 here.
         */
        if (w < 8*(IPaddrlen-IPv4addrlen))
            return 6;
    } else {
        /* as a straight v4 bit mask */
        x = parseip(to, from);
        if (x != -1)
            x = (ulong)nhgetl(to + IPv4off);
        if(memcmp(to, v4prefix, IPv4off) == 0)
            memset(to, 0xff, IPv4off);
    }
    return x;
}
```

⟨lib_networking/libip/parseip.c 199a⟩≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>
#include <ip.h>

⟨function v4parseip 75a⟩

⟨function ipcharok 198b⟩

⟨function delimchar 198c⟩

⟨function parseip 75b⟩

⟨function parseipmask 198d⟩
```

I.2.3 lib_networking/libip/myetheraddr.c

⟨lib_networking/libip/myetheraddr.c 199b⟩≡

```
#include <u.h>
#include <libc.h>
#include <ip.h>

⟨function myetheraddr 73b⟩
```

I.2.4 lib_networking/libip/myipaddr.c

⟨*global* loopbacknet 199c⟩≡ (200b)

```
static ipaddr loopbacknet = {
    0, 0, 0, 0,
```

```
    0, 0, 0, 0,
    0, 0, 0xff, 0xff,
    127, 0, 0, 0
};
```

⟨*global* loopbackmask 200a⟩≡ (200b)

```
static ipaddr loopbackmask = {
    0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff,
    0xff, 0, 0, 0
};
```

⟨lib_networking/libip/myipaddr.c 200b⟩≡

```
#include <u.h>
#include <libc.h>
#include <ip.h>

⟨global loopbacknet 199c⟩
⟨global loopbackmask 200a⟩

⟨function myipaddr 64c⟩
```

I.2.5 lib_networking/libip/eipfmt.c

⟨*enum* _anon_ (lib_networking/libip/eipfmt.c) 200c⟩≡ (200e)

```
enum
{
    Isprefix= 16,
};
```

⟨*global* prefixvals 200d⟩≡ (200e)

```
uchar prefixvals[256] =
{
[0x00] 0 | Isprefix,
[0x80] 1 | Isprefix,
[0xC0] 2 | Isprefix,
[0xE0] 3 | Isprefix,
[0xF0] 4 | Isprefix,
[0xF8] 5 | Isprefix,
[0xFC] 6 | Isprefix,
[0xFE] 7 | Isprefix,
[0xFF] 8 | Isprefix,
};
```

⟨lib_networking/libip/eipfmt.c 200e⟩≡

```
#include <u.h>
#include <libc.h>
#include <ip.h>

⟨enum _anon_ (lib_networking/libip/eipfmt.c) 200c⟩

⟨global prefixvals 200d⟩

⟨function eipfmt 163⟩
```

I.2.6 lib_networking/libip/equivip.c

⟨lib_networking/libip/equivip.c 201a⟩≡
```
#include <u.h>
#include <libc.h>
#include <ip.h>

⟨function equivip4 77c⟩

⟨function equivip6 470a⟩
```

I.2.7 lib_networking/libip/ipaux.c

⟨global IPv4allsys 201b⟩≡ (201e)
```
ipaddr IPv4allsys = {
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0xff, 0xff,

    0xe0, 0, 0, 0x01
};
```

⟨global IPv4allrouter 201c⟩≡ (201e)
```
ipaddr IPv4allrouter = {
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0xff, 0xff,

    0xe0, 0, 0, 0x02
};
```

⟨global IPallbits 201d⟩≡ (201e)
```
ipaddr IPallbits = {
    0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff
};
```

⟨lib_networking/libip/ipaux.c 201e⟩≡
```
#include <u.h>
#include <libc.h>
#include <ip.h>

⟨global IPv4bcast 158a⟩
⟨global IPv4allsys 201b⟩
⟨global IPv4allrouter 201c⟩
⟨global IPallbits 201d⟩
⟨global IPnoaddr 15e⟩

⟨global v4prefix 13e⟩

⟨function isv4 13g⟩

⟨function v4tov6 14a⟩

⟨function v6tov4 14b⟩
```

I.2.8 lib_networking/libip/bo.c

⟨*function* hnputv 202a⟩≡ (203c)

```
void
hnputv(void *p, uvlong v)
{
    uchar *a;

    a = p;
    a[0] = v>>56;
    a[1] = v>>48;
    a[2] = v>>40;
    a[3] = v>>32;
    a[4] = v>>24;
    a[5] = v>>16;
    a[6] = v>>8;
    a[7] = v;
}
```

⟨*function* hnputl 202b⟩≡ (203c)

```
void
hnputl(void *p, uint v)
{
    uchar *a;

    a = p;
    a[0] = v>>24;
    a[1] = v>>16;
    a[2] = v>>8;
    a[3] = v;
}
```

⟨*function* hnputs 202c⟩≡ (203c)

```
void
hnputs(void *p, ushort v)
{
    uchar *a;

    a = p;
    a[0] = v>>8;
    a[1] = v;
}
```

⟨*function* nhgetv 202d⟩≡ (203c)

```
uvlong
nhgetv(void *p)
{
    uchar *a;
    uvlong v;

    a = p;
    v = (uvlong)a[0]<<56;
    v |= (uvlong)a[1]<<48;
    v |= (uvlong)a[2]<<40;
    v |= (uvlong)a[3]<<32;
    v |= a[4]<<24;
    v |= a[5]<<16;
    v |= a[6]<<8;
    v |= a[7]<<0;
    return v;
}
```

⟨*function* nhgetl 203a⟩≡ (203c)

```
uint
nhgetl(void *p)
{
    uchar *a;

    a = p;
    return (a[0]<<24)|(a[1]<<16)|(a[2]<<8)|(a[3]<<0);
}
```

⟨*function* nhgets 203b⟩≡ (203c)

```
ushort
nhgets(void *p)
{
    uchar *a;

    a = p;
    return (a[0]<<8)|(a[1]<<0);
}
```

⟨lib_networking/libip/bo.c 203c⟩≡

```
#include <u.h>
#include <libc.h>
#include <ip.h>

⟨function hnputv 202a⟩

⟨function hnputl 202b⟩

⟨function hnputs 202c⟩

⟨function nhgetv 202d⟩

⟨function nhgetl 203a⟩

⟨function nhgets 203b⟩
```

I.2.9 lib_networking/libip/classmask.c

⟨lib_networking/libip/classmask.c 203d⟩≡

```
#include <u.h>
#include <libc.h>
#include <ip.h>

⟨global classmask 15b⟩

⟨global v6loopback 470b⟩

⟨global v6linklocal 470c⟩
⟨global v6linklocalmask 470d⟩
⟨global v6llpreflen 470e⟩

⟨global v6multicast 470f⟩
⟨global v6multicastmask 470g⟩
⟨global v6mcpreflen 471a⟩

⟨global v6solicitednode 471b⟩
⟨global v6solicitednodemask 471c⟩
⟨global v6snpreflen 471d⟩
```

⟨*function* defmask 15a⟩

⟨*function* maskip 15c⟩

I.2.10 lib_networking/libip/ptclbsum.c

⟨*global* endian 204a⟩≡ (205a)
```
static short endian = 1;
```

⟨*global* aendian 204b⟩≡ (205a)
```
static uchar* aendian = (uchar*)&endian;
```

⟨*constant* LITTLE 204c⟩≡ (205a)
```
#define LITTLE *aendian
```

⟨*function* ptclbsum 204d⟩≡ (205a)
```
ushort
ptclbsum(uchar *addr, int len)
{
    ulong losum, hisum, mdsum, x;
    ulong t1, t2;

    losum = 0;
    hisum = 0;
    mdsum = 0;

    x = 0;
    if((uintptr)addr & 1) {
        if(len) {
            hisum += addr[0];
            len--;
            addr++;
        }
        x = 1;
    }
    while(len >= 16) {
        t1 = *(ushort*)(addr+0);
        t2 = *(ushort*)(addr+2); mdsum += t1;
        t1 = *(ushort*)(addr+4); mdsum += t2;
        t2 = *(ushort*)(addr+6); mdsum += t1;
        t1 = *(ushort*)(addr+8); mdsum += t2;
        t2 = *(ushort*)(addr+10); mdsum += t1;
        t1 = *(ushort*)(addr+12); mdsum += t2;
        t2 = *(ushort*)(addr+14); mdsum += t1;
        mdsum += t2;
        len -= 16;
        addr += 16;
    }
    while(len >= 2) {
        mdsum += *(ushort*)addr;
        len -= 2;
        addr += 2;
    }
    if(x) {
        if(len)
            losum += addr[0];
        if(LITTLE)
            losum += mdsum;
```

```
        else
            hisum += mdsum;
    } else {
        if(len)
            hisum += addr[0];
        if(LITTLE)
            hisum += mdsum;
        else
            losum += mdsum;
    }

    losum += hisum >> 8;
    losum += (hisum & 0xff) << 8;
    while(hisum = losum>>16)
        losum = hisum + (losum & 0xffff);

    return losum & 0xffff;
}
```

⟨lib_networking/libip/ptclbsum.c 205a⟩≡

```
#include <u.h>
#include <libc.h>
#include <ip.h>

⟨global endian 204a⟩
⟨global aendian 204b⟩
⟨constant LITTLE 204c⟩

⟨function ptclbsum 204d⟩
```

I.2.11 lib_networking/libip/readipifc.c

⟨lib_networking/libip/readipifc.c 205b⟩≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>
#include <ip.h>

⟨function findfield 64b⟩

⟨function _readipifc 62b⟩

⟨function _freeifc 64a⟩

⟨function readipifc 62a⟩
```

I.3 lib_networking/libip/tests/

I.3.1 lib_networking/libip/tests/testreadipifc.c

⟨lib_networking/libip/tests/testreadipifc.c 205c⟩≡

```
#include <u.h>
#include <libc.h>
#include <ip.h>

⟨function main 164⟩
```

I.4 kernel/network/ip/

I.4.1 kernel/network/ip/ip.h

⟨*constant* Nhash 206a⟩≡ (206b)

```
Nhash=    64,
```

⟨*enum* _anon_ (kernel/network/ip/ip.h) 206b⟩≡ (208)

```
enum
{
  Addrlen=  64,
  ⟨constant Maxproto 21d⟩
  ⟨constant Nhash 206a⟩
  Maxincall=  32, /* max. conn.s in listen q not accepted yet */
  ⟨constant Nchans 116c⟩
  MAClen=   16,   /* longest mac address */

  ⟨constant MAXTTL 69e⟩
  DFLTTOS=  0,

  ⟨constant IPaddrlen 13c⟩
  ⟨constant IPv4addrlen 13a⟩
  ⟨constant IPv4off 13f⟩
  IPllen=   4,

  ⟨constant IP_VER4 80a⟩
  ⟨constant IP_HLEN4 80b⟩
  IP_VER6=  0x60,

  IP_DF=    0x4000,   /* v4: Don't fragment */
  IP_MF=    0x2000,   /* v4: More fragments */
  IP4HDR=   20,   /* sizeof(Ip4hdr) */

  ⟨constant IP_MAX 83a⟩

  ⟨constant Lroot 30c⟩

  Maxpath = 64,
};
```

⟨*struct* Ipfrag 206c⟩≡ (208)

```
//@Scheck: used only for its macro below, could maybe simplify?
struct Ipfrag
{
  ushort  foff;
  ushort  flen;

  uchar payload[];
};
```

⟨*constant* IPFRAGSZ 206d⟩≡ (208)

```
#define IPFRAGSZ offsetof(Ipfrag, payload[0])
```

⟨*struct* Routerparams 206e⟩≡ (208)

```
/* default values, one per stack */
struct Routerparams {
  int mflag;    /* flag: managed address configuration */
  int oflag;    /* flag: other stateful configuration */
  int   maxraint; /* max. router adv interval (ms) */
  int minraint; /* min. router adv interval (ms) */
```

```
    int linkmtu;  /* mtu options */
    int reachtime;  /* reachable time */
    int rxmitra;  /* retransmit interval */
    int ttl;    /* cur hop count limit */
    int routerlt; /* router lifetime */
  };
```

⟨*struct* Hostparams 207a⟩≡ (208)

```
  struct Hostparams {
    int rxmithost;
  };
```

⟨*enum* _anon_ (kernel/network/ip/ip.h)4 207b⟩≡ (208)

```
  enum
  {
    ⟨constant Nipht 117c⟩
  };
```

⟨*enum* _anon_ (kernel/network/ip/ip.h)5 207c⟩≡ (208)

```
  /*
   *  logging
   */
  enum
  {
    Logip=    1<<1,
    Logtcp=   1<<2,
    Logfs=    1<<3,
    Logicmp=  1<<5,
    Logudp=   1<<6,
    Logcompress=  1<<7,
    Loggre=   1<<9,
    Logppp=   1<<10,
    Logtcprxmt= 1<<11,
    Logigmp=  1<<12,
    Logudpmsg=  1<<13,
    Logipmsg= 1<<14,
    Logrudp=  1<<15,
    Logrudpmsg= 1<<16,
    Logesp=   1<<17,
    Logtcpwin=  1<<18,
  };
```

⟨*struct* Routewalk 207d⟩≡ (208)

```
  struct Routewalk
  {
    int o;
    int h;
    char* p;
    char* e;
    void* state;
    void  (*walk)(Route*, Routewalk*);
  };
```

⟨*constant* NOW 207e⟩≡ (208)

```
  #define NOW TK2MS(CPUS(0)->ticks)
```

⟨kernel/network/ip/ip.h 208⟩≡

```
// coupling: include/ip.h
// This file references also code in lib_networking (linked with the kernel).
// Those functions are also exported in include/ip.h.
// Some types are duplicated with include/ip.h.

// forward decls
typedef struct  Conv  Conv;
typedef struct  Fragment4 Fragment4;
typedef struct  Fragment6 Fragment6;
typedef struct  Fs  Fs;
typedef struct  IP  IP;
typedef struct  IPaux IPaux;
typedef struct  Ip4hdr  Ip4hdr;
typedef struct  Ipfrag  Ipfrag;
typedef struct  Ipself  Ipself;
typedef struct  Ipselftab Ipselftab;
typedef struct  Iplink  Iplink;
typedef struct  Iplifc  Iplifc;
typedef struct  Ipmulti Ipmulti;
typedef struct  Ipifc Ipifc;
typedef struct  Iphash  Iphash;
typedef struct  Ipht  Ipht;
typedef struct  Netlog  Netlog;
typedef struct  Medium  Medium;
typedef struct  Proto Proto;
typedef struct  Arpent  Arpent;
typedef struct  Arp Arp;
typedef struct  Route Route;
typedef struct  Routerparams  Routerparams;
typedef struct  Hostparams  Hostparams;
typedef struct  v6router  v6router;
typedef struct  v6params  v6params;

#pragma incomplete Arp
#pragma incomplete Ipself
#pragma incomplete Ipselftab
#pragma incomplete IP
#pragma incomplete Netlog

⟨enum _anon_ (kernel/network/ip/ip.h) 206b⟩

⟨enum ip_version 14e⟩

⟨typedef ipv4 13b⟩
⟨typedef ipaddr 13d⟩
⟨typedef iplong 15d⟩
⟨typedef ipv4or6 14c⟩
⟨typedef ipv4p 14d⟩

⟨enum conversation_state 25c⟩

⟨enum _anon_ (kernel/network/ip/ip.h)3 22e⟩

⟨struct Fragment4 22b⟩

⟨struct Fragment6 471h⟩

⟨struct Ipfrag 206c⟩
```

⟨*constant* IPFRAGSZ 206d⟩

⟨*struct IP (kernel)* 22a⟩

⟨*struct* Ip4hdr 16a⟩

⟨*struct Conv (kernel)* 25a⟩

⟨*struct Medium (kernel)* 18c⟩

⟨*struct Iplifc (kernel)* 18a⟩

⟨*struct* Iplink 109b⟩

```
/* rfc 2461, pp.40|43. */
```

⟨*struct* Routerparams 206e⟩

⟨*struct* Hostparams 207a⟩

⟨*struct Ipifc (kernel)* 17c⟩

⟨*struct* Ipmulti 158b⟩

⟨*enum* _anon_ (kernel/network/ip/ip.h)4 207b⟩

⟨*enum* matchtype 117e⟩

⟨*struct* Iphash 117d⟩
⟨*struct* Ipht 117b⟩

```
void iphtadd(Ipht*, Conv*);
void iphtrem(Ipht*, Conv*);
Conv* iphtlook(Ipht *ht, uchar *sa, ushort sp, uchar *da, ushort dp);
```

⟨*struct Proto (kernel)* 23b⟩

⟨*struct Fs (kernel)* 21c⟩

⟨*struct* v6router 471i⟩

⟨*struct* v6params 472a⟩

```
int Fsconnected(Conv*, char*);
Conv* Fsnewcall(Conv*, uchar*, ushort, uchar*, ushort, uchar);
//int Fspcolstats(char*, int);
int Fsproto(Fs*, Proto*);
//int Fsbuiltinproto(Fs*, uchar);
//Conv* Fsprotoclone(Proto*, char*);
Proto*  Fsrcvpcol(Fs*, uchar);
Proto*  Fsrcvpcolx(Fs*, uchar);
char* Fsstdconnect(Conv*, char**, int);
char* Fsstdannounce(Conv*, char**, int);
//char* Fsstdbind(Conv*, char**, int);
ulong scaledconv(void);
//void  closeconv(Conv*);
```
⟨*enum* _anon_ (kernel/network/ip/ip.h)5 207c⟩

```
void  netloginit(Fs*);
void  netlogopen(Fs*);
void  netlogclose(Fs*);
void  netlogctl(Fs*, char*, int);
long  netlogread(Fs*, void*, ulong, long);
void  netlog(Fs*, int, char*, ...);
//void  ifcloginit(Fs*);
//long  ifclogread(Fs*, Chan *,void*, ulong, long);
//void  ifclog(Fs*, uchar *, int);
//void  ifclogopen(Fs*, Chan*);
//void  ifclogclose(Fs*, Chan*);

#pragma varargck argpos netlog  3

/*
 *  iproute.c
 */
typedef struct RouteTree RouteTree;
typedef struct Routewalk Routewalk;
typedef struct V4route V4route;
typedef struct V6route V6route;
```

⟨*enum* _anon_ (kernel/network/ip/ip.h)6 29g⟩

⟨*struct* Routewalk 207d⟩

⟨*struct RouteTree (kernel)* 29d⟩

⟨*struct* V4route 29c⟩

⟨*struct* V6route 472b⟩

⟨*struct Route (kernel)* 29b⟩

```
extern void v4addroute(Fs *f, char *tag, uchar *a, uchar *mask, uchar *gate, int type);
extern void v4delroute(Fs *f, uchar *a, uchar *mask, int dolock);
extern Route* v4lookup(Fs *f, uchar *a, Conv *c);

extern void v6addroute(Fs *f, char *tag, uchar *a, uchar *mask, uchar *gate, int type);
extern void v6delroute(Fs *f, uchar *a, uchar *mask, int dolock);
extern Route* v6lookup(Fs *f, uchar *a, Conv *c);

extern long routeread(Fs *f, char*, ulong, int);
extern long routewrite(Fs *f, Chan*, char*, int);
extern void routetype(int, char*);
//extern void ipwalkroutes(Fs*, Routewalk*);
//extern void convroute(Route*, uchar*, uchar*, uchar*, char*, int*);

/*
 *  devip.c
 */
```

⟨*struct* IPaux 32a⟩

```
extern IPaux* newipaux(char*, char*);

/*
 *  arp.c
 */
```

⟨*struct* Arpent 91d⟩

```
extern void arpinit(Fs*);
extern int  arpread(Arp*, char*, ulong, int);
extern int  arpwrite(Fs*, char*, int);
extern Arpent*  arpget(Arp*, Block *bp, int version, Ipifc *ifc, uchar *ip, uchar *h);
extern void arprelease(Arp*, Arpent *a);
extern Block* arpresolve(Arp*, Arpent *a, Medium *type, uchar *mac);
extern void arpenter(Fs*, int version, uchar *ip, uchar *mac, int len, int norefresh);

/*
 * ipaux.c
 */

//extern int  myetheraddr(uchar*, char*);
extern vlong  parseip(uchar*, char*);
extern vlong  parseipmask(uchar*, char*);
//extern char*  v4parseip(uchar*, char*);
extern void maskip(uchar *from, uchar *mask, uchar *to);
extern int  parsemac(uchar *to, char *from, int len);
extern uchar* defmask(uchar*);
extern int  isv4(uchar*);
extern void v4tov6(uchar *v6, uchar *v4);
extern int  v6tov4(uchar *v4, uchar *v6);
extern int  eipfmt(Fmt*);
```

⟨*macro* ipmove((kernel/network/ip/ip.h)) 15f⟩
⟨*macro* ipcmp((kernel/network/ip/ip.h)) 15g⟩

```
extern ipaddr IPv4bcast;
extern ipaddr IPnoaddr;
extern ipaddr v4prefix;
extern ipaddr IPallbits;
//extern ipaddr IPv4bcastobs;
//extern ipaddr IPv4allsys;
//extern ipaddr IPv4allrouter;
```

⟨*constant* NOW 207e⟩

```
/*
 *  media
 */
//extern Medium ethermedium;
//extern Medium nullmedium;
//extern Medium pktmedium;

/*
 *  ipifc.c
 */
extern Medium*  ipfindmedium(char *name);
extern void addipmedium(Medium *med);
extern int  ipforme(Fs*, uchar *addr);
extern int  iptentative(Fs*, uchar *addr);
//extern int  ipisbm(uchar *);
extern int  ipismulticast(uchar *);
extern Ipifc* findipifc(Fs*, uchar *remote, int type);
extern void findlocalip(Fs*, uchar *local, uchar *remote);
extern int  ipv4local(Ipifc *ifc, uchar *addr);
extern int  ipv6local(Ipifc *ifc, uchar *addr);
extern int  ipv6anylocal(Ipifc *ifc, uchar *addr);
extern Iplifc*  iplocalonifc(Ipifc *ifc, uchar *ip);
extern int  ipproxyifc(Fs *f, Ipifc *ifc, uchar *ip);
```

```
extern int  ipismulticast(uchar *ip);
//extern int  ipisbooting(void);
//extern int  ipifccheckin(Ipifc *ifc, Medium *med);
//extern void ipifccheckout(Ipifc *ifc);
//extern int  ipifcgrab(Ipifc *ifc);
extern void ipifcaddroute(Fs*, int, uchar*, uchar*, uchar*, int);
extern void ipifcremroute(Fs*, int, uchar*, uchar*);
extern void ipifcremmulti(Conv *c, uchar *ma, uchar *ia);
extern void ipifcaddmulti(Conv *c, uchar *ma, uchar *ia);
//extern char*  ipifcrem(Ipifc *ifc, char **argv, int argc);
//extern char*  ipifcadd(Ipifc *ifc, char **argv, int argc, int tentative, Iplifc *lifcp);
extern long ipselftabread(Fs*, char *a, ulong offset, int n);
//extern char*  ipifcadd6(Ipifc *ifc, char**argv, int argc);

/*
 *  ip.c
 */
extern void iprouting(Fs*, int);
extern void icmpnoconv(Fs*, Block*);
extern void icmpcantfrag(Fs*, Block*, int);
extern void icmpttlexceeded(Fs*, uchar*, Block*);
extern ushort ipcsum(uchar*);
extern void ipiput4(Fs*, Ipifc*, Block*);
extern void ipiput6(Fs*, Ipifc*, Block*);
extern int  ipoput4(Fs*, Block*, int, int, int, Conv*);
extern int  ipoput6(Fs*, Block*, int, int, int, Conv*);
extern int  ipstats(Fs*, char*, int);
extern ushort ptclbsum(uchar*, int);
extern ushort ptclcsum(Block*, int, int);
extern void ip_init(Fs*);
//extern void update_mtucache(uchar*, ulong);
//extern ulong  restrict_mtu(uchar*, ulong);

/*
 * bootp.c
 */
extern int  bootpread(char*, ulong, int);

/*
 * chandial.c
 */
extern Chan*  chandial(char*, char*, char*, Chan**);

/*
 *  global to all of the stack
 */
//extern void (*igmpreportfn)(Ipifc*, uchar*);
```

I.4.2 kernel/network/ip/ip.c

⟨*macro* BKFG 212a⟩≡ (212b)

```
/*
 * This sleazy macro relies on the media header size being
 * larger than sizeof(Ipfrag). ipreassemble checks this is true
 */
#define BKFG(xp)    ((Ipfrag*)((xp)->base))
```

⟨kernel/network/ip/ip.c 212b⟩≡

```
#include    "u.h"
```

```
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
#include    "../port/error.h"

#include    "ip.h"
```

⟨*macro* BLKIPVER 30d⟩

⟨*global* statnames 23a⟩

⟨*macro* BLKIP 30e⟩
⟨*macro* BKFG 212a⟩

```
ushort      ipcsum(uchar*);
Block*      ip4reassemble(IP*, int, Block*, Ip4hdr*);
void        ipfragfree4(IP*, Fragment4*);
Fragment4*  ipfragallo4(IP*);
```

⟨*function* ip_init_6 468d⟩

⟨*function* initfrag 33a⟩

⟨*function* ip_init 32d⟩

⟨*function* iprouting 68a⟩

⟨*function* ipoput4 79b⟩

⟨*function* ipiput4 83f⟩

⟨*function* ipstats 65b⟩

⟨*function* ip4reassemble 85e⟩

⟨*function* ipfragfree4 78b⟩

⟨*function* ipfragallo4 79a⟩

⟨*function* ipcsum 78a⟩

I.5 kernel/network/

I.5.1 kernel/network/portfns_network.h

⟨kernel/network/portfns_network.h 213⟩≡

```
void   hnputl(void*, uint);
void   hnputs(void*, ushort);
uint   nhgetl(void*);
ushort nhgets(void*);

//void    hnputv(void*, uvlong);
//uvlong    nhgetv(void*);
```

I.5.2 kernel/network/netif.h

⟨*enum* _anon_ (kernel/network/netif.h) 214a⟩≡ (215a)

```
enum
{
  Nmaxaddr= 64,
  Nmhash=   31,

  Ncloneqid=  1,
  Naddrqid,
  N2ndqid,
  N3rdqid,
  Ndataqid,
  Nctlqid,
  Nstatqid,
  Ntypeqid,
  Nifstatqid,
  Nmtuqid,
};
```

⟨*macro* NETTYPE 214b⟩≡ (215a)

```
/*
 *  Macros to manage Qid's used for multiplexed devices
 */
#define NETTYPE(x)  (((ulong)x)&0x1f)
```

⟨*macro* NETID 214c⟩≡ (215a)

```
#define NETID(x)  ((((ulong)x))>>5)
```

⟨*macro* NETQID 214d⟩≡ (215a)

```
#define NETQID(i,t) ((((ulong)i)<<5)|(t))
```

⟨*struct* Netaddr 214e⟩≡ (215a)

```
/*
 *  a network address
 */
struct Netaddr
{
  Netaddr *next;    /* allocation chain */
  Netaddr *hnext;
  uchar addr[Nmaxaddr];
  int ref;
};
```

⟨*enum* _anon_ (kernel/network/netif.h)2 214f⟩≡ (215a)

```
/*
 *  Ethernet specific
 */
enum
{
  ⟨constant Eaddrlen 19e⟩
  ETHERMINTU =  60,   /* minimum transmit size */
  ETHERMAXTU =  1514,   /* maximum transmit size */
  ETHERHDRSIZE =  14,   /* size of an ethernet header */

  /* ethernet packet types */
  ETARP   = 0x0806,
  ETIP4   = 0x0800,

  ETIP6   = 0x86DD,
};
```

⟨kernel/network/netif.h 215a⟩≡

```
// todo: split in portdat_network.h? and portfns_network.h

typedef struct Etherpkt Etherpkt;
typedef struct Netaddr  Netaddr;
typedef struct Netfile  Netfile;
typedef struct Netif  Netif;

⟨enum _anon_ (kernel/network/netif.h) 214a⟩

⟨macro NETTYPE 214b⟩
⟨macro NETID 214c⟩
⟨macro NETQID 214d⟩

⟨struct Netfile 28c⟩

⟨struct Netaddr 214e⟩

⟨struct Netif (kernel) 27d⟩

void  netifinit(Netif*, char*, int, ulong);
Walkqid*  netifwalk(Netif*, Chan*, Chan*, char **, int);
Chan* netifopen(Netif*, Chan*, int);
void  netifclose(Netif*, Chan*);
long  netifread(Netif*, Chan*, void*, long, ulong);
Block*  netifbread(Netif*, Chan*, long, ulong);
long  netifwrite(Netif*, Chan*, void*, long);
int netifwstat(Netif*, Chan*, uchar*, int);
int netifstat(Netif*, Chan*, uchar*, int);
int activemulti(Netif*, uchar*, int);

⟨enum _anon_ (kernel/network/netif.h)2 214f⟩

⟨typedef eaddr 19f⟩

⟨struct Etherpkt 74b⟩
```

I.5.3 kernel/network/netif.c

⟨function netifinit 215b⟩≡ (227)

```
/*
 *  set up a new network interface
 */
void
netifinit(Netif *nif, char *name, int nfile, ulong limit)
{
    strncpy(nif->name, name, KNAMELEN-1);
    nif->name[KNAMELEN-1] = 0;
    nif->nfile = nfile;
    nif->f = xalloc(nfile*sizeof(Netfile*));
    if (nif->f == nil)
        panic("netifinit: no memory");
    memset(nif->f, 0, nfile*sizeof(Netfile*));
    nif->limit = limit;
}
```

⟨*function* netifgen 216⟩≡ (227)

```
/*
 *  generate a 3 level directory
 */
static int
netifgen(Chan *c, char*, Dirtab *vp, int, int i, Dir *dp)
{
    Qid q;
    Netif *nif = (Netif*)vp;
    Netfile *f;
    int t;
    int perm;
    char *o;

    q.type = QTFILE;
    q.vers = 0;

    /* top level directory contains the name of the network */
    if(c->qid.path == 0){
        switch(i){
        case DEVDOTDOT:
            q.path = 0;
            q.type = QTDIR;
            devdir(c, q, ".", 0, eve, 0555, dp);
            break;
        case 0:
            q.path = N2ndqid;
            q.type = QTDIR;
            strcpy(up->genbuf, nif->name);
            devdir(c, q, up->genbuf, 0, eve, 0555, dp);
            break;
        default:
            return -1;
        }
        return 1;
    }

    /* second level contains clone plus all the conversations */
    t = NETTYPE(c->qid.path);
    if(t == N2ndqid || t == Ncloneqid || t == Naddrqid){
        switch(i) {
        case DEVDOTDOT:
            q.type = QTDIR;
            q.path = 0;
            devdir(c, q, ".", 0, eve, DMDIR|0555, dp);
            break;
        case 0:
            q.path = Ncloneqid;
            devdir(c, q, "clone", 0, eve, 0666, dp);
            break;
        case 1:
            q.path = Naddrqid;
            devdir(c, q, "addr", 0, eve, 0666, dp);
            break;
        case 2:
            q.path = Nstatqid;
            devdir(c, q, "stats", 0, eve, 0444, dp);
            break;
        case 3:
            q.path = Nifstatqid;
```

```
            devdir(c, q, "ifstats", 0, eve, 0444, dp);
            break;
        default:
            i -= 4;
            if(i >= nif->nfile)
                return -1;
            if(nif->f[i] == 0)
                return 0;
            q.type = QTDIR;
            q.path = NETQID(i, N3rdqid);
            snprint(up->genbuf, sizeof up->genbuf, "%d", i);
            devdir(c, q, up->genbuf, 0, eve, DMDIR|0555, dp);
            break;
        }
        return 1;
    }

    /* third level */
    f = nif->f[NETID(c->qid.path)];
    if(f == 0)
        return 0;
    if(*f->owner){
        o = f->owner;
        perm = f->mode;
    } else {
        o = eve;
        perm = 0666;
    }
    switch(i){
    case DEVDOTDOT:
        q.type = QTDIR;
        q.path = N2ndqid;
        strcpy(up->genbuf, nif->name);
        devdir(c, q, up->genbuf, 0, eve, DMDIR|0555, dp);
        break;
    case 0:
        q.path = NETQID(NETID(c->qid.path), Ndataqid);
        devdir(c, q, "data", 0, o, perm, dp);
        break;
    case 1:
        q.path = NETQID(NETID(c->qid.path), Nctlqid);
        devdir(c, q, "ctl", 0, o, perm, dp);
        break;
    case 2:
        q.path = NETQID(NETID(c->qid.path), Nstatqid);
        devdir(c, q, "stats", 0, eve, 0444, dp);
        break;
    case 3:
        q.path = NETQID(NETID(c->qid.path), Ntypeqid);
        devdir(c, q, "type", 0, eve, 0444, dp);
        break;
    case 4:
        q.path = NETQID(NETID(c->qid.path), Nifstatqid);
        devdir(c, q, "ifstats", 0, eve, 0444, dp);
        break;
    default:
        return -1;
    }
    return 1;
}
```

⟨*function* netifwalk 218a⟩≡ (227)

```
Walkqid*
netifwalk(Netif *nif, Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, (Dirtab *)nif, 0, netifgen);
}
```

⟨*function* netifopen 218b⟩≡ (227)

```
Chan*
netifopen(Netif *nif, Chan *c, int omode)
{
    int id;
    Netfile *f;

    id = 0;
    if(c->qid.type & QTDIR){
        if(omode != OREAD)
            error(Eperm);
    } else {
        switch(NETTYPE(c->qid.path)){
        case Ndataqid:
        case Nctlqid:
            id = NETID(c->qid.path);
            openfile(nif, id);
            break;
        case Ncloneqid:
            id = openfile(nif, -1);
            c->qid.path = NETQID(id, Nctlqid);
            break;
        default:
            if(omode != OREAD)
                error(Ebadarg);
        }
        switch(NETTYPE(c->qid.path)){
        case Ndataqid:
        case Nctlqid:
            f = nif->f[id];
            if(netown(f, up->user, omode&7) < 0)
                error(Eperm);
            break;
        }
    }
    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    c->iounit = qiomaxatomic;
    return c;
}
```

⟨*function* netifread 218c⟩≡ (227)

```
long
netifread(Netif *nif, Chan *c, void *a, long n, ulong offset)
{
    int i, j;
    Netfile *f;
    char *p;

    if(c->qid.type&QTDIR)
        return devdirread(c, a, n, (Dirtab*)nif, 0, netifgen);
```

```
    switch(NETTYPE(c->qid.path)){
    case Ndataqid:
        f = nif->f[NETID(c->qid.path)];
        return qread(f->in, a, n);
    case Nctlqid:
        return readnum(offset, a, n, NETID(c->qid.path), NUMSIZE);
    case Nstatqid:
        p = malloc(READSTR);
        if(p == nil)
            error(Enomem);
        j = snprint(p, READSTR, "in: %llud\n", nif->inpackets);
        j += snprint(p+j, READSTR-j, "link: %d\n", nif->link);
        j += snprint(p+j, READSTR-j, "out: %llud\n", nif->outpackets);
        j += snprint(p+j, READSTR-j, "crc errs: %d\n", nif->crcs);
        j += snprint(p+j, READSTR-j, "overflows: %d\n", nif->overflows);
        j += snprint(p+j, READSTR-j, "soft overflows: %d\n", nif->soverflows);
        j += snprint(p+j, READSTR-j, "framing errs: %d\n", nif->frames);
        j += snprint(p+j, READSTR-j, "buffer errs: %d\n", nif->buffs);
        j += snprint(p+j, READSTR-j, "output errs: %d\n", nif->oerrs);
        j += snprint(p+j, READSTR-j, "prom: %d\n", nif->prom);
        j += snprint(p+j, READSTR-j, "mbps: %d\n", nif->mbps);
        j += snprint(p+j, READSTR-j, "addr: ");
        for(i = 0; i < nif->alen; i++)
            j += snprint(p+j, READSTR-j, "%2.2ux", nif->addr[i]);
        snprint(p+j, READSTR-j, "\n");
        n = readstr(offset, a, n, p);
        free(p);
        return n;
    case Naddrqid:
        p = malloc(READSTR);
        if(p == nil)
            error(Enomem);
        j = 0;
        for(i = 0; i < nif->alen; i++)
            j += snprint(p+j, READSTR-j, "%2.2ux", nif->addr[i]);
        n = readstr(offset, a, n, p);
        free(p);
        return n;
    case Ntypeqid:
        f = nif->f[NETID(c->qid.path)];
        return readnum(offset, a, n, f->type, NUMSIZE);
    case Nifstatqid:
        return 0;
    }
    error(Ebadarg);
    return -1;  /* not reached */
}
```

⟨*function* netifbread 219a⟩≡ (227)

```
Block*
netifbread(Netif *nif, Chan *c, long n, ulong offset)
{
    if((c->qid.type & QTDIR) || NETTYPE(c->qid.path) != Ndataqid)
        return devbread(c, n, offset);

    return qbread(nif->f[NETID(c->qid.path)]->in, n);
}
```

⟨*function* typeinuse 219b⟩≡ (227)

```
/*
```

```
 *  make sure this type isn't already in use on this device
 */
static int
typeinuse(Netif *nif, int type)
{
    Netfile *f, **fp, **efp;

    if(type <= 0)
        return 0;

    efp = &nif->f[nif->nfile];
    for(fp = nif->f; fp < efp; fp++){
        f = *fp;
        if(f == 0)
            continue;
        if(f->type == type)
            return 1;
    }
    return 0;
}
```

⟨*function* netifwrite 220⟩≡ (227)

```
/*
 *  the devxxx.c that calls us handles writing data, it knows best
 */
long
netifwrite(Netif *nif, Chan *c, void *a, long n)
{
    Netfile *f;
    int type;
    char *p, buf[64];
    uchar binaddr[Nmaxaddr];

    if(NETTYPE(c->qid.path) != Nctlqid)
        error(Eperm);

    if(n >= sizeof(buf))
        n = sizeof(buf)-1;
    memmove(buf, a, n);
    buf[n] = 0;

    if(waserror()){
        qunlock(nif);
        nexterror();
    }

    qlock(nif);
    f = nif->f[NETID(c->qid.path)];
    if((p = matchtoken(buf, "connect")) != 0){
        type = atoi(p);
        if(typeinuse(nif, type))
            error(Einuse);
        f->type = type;
        if(f->type < 0)
            nif->all++;
    } else if(matchtoken(buf, "promiscuous")){
        if(f->prom == 0){
            if(nif->prom == 0 && nif->promiscuous != nil)
                nif->promiscuous(nif->arg, 1);
            f->prom = 1;
```

```
            nif->prom++;
        }
    } else if((p = matchtoken(buf, "scanbs")) != 0){
        /* scan for base stations */
        if(f->scan == 0){
            type = atoi(p);
            if(type < 5)
                type = 5;
            if(nif->scanbs != nil)
                nif->scanbs(nif->arg, type);
            f->scan = type;
            nif->scan++;
        }
    } else if(matchtoken(buf, "bridge")){
        f->bridge = 1;
    } else if(matchtoken(buf, "headersonly")){
        f->headersonly = 1;
    } else if((p = matchtoken(buf, "addmulti")) != 0){
        if(parseaddr(binaddr, p, nif->alen) < 0)
            error("bad address");
        p = netmulti(nif, f, binaddr, 1);
        if(p)
            error(p);
    } else if((p = matchtoken(buf, "remmulti")) != 0){
        if(parseaddr(binaddr, p, nif->alen) < 0)
            error("bad address");
        p = netmulti(nif, f, binaddr, 0);
        if(p)
            error(p);
    } else
        n = -1;
    qunlock(nif);
    poperror();
    return n;
}
```

⟨*function* netifwstat 221⟩≡ (227)

```
int
netifwstat(Netif *nif, Chan *c, uchar *db, int n)
{
    Dir *dir;
    Netfile *f;
    int m;

    f = nif->f[NETID(c->qid.path)];
    if(f == 0)
        error(Enonexist);

    if(netown(f, up->user, OWRITE) < 0)
        error(Eperm);

    dir = smalloc(sizeof(Dir)+n);
    m = convM2D(db, n, &dir[0], (char*)&dir[1]);
    if(m == 0){
        free(dir);
        error(Eshortstat);
    }
    if(!emptystr(dir[0].uid))
        strncpy(f->owner, dir[0].uid, KNAMELEN);
    if(dir[0].mode != ~0UL)
```

```
        f->mode = dir[0].mode;
    free(dir);
    return m;
}
```

⟨*function* netifstat 222a⟩≡ (227)

```
int
netifstat(Netif *nif, Chan *c, uchar *db, int n)
{
    return devstat(c, db, n, (Dirtab *)nif, 0, netifgen);
}
```

⟨*function* netifclose 222b⟩≡ (227)

```
void
netifclose(Netif *nif, Chan *c)
{
    Netfile *f;
    int t;
    Netaddr *ap;

    if((c->flag & COPEN) == 0)
        return;

    t = NETTYPE(c->qid.path);
    if(t != Ndataqid && t != Nctlqid)
        return;

    f = nif->f[NETID(c->qid.path)];
    qlock(f);
    if(--(f->inuse) == 0){
        if(f->prom){
            qlock(nif);
            if(--(nif->prom) == 0 && nif->promiscuous != nil)
                nif->promiscuous(nif->arg, 0);
            qunlock(nif);
            f->prom = 0;
        }
        if(f->scan){
            qlock(nif);
            if(--(nif->scan) == 0 && nif->scanbs != nil)
                nif->scanbs(nif->arg, 0);
            qunlock(nif);
            f->prom = 0;
            f->scan = 0;
        }
        if(f->nmaddr){
            qlock(nif);
            t = 0;
            for(ap = nif->maddr; ap; ap = ap->next){
                if(f->maddr[t/8] & (1<<(t%8)))
                    netmulti(nif, f, ap->addr, 0);
            }
            qunlock(nif);
            f->nmaddr = 0;
        }
        if(f->type < 0){
            qlock(nif);
            --(nif->all);
            qunlock(nif);
        }
```

```
        f->owner[0] = 0;
        f->type = 0;
        f->bridge = 0;
        f->headersonly = 0;
        qclose(f->in);
    }
    qunlock(f);
}
```

⟨*global* netlock 223a⟩≡ (227)

```
Lock netlock;
```

⟨*function* netown 223b⟩≡ (227)

```
static int
netown(Netfile *p, char *o, int omode)
{
    static int access[] = { 0400, 0200, 0600, 0100 };
    int mode;
    int t;

    lock(&netlock);
    if(*p->owner){
        if(strncmp(o, p->owner, KNAMELEN) == 0) /* User */
            mode = p->mode;
        else if(strncmp(o, eve, KNAMELEN) == 0) /* Bootes is group */
            mode = p->mode<<3;
        else
            mode = p->mode<<6;      /* Other */

        t = access[omode&3];
        if((t & mode) == t){
            unlock(&netlock);
            return 0;
        } else {
            unlock(&netlock);
            return -1;
        }
    }
    strncpy(p->owner, o, KNAMELEN);
    p->mode = 0660;
    unlock(&netlock);
    return 0;
}
```

⟨*function* openfile 223c⟩≡ (227)

```
/*
 *  Increment the reference count of a network device.
 *  If id < 0, return an unused ether device.
 */
static int
openfile(Netif *nif, int id)
{
    Netfile *f, **fp, **efp;

    if(id >= 0){
        f = nif->f[id];
        if(f == 0)
            error(Enodev);
        qlock(f);
        qreopen(f->in);
```

```
        f->inuse++;
        qunlock(f);
        return id;
    }

    qlock(nif);
    if(waserror()){
        qunlock(nif);
        nexterror();
    }
    efp = &nif->f[nif->nfile];
    for(fp = nif->f; fp < efp; fp++){
        f = *fp;
        if(f == 0){
            f = malloc(sizeof(Netfile));
            if(f == 0)
                exhausted("memory");
            f->in = qopen(nif->limit, Qmsg, 0, 0);
            if(f->in == nil){
                free(f);
                exhausted("memory");
            }
            *fp = f;
            qlock(f);
        } else {
            qlock(f);
            if(f->inuse){
                qunlock(f);
                continue;
            }
        }
        f->inuse = 1;
        qreopen(f->in);
        netown(f, up->user, 0);
        qunlock(f);
        qunlock(nif);
        poperror();
        return fp - nif->f;
    }
    error(Enodev);
    return -1;  /* not reached */
}
```

⟨*function* matchtoken 224⟩≡ (227)

```
/*
 *  look for a token starting a string,
 *  return a pointer to first non-space char after it
 */
static char*
matchtoken(char *p, char *token)
{
    int n;

    n = strlen(token);
    if(strncmp(p, token, n))
        return 0;
    p += n;
    if(*p == 0)
        return p;
    if(*p != ' ' && *p != '\t' && *p != '\n')
```

```
        return 0;
    while(*p == ' ' || *p == '\t' || *p == '\n')
        p++;
    return p;
}
```

⟨*function* hnputl*(*(kernel/network/netif.c)*)* 225a⟩≡ (227)

```
void
hnputl(void *p, uint v)
{
    uchar *a;

    a = p;
    a[0] = v>>24;
    a[1] = v>>16;
    a[2] = v>>8;
    a[3] = v;
}
```

⟨*function* hnputs*(*(kernel/network/netif.c)*)* 225b⟩≡ (227)

```
void
hnputs(void *p, ushort v)
{
    uchar *a;

    a = p;
    a[0] = v>>8;
    a[1] = v;
}
```

⟨*function* nhgetl*(*(kernel/network/netif.c)*)* 225c⟩≡ (227)

```
uint
nhgetl(void *p)
{
    uchar *a;

    a = p;
    return (a[0]<<24)|(a[1]<<16)|(a[2]<<8)|(a[3]<<0);
}
```

⟨*function* nhgets*(*(kernel/network/netif.c)*)* 225d⟩≡ (227)

```
ushort
nhgets(void *p)
{
    uchar *a;

    a = p;
    return (a[0]<<8)|(a[1]<<0);
}
```

⟨*function* hash 225e⟩≡ (227)

```
static ulong
hash(uchar *a, int len)
{
    ulong sum = 0;

    while(len-- > 0)
        sum = (sum << 1) + *a++;
    return sum%Nmhash;
}
```

⟨*function* activemulti 226a⟩≡ (227)

```
int
activemulti(Netif *nif, uchar *addr, int alen)
{
    Netaddr *hp;

    for(hp = nif->mhash[hash(addr, alen)]; hp; hp = hp->hnext)
        if(memcmp(addr, hp->addr, alen) == 0){
            if(hp->ref)
                return 1;
            else
                break;
        }
    return 0;
}
```

⟨*function* parseaddr 226b⟩≡ (227)

```
static int
parseaddr(uchar *to, char *from, int alen)
{
    char nip[4];
    char *p;
    int i;

    p = from;
    for(i = 0; i < alen; i++){
        if(*p == 0)
            return -1;
        nip[0] = *p++;
        if(*p == 0)
            return -1;
        nip[1] = *p++;
        nip[2] = 0;
        to[i] = strtoul(nip, 0, 16);
        if(*p == ':')
            p++;
    }
    return 0;
}
```

⟨*function* netmulti 226c⟩≡ (227)

```
/*
 *  keep track of multicast addresses
 */
static char*
netmulti(Netif *nif, Netfile *f, uchar *addr, int add)
{
    Netaddr **l, *ap;
    int i;
    ulong h;

    if(nif->multicast == nil)
        return "interface does not support multicast";

    l = &nif->maddr;
    i = 0;
    for(ap = *l; ap; ap = *l){
        if(memcmp(addr, ap->addr, nif->alen) == 0)
            break;
        i++;
```

```
        l = &ap->next;
    }

    if(add){
        if(ap == 0){
            *l = ap = smalloc(sizeof(*ap));
            memmove(ap->addr, addr, nif->alen);
            ap->next = 0;
            ap->ref = 1;
            h = hash(addr, nif->alen);
            ap->hnext = nif->mhash[h];
            nif->mhash[h] = ap;
        } else {
            ap->ref++;
        }
        if(ap->ref == 1){
            nif->nmaddr++;
            nif->multicast(nif->arg, addr, 1);
        }
        if(i < 8*sizeof(f->maddr)){
            if((f->maddr[i/8] & (1<<(i%8))) == 0)
                f->nmaddr++;
            f->maddr[i/8] |= 1<<(i%8);
        }
    } else {
        if(ap == 0 || ap->ref == 0)
            return 0;
        ap->ref--;
        if(ap->ref == 0){
            nif->nmaddr--;
            nif->multicast(nif->arg, addr, 0);
        }
        if(i < 8*sizeof(f->maddr)){
            if((f->maddr[i/8] & (1<<(i%8))) != 0)
                f->nmaddr--;
            f->maddr[i/8] &= ~(1<<(i%8));
        }
    }
    return 0;
}
```

⟨kernel/network/netif.c 227⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
#include    "../port/error.h"
#include    "../port/netif.h"

static int netown(Netfile*, char*, int);
static int openfile(Netif*, int);
static char* matchtoken(char*, char*);
static char* netmulti(Netif*, Netfile*, uchar*, int);
static int parseaddr(uchar*, char*, int);
```

⟨*function* netifinit 215b⟩

⟨*function* netifgen 216⟩

⟨*function* netifwalk 218a⟩

⟨*function* netifopen 218b⟩

⟨*function* netifread 218c⟩

⟨*function* netifbread 219a⟩

⟨*function* typeinuse 219b⟩

⟨*function* netifwrite 220⟩

⟨*function* netifwstat 221⟩

⟨*function* netifstat 222a⟩

⟨*function* netifclose 222b⟩

⟨*global* netlock 223a⟩

⟨*function* netown 223b⟩

⟨*function* openfile 223c⟩

⟨*function* matchtoken 224⟩

⟨*function* hnputl*(*(kernel/network/netif.c)*)* 225a⟩

⟨*function* hnputs*(*(kernel/network/netif.c)*)* 225b⟩

⟨*function* nhgetl*(*(kernel/network/netif.c)*)* 225c⟩

⟨*function* nhgets*(*(kernel/network/netif.c)*)* 225d⟩

⟨*function* hash 225e⟩

⟨*function* activemulti 226a⟩

⟨*function* parseaddr 226b⟩

⟨*function* netmulti 226c⟩

I.6 kernel/network/386/

I.6.1 kernel/network/etherif.h

⟨*enum* _anon_ (kernel/network/etherif.h) 228a⟩≡ (229a)

```
enum {
  ⟨constant MaxEther 20d⟩
  Ntypes    = 8,
};
```

⟨*macro* NEXT 228b⟩≡ (229a)

```
#define NEXT(x, l)  (((x)+1)%(l))
```

⟨*macro* PREV 228c⟩≡ (229a)

```
#define PREV(x, l)  (((x) == 0) ? (l)-1: (x)-1)
```

⟨kernel/network/etherif.h 229a⟩≡

```
⟨enum _anon_ (kernel/network/etherif.h) 228a⟩

typedef struct Ether Ether;
⟨struct Ether (kernel) 20e⟩

extern Block* etheriq(Ether*, Block*, int);
extern void addethercard(char*, int(*)(Ether*));
extern ulong ethercrc(uchar*, int);
//extern int parseether(uchar*, char*);

⟨macro NEXT 228b⟩
⟨macro PREV 228c⟩
```

I.6.2 kernel/network/386/devether.c

⟨function etherwalk 229b⟩≡ (237b)

```
static Walkqid*
etherwalk(Chan* chan, Chan* nchan, char** name, int nname)
{
    return netifwalk(etherxx[chan->dev], chan, nchan, name, nname);
}
```

⟨function etherstat 229c⟩≡ (237b)

```
static int
etherstat(Chan* chan, uchar* dp, int n)
{
    return netifstat(etherxx[chan->dev], chan, dp, n);
}
```

⟨function etheropen 229d⟩≡ (237b)

```
static Chan*
etheropen(Chan* chan, int omode)
{
    return netifopen(etherxx[chan->dev], chan, omode);
}
```

⟨function ethercreate 229e⟩≡ (237b)

```
static void
ethercreate(Chan*, char*, int, ulong)
{
}
```

⟨function etherclose 229f⟩≡ (237b)

```
static void
etherclose(Chan* chan)
{
    netifclose(etherxx[chan->dev], chan);
}
```

⟨function etherread 229g⟩≡ (237b)

```
static long
etherread(Chan* chan, void* buf, long n, vlong off)
{
    Ether *ether;
    ulong offset = off;

    ether = etherxx[chan->dev];
    if((chan->qid.type & QTDIR) == 0 && ether->ifstat){
```

```
        /*
         * With some controllers it is necessary to reach
         * into the chip to extract statistics.
         */
        if(NETTYPE(chan->qid.path) == Nifstatqid)
            return ether->ifstat(ether, buf, n, offset);
        else if(NETTYPE(chan->qid.path) == Nstatqid)
            ether->ifstat(ether, buf, 0, offset);
    }

    return netifread(ether, chan, buf, n, offset);
}
```

⟨*function* etherbread 230a⟩≡ (237b)

```
static Block*
etherbread(Chan* chan, long n, ulong offset)
{
    return netifbread(etherxx[chan->dev], chan, n, offset);
}
```

⟨*function* etherwstat 230b⟩≡ (237b)

```
static int
etherwstat(Chan* chan, uchar* dp, int n)
{
    return netifwstat(etherxx[chan->dev], chan, dp, n);
}
```

⟨*function* etherrtrace 230c⟩≡ (237b)

```
static void
etherrtrace(Netfile* f, Etherpkt* pkt, int len)
{
    int i, n;
    Block *bp;

    if(qwindow(f->in) <= 0)
        return;
    if(len > 58)
        n = 58;
    else
        n = len;
    bp = iallocb(64);
    if(bp == nil)
        return;
    memmove(bp->wp, pkt->d, n);
    i = TK2MS(CPUS(0)->ticks);
    bp->wp[58] = len>>8;
    bp->wp[59] = len;
    bp->wp[60] = i>>24;
    bp->wp[61] = i>>16;
    bp->wp[62] = i>>8;
    bp->wp[63] = i;
    bp->wp += 64;
    qpass(f->in, bp);
}
```

⟨*function* etheriq 230d⟩≡ (237b)

```
Block*
etheriq(Ether* ether, Block* bp, int fromwire)
{
    Etherpkt *pkt;
```

```
ushort type;
int len, multi, tome, fromme;
Netfile **ep, *f, **fp, *fx;
Block *xbp;

ether->inpackets++;

pkt = (Etherpkt*)bp->rp;
len = BLEN(bp);
type = (pkt->type[0]<<8)|pkt->type[1];
fx = 0;
ep = &ether->f[Ntypes];

multi = pkt->d[0] & 1;
/* check for valid multicast addresses */
if(multi && memcmp(pkt->d, ether->bcast, sizeof(pkt->d)) != 0 && ether->prom == 0){
    if(!activemulti(ether, pkt->d, sizeof(pkt->d))){
        if(fromwire){
            freeb(bp);
            bp = 0;
        }
        return bp;
    }
}

/* is it for me? */
tome = memcmp(pkt->d, ether->ea, sizeof(pkt->d)) == 0;
fromme = memcmp(pkt->s, ether->ea, sizeof(pkt->s)) == 0;

/*
 * Multiplex the packet to all the connections which want it.
 * If the packet is not to be used subsequently (fromwire != 0),
 * attempt to simply pass it into one of the connections, thereby
 * saving a copy of the data (usual case hopefully).
 */
for(fp = ether->f; fp < ep; fp++){
    if(f = *fp)
    if(f->type == type || f->type < 0)
    if(tome || multi || f->prom){
        /* Don't want to hear bridged packets */
        if(f->bridge && !fromwire && !fromme)
            continue;
        if(!f->headersonly){
            if(fromwire && fx == 0)
                fx = f;
            else if(xbp = iallocb(len)){
                memmove(xbp->wp, pkt, len);
                xbp->wp += len;
                if(qpass(f->in, xbp) < 0){
                    // print("soverflow for f->in\n");
                    ether->soverflows++;
                }
            }
            else{
                // print("soverflow iallocb\n");
                ether->soverflows++;
            }
        }
        else
            etherrtrace(f, pkt, len);
```

```
        }
    }

    if(fx){
        if(qpass(fx->in, bp) < 0){
            // print("soverflow for fx->in\n");
            ether->soverflows++;
        }
        return 0;
    }
    if(fromwire){
        freeb(bp);
        return 0;
    }

    return bp;
}
```

⟨*function* etheroq 232a⟩≡ (237b)

```
static int
etheroq(Ether* ether, Block* bp)
{
    int len, loopback, s;
    Etherpkt *pkt;

    ether->outpackets++;

    /*
     * Check if the packet has to be placed back onto the input queue,
     * i.e. if it's a loopback or broadcast packet or the interface is
     * in promiscuous mode.
     * If it's a loopback packet indicate to etheriq that the data isn't
     * needed and return, etheriq will pass-on or free the block.
     * To enable bridging to work, only packets that were originated
     * by this interface are fed back.
     */
    pkt = (Etherpkt*)bp->rp;
    len = BLEN(bp);
    loopback = memcmp(pkt->d, ether->ea, sizeof(pkt->d)) == 0;
    if(loopback || memcmp(pkt->d, ether->bcast, sizeof(pkt->d)) == 0 || ether->prom){
        s = arch_splhi();
        etheriq(ether, bp, 0);
        arch_splx(s);
    }

    if(!loopback){
        if(qfull(ether->oq))
            print("etheroq: WARNING: ether->oq full!\n");
        qbwrite(ether->oq, bp);
        if(ether->transmit != nil)
            ether->transmit(ether);
    } else
        freeb(bp);

    return len;
}
```

⟨*function* etherwrite 232b⟩≡ (237b)

```
static long
etherwrite(Chan* chan, void* buf, long n, vlong)
```

```
{
    Ether *ether;
    Block *bp;
    int nn, onoff;
    Cmdbuf *cb;

    ether = etherxx[chan->dev];
    if(NETTYPE(chan->qid.path) != Ndataqid) {
        nn = netifwrite(ether, chan, buf, n);
        if(nn >= 0)
            return nn;
        cb = parsecmd(buf, n);
        if(cb->f[0] && strcmp(cb->f[0], "nonblocking") == 0){
            if(cb->nf <= 1)
                onoff = 1;
            else
                onoff = atoi(cb->f[1]);
            qnoblock(ether->oq, onoff);
            free(cb);
            return n;
        }
        free(cb);
        if(ether->ctl != nil)
            return ether->ctl(ether, buf, n);

        error(Ebadctl);
    }

    if(n > ether->mtu)
        error(Etoobig);
    if(n < ether->minmtu)
        error(Etoosmall);

    bp = allocb(n);
    if(waserror()){
        freeb(bp);
        nexterror();
    }
    memmove(bp->rp, buf, n);
    memmove(bp->rp+Eaddrlen, ether->ea, Eaddrlen);
    poperror();
    bp->wp += n;

    return etheroq(ether, bp);
}
```

⟨*function* etherbwrite 233⟩≡ (237b)

```
static long
etherbwrite(Chan* chan, Block* bp, ulong)
{
    Ether *ether;
    long n;

    n = BLEN(bp);
    if(NETTYPE(chan->qid.path) != Ndataqid){
        if(waserror()) {
            freeb(bp);
            nexterror();
        }
        n = etherwrite(chan, bp->rp, n, 0);
```

```
        poperror();
        freeb(bp);
        return n;
    }
    ether = etherxx[chan->dev];

    if(n > ether->mtu){
        freeb(bp);
        error(Etoobig);
    }
    if(n < ether->minmtu){
        freeb(bp);
        error(Etoosmall);
    }

    return etheroq(ether, bp);
}
```

⟨*global* cards 234a⟩≡ (237b)

```
static struct {
    char*   type;
    int (*reset)(Ether*);
} cards[MaxEther+1];
```

⟨*function* addethercard 234b⟩≡ (237b)

```
void
addethercard(char* t, int (*r)(Ether*))
{
    static int ncard;

    if(ncard == MaxEther)
        panic("too many ether cards");
    cards[ncard].type = t;
    cards[ncard].reset = r;
    ncard++;
}
```

⟨*function* parseether*(*(kernel/network/386/devether.c)*)* 234c⟩≡ (237b)

```
int
parseether(uchar *to, char *from)
{
    char nip[4];
    char *p;
    int i;

    p = from;
    for(i = 0; i < Eaddrlen; i++){
        if(*p == 0)
            return -1;
        nip[0] = *p++;
        if(*p == 0)
            return -1;
        nip[1] = *p++;
        nip[2] = 0;
        to[i] = strtoul(nip, 0, 16);
        if(*p == ':')
            p++;
    }
    return 0;
}
```

⟨*function* etherprobe 235⟩≡ (237b)

```
static Ether*
etherprobe(int cardno, int ctlrno)
{
    int i, lg;
    ulong mb, bsz;
    Ether *ether;
    char buf[128], name[32];

    ether = malloc(sizeof(Ether));
    if(ether == nil)
        error(Enomem);
    memset(ether, 0, sizeof(Ether));
    ether->ctlrno = ctlrno;
    ether->tbdf = BUSUNKNOWN;
    ether->mbps = 10;
    ether->minmtu = ETHERMINTU;
    ether->maxmtu = ETHERMAXTU;
    ether->mtu = ETHERMAXTU;

    if(cardno < 0){
        if(arch_isaconfig("ether", ctlrno, ether) == 0){
            free(ether);
            return nil;
        }
        for(cardno = 0; cards[cardno].type; cardno++){
            if(cistrcmp(cards[cardno].type, ether->type))
                continue;
            for(i = 0; i < ether->nopt; i++){
                if(strncmp(ether->opt[i], "ea=", 3))
                    continue;
                if(parseether(ether->ea, &ether->opt[i][3]))
                    memset(ether->ea, 0, Eaddrlen);
            }
            break;
        }
    }

    if(cardno >= MaxEther || cards[cardno].type == nil){
        free(ether);
        return nil;
    }
    if(cards[cardno].reset(ether) < 0){
        free(ether);
        return nil;
    }

    /*
     * IRQ2 doesn't really exist, it's used to gang the interrupt
     * controllers together. A device set to IRQ2 will appear on
     * the second interrupt controller as IRQ9.
     */
    if(ether->irq == 2)
        ether->irq = 9;
    snprint(name, sizeof(name), "ether%d", ctlrno);

    /*
     * If ether->irq is <0, it is a hack to indicate no interrupt
     * used by ethersink.
     */
```

```
    if(ether->irq >= 0)
        arch_intrenable(ether->irq, ether->interrupt, ether, ether->tbdf, name);

    i = sprint(buf, "#l%d: %s: ", ctlrno, cards[cardno].type);
    if(ether->mbps >= 1000)
        i += sprint(buf+i, "%dGbps", ether->mbps/1000);
    else
        i += sprint(buf+i, "%dMbps", ether->mbps);
    i += sprint(buf+i, " port 0x%luX irq %d", ether->port, ether->irq);
    if(ether->mem)
        i += sprint(buf+i, " addr 0x%luX", ether->mem);
    if(ether->size)
        i += sprint(buf+i, " size 0x%luX", ether->size);
    i += sprint(buf+i, ": %2.2ux%2.2ux%2.2ux%2.2ux%2.2ux%2.2ux",
        ether->ea[0], ether->ea[1], ether->ea[2],
        ether->ea[3], ether->ea[4], ether->ea[5]);
    sprint(buf+i, "\n");
    print(buf);

    /*
     * input queues are allocated by ../port/netif.c:/^openfile.
     * the size will be the last argument to netifinit() below.
     *
     * output queues should be small, to minimise ‘bufferbloat’,
     * which confuses tcp’s feedback loop.  at 1Gb/s, it only takes
     * ~15µs to transmit a full-sized non-jumbo packet.
     */

    /* compute log10(ether->mbps) into lg */
    for(lg = 0, mb = ether->mbps; mb >= 10; lg++)
        mb /= 10;
    if (lg > 14)            /* sanity cap; 2**(14+15) = 2^29 */
        lg = 14;

    /* allocate larger input queues for higher-speed interfaces */
    bsz = 1UL << (lg + 15);     /* 2^15 = 32K, bsz = 2^n × 32K */
    while (bsz > mainmem->maxsize / 8 && bsz > 128*1024)    /* sanity */
        bsz /= 2;
    netifinit(ether, name, Ntypes, bsz);

    if(ether->oq == nil)
        ether->oq = qopen(1 << (lg + 13), Qmsg, 0, 0);
    if(ether->oq == nil)
        panic("etherreset %s: can’t allocate output queue", name);

    ether->alen = Eaddrlen;
    memmove(ether->addr, ether->ea, Eaddrlen);
    memset(ether->bcast, 0xFF, Eaddrlen);

    return ether;
}
```

⟨*function* etherreset 236⟩≡ (237b)

```
static void
etherreset(void)
{
    Ether *ether;
    int cardno, ctlrno;

    for(ctlrno = 0; ctlrno < MaxEther; ctlrno++){
```

```
        if((ether = etherprobe(-1, ctlrno)) == nil)
            continue;
        etherxx[ctlrno] = ether;
    }

    if(getconf("*noetherprobe"))
        return;

    cardno = ctlrno = 0;
    while(cards[cardno].type != nil && ctlrno < MaxEther){
        if(etherxx[ctlrno] != nil){
            ctlrno++;
            continue;
        }
        if((ether = etherprobe(cardno, ctlrno)) == nil){
            cardno++;
            continue;
        }
        etherxx[ctlrno] = ether;
        ctlrno++;
    }
}
```

⟨*function* ethershutdown 237a⟩≡ (237b)

```
static void
ethershutdown(void)
{
    Ether *ether;
    int i;

    for(i = 0; i < MaxEther; i++){
        ether = etherxx[i];
        if(ether == nil)
            continue;
        if(ether->shutdown == nil) {
            print("#l%d: no shutdown function\n", i);
            continue;
        }
        (*ether->shutdown)(ether);
    }
}
```

⟨kernel/network/386/devether.c 237b⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "../port/error.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"

#include "io.h"
#include "pool.h"
#include <ureg.h>
#include "../port/netif.h"
#include "../port/etherif.h"
```

⟨*global etherxx (kernel)* 20c⟩

⟨*function* etherattach 33b⟩

⟨*function* etherwalk 229b⟩

⟨*function* etherstat 229c⟩

⟨*function* etheropen 229d⟩

⟨*function* ethercreate 229e⟩

⟨*function* etherclose 229f⟩

⟨*function* etherread 229g⟩

⟨*function* etherbread 230a⟩

⟨*function* etherwstat 230b⟩

⟨*function* etherrtrace 230c⟩

⟨*function* etheriq 230d⟩

⟨*function* etheroq 232a⟩

⟨*function* etherwrite 232b⟩

⟨*function* etherbwrite 233⟩

⟨*global* cards 234a⟩

⟨*function* addethercard 234b⟩

⟨*function* parseether*(*(kernel/network/386/devether.c)*)* 234c⟩

⟨*function* etherprobe 235⟩

⟨*function* etherreset 236⟩

⟨*function* ethershutdown 237a⟩

⟨*global* etherdevtab 54g⟩

⟨*constant* POLY 238a⟩

⟨*function* ethercrc 238b⟩

⟨*constant* POLY 238a⟩≡ (237b)

```
#define POLY 0xedb88320
```

⟨*function* ethercrc 238b⟩≡ (237b)

```
/* really slow 32 bit crc for ethers */
ulong
ethercrc(uchar *p, int len)
{
  int i, j;
  ulong crc, b;

  crc = 0xffffffff;
  for(i = 0; i < len; i++){
      b = *p++;
      for(j = 0; j < 8; j++){
          crc = (crc>>1) ^ (((crc^b) & 1) ? POLY : 0);
```

```
            b >>= 1;
        }
    }
    return crc;
}
```

I.6.3 kernel/network/386/ether8390.h

⟨*constant* Dp8390BufSz 239a⟩≡ (239c)

```
#define Dp8390BufSz 256
```

⟨*function* rdread 239b⟩≡ (239c)

```
static void
rdread(Dp8390* ctlr, void* to, int len)
{
    switch(ctlr->width){
    default:
        panic("dp8390 rdread: width %d\n", ctlr->width);
        break;

    case 2:
        inss(ctlr->data, to, len/2);
        break;

    case 1:
        insb(ctlr->data, to, len);
        break;
    }
}
```

⟨kernel/network/386/ether8390.h 239c⟩≡

```
typedef struct Dp8390 Dp8390;
⟨struct Dp8390 172a⟩

⟨constant Dp8390BufSz 239a⟩

extern int dp8390reset(Ether*);
extern void *dp8390read(Dp8390*, void*, ulong, ulong);
extern void dp8390getea(Ether*, uchar*);
extern void dp8390setea(Ether*);

⟨macro regr 177d⟩
⟨macro regw 177c⟩

⟨function rdread 239b⟩

⟨function rdwrite 180a⟩
```

I.6.4 kernel/network/386/ether8390.c

⟨*enum* _anon_ (kernel/network/386/ether8390.c)0 239d⟩≡ (242b)

```
enum {      /* NIC core registers */
    Cr  = 0x00,  /* command register, all pages */

                 /* Page 0, read */
    Clda0  = 0x01,  /* current local DMA address 0 */
    Clda1  = 0x02,  /* current local DMA address 1 */
```

```
    Bnry  = 0x03,  /* boundary pointer (R/W) */
    Tsr  = 0x04,  /* transmit status register */
    Ncr  = 0x05,  /* number of collisions register */
    Fifo  = 0x06,  /* FIFO */
    Isr  = 0x07,  /* interrupt status register (R/W) */
    Crda0  = 0x08,  /* current remote DMA address 0 */
    Crda1  = 0x09,  /* current remote DMA address 1 */
    Rsr  = 0x0C,  /* receive status register */
    Ref0  = 0x0D,  /* frame alignment errors */
    Ref1  = 0x0E,  /* CRC errors */
    Ref2  = 0x0F,  /* missed packet errors */

                   /* Page 0, write */
    Pstart  = 0x01,  /* page start register */
    Pstop  = 0x02,  /* page stop register */
    Tpsr  = 0x04,  /* transmit page start address */
    Tbcr0  = 0x05,  /* transmit byte count register 0 */
    Tbcr1  = 0x06,  /* transmit byte count register 1 */
    Rsar0  = 0x08,  /* remote start address register 0 */
    Rsar1  = 0x09,  /* remote start address register 1 */
    Rbcr0  = 0x0A,  /* remote byte count register 0 */
    Rbcr1  = 0x0B,  /* remote byte count register 1 */
    Rcr  = 0x0C,  /* receive configuration register */
    Tcr  = 0x0D,  /* transmit configuration register */
    Dcr  = 0x0E,  /* data configuration register */
    Imr  = 0x0F,  /* interrupt mask */

                   /* Page 1, read/write */
    Par0  = 0x01,  /* physical address register 0 */
    Curr  = 0x07,  /* current page register */
    Mar0  = 0x08,  /* multicast address register 0 */
};
```

⟨*enum* _anon_ (kernel/network/386/ether8390.c)1 240a⟩≡ (242b)

```
enum {     /* Cr */
    Stp  = 0x01,  /* stop */
    Sta  = 0x02,  /* start */
    Txp  = 0x04,  /* transmit packet */
    Rd0  = 0x08,  /* remote DMA command */
    Rd1  = 0x10,
    Rd2  = 0x20,
    RdREAD  = Rd0,  /* remote read */
    RdWRITE  = Rd1,  /* remote write */
    RdSEND  = Rd1|Rd0, /* send packet */
    RdABORT  = Rd2,  /* abort/complete remote DMA */
    Ps0  = 0x40,  /* page select */
    Ps1  = 0x80,
    Page0  = 0x00,
    Page1  = Ps0,
    Page2  = Ps1,
};
```

⟨*enum* _anon_ (kernel/network/386/ether8390.c)2 240b⟩≡ (242b)

```
enum {     /* Isr/Imr */
    Prx  = 0x01,  /* packet received */
    Ptx  = 0x02,  /* packet transmitted */
    Rxe  = 0x04,  /* receive error */
    Txe  = 0x08,  /* transmit error */
    Ovw  = 0x10,  /* overwrite warning */
    Cnt  = 0x20,  /* counter overflow */
```

```
    Rdc  = 0x40,  /* remote DMA complete */
    Rst  = 0x80,  /* reset status */
};
```

⟨*enum* _anon_ (kernel/network/386/ether8390.c)3 241a⟩≡ (242b)

```
enum {     /* Dcr */
    Wts  = 0x01,  /* word transfer select */
    Bos  = 0x02,  /* byte order select */
    Las  = 0x04,  /* long address select */
    Ls  = 0x08,  /* loopback select */
    Arm  = 0x10,  /* auto-initialise remote */
    Ft0  = 0x20,  /* FIFO threshold select */
    Ft1  = 0x40,
    Ft1WORD  = 0x00,
    Ft2WORD  = Ft0,
    Ft4WORD  = Ft1,
    Ft6WORD  = Ft1|Ft0,
};
```

⟨*enum* _anon_ (kernel/network/386/ether8390.c)4 241b⟩≡ (242b)

```
enum {     /* Tcr */
    Crc  = 0x01,  /* inhibit CRC */
    Lb0  = 0x02,  /* encoded loopback control */
    Lb1  = 0x04,
    LpbkNORMAL = 0x00,  /* normal operation */
    LpbkNIC  = Lb0,  /* internal NIC module loopback */
    LpbkENDEC = Lb1,  /* internal ENDEC module loopback */
    LpbkEXTERNAL = Lb1|Lb0, /* external loopback */
    Atd  = 0x08,  /* auto transmit disable */
    Ofst  = 0x10,  /* collision offset enable */
};
```

⟨*enum* _anon_ (kernel/network/386/ether8390.c)5 241c⟩≡ (242b)

```
enum {     /* Tsr */
    Ptxok  = 0x01,  /* packet transmitted */
    Col  = 0x04,  /* transmit collided */
    Abt  = 0x08,  /* tranmit aborted */
    Crs  = 0x10,  /* carrier sense lost */
    Fu  = 0x20,  /* FIFO underrun */
    Cdh  = 0x40,  /* CD heartbeat */
    Owc  = 0x80,  /* out of window collision */
};
```

⟨*enum* _anon_ (kernel/network/386/ether8390.c)6 241d⟩≡ (242b)

```
enum {     /* Rcr */
    Sep  = 0x01,  /* save errored packets */
    Ar  = 0x02,  /* accept runt packets */
    Ab  = 0x04,  /* accept broadcast */
    Am  = 0x08,  /* accept multicast */
    Pro  = 0x10,  /* promiscuous physical */
    Mon  = 0x20,  /* monitor mode */
};
```

⟨*enum* _anon_ (kernel/network/386/ether8390.c)7 241e⟩≡ (242b)

```
enum {     /* Rsr */
    Prxok  = 0x01,  /* packet received intact */
    Crce  = 0x02,  /* CRC error */
    Fae  = 0x04,  /* frame alignment error */
    Fo  = 0x08,  /* FIFO overrun */
    Mpa  = 0x10,  /* missed packet */
```

```
    Phy  = 0x20,  /* physical/multicast address */
    Dis  = 0x40,  /* receiver disabled */
    Dfr  = 0x80,  /* deferring */
};
```

⟨*struct* Hdr*(*(kernel/network/386/ether8390.c)*)* 242a⟩≡ (242b)

```
struct Hdr {
    uchar status;
    uchar next;
    uchar len0;
    uchar len1;
};
```

⟨kernel/network/386/ether8390.c 242b⟩≡

```
/*
 * National Semiconductor DP8390 and clone
 * Network Interface Controller.
 */
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "io.h"
#include "../port/error.h"
#include "../port/netif.h"
#include "../port/etherif.h"

#include "ether8390.h"
```

⟨*enum* _anon_ (kernel/network/386/ether8390.c)0 239d⟩

⟨*enum* _anon_ (kernel/network/386/ether8390.c)1 240a⟩

⟨*enum* _anon_ (kernel/network/386/ether8390.c)2 240b⟩

⟨*enum* _anon_ (kernel/network/386/ether8390.c)3 241a⟩

⟨*enum* _anon_ (kernel/network/386/ether8390.c)4 241b⟩

⟨*enum* _anon_ (kernel/network/386/ether8390.c)5 241c⟩

⟨*enum* _anon_ (kernel/network/386/ether8390.c)6 241d⟩

⟨*enum* _anon_ (kernel/network/386/ether8390.c)7 241e⟩

```
typedef struct Hdr Hdr;
```

⟨*struct* Hdr*(*(kernel/network/386/ether8390.c)*)* 242a⟩

⟨*function* dp8390setea 176a⟩

⟨*function* _dp8390read 183b⟩

⟨*function* dp8390read 174a⟩

⟨*function* dp8390write 179⟩

⟨*function* ringinit 175⟩

⟨*function* getcurr 183a⟩

⟨*function* receive*(*(kernel/network/386/ether8390.c)*)* 181⟩

⟨*function* txstart 178b⟩

⟨*function* transmit*(*(kernel/network/386/ether8390.c)*)* 178a⟩

⟨*function* overflow 184⟩

⟨*function* interrupt*(*(kernel/network/386/ether8390.c)*)* 180b⟩

⟨*global* allmar 185d⟩

⟨*function* setfilter 186a⟩

⟨*function* promiscuous 186b⟩

⟨*function* setbit 185e⟩

⟨*global* reverse 185b⟩

⟨*function* multicast 185c⟩

⟨*function* attach 177b⟩

⟨*function* disable 176b⟩

⟨*function* shutdown 177a⟩

⟨*function* dp8390reset 174b⟩

I.6.5 kernel/network/386/ether2000.c

⟨*enum* _anon_ (kernel/network/386/ether2000.c) 243a⟩≡ (243b)

```
enum {
    Data  = 0x10,  /* offset from I/O base of data port */
    Reset  = 0x1F,  /* offset from I/O base of reset port */
};
```

⟨kernel/network/386/ether2000.c 243b⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "../port/error.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"

#include "io.h"
#include "../port/netif.h"
#include "../port/etherif.h"

#include "ether8390.h"

/*
 * Driver written for the 'Notebook Computer Ethernet LAN Adapter',
 * a plug-in to the bus-slot on the rear of the Gateway NOMAD 425DXL
```

```
 * laptop. The manual says NE2000 compatible.
 * The interface appears to be pretty well described in the National
 * Semiconductor Local Area Network Databook (1992) as one of the
 * AT evaluation cards.
 *
 * The NE2000 is really just a DP8390[12] plus a data port
 * and a reset port.
 */
```

⟨*enum* _anon_ (kernel/network/386/ether2000.c) 243a⟩

```
typedef struct Ctlr Ctlr;
```
⟨*struct* Ctlr*(*(kernel/network/386/ether2000.c)*)* 186c⟩

⟨*global* ctlrhead*(*(kernel/network/386/ether2000.c)*)* 187a⟩
⟨*global* ctlrtail*(*(kernel/network/386/ether2000.c)*)* 187b⟩

⟨*global* ne2000pci 187d⟩

⟨*function* ne2000match 188⟩

⟨*function* ne2000pnp 187c⟩

⟨*function* ne2000reset 172c⟩

⟨*function* ether2000link 172b⟩

I.6.6 kernel/network/386/etherigbe.c

⟨*enum* _anon_ (kernel/network/386/etherigbe.c) 244a⟩≡ (279b)

```
enum {
    i82542      = (0x1000<<16)|0x8086,
    i82543gc    = (0x1004<<16)|0x8086,
    i82544ei    = (0x1008<<16)|0x8086,
    i82544eif   = (0x1009<<16)|0x8086,
    i82544gc    = (0x100d<<16)|0x8086,
    i82540em    = (0x100E<<16)|0x8086,
    i82540eplp  = (0x101E<<16)|0x8086,
    i82545em    = (0x100F<<16)|0x8086,
    i82545gmc   = (0x1026<<16)|0x8086,
    i82547ei    = (0x1019<<16)|0x8086,
    i82547gi    = (0x1075<<16)|0x8086,
    i82541ei    = (0x1013<<16)|0x8086,
    i82541gi    = (0x1076<<16)|0x8086,
    i82541gi2   = (0x1077<<16)|0x8086,
    i82541pi    = (0x107c<<16)|0x8086,
    i82546gb    = (0x1079<<16)|0x8086,
    i82546eb    = (0x1010<<16)|0x8086,
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)2 244b⟩≡ (279b)

```
enum {
    Ctrl        = 0x00000000,   /* Device Control */
    Ctrldup     = 0x00000004,   /* Device Control Duplicate */
    Status      = 0x00000008,   /* Device Status */
    Eecd        = 0x00000010,   /* EEPROM/Flash Control/Data */
    Ctrlext     = 0x00000018,   /* Extended Device Control */
    Mdic        = 0x00000020,   /* MDI Control */
    Fcal        = 0x00000028,   /* Flow Control Address Low */
```

```
    Fcah        = 0x0000002C,   /* Flow Control Address High */
    Fct     = 0x00000030,   /* Flow Control Type */
    Icr     = 0x000000C0,   /* Interrupt Cause Read */
    Ics     = 0x000000C8,   /* Interrupt Cause Set */
    Ims     = 0x000000D0,   /* Interrupt Mask Set/Read */
    Imc     = 0x000000D8,   /* Interrupt mask Clear */
    Rctl        = 0x00000100,   /* Receive Control */
    Fcttv       = 0x00000170,   /* Flow Control Transmit Timer Value */
    Txcw        = 0x00000178,   /* Transmit Configuration Word */
    Rxcw        = 0x00000180,   /* Receive Configuration Word */
    /* on the oldest cards (8254[23]), the Mta register is at 0x200 */
    Tctl        = 0x00000400,   /* Transmit Control */
    Tipg        = 0x00000410,   /* Transmit IPG */
    Tbt     = 0x00000448,   /* Transmit Burst Timer */
    Ait     = 0x00000458,   /* Adaptive IFS Throttle */
    Fcrtl       = 0x00002160,   /* Flow Control RX Threshold Low */
    Fcrth       = 0x00002168,   /* Flow Control Rx Threshold High */
    Rdfh        = 0x00002410,   /* Receive data fifo head */
    Rdft        = 0x00002418,   /* Receive data fifo tail */
    Rdfhs       = 0x00002420,   /* Receive data fifo head saved */
    Rdfts       = 0x00002428,   /* Receive data fifo tail saved */
    Rdfpc       = 0x00002430,   /* Receive data fifo packet count */
    Rdbal       = 0x00002800,   /* Rd Base Address Low */
    Rdbah       = 0x00002804,   /* Rd Base Address High */
    Rdlen       = 0x00002808,   /* Receive Descriptor Length */
    Rdh     = 0x00002810,   /* Receive Descriptor Head */
    Rdt     = 0x00002818,   /* Receive Descriptor Tail */
    Rdtr        = 0x00002820,   /* Receive Descriptor Timer Ring */
    Rxdctl      = 0x00002828,   /* Receive Descriptor Control */
    Radv        = 0x0000282C,   /* Receive Interrupt Absolute Delay Timer */
    Txdmac      = 0x00003000,   /* Transfer DMA Control */
    Ett     = 0x00003008,   /* Early Transmit Control */
    Tdfh        = 0x00003410,   /* Transmit data fifo head */
    Tdft        = 0x00003418,   /* Transmit data fifo tail */
    Tdfhs       = 0x00003420,   /* Transmit data Fifo Head saved */
    Tdfts       = 0x00003428,   /* Transmit data fifo tail saved */
    Tdfpc       = 0x00003430,   /* Trasnmit data Fifo packet count */
    Tdbal       = 0x00003800,   /* Td Base Address Low */
    Tdbah       = 0x00003804,   /* Td Base Address High */
    Tdlen       = 0x00003808,   /* Transmit Descriptor Length */
    Tdh     = 0x00003810,   /* Transmit Descriptor Head */
    Tdt     = 0x00003818,   /* Transmit Descriptor Tail */
    Tidv        = 0x00003820,   /* Transmit Interrupt Delay Value */
    Txdctl      = 0x00003828,   /* Transmit Descriptor Control */
    Tadv        = 0x0000382C,   /* Transmit Interrupt Absolute Delay Timer */

    Statistics  = 0x00004000,   /* Start of Statistics Area */
    Gorcl       = 0x88/4,   /* Good Octets Received Count */
    Gotcl       = 0x90/4,   /* Good Octets Transmitted Count */
    Torl        = 0xC0/4,   /* Total Octets Received */
    Totl        = 0xC8/4,   /* Total Octets Transmitted */
    Nstatistics = 64,

    Rxcsum      = 0x00005000,   /* Receive Checksum Control */
    Mta     = 0x00005200,   /* Multicast Table Array */
    Ral     = 0x00005400,   /* Receive Address Low */
    Rah     = 0x00005404,   /* Receive Address High */
    Manc        = 0x00005820,   /* Management Control */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)3 246a⟩≡ (279b)

```
enum {                  /* Ctrl */
    Bem     = 0x00000002,   /* Big Endian Mode */
    Prior       = 0x00000004,   /* Priority on the PCI bus */
    Lrst        = 0x00000008,   /* Link Reset */
    Asde        = 0x00000020,   /* Auto-Speed Detection Enable */
    Slu     = 0x00000040,   /* Set Link Up */
    Ilos        = 0x00000080,   /* Invert Loss of Signal (LOS) */
    SspeedMASK  = 0x00000300,   /* Speed Selection */
    SspeedSHIFT = 8,
    Sspeed10    = 0x00000000,   /* 10Mb/s */
    Sspeed100   = 0x00000100,   /* 100Mb/s */
    Sspeed1000  = 0x00000200,   /* 1000Mb/s */
    Frcspd      = 0x00000800,   /* Force Speed */
    Frcdplx     = 0x00001000,   /* Force Duplex */
    SwdpinsloMASK   = 0x003C0000,   /* Software Defined Pins - lo nibble */
    SwdpinsloSHIFT  = 18,
    SwdpioloMASK    = 0x03C00000,   /* Software Defined Pins - I or O */
    SwdpioloSHIFT   = 22,
    Devrst      = 0x04000000,   /* Device Reset */
    Rfce        = 0x08000000,   /* Receive Flow Control Enable */
    Tfce        = 0x10000000,   /* Transmit Flow Control Enable */
    Vme     = 0x40000000,   /* VLAN Mode Enable */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)4 246b⟩≡ (279b)

```
/*
 * can't find Tckok nor Rbcok in any Intel docs,
 * but even 82543gc docs define Lanid.
 */
enum {                  /* Status */
    Lu      = 0x00000002,   /* Link Up */
    Lanid       = 0x0000000C,   /* mask for Lan ID. (function id) */
//  Tckok       = 0x00000004,   /* Transmit clock is running */
//  Rbcok       = 0x00000008,   /* Receive clock is running */
    Txoff       = 0x00000010,   /* Transmission Paused */
    Tbimode     = 0x00000020,   /* TBI Mode Indication */
    LspeedMASK  = 0x000000C0,   /* Link Speed Setting */
    LspeedSHIFT = 6,
    Lspeed10    = 0x00000000,   /* 10Mb/s */
    Lspeed100   = 0x00000040,   /* 100Mb/s */
    Lspeed1000  = 0x00000080,   /* 1000Mb/s */
    Mtxckok     = 0x00000400,   /* MTX clock is running */
    Pci66       = 0x00000800,   /* PCI Bus speed indication */
    Bus64       = 0x00001000,   /* PCI Bus width indication */
    Pcixmode    = 0x00002000,   /* PCI-X mode */
    PcixspeedMASK   = 0x0000C000,   /* PCI-X bus speed */
    PcixspeedSHIFT  = 14,
    Pcix66      = 0x00000000,   /* 50-66MHz */
    Pcix100     = 0x00004000,   /* 66-100MHz */
    Pcix133     = 0x00008000,   /* 100-133MHz */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)5 246c⟩≡ (279b)

```
enum {                  /* Ctrl and Status */
    Fd      = 0x00000001,   /* Full-Duplex */
    AsdvMASK    = 0x00000300,
    AsdvSHIFT   = 8,
    Asdv10      = 0x00000000,   /* 10Mb/s */
    Asdv100     = 0x00000100,   /* 100Mb/s */
```

```
    Asdv1000    = 0x00000200,   /* 1000Mb/s */
  };
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)6 247a⟩≡ (279b)

```
  enum {                  /* Eecd */
      Sk      = 0x00000001,   /* Clock input to the EEPROM */
      Cs      = 0x00000002,   /* Chip Select */
      Di      = 0x00000004,   /* Data Input to the EEPROM */
      Do      = 0x00000008,   /* Data Output from the EEPROM */
      Areq        = 0x00000040,   /* EEPROM Access Request */
      Agnt        = 0x00000080,   /* EEPROM Access Grant */
      Eepresent   = 0x00000100,   /* EEPROM Present */
      Eesz256     = 0x00000200,   /* EEPROM is 256 words not 64 */
      Eeszaddr    = 0x00000400,   /* EEPROM size for 8254[17] */
      Spi     = 0x00002000,   /* EEPROM is SPI not Microwire */
  };
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)7 247b⟩≡ (279b)

```
  enum {                  /* Ctrlext */
      Gpien       = 0x0000000F,   /* General Purpose Interrupt Enables */
      SwdpinshiMASK   = 0x000000F0,   /* Software Defined Pins - hi nibble */
      SwdpinshiSHIFT  = 4,
      SwdpiohiMASK    = 0x00000F00,   /* Software Defined Pins - I or O */
      SwdpiohiSHIFT   = 8,
      Asdchk      = 0x00001000,   /* ASD Check */
      Eerst       = 0x00002000,   /* EEPROM Reset */
      Ips     = 0x00004000,   /* Invert Power State */
      Spdbyps     = 0x00008000,   /* Speed Select Bypass */
  };
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)8 247c⟩≡ (279b)

```
  enum {                  /* EEPROM content offsets */
      Ea      = 0x00,     /* Ethernet Address */
      Cf      = 0x03,     /* Compatibility Field */
      Pba     = 0x08,     /* Printed Board Assembly number */
      Icw1        = 0x0A,     /* Initialization Control Word 1 */
      Sid     = 0x0B,     /* Subsystem ID */
      Svid        = 0x0C,     /* Subsystem Vendor ID */
      Did     = 0x0D,     /* Device ID */
      Vid     = 0x0E,     /* Vendor ID */
      Icw2        = 0x0F,     /* Initialization Control Word 2 */
  };
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)9 247d⟩≡ (279b)

```
  enum {                  /* Mdic */
      MDIdMASK    = 0x0000FFFF,   /* Data */
      MDIdSHIFT   = 0,
      MDIrMASK    = 0x001F0000,   /* PHY Register Address */
      MDIrSHIFT   = 16,
      MDIpMASK    = 0x03E00000,   /* PHY Address */
      MDIpSHIFT   = 21,
      MDIwop      = 0x04000000,   /* Write Operation */
      MDIrop      = 0x08000000,   /* Read Operation */
      MDIready    = 0x10000000,   /* End of Transaction */
      MDIie       = 0x20000000,   /* Interrupt Enable */
      MDIe        = 0x40000000,   /* Error */
  };
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)10 248a⟩≡ (279b)

```
enum {                  /* Icr, Ics, Ims, Imc */
    Txdw        = 0x00000001,   /* Transmit Descriptor Written Back */
    Txqe        = 0x00000002,   /* Transmit Queue Empty */
    Lsc     = 0x00000004,   /* Link Status Change */
    Rxseq       = 0x00000008,   /* Receive Sequence Error */
    Rxdmt0      = 0x00000010,   /* Rd Minimum Threshold Reached */
    Rxo     = 0x00000040,   /* Receiver Overrun */
    Rxt0        = 0x00000080,   /* Receiver Timer Interrupt */
    Mdac        = 0x00000200,   /* MDIO Access Completed */
    Rxcfg       = 0x00000400,   /* Receiving /C/ ordered sets */
    Gpi0        = 0x00000800,   /* General Purpose Interrupts */
    Gpi1        = 0x00001000,
    Gpi2        = 0x00002000,
    Gpi3        = 0x00004000,
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)11 248b⟩≡ (279b)

```
/*
 * The Mdic register isn't implemented on the 82543GC,
 * the software defined pins are used instead.
 * These definitions work for the Intel PRO/1000 T Server Adapter.
 * The direction pin bits are read from the EEPROM.
 */
enum {
    Mdd     = ((1<<2)<<SwdpinsloSHIFT), /* data */
    Mddo        = ((1<<2)<<SwdpioloSHIFT),  /* pin direction */
    Mdc     = ((1<<3)<<SwdpinsloSHIFT), /* clock */
    Mdco        = ((1<<3)<<SwdpioloSHIFT),  /* pin direction */
    Mdr     = ((1<<0)<<SwdpinshiSHIFT), /* reset */
    Mdro        = ((1<<0)<<SwdpiohiSHIFT),  /* pin direction */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)12 248c⟩≡ (279b)

```
enum {                  /* Txcw */
    TxcwFd      = 0x00000020,   /* Full Duplex */
    TxcwHd      = 0x00000040,   /* Half Duplex */
    TxcwPauseMASK   = 0x00000180,   /* Pause */
    TxcwPauseSHIFT  = 7,
    TxcwPs      = (1<<TxcwPauseSHIFT),  /* Pause Supported */
    TxcwAs      = (2<<TxcwPauseSHIFT),  /* Asymmetric FC desired */
    TxcwRfiMASK = 0x00003000,   /* Remote Fault Indication */
    TxcwRfiSHIFT    = 12,
    TxcwNpr     = 0x00008000,   /* Next Page Request */
    TxcwConfig  = 0x40000000,   /* Transmit COnfig Control */
    TxcwAne     = 0x80000000,   /* Auto-Negotiation Enable */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)13 248d⟩≡ (279b)

```
enum {                  /* Rxcw */
    Rxword      = 0x0000FFFF,   /* Data from auto-negotiation process */
    Rxnocarrier = 0x04000000,   /* Carrier Sense indication */
    Rxinvalid   = 0x08000000,   /* Invalid Symbol during configuration */
    Rxchange    = 0x10000000,   /* Change to the Rxword indication */
    Rxconfig    = 0x20000000,   /* /C/ order set reception indication */
    Rxsync      = 0x40000000,   /* Lost bit synchronization indication */
    Anc     = 0x80000000,   /* Auto Negotiation Complete */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)14 249a⟩≡ (279b)

```
enum {                  /* Rctl */
    Rrst        = 0x00000001,   /* Receiver Software Reset */
    Ren     = 0x00000002,   /* Receiver Enable */
    Sbp     = 0x00000004,   /* Store Bad Packets */
    Upe     = 0x00000008,   /* Unicast Promiscuous Enable */
    Mpe     = 0x00000010,   /* Multicast Promiscuous Enable */
    Lpe     = 0x00000020,   /* Long Packet Reception Enable */
    LbmMASK     = 0x000000C0,   /* Loopback Mode */
    LbmOFF      = 0x00000000,   /* No Loopback */
    LbmTBI      = 0x00000040,   /* TBI Loopback */
    LbmMII      = 0x00000080,   /* GMII/MII Loopback */
    LbmXCVR     = 0x000000C0,   /* Transceiver Loopback */
    RdtmsMASK   = 0x00000300,   /* Rd Minimum Threshold Size */
    RdtmsHALF   = 0x00000000,   /* Threshold is 1/2 Rdlen */
    RdtmsQUARTER    = 0x00000100,   /* Threshold is 1/4 Rdlen */
    RdtmsEIGHTH = 0x00000200,   /* Threshold is 1/8 Rdlen */
    MoMASK      = 0x00003000,   /* Multicast Offset */
    Mo47b36     = 0x00000000,   /* bits [47:36] of received address */
    Mo46b35     = 0x00001000,   /* bits [46:35] of received address */
    Mo45b34     = 0x00002000,   /* bits [45:34] of received address */
    Mo43b32     = 0x00003000,   /* bits [43:32] of received address */
    Bam     = 0x00008000,   /* Broadcast Accept Mode */
    BsizeMASK   = 0x00030000,   /* Receive Buffer Size */
    Bsize2048   = 0x00000000,   /* Bsex = 0 */
    Bsize1024   = 0x00010000,   /* Bsex = 0 */
    Bsize512    = 0x00020000,   /* Bsex = 0 */
    Bsize256    = 0x00030000,   /* Bsex = 0 */
    Bsize16384  = 0x00010000,   /* Bsex = 1 */
    Vfe     = 0x00040000,   /* VLAN Filter Enable */
    Cfien       = 0x00080000,   /* Canonical Form Indicator Enable */
    Cfi     = 0x00100000,   /* Canonical Form Indicator value */
    Dpf     = 0x00400000,   /* Discard Pause Frames */
    Pmcf        = 0x00800000,   /* Pass MAC Control Frames */
    Bsex        = 0x02000000,   /* Buffer Size Extension */
    Secrc       = 0x04000000,   /* Strip CRC from incoming packet */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)15 249b⟩≡ (279b)

```
enum {                  /* Tctl */
    Trst        = 0x00000001,   /* Transmitter Software Reset */
    Ten     = 0x00000002,   /* Transmit Enable */
    Psp     = 0x00000008,   /* Pad Short Packets */
    CtMASK      = 0x00000FF0,   /* Collision Threshold */
    CtSHIFT     = 4,
    ColdMASK    = 0x003FF000,   /* Collision Distance */
    ColdSHIFT   = 12,
    Swxoff      = 0x00400000,   /* Sofware XOFF Transmission */
    Pbe     = 0x00800000,   /* Packet Burst Enable */
    Rtlc        = 0x01000000,   /* Re-transmit on Late Collision */
    Nrtu        = 0x02000000,   /* No Re-transmit on Underrrun */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)16 249c⟩≡ (279b)

```
enum {                  /* [RT]xdctl */
    PthreshMASK = 0x0000003F,   /* Prefetch Threshold */
    PthreshSHIFT    = 0,
    HthreshMASK = 0x00003F00,   /* Host Threshold */
    HthreshSHIFT    = 8,
    WthreshMASK = 0x003F0000,   /* Writeback Threshold */
```

```
    WthreshSHIFT    = 16,
    Gran        = 0x01000000,   /* Granularity */
    LthreshMASK = 0xFE000000,   /* Low Threshold */
    LthreshSHIFT    = 25,
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)17 250a⟩≡ (279b)

```
enum {                  /* Rxcsum */
    PcssMASK    = 0x000000FF,   /* Packet Checksum Start */
    PcssSHIFT   = 0,
    Ipofl       = 0x00000100,   /* IP Checksum Off-load Enable */
    Tuofl       = 0x00000200,   /* TCP/UDP Checksum Off-load Enable */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)18 250b⟩≡ (279b)

```
enum {                  /* Manc */
    Arpen       = 0x00002000,   /* Enable ARP Request Filtering */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)19 250c⟩≡ (279b)

```
enum {                  /* Receive Delay Timer Ring */
    DelayMASK   = 0x0000FFFF,   /* delay timer in 1.024nS increments */
    DelaySHIFT  = 0,
    Fpd     = 0x80000000,   /* Flush partial Descriptor Block */
};
```

⟨*struct* Rd 250d⟩≡ (279b)

```
typedef struct Rd {         /* Receive Descriptor */
    uint    addr[2];
    ushort  length;
    ushort  checksum;
    uchar   status;
    uchar   errors;
    ushort  special;
} Rd;
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)20 250e⟩≡ (279b)

```
enum {                  /* Rd status */
    Rdd     = 0x01,     /* Descriptor Done */
    Reop        = 0x02,     /* End of Packet */
    Ixsm        = 0x04,     /* Ignore Checksum Indication */
    Vp      = 0x08,     /* Packet is 802.1Q (matched VET) */
    Tcpcs       = 0x20,     /* TCP Checksum Calculated on Packet */
    Ipcs        = 0x40,     /* IP Checksum Calculated on Packet */
    Pif     = 0x80,     /* Passed in-exact filter */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)21 250f⟩≡ (279b)

```
enum {                  /* Rd errors */
    Ce      = 0x01,     /* CRC Error or Alignment Error */
    Se      = 0x02,     /* Symbol Error */
    Seq     = 0x04,     /* Sequence Error */
    Cxe     = 0x10,     /* Carrier Extension Error */
    Tcpe        = 0x20,     /* TCP/UDP Checksum Error */
    Ipe     = 0x40,     /* IP Checksum Error */
    Rxe     = 0x80,     /* RX Data Error */
};
```

⟨*struct* Td 251a⟩≡ (279b)

```
struct Td {             /* Transmit Descriptor */
    union {
        uint    addr[2];    /* Data */
        struct {        /* Context */
            uchar   ipcss;
            uchar   ipcso;
            ushort  ipcse;
            uchar   tucss;
            uchar   tucso;
            ushort  tucse;
        };
    };
    uint    control;
    uint    status;
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)22 251b⟩≡ (279b)

```
enum {                  /* Td control */
    LenMASK     = 0x000FFFFF,   /* Data/Packet Length Field */
    LenSHIFT    = 0,
    DtypeCD     = 0x00000000,   /* Data Type 'Context Descriptor' */
    DtypeDD     = 0x00100000,   /* Data Type 'Data Descriptor' */
    PtypeTCP    = 0x01000000,   /* TCP/UDP Packet Type (CD) */
    Teop        = 0x01000000,   /* End of Packet (DD) */
    PtypeIP     = 0x02000000,   /* IP Packet Type (CD) */
    Ifcs        = 0x02000000,   /* Insert FCS (DD) */
    Tse     = 0x04000000,   /* TCP Segmentation Enable */
    Rs      = 0x08000000,   /* Report Status */
    Rps     = 0x10000000,   /* Report Status Sent */
    Dext        = 0x20000000,   /* Descriptor Extension */
    Vle     = 0x40000000,   /* VLAN Packet Enable */
    Ide     = 0x80000000,   /* Interrupt Delay Enable */
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)23 251c⟩≡ (279b)

```
enum {                  /* Td status */
    Tdd     = 0x00000001,   /* Descriptor Done */
    Ec      = 0x00000002,   /* Excess Collisions */
    Lc      = 0x00000004,   /* Late Collision */
    Tu      = 0x00000008,   /* Transmit Underrun */
    Iixsm       = 0x00000100,   /* Insert IP Checksum */
    Itxsm       = 0x00000200,   /* Insert TCP/UDP Checksum */
    HdrlenMASK  = 0x0000FF00,   /* Header Length (Tse) */
    HdrlenSHIFT = 8,
    VlanMASK    = 0x0FFF0000,   /* VLAN Identifier */
    VlanSHIFT   = 16,
    Tcfi        = 0x10000000,   /* Canonical Form Indicator */
    PriMASK     = 0xE0000000,   /* User Priority */
    PriSHIFT    = 29,
    MssMASK     = 0xFFFF0000,   /* Maximum Segment Size (Tse) */
    MssSHIFT    = 16,
};
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)24 251d⟩≡ (279b)

```
enum {
    Nrd     = 256,      /* multiple of 8 */
    Ntd     = 64,       /* multiple of 8 */
    Nrb     = 1024,     /* private receive buffers per Ctlr */
    Rbsz        = 2048,
};
```

⟨*struct* CtlrEtherIgbe 252⟩≡ (279b)

```
struct CtlrEtherIgbe {
    int port;
    Pcidev* pcidev;
    CtlrEtherIgbe*  next;
    Ether*  edev;
    int active;
    int started;
    int id;
    int cls;
    ushort  eeprom[0x40];

    QLock   alock;          /* attach */
    void*   alloc;          /* receive/transmit descriptors */
    int nrd;
    int ntd;
    int nrb;            /* # bufs this Ctlr has in the pool */

    int*    nic;
    Lock    imlock;
    int im;         /* interrupt mask */

    Mii*    mii;
    Rendez  lrendez;
    int lim;

    int link;

    QLock   slock;
    uint    statistics[Nstatistics];
    uint    lsleep;
    uint    lintr;
    uint    rsleep;
    uint    rintr;
    uint    txdw;
    uint    tintr;
    uint    ixsm;
    uint    ipcs;
    uint    tcpcs;

    eaddr   ra;       /* receive address */
    ulong   mta[128];       /* multicast table array */

    Rendez  rrendez;
    int rim;
    int rdfree;         /* rx descriptors awaiting packets */
    Rd* rdba;           /* receive descriptor base address */
    Block** rb;         /* receive buffers */
    int rdh;            /* receive descriptor head */
    int rdt;            /* receive descriptor tail */
    int rdtr;           /* receive delay timer ring value */

    Lock    tlock;
    int tdfree;
    Td* tdba;           /* transmit descriptor base address */
    Block** tb;         /* transmit buffers */
    int tdh;            /* transmit descriptor head */
    int tdt;            /* transmit descriptor tail */

    int txcw;
```

```
    int fcrtl;
    int fcrth;
};
```

⟨*macro* csr32r 253a⟩≡ (279b)

```
#define csr32r(c, r)    (*((c)->nic+((r)/4)))
```

⟨*macro* csr32w 253b⟩≡ (279b)

```
#define csr32w(c, r, v) (*((c)->nic+((r)/4)) = (v))
```

⟨*global* igbectlrhead 253c⟩≡ (279b)

```
static CtlrEtherIgbe* igbectlrhead;
```

⟨*global* igbectlrtail 253d⟩≡ (279b)

```
static CtlrEtherIgbe* igbectlrtail;
```

⟨*global* igberblock 253e⟩≡ (279b)

```
static Lock igberblock;     /* free receive Blocks */
```

⟨*global* igberbpool 253f⟩≡ (279b)

```
static Block* igberbpool;   /* receive Blocks for all igbe controllers */
```

⟨*global* statistics 253g⟩≡ (279b)

```
static char* statistics[Nstatistics] = {
    "CRC Error",
    "Alignment Error",
    "Symbol Error",
    "RX Error",
    "Missed Packets",
    "Single Collision",
    "Excessive Collisions",
    "Multiple Collision",
    "Late Collisions",
    nil,
    "Collision",
    "Transmit Underrun",
    "Defer",
    "Transmit - No CRS",
    "Sequence Error",
    "Carrier Extension Error",
    "Receive Error Length",
    nil,
    "XON Received",
    "XON Transmitted",
    "XOFF Received",
    "XOFF Transmitted",
    "FC Received Unsupported",
    "Packets Received (64 Bytes)",
    "Packets Received (65-127 Bytes)",
    "Packets Received (128-255 Bytes)",
    "Packets Received (256-511 Bytes)",
    "Packets Received (512-1023 Bytes)",
    "Packets Received (1024-1522 Bytes)",
    "Good Packets Received",
    "Broadcast Packets Received",
    "Multicast Packets Received",
    "Good Packets Transmitted",
    nil,
    "Good Octets Received",
    nil,
```

```
    "Good Octets Transmitted",
    nil,
    nil,
    nil,
    "Receive No Buffers",
    "Receive Undersize",
    "Receive Fragment",
    "Receive Oversize",
    "Receive Jabber",
    nil,
    nil,
    nil,
    "Total Octets Received",
    nil,
    "Total Octets Transmitted",
    nil,
    "Total Packets Received",
    "Total Packets Transmitted",
    "Packets Transmitted (64 Bytes)",
    "Packets Transmitted (65-127 Bytes)",
    "Packets Transmitted (128-255 Bytes)",
    "Packets Transmitted (256-511 Bytes)",
    "Packets Transmitted (512-1023 Bytes)",
    "Packets Transmitted (1024-1522 Bytes)",
    "Multicast Packets Transmitted",
    "Broadcast Packets Transmitted",
    "TCP Segmentation Context Transmitted",
    "TCP Segmentation Context Fail",
};
```

⟨*function* igbeifstat 254⟩≡ (279b)

```
static long
igbeifstat(Ether* edev, void* a, long n, ulong offset)
{
    CtlrEtherIgbe *ctlr;
    char *p, *s;
    int i, l, r;
    uvlong tuvl, ruvl;

    ctlr = edev->ctlr;
    qlock(&ctlr->slock);
    p = malloc(READSTR);
    if(p == nil) {
        qunlock(&ctlr->slock);
        error(Enomem);
    }
    l = 0;
    for(i = 0; i < Nstatistics; i++){
        r = csr32r(ctlr, Statistics+i*4);
        if((s = statistics[i]) == nil)
            continue;
        switch(i){
        case Gorcl:
        case Gotcl:
        case Torl:
        case Totl:
            ruvl = r;
            ruvl += ((uvlong)csr32r(ctlr, Statistics+(i+1)*4))<<32;
            tuvl = ruvl;
            tuvl += ctlr->statistics[i];
```

```
            tuvl += ((uvlong)ctlr->statistics[i+1])<<32;
            if(tuvl == 0)
                continue;
            ctlr->statistics[i] = tuvl;
            ctlr->statistics[i+1] = tuvl>>32;
            l += snprint(p+l, READSTR-l, "%s: %llud %llud\n",
                s, tuvl, ruvl);
            i++;
            break;

        default:
            ctlr->statistics[i] += r;
            if(ctlr->statistics[i] == 0)
                continue;
            l += snprint(p+l, READSTR-l, "%s: %ud %ud\n",
                s, ctlr->statistics[i], r);
            break;
        }
    }

    l += snprint(p+l, READSTR-l, "lintr: %ud %ud\n",
        ctlr->lintr, ctlr->lsleep);
    l += snprint(p+l, READSTR-l, "rintr: %ud %ud\n",
        ctlr->rintr, ctlr->rsleep);
    l += snprint(p+l, READSTR-l, "tintr: %ud %ud\n",
        ctlr->tintr, ctlr->txdw);
    l += snprint(p+l, READSTR-l, "ixcs: %ud %ud %ud\n",
        ctlr->ixsm, ctlr->ipcs, ctlr->tcpcs);
    l += snprint(p+l, READSTR-l, "rdtr: %ud\n", ctlr->rdtr);
    l += snprint(p+l, READSTR-l, "Ctrlext: %08x\n", csr32r(ctlr, Ctrlext));

    l += snprint(p+l, READSTR-l, "eeprom:");
    for(i = 0; i < 0x40; i++){
        if(i && ((i & 0x07) == 0))
            l += snprint(p+l, READSTR-l, "\n       ");
        l += snprint(p+l, READSTR-l, " %4.4uX", ctlr->eeprom[i]);
    }
    l += snprint(p+l, READSTR-l, "\n");

    if(ctlr->mii != nil && ctlr->mii->curphy != nil){
        l += snprint(p+l, READSTR-l, "phy:   ");
        for(i = 0; i < NMiiPhyr; i++){
            if(i && ((i & 0x07) == 0))
                l += snprint(p+l, READSTR-l, "\n       ");
            r = miimir(ctlr->mii, i);
            l += snprint(p+l, READSTR-l, " %4.4uX", r);
        }
        snprint(p+l, READSTR-l, "\n");
    }
    n = readstr(offset, a, n, p);
    free(p);
    qunlock(&ctlr->slock);

    return n;
}
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)25 255⟩≡ (279b)

```
enum {
    CMrdtr,
};
```

⟨*global* igbectlmsg 256a⟩≡ (279b)

```
static Cmdtab igbectlmsg[] = {
    CMrdtr, "rdtr", 2,
};
```

⟨*function* igbectl 256b⟩≡ (279b)

```
static long
igbectl(Ether* edev, void* buf, long n)
{
    int v;
    char *p;
    CtlrEtherIgbe *ctlr;
    Cmdbuf *cb;
    Cmdtab *ct;

    if((ctlr = edev->ctlr) == nil)
        error(Enonexist);

    cb = parsecmd(buf, n);
    if(waserror()){
        free(cb);
        nexterror();
    }

    ct = lookupcmd(cb, igbectlmsg, nelem(igbectlmsg));
    switch(ct->index){
    case CMrdtr:
        v = strtol(cb->f[1], &p, 0);
        if(v < 0 || p == cb->f[1] || v > 0xFFFF)
            error(Ebadarg);
        ctlr->rdtr = v;
        csr32w(ctlr, Rdtr, Fpd|v);
        break;
    }
    free(cb);
    poperror();

    return n;
}
```

⟨*function* igbepromiscuous 256c⟩≡ (279b)

```
static void
igbepromiscuous(void* arg, int on)
{
    int rctl;
    CtlrEtherIgbe *ctlr;
    Ether *edev;

    edev = arg;
    ctlr = edev->ctlr;

    rctl = csr32r(ctlr, Rctl);
    rctl &= ~MoMASK;
    rctl |= Mo47b36;
    if(on)
        rctl |= Upe|Mpe;
    else
        rctl &= ~(Upe|Mpe);
    csr32w(ctlr, Rctl, rctl|Mpe);   /* temporarily keep Mpe on */
}
```

⟨*function* igbemulticast 257a⟩≡ (279b)

```
static void
igbemulticast(void* arg, uchar* addr, int add)
{
    int bit, x;
    CtlrEtherIgbe *ctlr;
    Ether *edev;

    edev = arg;
    ctlr = edev->ctlr;

    x = addr[5]>>1;
    bit = ((addr[5] & 1)<<4)|(addr[4]>>4);
    /*
     * multiple ether addresses can hash to the same filter bit,
     * so it's never safe to clear a filter bit.
     * if we want to clear filter bits, we need to keep track of
     * all the multicast addresses in use, clear all the filter bits,
     * then set the ones corresponding to in-use addresses.
     */
    if(add)
        ctlr->mta[x] |= 1<<bit;
//  else
//      ctlr->mta[x] &= ~(1<<bit);

    csr32w(ctlr, Mta+x*4, ctlr->mta[x]);
}
```

⟨*function* igberballoc 257b⟩≡ (279b)

```
static Block*
igberballoc(void)
{
    Block *bp;

    ilock(&igberblock);
    if((bp = igberbpool) != nil){
        igberbpool = bp->next;
        bp->next = nil;
        arch_xinc(&bp->ref);    /* prevent bp from being freed */
    }
    iunlock(&igberblock);

    return bp;
}
```

⟨*function* igberbfree 257c⟩≡ (279b)

```
static void
igberbfree(Block* bp)
{
    bp->rp = bp->lim - Rbsz;
    bp->wp = bp->rp;
    bp->flag &= ~(Bipck | Budpck | Btcpck | Bpktck);

    ilock(&igberblock);
    bp->next = igberbpool;
    igberbpool = bp;
    iunlock(&igberblock);
}
```

⟨*function* igbeim 258a⟩≡ (279b)

```
static void
igbeim(CtlrEtherIgbe* ctlr, int im)
{
    ilock(&ctlr->imlock);
    ctlr->im |= im;
    csr32w(ctlr, Ims, ctlr->im);
    iunlock(&ctlr->imlock);
}
```

⟨*function* igbelim 258b⟩≡ (279b)

```
static int
igbelim(void* ctlr)
{
    return ((CtlrEtherIgbe*)ctlr)->lim != 0;
}
```

⟨*function* igbelproc 258c⟩≡ (279b)

```
static void
igbelproc(void* arg)
{
    CtlrEtherIgbe *ctlr;
    Ether *edev;
    MiiPhy *phy;
    int ctrl, r;

    edev = arg;
    ctlr = edev->ctlr;
    for(;;){
        if(ctlr->mii == nil || ctlr->mii->curphy == nil) {
            sched();
            continue;
        }

        /*
         * To do:
         *  logic to manage status change,
         *  this is incomplete but should work
         *  one time to set up the hardware.
         *
         *  MiiPhy.speed, etc. should be in Mii.
         */
        if(miistatus(ctlr->mii) < 0)
            //continue;
            goto enable;

        phy = ctlr->mii->curphy;
        ctrl = csr32r(ctlr, Ctrl);

        switch(ctlr->id){
        case i82543gc:
        case i82544ei:
        case i82544eif:
        default:
            if(!(ctrl & Asde)){
                ctrl &= ~(SspeedMASK|Ilos|Fd);
                ctrl |= Frcdplx|Frcspd;
                if(phy->speed == 1000)
                    ctrl |= Sspeed1000;
                else if(phy->speed == 100)
```

```
                    ctrl |= Sspeed100;
                if(phy->fd)
                    ctrl |= Fd;
            }
            break;

        case i82540em:
        case i82540eplp:
        case i82547gi:
        case i82541gi:
        case i82541gi2:
        case i82541pi:
            break;
        }

        /*
         * Collision Distance.
         */
        r = csr32r(ctlr, Tctl);
        r &= ~ColdMASK;
        if(phy->fd)
            r |= 64<<ColdSHIFT;
        else
            r |= 512<<ColdSHIFT;
        csr32w(ctlr, Tctl, r);

        /*
         * Flow control.
         */
        if(phy->rfc)
            ctrl |= Rfce;
        if(phy->tfc)
            ctrl |= Tfce;
        csr32w(ctlr, Ctrl, ctrl);

enable:
        ctlr->lim = 0;
        igbeim(ctlr, Lsc);

        ctlr->lsleep++;
        sleep(&ctlr->lrendez, igbelim, ctlr);
    }
}
```

⟨*function* igbetxinit 259⟩≡ (279b)

```
static void
igbetxinit(CtlrEtherIgbe* ctlr)
{
    int i, r;
    Block *bp;

    csr32w(ctlr, Tctl, (0x0F<<CtSHIFT)|Psp|(66<<ColdSHIFT));
    switch(ctlr->id){
    default:
        r = 6;
        break;
    case i82543gc:
    case i82544ei:
    case i82544eif:
    case i82544gc:
```

```
case i82540em:
case i82540eplp:
case i82541ei:
case i82541gi:
case i82541gi2:
case i82541pi:
case i82545em:
case i82545gmc:
case i82546gb:
case i82546eb:
case i82547ei:
case i82547gi:
    r = 8;
    break;
}
csr32w(ctlr, Tipg, (6<<20)|(8<<10)|r);
csr32w(ctlr, Ait, 0);
csr32w(ctlr, Txdmac, 0);

csr32w(ctlr, Tdbal, PCIWADDR(ctlr->tdba));
csr32w(ctlr, Tdbah, 0);
csr32w(ctlr, Tdlen, ctlr->ntd*sizeof(Td));
ctlr->tdh = PREV(0, ctlr->ntd);
csr32w(ctlr, Tdh, 0);
ctlr->tdt = 0;
csr32w(ctlr, Tdt, 0);

for(i = 0; i < ctlr->ntd; i++){
    if((bp = ctlr->tb[i]) != nil){
        ctlr->tb[i] = nil;
        freeb(bp);
    }
    memset(&ctlr->tdba[i], 0, sizeof(Td));
}
ctlr->tdfree = ctlr->ntd;

csr32w(ctlr, Tidv, 128);
r = (4<<WthreshSHIFT)|(4<<HthreshSHIFT)|(8<<PthreshSHIFT);

switch(ctlr->id){
default:
    break;
case i82540em:
case i82540eplp:
case i82547gi:
case i82545em:
case i82545gmc:
case i82546gb:
case i82546eb:
case i82541gi:
case i82541gi2:
case i82541pi:
    r = csr32r(ctlr, Txdctl);
    r &= ~WthreshMASK;
    r |= Gran|(4<<WthreshSHIFT);

    csr32w(ctlr, Tadv, 64);
    break;
}
```

```
    csr32w(ctlr, Txdctl, r);

    r = csr32r(ctlr, Tctl);
    r |= Ten;
    csr32w(ctlr, Tctl, r);
}
```

⟨*function* igbetransmit 261⟩≡ (279b)

```
static void
igbetransmit(Ether* edev)
{
    Td *td;
    Block *bp;
    CtlrEtherIgbe *ctlr;
    int tdh, tdt;

    ctlr = edev->ctlr;

    ilock(&ctlr->tlock);

    /*
     * Free any completed packets
     */
    tdh = ctlr->tdh;
    while(NEXT(tdh, ctlr->ntd) != csr32r(ctlr, Tdh)){
        if((bp = ctlr->tb[tdh]) != nil){
            ctlr->tb[tdh] = nil;
            freeb(bp);
        }
        memset(&ctlr->tdba[tdh], 0, sizeof(Td));
        tdh = NEXT(tdh, ctlr->ntd);
    }
    ctlr->tdh = tdh;

    /*
     * Try to fill the ring back up.
     */
    tdt = ctlr->tdt;
    while(NEXT(tdt, ctlr->ntd) != tdh){
        if((bp = qget(edev->oq)) == nil)
            break;
        td = &ctlr->tdba[tdt];
        td->addr[0] = PCIWADDR(bp->rp);
        td->control = ((BLEN(bp) & LenMASK)<<LenSHIFT);
        td->control |= Dext|Ifcs|Teop|DtypeDD;
        ctlr->tb[tdt] = bp;
        tdt = NEXT(tdt, ctlr->ntd);
        if(NEXT(tdt, ctlr->ntd) == tdh){
            td->control |= Rs;
            ctlr->txdw++;
            ctlr->tdt = tdt;
            csr32w(ctlr, Tdt, tdt);
            igbeim(ctlr, Txdw);
            break;
        }
        ctlr->tdt = tdt;
        csr32w(ctlr, Tdt, tdt);
    }

    iunlock(&ctlr->tlock);
```

```
}
```

⟨*function* igbereplenish 262a⟩≡ (279b)

```
static void
igbereplenish(CtlrEtherIgbe* ctlr)
{
    Rd *rd;
    int rdt;
    Block *bp;

    rdt = ctlr->rdt;
    while(NEXT(rdt, ctlr->nrd) != ctlr->rdh){
        rd = &ctlr->rdba[rdt];
        if(ctlr->rb[rdt] == nil){
            bp = igberballoc();
            if(bp == nil){
                iprint("#l%d: igbereplenish: no available buffers\n",
                    ctlr->edev->ctlrno);
                break;
            }
            ctlr->rb[rdt] = bp;
            rd->addr[0] = PCIWADDR(bp->rp);
            rd->addr[1] = 0;
        }
        arch_coherence();
        rd->status = 0;
        rdt = NEXT(rdt, ctlr->nrd);
        ctlr->rdfree++;
    }
    ctlr->rdt = rdt;
    csr32w(ctlr, Rdt, rdt);
}
```

⟨*function* igberxinit 262b⟩≡ (279b)

```
static void
igberxinit(CtlrEtherIgbe* ctlr)
{
    int i;
    Block *bp;

    /* temporarily keep Mpe on */
    csr32w(ctlr, Rctl, Dpf|Bsize2048|Bam|RdtmsHALF|Mpe);

    csr32w(ctlr, Rdbal, PCIWADDR(ctlr->rdba));
    csr32w(ctlr, Rdbah, 0);
    csr32w(ctlr, Rdlen, ctlr->nrd*sizeof(Rd));
    ctlr->rdh = 0;
    csr32w(ctlr, Rdh, 0);
    ctlr->rdt = 0;
    csr32w(ctlr, Rdt, 0);
    ctlr->rdtr = 0;
    csr32w(ctlr, Rdtr, Fpd|0);

    for(i = 0; i < ctlr->nrd; i++){
        if((bp = ctlr->rb[i]) != nil){
            ctlr->rb[i] = nil;
            freeb(bp);
        }
    }
    igbereplenish(ctlr);
```

```
    switch(ctlr->id){
    case i82540em:
    case i82540eplp:
    case i82541gi:
    case i82541gi2:
    case i82541pi:
    case i82545em:
    case i82545gmc:
    case i82546gb:
    case i82546eb:
    case i82547gi:
        csr32w(ctlr, Radv, 64);
        break;
    }
    csr32w(ctlr, Rxdctl, (8<<WthreshSHIFT)|(8<<HthreshSHIFT)|4);

    /*
     * Disable checksum offload as it has known bugs.
     */
    csr32w(ctlr, Rxcsum, ETHERHDRSIZE<<PcssSHIFT);
}
```

⟨*function* igberim 263a⟩≡ (279b)

```
static int
igberim(void* ctlr)
{
    return ((CtlrEtherIgbe*)ctlr)->rim != 0;
}
```

⟨*function* igberproc 263b⟩≡ (279b)

```
static void
igberproc(void* arg)
{
    Rd *rd;
    Block *bp;
    CtlrEtherIgbe *ctlr;
    int r, rdh;
    Ether *edev;

    edev = arg;
    ctlr = edev->ctlr;

    igberxinit(ctlr);
    r = csr32r(ctlr, Rctl);
    r |= Ren;
    csr32w(ctlr, Rctl, r);

    for(;;){
        ctlr->rim = 0;
        igbeim(ctlr, Rxt0|Rxo|Rxdmt0|Rxseq);
        ctlr->rsleep++;
        sleep(&ctlr->rrendez, igberim, ctlr);

        rdh = ctlr->rdh;
        for(;;){
            rd = &ctlr->rdba[rdh];

            if(!(rd->status & Rdd))
                break;
```

```
            /*
             * Accept eop packets with no errors.
             * With no errors and the Ixsm bit set,
             * the descriptor status Tpcs and Ipcs bits give
             * an indication of whether the checksums were
             * calculated and valid.
             */
            if((rd->status & Reop) && rd->errors == 0){
                bp = ctlr->rb[rdh];
                ctlr->rb[rdh] = nil;
                bp->wp += rd->length;
                bp->next = nil;
                if(!(rd->status & Ixsm)){
                    ctlr->ixsm++;
                    if(rd->status & Ipcs){
                        /*
                         * IP checksum calculated
                         * (and valid as errors == 0).
                         */
                        ctlr->ipcs++;
                        bp->flag |= Bipck;
                    }
                    if(rd->status & Tcpcs){
                        /*
                         * TCP/UDP checksum calculated
                         * (and valid as errors == 0).
                         */
                        ctlr->tcpcs++;
                        bp->flag |= Btcpck|Budpck;
                    }
                    bp->checksum = rd->checksum;
                    bp->flag |= Bpktck;
                }
                etheriq(edev, bp, 1);
            }
            else if(ctlr->rb[rdh] != nil){
                freeb(ctlr->rb[rdh]);
                ctlr->rb[rdh] = nil;
            }

            memset(rd, 0, sizeof(Rd));
            arch_coherence();
            ctlr->rdfree--;
            rdh = NEXT(rdh, ctlr->nrd);
        }
        ctlr->rdh = rdh;

        if(ctlr->rdfree < ctlr->nrd/2 || (ctlr->rim & Rxdmt0))
            igbereplenish(ctlr);
    }
}
```

⟨*function* igbeattach 264⟩≡ (279b)

```
static void
igbeattach(Ether* edev)
{
    Block *bp;
    CtlrEtherIgbe *ctlr;
    char name[KNAMELEN];
```

```
ctlr = edev->ctlr;
ctlr->edev = edev;          /* point back to Ether* */
qlock(&ctlr->alock);
if(ctlr->alloc != nil){         /* already allocated? */
    qunlock(&ctlr->alock);
    return;
}

ctlr->tb = nil;
ctlr->rb = nil;
ctlr->alloc = nil;
ctlr->nrb = 0;
if(waserror()){
    while(ctlr->nrb > 0){
        bp = igberballoc();
        bp->free = nil;
        freeb(bp);
        ctlr->nrb--;
    }
    free(ctlr->tb);
    ctlr->tb = nil;
    free(ctlr->rb);
    ctlr->rb = nil;
    free(ctlr->alloc);
    ctlr->alloc = nil;
    qunlock(&ctlr->alock);
    nexterror();
}

ctlr->nrd = ROUND(Nrd, 8);
ctlr->ntd = ROUND(Ntd, 8);
ctlr->alloc = malloc(ctlr->nrd*sizeof(Rd)+ctlr->ntd*sizeof(Td) + 127);
if(ctlr->alloc == nil) {
    print("igbe: can't allocate ctlr->alloc\n");
    error(Enomem);
}
ctlr->rdba = (Rd*)ROUNDUP((uintptr)ctlr->alloc, 128);
ctlr->tdba = (Td*)(ctlr->rdba+ctlr->nrd);

ctlr->rb = malloc(ctlr->nrd*sizeof(Block*));
ctlr->tb = malloc(ctlr->ntd*sizeof(Block*));
if (ctlr->rb == nil || ctlr->tb == nil) {
    print("igbe: can't allocate ctlr->rb or ctlr->tb\n");
    error(Enomem);
}

for(ctlr->nrb = 0; ctlr->nrb < Nrb; ctlr->nrb++){
    if((bp = allocb(Rbsz)) == nil)
        break;
    bp->free = igberbfree;
    freeb(bp);
}

snprint(name, KNAMELEN, "#l%dlproc", edev->ctlrno);
kproc(name, igbelproc, edev);

snprint(name, KNAMELEN, "#l%drproc", edev->ctlrno);
kproc(name, igberproc, edev);
```

```
    igbetxinit(ctlr);

    qunlock(&ctlr->alock);
    poperror();
}
```

⟨*function* igbeinterrupt 266a⟩≡ (279b)

```
static void
igbeinterrupt(Ureg*, void* arg)
{
    CtlrEtherIgbe *ctlr;
    Ether *edev;
    int icr, im, txdw;

    edev = arg;
    ctlr = edev->ctlr;

    ilock(&ctlr->imlock);
    csr32w(ctlr, Imc, ~0);
    im = ctlr->im;
    txdw = 0;

    while((icr = csr32r(ctlr, Icr) & ctlr->im) != 0){
        if(icr & Lsc){
            im &= ~Lsc;
            ctlr->lim = icr & Lsc;
            wakeup(&ctlr->lrendez);
            ctlr->lintr++;
        }
        if(icr & (Rxt0|Rxo|Rxdmt0|Rxseq)){
            im &= ~(Rxt0|Rxo|Rxdmt0|Rxseq);
            ctlr->rim = icr & (Rxt0|Rxo|Rxdmt0|Rxseq);
            wakeup(&ctlr->rrendez);
            ctlr->rintr++;
        }
        if(icr & Txdw){
            im &= ~Txdw;
            txdw++;
            ctlr->tintr++;
        }
    }

    ctlr->im = im;
    csr32w(ctlr, Ims, im);
    iunlock(&ctlr->imlock);

    if(txdw)
        igbetransmit(edev);
}
```

⟨*function* i82543mdior 266b⟩≡ (279b)

```
static int
i82543mdior(CtlrEtherIgbe* ctlr, int n)
{
    int ctrl, data, i, r;

    /*
     * Read n bits from the Management Data I/O Interface.
     */
    ctrl = csr32r(ctlr, Ctrl);
```

```
    r = (ctrl & ~Mddo)|Mdco;
    data = 0;
    for(i = n-1; i >= 0; i--){
        if(csr32r(ctlr, Ctrl) & Mdd)
            data |= (1<<i);
        csr32w(ctlr, Ctrl, Mdc|r);
        csr32w(ctlr, Ctrl, r);
    }
    csr32w(ctlr, Ctrl, ctrl);

    return data;
}
```

⟨*function* i82543mdiow 267a⟩≡ (279b)

```
static int
i82543mdiow(CtlrEtherIgbe* ctlr, int bits, int n)
{
    int ctrl, i, r;

    /*
     * Write n bits to the Management Data I/O Interface.
     */
    ctrl = csr32r(ctlr, Ctrl);
    r = Mdco|Mddo|ctrl;
    for(i = n-1; i >= 0; i--){
        if(bits & (1<<i))
            r |= Mdd;
        else
            r &= ~Mdd;
        csr32w(ctlr, Ctrl, Mdc|r);
        csr32w(ctlr, Ctrl, r);
    }
    csr32w(ctlr, Ctrl, ctrl);

    return 0;
}
```

⟨*function* i82543miimir 267b⟩≡ (279b)

```
static int
i82543miimir(Mii* mii, int pa, int ra)
{
    int data;
    CtlrEtherIgbe *ctlr;

    ctlr = mii->ctlr;

    /*
     * MII Management Interface Read.
     *
     * Preamble;
     * ST+OP+PHYAD+REGAD;
     * TA + 16 data bits.
     */
    i82543mdiow(ctlr, 0xFFFFFFFF, 32);
    i82543mdiow(ctlr, 0x1800|(pa<<5)|ra, 14);
    data = i82543mdior(ctlr, 18);

    if(data & 0x10000)
        return -1;
```

```
    return data & 0xFFFF;
}
```

⟨*function* i82543miimiw 268a⟩≡ (279b)

```
static int
i82543miimiw(Mii* mii, int pa, int ra, int data)
{
    CtlrEtherIgbe *ctlr;

    ctlr = mii->ctlr;

    /*
     * MII Management Interface Write.
     *
     * Preamble;
     * ST+OP+PHYAD+REGAD+TA + 16 data bits;
     * Z.
     */
    i82543mdiow(ctlr, 0xFFFFFFFF, 32);
    data &= 0xFFFF;
    data |= (0x05<<(5+5+2+16))|(pa<<(5+2+16))|(ra<<(2+16))|(0x02<<16);
    i82543mdiow(ctlr, data, 32);

    return 0;
}
```

⟨*function* igbemiimir 268b⟩≡ (279b)

```
static int
igbemiimir(Mii* mii, int pa, int ra)
{
    CtlrEtherIgbe *ctlr;
    int mdic, timo;

    ctlr = mii->ctlr;

    csr32w(ctlr, Mdic, MDIrop|(pa<<MDIpSHIFT)|(ra<<MDIrSHIFT));
    mdic = 0;
    for(timo = 64; timo; timo--){
        mdic = csr32r(ctlr, Mdic);
        if(mdic & (MDIe|MDIready))
            break;
        arch_microdelay(1);
    }

    if((mdic & (MDIe|MDIready)) == MDIready)
        return mdic & 0xFFFF;
    return -1;
}
```

⟨*function* igbemiimiw 268c⟩≡ (279b)

```
static int
igbemiimiw(Mii* mii, int pa, int ra, int data)
{
    CtlrEtherIgbe *ctlr;
    int mdic, timo;

    ctlr = mii->ctlr;

    data &= MDIdMASK;
    csr32w(ctlr, Mdic, MDIwop|(pa<<MDIpSHIFT)|(ra<<MDIrSHIFT)|data);
```

```
    mdic = 0;
    for(timo = 64; timo; timo--){
        mdic = csr32r(ctlr, Mdic);
        if(mdic & (MDIe|MDIready))
            break;
        arch_microdelay(1);
    }
    if((mdic & (MDIe|MDIready)) == MDIready)
        return 0;
    return -1;
}
```

⟨*function* igbemii 269⟩≡ (279b)

```
static int
igbemii(CtlrEtherIgbe* ctlr)
{
    MiiPhy *phy;
    int ctrl, p, r;

    r = csr32r(ctlr, Status);
    if(r & Tbimode)
        return -1;
    if((ctlr->mii = malloc(sizeof(Mii))) == nil)
        return -1;
    ctlr->mii->ctlr = ctlr;

    ctrl = csr32r(ctlr, Ctrl);
    ctrl |= Slu;

    switch(ctlr->id){
    case i82543gc:
        ctrl |= Frcdplx|Frcspd;
        csr32w(ctlr, Ctrl, ctrl);

        /*
         * The reset pin direction (Mdro) should already
         * be set from the EEPROM load.
         * If it's not set this configuration is unexpected
         * so bail.
         */
        r = csr32r(ctlr, Ctrlext);
        if(!(r & Mdro)) {
            print("igbe: 82543gc Mdro not set\n");
            return -1;
        }
        csr32w(ctlr, Ctrlext, r);
        arch_delay(20);
        r = csr32r(ctlr, Ctrlext);
        r &= ~Mdr;
        csr32w(ctlr, Ctrlext, r);
        arch_delay(20);
        r = csr32r(ctlr, Ctrlext);
        r |= Mdr;
        csr32w(ctlr, Ctrlext, r);
        arch_delay(20);

        ctlr->mii->mir = i82543miimir;
        ctlr->mii->miw = i82543miimiw;
        break;
    case i82544ei:
```

```
case i82544eif:
case i82544gc:
case i82540em:
case i82540eplp:
case i82547ei:
case i82547gi:
case i82541ei:
case i82541gi:
case i82541gi2:
case i82541pi:
case i82545em:
case i82545gmc:
case i82546gb:
case i82546eb:
    ctrl &= ~(Frcdplx|Frcspd);
    csr32w(ctlr, Ctrl, ctrl);
    ctlr->mii->mir = igbemiimir;
    ctlr->mii->miw = igbemiimiw;
    break;
default:
    free(ctlr->mii);
    ctlr->mii = nil;
    return -1;
}

if(mii(ctlr->mii, ~0) == 0 || (phy = ctlr->mii->curphy) == nil){
    free(ctlr->mii);
    ctlr->mii = nil;
    return -1;
}
USED(phy);
// print("oui %X phyno %d\n", phy->oui, phy->phyno);

/*
 * 8254X-specific PHY registers not in 802.3:
 *  0x10    PHY specific control
 *  0x14    extended PHY specific control
 * Set appropriate values then reset the PHY to have
 * changes noted.
 */
switch(ctlr->id){
case i82547gi:
case i82541gi:
case i82541gi2:
case i82541pi:
case i82545em:
case i82545gmc:
case i82546gb:
case i82546eb:
    break;
default:
    r = miimir(ctlr->mii, 16);
    r |= 0x0800;            /* assert CRS on Tx */
    r |= 0x0060;            /* auto-crossover all speeds */
    r |= 0x0002;            /* polarity reversal enabled */
    miimiw(ctlr->mii, 16, r);

    r = miimir(ctlr->mii, 20);
    r |= 0x0070;            /* +25MHz clock */
    r &= ~0x0F00;
```

```
        r |= 0x0100;            /* 1x downshift */
        miimiw(ctlr->mii, 20, r);

        miireset(ctlr->mii);
        p = 0;
        if(ctlr->txcw & TxcwPs)
            p |= AnaP;
        if(ctlr->txcw & TxcwAs)
            p |= AnaAP;
        miiane(ctlr->mii, ~0, p, ~0);
        break;
    }
    return 0;
}
```

⟨*function* at93c46io 271⟩≡ (279b)

```
static int
at93c46io(CtlrEtherIgbe* ctlr, char* op, int data)
{
    char *lp, *p;
    int i, loop, eecd, r;

    eecd = csr32r(ctlr, Eecd);

    r = 0;
    loop = -1;
    lp = nil;
    for(p = op; *p != '\0'; p++){
        switch(*p){
        default:
            return -1;
        case ' ':
            continue;
        case ':':           /* start of loop */
            loop = strtol(p+1, &lp, 0)-1;
            lp--;
            if(p == lp)
                loop = 7;
            p = lp;
            continue;
        case ';':           /* end of loop */
            if(lp == nil)
                return -1;
            loop--;
            if(loop >= 0)
                p = lp;
            else
                lp = nil;
            continue;
        case 'C':           /* assert clock */
            eecd |= Sk;
            break;
        case 'c':           /* deassert clock */
            eecd &= ~Sk;
            break;
        case 'D':           /* next bit in 'data' byte */
            if(loop < 0)
                return -1;
            if(data & (1<<loop))
                eecd |= Di;
```

```
            else
                eecd &= ~Di;
            break;
        case 'O':           /* collect data output */
            i = (csr32r(ctlr, Eecd) & Do) != 0;
            if(loop >= 0)
                r |= (i<<loop);
            else
                r = i;
            continue;
        case 'I':           /* assert data input */
            eecd |= Di;
            break;
        case 'i':           /* deassert data input */
            eecd &= ~Di;
            break;
        case 'S':           /* enable chip select */
            eecd |= Cs;
            break;
        case 's':           /* disable chip select */
            eecd &= ~Cs;
            break;
        }
        csr32w(ctlr, Eecd, eecd);
        arch_microdelay(50);
    }
    if(loop >= 0)
        return -1;
    return r;
}
```

⟨*function* at93c46r 272⟩≡ (279b)

```
static int
at93c46r(CtlrEtherIgbe* ctlr)
{
    ushort sum;
    char rop[20];
    int addr, areq, bits, data, eecd, i;

    eecd = csr32r(ctlr, Eecd);
    if(eecd & Spi){
        print("igbe: SPI EEPROM access not implemented\n");
        return 0;
    }
    if(eecd & (Eeszaddr|Eesz256))
        bits = 8;
    else
        bits = 6;

    sum = 0;

    switch(ctlr->id){
    default:
        areq = 0;
        break;
    case i82540em:
    case i82540eplp:
    case i82541ei:
    case i82541gi:
    case i82541gi2:
```

```
    case i82541pi:
    case i82545em:
    case i82545gmc:
    case i82546gb:
    case i82546eb:
    case i82547ei:
    case i82547gi:
        areq = 1;
        csr32w(ctlr, Eecd, eecd|Areq);
        for(i = 0; i < 1000; i++){
            if((eecd = csr32r(ctlr, Eecd)) & Agnt)
                break;
            arch_microdelay(5);
        }
        if(!(eecd & Agnt)){
            print("igbe: not granted EEPROM access\n");
            goto release;
        }
        break;
    }
    snprint(rop, sizeof(rop), "S :%dDCc;", bits+3);

    for(addr = 0; addr < 0x40; addr++){
        /*
         * Read a word at address 'addr' from the Atmel AT93C46
         * 3-Wire Serial EEPROM or compatible. The EEPROM access is
         * controlled by 4 bits in Eecd. See the AT93C46 datasheet
         * for protocol details.
         */
        if(at93c46io(ctlr, rop, (0x06<<bits)|addr) != 0){
            print("igbe: can't set EEPROM address 0x%2.2X\n", addr);
            goto release;
        }
        data = at93c46io(ctlr, ":16COc;", 0);
        at93c46io(ctlr, "sic", 0);
        ctlr->eeprom[addr] = data;
        sum += data;
    }

release:
    if(areq)
        csr32w(ctlr, Eecd, eecd & ~Areq);
    return sum;
}
```

⟨*function* igbedetach 273⟩≡ (279b)

```
static int
igbedetach(CtlrEtherIgbe* ctlr)
{
    int r, timeo;

    /*
     * Perform a device reset to get the chip back to the
     * power-on state, followed by an EEPROM reset to read
     * the defaults for some internal registers.
     */
    csr32w(ctlr, Imc, ~0);
    csr32w(ctlr, Rctl, 0);
    csr32w(ctlr, Tctl, 0);
```

```
    arch_delay(10);

    csr32w(ctlr, Ctrl, Devrst);
    arch_delay(1);
    for(timeo = 0; timeo < 1000; timeo++){
        if(!(csr32r(ctlr, Ctrl) & Devrst))
            break;
        arch_delay(1);
    }
    if(csr32r(ctlr, Ctrl) & Devrst)
        return -1;
    r = csr32r(ctlr, Ctrlext);
    csr32w(ctlr, Ctrlext, r|Eerst);
    arch_delay(1);
    for(timeo = 0; timeo < 1000; timeo++){
        if(!(csr32r(ctlr, Ctrlext) & Eerst))
            break;
        arch_delay(1);
    }
    if(csr32r(ctlr, Ctrlext) & Eerst)
        return -1;

    switch(ctlr->id){
    default:
        break;
    case i82540em:
    case i82540eplp:
    case i82541gi:
    case i82541gi2:
    case i82541pi:
    case i82545em:
    case i82545gmc:
    case i82547gi:
    case i82546gb:
    case i82546eb:
        r = csr32r(ctlr, Manc);
        r &= ~Arpen;
        csr32w(ctlr, Manc, r);
        break;
    }

    csr32w(ctlr, Imc, ~0);
    arch_delay(1);
    for(timeo = 0; timeo < 1000; timeo++){
        if(!csr32r(ctlr, Icr))
            break;
        arch_delay(1);
    }
    if(csr32r(ctlr, Icr))
        return -1;

    return 0;
}
```

⟨*function* igbeshutdown 274⟩≡ (279b)

```
static void
igbeshutdown(Ether* ether)
{
    igbedetach(ether->ctlr);
}
```

⟨*function* igbereset 275⟩≡ (279b)

```
static int
igbereset(CtlrEtherIgbe* ctlr)
{
    int ctrl, i, pause, r, swdpio, txcw;

    if(igbedetach(ctlr))
        return -1;

    /*
     * Read the EEPROM, validate the checksum
     * then get the device back to a power-on state.
     */
    if((r = at93c46r(ctlr)) != 0xBABA){
        print("igbe: bad EEPROM checksum - 0x%4.4uX\n", r);
        return -1;
    }

    /*
     * Snarf and set up the receive addresses.
     * There are 16 addresses. The first should be the MAC address.
     * The others are cleared and not marked valid (MS bit of Rah).
     */
    if ((ctlr->id == i82546gb || ctlr->id == i82546eb) &&
        BUSFNO(ctlr->pcidev->tbdf) == 1)
        ctlr->eeprom[Ea+2] += 0x100;        /* second interface */
    if(ctlr->id == i82541gi && ctlr->eeprom[Ea] == 0xFFFF)
        ctlr->eeprom[Ea] = 0xD000;
    for(i = Ea; i < Eaddrlen/2; i++){
        ctlr->ra[2*i] = ctlr->eeprom[i];
        ctlr->ra[2*i+1] = ctlr->eeprom[i]>>8;
    }
    /* lan id seems to vary on 82543gc; don't use it */
    if (ctlr->id != i82543gc) {
        r = (csr32r(ctlr, Status) & Lanid) >> 2;
        ctlr->ra[5] += r;       /* ea ctlr[1] = ea ctlr[0]+1 */
    }

    r = (ctlr->ra[3]<<24)|(ctlr->ra[2]<<16)|(ctlr->ra[1]<<8)|ctlr->ra[0];
    csr32w(ctlr, Ral, r);
    r = 0x80000000|(ctlr->ra[5]<<8)|ctlr->ra[4];
    csr32w(ctlr, Rah, r);
    for(i = 1; i < 16; i++){
        csr32w(ctlr, Ral+i*8, 0);
        csr32w(ctlr, Rah+i*8, 0);
    }

    /*
     * Clear the Multicast Table Array.
     * It's a 4096 bit vector accessed as 128 32-bit registers.
     */
    memset(ctlr->mta, 0, sizeof(ctlr->mta));
    for(i = 0; i < 128; i++)
        csr32w(ctlr, Mta+i*4, 0);

    /*
     * Just in case the Eerst didn't load the defaults
     * (doesn't appear to fully on the 82543GC), do it manually.
     */
    if (ctlr->id == i82543gc) {
```

```
    txcw = csr32r(ctlr, Txcw);
    txcw &= ~(TxcwAne|TxcwPauseMASK|TxcwFd);
    ctrl = csr32r(ctlr, Ctrl);
    ctrl &= ~(SwdpioloMASK|Frcspd|Ilos|Lrst|Fd);

    if(ctlr->eeprom[Icw1] & 0x0400){
        ctrl |= Fd;
        txcw |= TxcwFd;
    }
    if(ctlr->eeprom[Icw1] & 0x0200)
        ctrl |= Lrst;
    if(ctlr->eeprom[Icw1] & 0x0010)
        ctrl |= Ilos;
    if(ctlr->eeprom[Icw1] & 0x0800)
        ctrl |= Frcspd;
    swdpio = (ctlr->eeprom[Icw1] & 0x01E0)>>5;
    ctrl |= swdpio<<SwdpioloSHIFT;
    csr32w(ctlr, Ctrl, ctrl);

    ctrl = csr32r(ctlr, Ctrlext);
    ctrl &= ~(Ips|SwdpiohiMASK);
    swdpio = (ctlr->eeprom[Icw2] & 0x00F0)>>4;
    if(ctlr->eeprom[Icw1] & 0x1000)
        ctrl |= Ips;
    ctrl |= swdpio<<SwdpiohiSHIFT;
    csr32w(ctlr, Ctrlext, ctrl);

    if(ctlr->eeprom[Icw2] & 0x0800)
        txcw |= TxcwAne;
    pause = (ctlr->eeprom[Icw2] & 0x3000)>>12;
    txcw |= pause<<TxcwPauseSHIFT;
    switch(pause){
    default:
        ctlr->fcrtl = 0x00002000;
        ctlr->fcrth = 0x00004000;
        txcw |= TxcwAs|TxcwPs;
        break;
    case 0:
        ctlr->fcrtl = 0x00002000;
        ctlr->fcrth = 0x00004000;
        break;
    case 2:
        ctlr->fcrtl = 0;
        ctlr->fcrth = 0;
        txcw |= TxcwAs;
        break;
    }
    ctlr->txcw = txcw;
    csr32w(ctlr, Txcw, txcw);
}


/*
 * Flow control - values from the datasheet.
 */
csr32w(ctlr, Fcal, 0x00C28001);
csr32w(ctlr, Fcah, 0x00000100);
csr32w(ctlr, Fct, 0x00008808);
csr32w(ctlr, Fcttv, 0x00000100);
```

```
    csr32w(ctlr, Fcrtl, ctlr->fcrtl);
    csr32w(ctlr, Fcrth, ctlr->fcrth);

    if(!(csr32r(ctlr, Status) & Tbimode) && igbemii(ctlr) < 0)
        return -1;

    return 0;
}
```

⟨*function* igbepci 277⟩≡ (279b)

```
static void
igbepci(void)
{
    int cls;
    Pcidev *p;
    CtlrEtherIgbe *ctlr;
    void *mem;

    p = nil;
    while(p = pcimatch(p, 0, 0)){
        if(p->ccrb != 0x02 || p->ccru != 0)
            continue;

        switch((p->did<<16)|p->vid){
        default:
            continue;
        case i82543gc:
        case i82544ei:
        case i82544eif:
        case i82544gc:
        case i82547ei:
        case i82547gi:
        case i82540em:
        case i82540eplp:
        case i82541ei:
        case i82541gi:
        case i82541gi2:
        case i82541pi:
        case i82545em:
        case i82545gmc:
        case i82546gb:
        case i82546eb:
            break;
        }

        mem = vmap(p->mem[0].bar & ~0x0F, p->mem[0].size);
        if(mem == nil){
            print("igbe: can't map %8.8luX\n", p->mem[0].bar);
            continue;
        }
        cls = pcicfgr8(p, PciCLS);
        switch(cls){
        default:
            print("igbe: p->cls %#ux, setting to 0x10\n", p->cls);
            p->cls = 0x10;
            pcicfgw8(p, PciCLS, p->cls);
            break;
        case 0x08:
        case 0x10:
            break;
```

```
        }
        ctlr = malloc(sizeof(CtlrEtherIgbe));
        if(ctlr == nil) {
            vunmap(mem, p->mem[0].size);
            error(Enomem);
        }
        ctlr->port = p->mem[0].bar & ~0x0F;
        ctlr->pcidev = p;
        ctlr->id = (p->did<<16)|p->vid;
        ctlr->cls = cls*4;
        ctlr->nic = mem;

        if(igbereset(ctlr)){
            free(ctlr);
            vunmap(mem, p->mem[0].size);
            continue;
        }
        pcisetbme(p);

        if(igbectlrhead != nil)
            igbectlrtail->next = ctlr;
        else
            igbectlrhead = ctlr;
        igbectlrtail = ctlr;
    }
}
```

⟨*function* igbepnp 278⟩≡ (279b)

```
static int
igbepnp(Ether* edev)
{
    CtlrEtherIgbe *ctlr;

    if(igbectlrhead == nil)
        igbepci();

    /*
     * Any adapter matches if no edev->port is supplied,
     * otherwise the ports must match.
     */
    for(ctlr = igbectlrhead; ctlr != nil; ctlr = ctlr->next){
        if(ctlr->active)
            continue;
        if(edev->port == 0 || edev->port == ctlr->port){
            ctlr->active = 1;
            break;
        }
    }
    if(ctlr == nil)
        return -1;

    edev->ctlr = ctlr;
    edev->port = ctlr->port;
    edev->irq = ctlr->pcidev->intl;
    edev->tbdf = ctlr->pcidev->tbdf;
    edev->mbps = 1000;
    memmove(edev->ea, ctlr->ra, Eaddrlen);

    /*
     * Linkage to the generic ethernet driver.
```

```
     */
    edev->attach = igbeattach;
    edev->transmit = igbetransmit;
    edev->interrupt = igbeinterrupt;
    edev->ifstat = igbeifstat;
    edev->ctl = igbectl;

    edev->arg = edev;
    edev->promiscuous = igbepromiscuous;
    edev->shutdown = igbeshutdown;
    edev->multicast = igbemulticast;

    return 0;
}
```

⟨*function* etherigbelink 279a⟩≡ (279b)

```
void
etherigbelink(void)
{
    addethercard("i82543", igbepnp);
    addethercard("igbe", igbepnp);
}
```

⟨kernel/network/386/etherigbe.c 279b⟩≡

```
/*
 * Intel 8254[340]NN Gigabit Ethernet PCI Controllers
 * as found on the Intel PRO/1000 series of adapters:
 *  82543GC Intel PRO/1000 T
 *  82544EI Intel PRO/1000 XT
 *  82540EM Intel PRO/1000 MT
 *  82541[GP]I
 *  82547GI
 *  82546GB
 *  82546EB
 * To Do:
 *  finish autonegotiation code;
 *  integrate fiber stuff back in (this ONLY handles
 *  the CAT5 cards at the moment);
 *  add tuning control via ctl file;
 *  this driver is little-endian specific.
 */
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"

#include "io.h"
#include "../port/error.h"
#include "../port/netif.h"
#include "../port/etherif.h"

#include "ethermii.h"

typedef struct Td Td;
typedef struct CtlrEtherIgbe CtlrEtherIgbe;


⟨enum _anon_ (kernel/network/386/etherigbe.c) 244a⟩
```

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)2 244b⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)3 246a⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)4 246b⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)5 246c⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)6 247a⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)7 247b⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)8 247c⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)9 247d⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)10 248a⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)11 248b⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)12 248c⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)13 248d⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)14 249a⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)15 249b⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)16 249c⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)17 250a⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)18 250b⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)19 250c⟩

⟨*struct* Rd 250d⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)20 250e⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)21 250f⟩

⟨*struct* Td 251a⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)22 251b⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)23 251c⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)24 251d⟩

⟨*struct* CtlrEtherIgbe 252⟩

⟨*macro* csr32r 253a⟩
⟨*macro* csr32w 253b⟩

⟨*global* igbectlrhead 253c⟩
⟨*global* igbectlrtail 253d⟩

⟨*global* igberblock 253e⟩
⟨*global* igberbpool 253f⟩

⟨*global* statistics 253g⟩

⟨*function* igbeifstat 254⟩

⟨*enum* _anon_ (kernel/network/386/etherigbe.c)25 255⟩

⟨*global* igbectlmsg 256a⟩

⟨*function* igbectl 256b⟩

⟨*function* igbepromiscuous 256c⟩

⟨*function* igbemulticast 257a⟩

⟨*function* igberballoc 257b⟩

⟨*function* igberbfree 257c⟩

⟨*function* igbeim 258a⟩

⟨*function* igbelim 258b⟩

⟨*function* igbelproc 258c⟩

⟨*function* igbetxinit 259⟩

⟨*function* igbetransmit 261⟩

⟨*function* igbereplenish 262a⟩

⟨*function* igberxinit 262b⟩

⟨*function* igberim 263a⟩

⟨*function* igberproc 263b⟩

⟨*function* igbeattach 264⟩

⟨*function* igbeinterrupt 266a⟩

⟨*function* i82543mdior 266b⟩

⟨*function* i82543mdiow 267a⟩

⟨*function* i82543miimir 267b⟩

⟨*function* i82543miimiw 268a⟩

⟨*function* igbemiimir 268b⟩

⟨*function* igbemiimiw 268c⟩

⟨*function* igbemii 269⟩

⟨*function* at93c46io 271⟩

⟨*function* at93c46r 272⟩

⟨*function* igbedetach 273⟩

⟨*function* igbeshutdown 274⟩

⟨*function* igbereset 275⟩

⟨*function* igbepci 277⟩

⟨*function* igbepnp 278⟩

⟨*function* etherigbelink 279a⟩

I.6.7 kernel/network/386/ethermii.c

⟨*function* mii 282⟩≡ (286a)

```
int
mii(Mii* mii, int mask)
{
    MiiPhy *miiphy;
    int bit, oui, phyno, r, rmask;

    /*
     * Probe through mii for PHYs in mask;
     * return the mask of those found in the current probe.
     * If the PHY has not already been probed, update
     * the Mii information.
     */
    rmask = 0;
    for(phyno = 0; phyno < NMiiPhy; phyno++){
        bit = 1<<phyno;
        if(!(mask & bit))
            continue;
        if(mii->mask & bit){
            rmask |= bit;
            continue;
        }
        if(mii->mir(mii, phyno, Bmsr) == -1)
            continue;
        r = mii->mir(mii, phyno, Phyidr1);
        oui = (r & 0x3FFF)<<6;
        r = mii->mir(mii, phyno, Phyidr2);
        oui |= r>>10;
        if(oui == 0xFFFFF || oui == 0)
            continue;

        if((miiphy = malloc(sizeof(MiiPhy))) == nil)
            continue;

        miiphy->mii = mii;
        miiphy->oui = oui;
        miiphy->phyno = phyno;

        miiphy->anar = ~0;
        miiphy->fc = ~0;
        miiphy->mscr = ~0;

        mii->phy[phyno] = miiphy;
        if(mii->curphy == nil)
            mii->curphy = miiphy;
```

```
        mii->mask |= bit;
        mii->nphy++;

        rmask |= bit;
    }
    return rmask;
}
```

⟨*function* miimir 283a⟩≡ (286a)

```
int
miimir(Mii* mii, int r)
{
    if(mii == nil || mii->ctlr == nil || mii->curphy == nil)
        return -1;
    return mii->mir(mii, mii->curphy->phyno, r);
}
```

⟨*function* miimiw 283b⟩≡ (286a)

```
int
miimiw(Mii* mii, int r, int data)
{
    if(mii == nil || mii->ctlr == nil || mii->curphy == nil)
        return -1;
    return mii->miw(mii, mii->curphy->phyno, r, data);
}
```

⟨*function* miireset 283c⟩≡ (286a)

```
int
miireset(Mii* mii)
{
    int bmcr;

    if(mii == nil || mii->ctlr == nil || mii->curphy == nil)
        return -1;
    bmcr = mii->mir(mii, mii->curphy->phyno, Bmcr);
    bmcr |= BmcrR;
    mii->miw(mii, mii->curphy->phyno, Bmcr, bmcr);
    arch_microdelay(1);

    return 0;
}
```

⟨*function* miiane 283d⟩≡ (286a)

```
int
miiane(Mii* mii, int a, int p, int e)
{
    int anar, bmsr, mscr, r, phyno;

    if(mii == nil || mii->ctlr == nil || mii->curphy == nil)
        return -1;
    phyno = mii->curphy->phyno;

    bmsr = mii->mir(mii, phyno, Bmsr);
    if(!(bmsr & BmsrAna))
        return -1;

    if(a != ~0)
        anar = (AnaTXFD|AnaTXHD|Ana10FD|Ana10HD) & a;
    else if(mii->curphy->anar != ~0)
        anar = mii->curphy->anar;
```

```
    else{
        anar = mii->mir(mii, phyno, Anar);
        anar &= ~(AnaAP|AnaP|AnaT4|AnaTXFD|AnaTXHD|Ana10FD|Ana10HD);
        if(bmsr & Bmsr10THD)
            anar |= Ana10HD;
        if(bmsr & Bmsr10TFD)
            anar |= Ana10FD;
        if(bmsr & Bmsr100TXHD)
            anar |= AnaTXHD;
        if(bmsr & Bmsr100TXFD)
            anar |= AnaTXFD;
    }
    mii->curphy->anar = anar;

    if(p != ~0)
        anar |= (AnaAP|AnaP) & p;
    else if(mii->curphy->fc != ~0)
        anar |= mii->curphy->fc;
    mii->curphy->fc = (AnaAP|AnaP) & anar;

    if(bmsr & BmsrEs){
        mscr = mii->mir(mii, phyno, Mscr);
        mscr &= ~(Mscr1000TFD|Mscr1000THD);
        if(e != ~0)
            mscr |= (Mscr1000TFD|Mscr1000THD) & e;
        else if(mii->curphy->mscr != ~0)
            mscr = mii->curphy->mscr;
        else{
            r = mii->mir(mii, phyno, Esr);
            if(r & Esr1000THD)
                mscr |= Mscr1000THD;
            if(r & Esr1000TFD)
                mscr |= Mscr1000TFD;
        }
        mii->curphy->mscr = mscr;
        mii->miw(mii, phyno, Mscr, mscr);
    }
    mii->miw(mii, phyno, Anar, anar);

    r = mii->mir(mii, phyno, Bmcr);
    if(!(r & BmcrR)){
        r |= BmcrAne|BmcrRan;
        mii->miw(mii, phyno, Bmcr, r);
    }

    return 0;
}
```

⟨*function* miistatus 284⟩≡ (286a)

```
int
miistatus(Mii* mii)
{
    MiiPhy *phy;
    int anlpar, bmsr, p, r, phyno;

    if(mii == nil || mii->ctlr == nil || mii->curphy == nil)
        return -1;
    phy = mii->curphy;
    phyno = phy->phyno;
```

```
/*
 * Check Auto-Negotiation is complete and link is up.
 * (Read status twice as the Ls bit is sticky).
 */
bmsr = mii->mir(mii, phyno, Bmsr);
if(!(bmsr & (BmsrAnc|BmsrAna))) {
    // print("miistatus: auto-neg incomplete\n");
    return -1;
}

bmsr = mii->mir(mii, phyno, Bmsr);
if(!(bmsr & BmsrLs)){
    // print("miistatus: link down\n");
    phy->link = 0;
    return -1;
}

phy->speed = phy->fd = phy->rfc = phy->tfc = 0;
if(phy->mscr){
    r = mii->mir(mii, phyno, Mssr);
    if((phy->mscr & Mscr1000TFD) && (r & Mssr1000TFD)){
        phy->speed = 1000;
        phy->fd = 1;
    }
    else if((phy->mscr & Mscr1000THD) && (r & Mssr1000THD))
        phy->speed = 1000;
}

anlpar = mii->mir(mii, phyno, Anlpar);
if(phy->speed == 0){
    r = phy->anar & anlpar;
    if(r & AnaTXFD){
        phy->speed = 100;
        phy->fd = 1;
    }
    else if(r & AnaTXHD)
        phy->speed = 100;
    else if(r & Ana10FD){
        phy->speed = 10;
        phy->fd = 1;
    }
    else if(r & Ana10HD)
        phy->speed = 10;
}
if(phy->speed == 0) {
    // print("miistatus: phy speed 0\n");
    return -1;
}

if(phy->fd){
    p = phy->fc;
    r = anlpar & (AnaAP|AnaP);
    if(p == AnaAP && r == (AnaAP|AnaP))
        phy->tfc = 1;
    else if(p == (AnaAP|AnaP) && r == AnaAP)
        phy->rfc = 1;
    else if((p & AnaP) && (r & AnaP))
        phy->rfc = phy->tfc = 1;
}
```

```
    phy->link = 1;

    return 0;
}
```

⟨kernel/network/386/ethermii.c 286a⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "io.h"
#include "../port/error.h"
#include "../port/netif.h"
#include "../port/etherif.h"

#include "ethermii.h"

⟨function mii 282⟩

⟨function miimir 283a⟩

⟨function miimiw 283b⟩

⟨function miireset 283c⟩

⟨function miiane 283d⟩

⟨function miistatus 284⟩
```

I.6.8 kernel/network/386/ethermii.h

⟨enum _anon_ (kernel/network/386/ethermii.h) 286b⟩≡ (288c)

```
enum {          /* registers */
  Bmcr    = 0x00,   /* Basic Mode Control */
  Bmsr    = 0x01,   /* Basic Mode Status */
  Phyidr1   = 0x02,   /* PHY Identifier #1 */
  Phyidr2   = 0x03,   /* PHY Identifier #2 */
  Anar    = 0x04,   /* Auto-Negotiation Advertisement */
  Anlpar    = 0x05,   /* AN Link Partner Ability */
  Aner    = 0x06,   /* AN Expansion */
  Annptr    = 0x07,   /* AN Next Page TX */
  Annprr    = 0x08,   /* AN Next Page RX */
  Mscr    = 0x09,   /* MASTER-SLAVE Control */
  Mssr    = 0x0A,   /* MASTER-SLAVE Status */
  Esr   = 0x0F,   /* Extended Status */

  NMiiPhyr  = 32,
  NMiiPhy   = 32,
};
```

⟨enum _anon_ (kernel/network/386/ethermii.h)2 286c⟩≡ (288c)

```
enum {          /* Bmcr */
  BmcrSs1   = 0x0040, /* Speed Select[1] */
  BmcrCte   = 0x0080, /* Collision Test Enable */
  BmcrDm    = 0x0100, /* Duplex Mode */
  BmcrRan   = 0x0200, /* Restart Auto-Negotiation */
  BmcrI   = 0x0400, /* Isolate */
  BmcrPd    = 0x0800, /* Power Down */
```

```
    BmcrAne   = 0x1000, /* Auto-Negotiation Enable */
    BmcrSs0   = 0x2000, /* Speed Select[0] */
    BmcrLe    = 0x4000, /* Loopback Enable */
    BmcrR   = 0x8000, /* Reset */
  };
```

⟨*enum* _anon_ (kernel/network/386/ethermii.h)3 287a⟩≡ (288c)

```
  enum {          /* Bmsr */
    BmsrEc    = 0x0001, /* Extended Capability */
    BmsrJd    = 0x0002, /* Jabber Detect */
    BmsrLs    = 0x0004, /* Link Status */
    BmsrAna   = 0x0008, /* Auto-Negotiation Ability */
    BmsrRf    = 0x0010, /* Remote Fault */
    BmsrAnc   = 0x0020, /* Auto-Negotiation Complete */
    BmsrPs    = 0x0040, /* Preamble Suppression Capable */
    BmsrEs    = 0x0100, /* Extended Status */
    Bmsr100T2HD = 0x0200, /* 100BASE-T2 HD Capable */
    Bmsr100T2FD = 0x0400, /* 100BASE-T2 FD Capable */
    Bmsr10THD = 0x0800, /* 10BASE-T HD Capable */
    Bmsr10TFD = 0x1000, /* 10BASE-T FD Capable */
    Bmsr100TXHD = 0x2000, /* 100BASE-TX HD Capable */
    Bmsr100TXFD = 0x4000, /* 100BASE-TX FD Capable */
    Bmsr100T4 = 0x8000, /* 100BASE-T4 Capable */
  };
```

⟨*enum* _anon_ (kernel/network/386/ethermii.h)4 287b⟩≡ (288c)

```
  enum {          /* Anar/Anlpar */
    Ana10HD   = 0x0020, /* Advertise 10BASE-T */
    Ana10FD   = 0x0040, /* Advertise 10BASE-T FD */
    AnaTXHD   = 0x0080, /* Advertise 100BASE-TX */
    AnaTXFD   = 0x0100, /* Advertise 100BASE-TX FD */
    AnaT4   = 0x0200, /* Advertise 100BASE-T4 */
    AnaP    = 0x0400, /* Pause */
    AnaAP   = 0x0800, /* Asymmetrical Pause */
    AnaRf   = 0x2000, /* Remote Fault */
    AnaAck    = 0x4000, /* Acknowledge */
    AnaNp   = 0x8000, /* Next Page Indication */
  };
```

⟨*enum* _anon_ (kernel/network/386/ethermii.h)5 287c⟩≡ (288c)

```
  enum {          /* Mscr */
    Mscr1000THD = 0x0100, /* Advertise 1000BASE-T HD */
    Mscr1000TFD = 0x0200, /* Advertise 1000BASE-T FD */
  };
```

⟨*enum* _anon_ (kernel/network/386/ethermii.h)6 287d⟩≡ (288c)

```
  enum {          /* Mssr */
    Mssr1000THD = 0x0400, /* Link Partner 1000BASE-T HD able */
    Mssr1000TFD = 0x0800, /* Link Partner 1000BASE-T FD able */
  };
```

⟨*enum* _anon_ (kernel/network/386/ethermii.h)7 287e⟩≡ (288c)

```
  enum {          /* Esr */
    Esr1000THD  = 0x1000, /* 1000BASE-T HD Capable */
    Esr1000TFD  = 0x2000, /* 1000BASE-T FD Capable */
    Esr1000XHD  = 0x4000, /* 1000BASE-X HD Capable */
    Esr1000XFD  = 0x8000, /* 1000BASE-X FD Capable */
  };
```

⟨*struct* Mii 288a⟩≡ (288c)

```
struct Mii {
  Lock;
  int nphy;
  int mask;
  MiiPhy* phy[NMiiPhy];
  MiiPhy* curphy;

  void* ctlr;
  int (*mir)(Mii*, int, int);
  int (*miw)(Mii*, int, int, int);
};
```

⟨*struct* MiiPhy 288b⟩≡ (288c)

```
struct MiiPhy {
  Mii*  mii;
  int oui;
  int phyno;

  int anar;
  int fc;
  int mscr;

  int link;
  int speed;
  int fd;
  int rfc;
  int tfc;
};
```

⟨kernel/network/386/ethermii.h 288c⟩≡

```
typedef struct Mii Mii;
typedef struct MiiPhy MiiPhy;
```

⟨*enum* _anon_ (kernel/network/386/ethermii.h) 286b⟩

⟨*enum* _anon_ (kernel/network/386/ethermii.h)2 286c⟩

⟨*enum* _anon_ (kernel/network/386/ethermii.h)3 287a⟩

⟨*enum* _anon_ (kernel/network/386/ethermii.h)4 287b⟩

⟨*enum* _anon_ (kernel/network/386/ethermii.h)5 287c⟩

⟨*enum* _anon_ (kernel/network/386/ethermii.h)6 287d⟩

⟨*enum* _anon_ (kernel/network/386/ethermii.h)7 287e⟩

⟨*struct* Mii 288a⟩

⟨*struct* MiiPhy 288b⟩

```
extern int mii(Mii*, int);
extern int miiane(Mii*, int, int, int);
extern int miimir(Mii*, int);
extern int miimiw(Mii*, int, int);
extern int miireset(Mii*);
extern int miistatus(Mii*);
```

I.7 kernel/network/arm/

I.7.1 network/arm/devether.c

⟨*global* etherxx*(arm)* 289a⟩≡ (298)

```
static Ether *etherxx[MaxEther];
```

⟨*function* etherattach*(arm)* 289b⟩≡ (298)

```
Chan*
etherattach(char* spec)
{
    int ctlrno;
    char *p;
    Chan *chan;

    ctlrno = 0;
    if(spec && *spec){
        ctlrno = strtoul(spec, &p, 0);
        if((ctlrno == 0 && p == spec) || *p != 0)
            error(Ebadarg);
        if(ctlrno < 0 || ctlrno >= MaxEther)
            error(Ebadarg);
    }
    if(etherxx[ctlrno] == 0)
        error(Enodev);

    chan = devattach('l', spec);
    if(waserror()){
        chanfree(chan);
        nexterror();
    }
    chan->dev = ctlrno;
    if(etherxx[ctlrno]->attach)
        etherxx[ctlrno]->attach(etherxx[ctlrno]);
    poperror();
    return chan;
}
```

⟨*function* etherwalk*(arm)* 289c⟩≡ (298)

```
static Walkqid*
etherwalk(Chan* chan, Chan* nchan, char** name, int nname)
{
    return netifwalk(etherxx[chan->dev], chan, nchan, name, nname);
}
```

⟨*function* etherstat*(arm)* 289d⟩≡ (298)

```
static int
etherstat(Chan* chan, uchar* dp, int n)
{
    return netifstat(etherxx[chan->dev], chan, dp, n);
}
```

⟨*function* etheropen*(arm)* 289e⟩≡ (298)

```
static Chan*
etheropen(Chan* chan, int omode)
{
    return netifopen(etherxx[chan->dev], chan, omode);
}
```

⟨*function* ethercreate*(arm)* 290a⟩≡ (298)

```
static void
ethercreate(Chan*, char*, int, ulong)
{
}
```

⟨*function* etherclose*(arm)* 290b⟩≡ (298)

```
static void
etherclose(Chan* chan)
{
    netifclose(etherxx[chan->dev], chan);
}
```

⟨*function* etherread*(arm)* 290c⟩≡ (298)

```
static long
etherread(Chan* chan, void* buf, long n, vlong off)
{
    Ether *ether;
    ulong offset = off;

    ether = etherxx[chan->dev];
    if((chan->qid.type & QTDIR) == 0 && ether->ifstat){
        /*
         * With some controllers it is necessary to reach
         * into the chip to extract statistics.
         */
        if(NETTYPE(chan->qid.path) == Nifstatqid)
            return ether->ifstat(ether, buf, n, offset);
        else if(NETTYPE(chan->qid.path) == Nstatqid)
            ether->ifstat(ether, buf, 0, offset);
    }

    return netifread(ether, chan, buf, n, offset);
}
```

⟨*function* etherbread*(arm)* 290d⟩≡ (298)

```
static Block*
etherbread(Chan* chan, long n, ulong offset)
{
    return netifbread(etherxx[chan->dev], chan, n, offset);
}
```

⟨*function* etherwstat*(arm)* 290e⟩≡ (298)

```
static int
etherwstat(Chan* chan, uchar* dp, int n)
{
    return netifwstat(etherxx[chan->dev], chan, dp, n);
}
```

⟨*function* etherrtrace*(arm)* 290f⟩≡ (298)

```
static void
etherrtrace(Netfile* f, Etherpkt* pkt, int len)
{
    int i, n;
    Block *bp;

    if(qwindow(f->in) <= 0)
        return;
    if(len > 58)
        n = 58;
```

```
    else
        n = len;
    bp = iallocb(64);
    if(bp == nil)
        return;
    memmove(bp->wp, pkt->d, n);
    i = TK2MS(CPUS(0)->ticks);
    bp->wp[58] = len>>8;
    bp->wp[59] = len;
    bp->wp[60] = i>>24;
    bp->wp[61] = i>>16;
    bp->wp[62] = i>>8;
    bp->wp[63] = i;
    bp->wp += 64;
    qpass(f->in, bp);
}
```

⟨*function* etheriq*(arm)* 291⟩≡ (298)

```
Block*
etheriq(Ether* ether, Block* bp, int fromwire)
{
    Etherpkt *pkt;
    ushort type;
    int len, multi, tome, fromme;
    Netfile **ep, *f, **fp, *fx;
    Block *xbp;

    ether->inpackets++;

    pkt = (Etherpkt*)bp->rp;
    len = BLEN(bp);
    type = (pkt->type[0]<<8)|pkt->type[1];
    fx = 0;
    ep = &ether->f[Ntypes];

    multi = pkt->d[0] & 1;
    /* check for valid multicast addresses */
    if(multi && memcmp(pkt->d, ether->bcast, sizeof(pkt->d)) != 0 &&
        ether->prom == 0){
        if(!activemulti(ether, pkt->d, sizeof(pkt->d))){
            if(fromwire){
                freeb(bp);
                bp = 0;
            }
            return bp;
        }
    }
    /* is it for me? */
    tome = memcmp(pkt->d, ether->ea, sizeof(pkt->d)) == 0;
    fromme = memcmp(pkt->s, ether->ea, sizeof(pkt->s)) == 0;

    /*
     * Multiplex the packet to all the connections which want it.
     * If the packet is not to be used subsequently (fromwire != 0),
     * attempt to simply pass it into one of the connections, thereby
     * saving a copy of the data (usual case hopefully).
     */
    for(fp = ether->f; fp < ep; fp++){
        if((f = *fp) != nil && (f->type == type || f->type < 0) &&
            (tome || multi || f->prom)){
```

```
            /* Don't want to hear bridged packets */
            if(f->bridge && !fromwire && !fromme)
                continue;
            if(!f->headersonly){
                if(fromwire && fx == 0)
                    fx = f;
                else if(xbp = iallocb(len)){
                    memmove(xbp->wp, pkt, len);
                    xbp->wp += len;
                    if(qpass(f->in, xbp) < 0)
                        ether->soverflows++;
                }
                else
                    ether->soverflows++;
            }
            else
                etherrtrace(f, pkt, len);
        }
    }

    if(fx){
        if(qpass(fx->in, bp) < 0)
            ether->soverflows++;
        return 0;
    }
    if(fromwire){
        freeb(bp);
        return 0;
    }
    return bp;
}
```

⟨*function* etheroq*(arm)* 292⟩≡ (298)

```
static int
etheroq(Ether* ether, Block* bp)
{
    int len, loopback, s;
    Etherpkt *pkt;

    ether->outpackets++;

    /*
     * Check if the packet has to be placed back onto the input queue,
     * i.e. if it's a loopback or broadcast packet or the interface is
     * in promiscuous mode.
     * If it's a loopback packet indicate to etheriq that the data isn't
     * needed and return, etheriq will pass-on or free the block.
     * To enable bridging to work, only packets that were originated
     * by this interface are fed back.
     */
    pkt = (Etherpkt*)bp->rp;
    len = BLEN(bp);
    loopback = memcmp(pkt->d, ether->ea, sizeof(pkt->d)) == 0;
    if(loopback || memcmp(pkt->d, ether->bcast, sizeof(pkt->d)) == 0 || ether->prom){
        s = arch_splhi();
        etheriq(ether, bp, 0);
        arch_splx(s);
    }

    if(!loopback){
```

```
        qbwrite(ether->oq, bp);
        if(ether->transmit != nil)
            ether->transmit(ether);
    } else
        freeb(bp);

    return len;
}
```

⟨*function* etherwrite*(arm)* 293a⟩≡ (298)

```
static long
etherwrite(Chan* chan, void* buf, long n, vlong)
{
    Ether *ether;
    Block *bp;
    int nn, onoff;
    Cmdbuf *cb;

    ether = etherxx[chan->dev];
    if(NETTYPE(chan->qid.path) != Ndataqid) {
        nn = netifwrite(ether, chan, buf, n);
        if(nn >= 0)
            return nn;
        cb = parsecmd(buf, n);
        if(cb->f[0] && strcmp(cb->f[0], "nonblocking") == 0){
            if(cb->nf <= 1)
                onoff = 1;
            else
                onoff = atoi(cb->f[1]);
            qnoblock(ether->oq, onoff);
            free(cb);
            return n;
        }
        free(cb);
        if(ether->ctl!=nil)
            return ether->ctl(ether,buf,n);

        error(Ebadctl);
    }

    if(n > ether->maxmtu)
        error(Etoobig);
    if(n < ether->minmtu)
        error(Etoosmall);

    bp = allocb(n);
    if(waserror()){
        freeb(bp);
        nexterror();
    }
    memmove(bp->rp, buf, n);
    memmove(bp->rp+Eaddrlen, ether->ea, Eaddrlen);
    poperror();
    bp->wp += n;

    return etheroq(ether, bp);
}
```

⟨*function* etherbwrite*(arm)* 293b⟩≡ (298)

```
static long
```

```
etherbwrite(Chan* chan, Block* bp, ulong)
{
    Ether *ether;
    long n;

    n = BLEN(bp);
    if(NETTYPE(chan->qid.path) != Ndataqid){
        if(waserror()) {
            freeb(bp);
            nexterror();
        }
        n = etherwrite(chan, bp->rp, n, 0);
        poperror();
        freeb(bp);
        return n;
    }
    ether = etherxx[chan->dev];

    if(n > ether->maxmtu){
        freeb(bp);
        error(Etoobig);
    }
    if(n < ether->minmtu){
        freeb(bp);
        error(Etoosmall);
    }

    return etheroq(ether, bp);
}
```

⟨*global* cards*(arm)* 294a⟩≡ (298)

```
static struct {
    char* type;
    int (*reset)(Ether*);
} cards[MaxEther+1];
```

⟨*function* addethercard*(arm)* 294b⟩≡ (298)

```
void
addethercard(char* t, int (*r)(Ether*))
{
    static int ncard;

    if(ncard == MaxEther)
        panic("too many ether cards");
    cards[ncard].type = t;
    cards[ncard].reset = r;
    ncard++;
}
```

⟨*function* parseether*(arm)* 294c⟩≡ (298)

```
int
parseether(uchar *to, char *from)
{
    char nip[4];
    char *p;
    int i;

    p = from;
    for(i = 0; i < Eaddrlen; i++){
        if(*p == 0)
```

```
            return -1;
        nip[0] = *p++;
        if(*p == 0)
            return -1;
        nip[1] = *p++;
        nip[2] = 0;
        to[i] = strtoul(nip, 0, 16);
        if(*p == ':')
            p++;
    }
    return 0;
}
```

⟨*function* etherreset*(arm)* 295⟩≡ (298)

```
static void
etherreset(void)
{
    Ether *ether;
    int i, n, ctlrno;
    char name[KNAMELEN], buf[128];

    for(ether = 0, ctlrno = 0; ctlrno < MaxEther; ctlrno++){
        if(ether == 0)
            ether = malloc(sizeof(Ether));
        memset(ether, 0, sizeof(Ether));
        ether->ctlrno = ctlrno;
        ether->mbps = 10;
        ether->minmtu = ETHERMINTU;
        ether->maxmtu = ETHERMAXTU;

        if(archether(ctlrno, ether) <= 0)
            continue;

        if(arch_isaconfig("ether", ctlrno, ether) == 0){
//   free(ether);
//   return nil;
            continue;
        }
        for(n = 0; cards[n].type; n++){
            if(cistrcmp(cards[n].type, ether->type))
                continue;
            for(i = 0; i < ether->nopt; i++)
                if(cistrncmp(ether->opt[i], "ea=", 3) == 0){
                    if(parseether(ether->ea,
                        &ether->opt[i][3]) == -1)
                        memset(ether->ea, 0, Eaddrlen);
                } else if(cistrcmp(ether->opt[i],
                    "100BASE-TXFD") == 0)
                    ether->mbps = 100;
            if(cards[n].reset(ether))
                break;
            snprint(name, sizeof(name), "ether%d", ctlrno);

            if(ether->interrupt != nil && ether->irq >= 0)
                arch_intrenable(ether->irq, ether->interrupt,
                    ether, 0, name);

            i = snprint(buf, sizeof buf,
                "#l%d: %s: %dMbps port %#lux irq %d",
                ctlrno, ether->type, ether->mbps, ether->port,
```

```
                ether->irq);
            if(ether->mem)
                i += snprint(buf+i, sizeof buf - i,
                    " addr %#lux", PADDR(ether->mem));
            if(ether->size)
                i += snprint(buf+i, sizeof buf - i,
                    " size %#luX", ether->size);
            i += snprint(buf+i, sizeof buf - i,
                ": %2.2ux%2.2ux%2.2ux%2.2ux%2.2ux%2.2ux",
                ether->ea[0], ether->ea[1], ether->ea[2],
                ether->ea[3], ether->ea[4], ether->ea[5]);
            snprint(buf+i, sizeof buf - i, "\n");
            iprint("%s", buf);  /* it may be too early for print */

            if(ether->mbps >= 1000)
                netifinit(ether, name, Ntypes, 4*1024*1024);
            else if(ether->mbps >= 100)
                netifinit(ether, name, Ntypes, 1024*1024);
            else
                netifinit(ether, name, Ntypes, 65*1024);
            if(ether->oq == 0)
                ether->oq = qopen(ether->limit, Qmsg, 0, 0);
            if(ether->oq == 0)
                panic("etherreset %s", name);
            ether->alen = Eaddrlen;
            memmove(ether->addr, ether->ea, Eaddrlen);
            memset(ether->bcast, 0xFF, Eaddrlen);

            etherxx[ctlrno] = ether;
            ether = 0;
            break;
        }
    }
    if(ether)
        free(ether);
}
```

⟨*function* ethershutdown*(arm)* 296a⟩≡ (298)

```
static void
ethershutdown(void)
{
    Ether *ether;
    int i;

    for(i = 0; i < MaxEther; i++){
        ether = etherxx[i];
        if(ether == nil)
            continue;
        if(ether->shutdown == nil) {
            print("#l%d: no shutdown function\n", i);
            continue;
        }
        (*ether->shutdown)(ether);
    }
}
```

⟨*constant* POLY*(arm)* 296b⟩≡ (298)

```
#define POLY 0xedb88320
```

⟨*function* ethercrc*(arm)* 297a⟩≡ (298)

```
/* really slow 32 bit crc for ethers */
ulong
ethercrc(uchar *p, int len)
{
    int i, j;
    ulong crc, b;

    crc = 0xffffffff;
    for(i = 0; i < len; i++){
        b = *p++;
        for(j = 0; j < 8; j++){
            crc = (crc>>1) ^ (((crc^b) & 1) ? POLY : 0);
            b >>= 1;
        }
    }
    return crc;
}
```

⟨*function* dumpoq*(arm)* 297b⟩≡ (298)

```
void
dumpoq(Queue *oq)
{
    if (oq == nil)
        print("no outq! ");
    else if (qisclosed(oq))
        print("outq closed ");
    else if (qfull(oq))
        print("outq full ");
    else
        print("outq %d ", qlen(oq));
}
```

⟨*function* dumpnetif*(arm)* 297c⟩≡ (298)

```
void
dumpnetif(Netif *netif)
{
    print("netif %s ", netif->name);
    print("limit %d mbps %d link %d ",
        netif->limit, netif->mbps, netif->link);
    print("inpkts %lld outpkts %lld errs %d\n",
        netif->inpackets, netif->outpackets,
        netif->crcs + netif->oerrs + netif->frames + netif->overflows +
        netif->buffs + netif->soverflows);
}
```

⟨*global* etherdevtab*(arm)* 297d⟩≡ (298)

```
Dev etherdevtab = {
    .dc = 'l',
    .name = "ether",

    .reset = etherreset,
    .init = devinit,
    .shutdown = ethershutdown,
    .attach = etherattach,
    .walk = etherwalk,
    .stat = etherstat,
    .open = etheropen,
    .create = ethercreate,
    .close = etherclose,
```

```
    .read = etherread,
    .bread = etherbread,
    .write = etherwrite,
    .bwrite = etherbwrite,
    .remove = devremove,
    .wstat = etherwstat,
};
```

⟨kernel/network/arm/devether.c 298⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "../port/error.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "io.h"

#include "../port/netif.h"
#include "../port/etherif.h"

extern int archether(unsigned ctlno, Ether *ether);

⟨global etherxx(arm) 289a⟩

⟨function etherattach(arm) 289b⟩

⟨function etherwalk(arm) 289c⟩

⟨function etherstat(arm) 289d⟩

⟨function etheropen(arm) 289e⟩

⟨function ethercreate(arm) 290a⟩

⟨function etherclose(arm) 290b⟩

⟨function etherread(arm) 290c⟩

⟨function etherbread(arm) 290d⟩

⟨function etherwstat(arm) 290e⟩

⟨function etherrtrace(arm) 290f⟩

⟨function etheriq(arm) 291⟩

⟨function etheroq(arm) 292⟩

⟨function etherwrite(arm) 293a⟩

⟨function etherbwrite(arm) 293b⟩

⟨global cards(arm) 294a⟩

⟨function addethercard(arm) 294b⟩

⟨function parseether(arm) 294c⟩

⟨function etherreset(arm) 295⟩
```

⟨*function* ethershutdown*(arm)* 296a⟩

⟨*constant* POLY*(arm)* 296b⟩

⟨*function* ethercrc*(arm)* 297a⟩

⟨*function* dumpoq*(arm)* 297b⟩

⟨*function* dumpnetif*(arm)* 297c⟩

⟨*global* etherdevtab*(arm)* 297d⟩

I.8 kernel/network/ip/

I.8.1 kernel/network/ip/devip.c

⟨*function* ipreset 299a⟩≡ (302c)

```
static void
ipreset(void)
{
    nullmediumlink();
    pktmediumlink();

    fmtinstall('i', eipfmt);
    fmtinstall('I', eipfmt);
    fmtinstall('E', eipfmt);
    fmtinstall('V', eipfmt);
    fmtinstall('M', eipfmt);
}
```

⟨*function* ipstat 299b⟩≡ (302c)

```
static int
ipstat(Chan* c, uchar* db, int n)
{
    return devstat(c, db, n, nil, 0, ipgen);
}
```

⟨*function* incoming 299c⟩≡ (302c)

```
static int
incoming(void* arg)
{
    Conv *conv;

    conv = arg;
    return conv->incall != nil;
}
```

⟨*function* ipwstat 299d⟩≡ (302c)

```
static int
ipwstat(Chan *c, uchar *dp, int n)
{
    Dir d;
    Conv *cv;
    Fs *f;
    Proto *p;

    f = ipfs[c->dev];
```

```
    switch(TYPE(c->qid)) {
    default:
        error(Eperm);
        break;
    case Qctl:
    case Qdata:
        break;
    }

    n = convM2D(dp, n, &d, nil);
    if(n > 0){
        p = f->p[PROTO(c->qid)];
        cv = p->conv[CONV(c->qid)];
        if(!iseve() && strcmp(ATTACHER(c), cv->owner) != 0)
            error(Eperm);
        if(d.uid[0])
            kstrdup(&cv->owner, d.uid);
        cv->perm = d.mode & 0777;
    }
    return n;
}
```

⟨*enum* _anon_ (kernel/network/ip/devip.c)2 300a⟩≡ (302c)

```
enum
{
    ⟨constant Statelen 53d⟩
};
```

⟨*function* ipbread 300b⟩≡ (302c)

```
static Block*
ipbread(Chan* ch, long n, ulong offset)
{
    Conv *c;
    Proto *x;
    Fs *f;

    switch(TYPE(ch->qid)){
    case Qdata:
        f = ipfs[ch->dev];
        x = f->p[PROTO(ch->qid)];
        c = x->conv[CONV(ch->qid)];
        return qbread(c->rq, n);
    default:
        return devbread(ch, n, offset);
    }
}
```

⟨*function* setladdr 300c⟩≡ (302c)

```
/*
 *  set local address to be that of the ifc closest to remote address
 */
static void
setladdr(Conv* c)
{
    findlocalip(c->p->f, c->laddr, c->raddr);
}
```

⟨*function* setluniqueport 300d⟩≡ (302c)

```
/*
 *  set a local port making sure the quad of raddr,rport,laddr,lport is unique
```

```
 */
char*
setluniqueport(Conv* c, int lport)
{
    Proto *p;
    Conv *xp;
    int x;

    p = c->p;

    qlock(p);
    for(x = 0; x < p->nc; x++){
        xp = p->conv[x];
        if(xp == nil)
            break;
        if(xp == c)
            continue;
        if((xp->state == Connected || xp->state == Announced)
        && xp->lport == lport
        && xp->rport == c->rport
        && ipcmp(xp->raddr, c->raddr) == 0
        && ipcmp(xp->laddr, c->laddr) == 0){
            qunlock(p);
            return "address in use";
        }
    }
    c->lport = lport;
    qunlock(p);
    return nil;
}
```

⟨*function* lportinuse 301a⟩≡ (302c)

```
/*
 * is lport in use by anyone?
 */
static int
lportinuse(Proto *p, ushort lport)
{
    int x;

    for(x = 0; x < p->nc && p->conv[x]; x++)
        if(p->conv[x]->lport == lport)
            return 1;
    return 0;
}
```

⟨*function* ipbwrite 301b⟩≡ (302c)

```
static long
ipbwrite(Chan* ch, Block* bp, ulong offset)
{
    Conv *c;
    Proto *x;
    Fs *f;
    int n;

    switch(TYPE(ch->qid)){
    case Qdata:
        f = ipfs[ch->dev];
        x = f->p[PROTO(ch->qid)];
        c = x->conv[CONV(ch->qid)];
```

```
        if(c->wq == nil)
            error(Eperm);

        if(bp->next)
            bp = concatblock(bp);
        n = BLEN(bp);
        qbwrite(c->wq, bp);
        return n;
    default:
        return devbwrite(ch, bp, offset);
    }
}
```

⟨*function* Fsrcvpcolx 302a⟩≡ (302c)

```
Proto*
Fsrcvpcolx(Fs *f, uchar proto)
{
    return f->t2p[proto];
}
```

⟨*function* ndbwrite 302b⟩≡ (302c)

```
long
ndbwrite(Fs *f, char *a, ulong off, int n)
{
    if(off > strlen(f->ndb))
        error(Eio);
    if(off+n >= sizeof(f->ndb))
        error(Eio);
    memmove(f->ndb+off, a, n);
    f->ndb[off+n] = 0;
    f->ndbvers++;
    f->ndbmtime = seconds();
    return n;
}
```

⟨kernel/network/ip/devip.c 302c⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
#include    "../port/error.h"
#include    "../ip/ip.h"

⟨enum qid((kernel/network/ip/devip.c)) 26f⟩

⟨enum misc((kernel/network/ip/devip.c)) 26g⟩

⟨macro TYPE 27c⟩
⟨macro CONV 27b⟩
⟨macro PROTO 27a⟩
⟨macro QID 26h⟩

⟨global network 36c⟩

⟨global fslock 21h⟩
⟨global ipfs 21e⟩
//Queue *qlog;
```

```
extern  void nullmediumlink(void);
extern  void pktmediumlink(void);
long ndbwrite(Fs *f, char *a, ulong off, int n);
Conv*   Fsprotoclone(Proto*, char*);
char*   Fsstdbind(Conv*, char**, int);
void    closeconv(Conv*);
```

⟨*function* ip3gen 38c⟩

⟨*function* ip2gen 37d⟩

⟨*function* ip1gen 37a⟩

⟨*function* ipgen 36a⟩

⟨*function* ipreset 299a⟩

⟨*function* ipgetfs 31b⟩

⟨*function* newipaux 32c⟩

⟨*macro* ATTACHER 32b⟩

⟨*function* ipattach 31a⟩

⟨*function* ipwalk 35d⟩

⟨*function* ipstat 299b⟩

⟨*function* incoming 299c⟩

⟨*global* m2p 39b⟩

⟨*function* ipopen 39c⟩

⟨*function* ipcreate 35b⟩

⟨*function* ipremove 35c⟩

⟨*function* ipwstat 299d⟩

⟨*function* closeconv 40d⟩

⟨*function* ipclose 40b⟩

⟨*enum* _anon_ (kernel/network/ip/devip.c)2 300a⟩

⟨*function* ipread 41a⟩

⟨*function* ipbread 300b⟩

⟨*function* setladdr 300c⟩

⟨*function* setluniqueport 300d⟩

⟨*function* lportinuse 301a⟩

⟨*function* setlport 48d⟩

⟨*function* setladdrport 49⟩

⟨*function* setraddrport 50a⟩

⟨*function* Fsstdconnect 46b⟩
⟨*function* connected 46a⟩
⟨*function* connectctlmsg 45b⟩

⟨*function* Fsstdannounce 47d⟩

⟨*function* announced 47c⟩
⟨*function* announcectlmsg 47b⟩

⟨*function* Fsstdbind 50d⟩

⟨*function* bindctlmsg 50c⟩

⟨*function* tosctlmsg 69a⟩

⟨*function* ttlctlmsg 69d⟩

⟨*function* ipwrite 41c⟩

⟨*function* ipbwrite 301b⟩

⟨*global* ipdevtab 35a⟩

⟨*function* Fsproto 56b⟩

⟨*function* Fsprotoclone 42b⟩

⟨*function* Fsconnected 46c⟩

⟨*function* Fsrcvpcol 85b⟩

⟨*function* Fsrcvpcolx 302a⟩

⟨*function* Fsnewcall 125⟩

⟨*function* ndbwrite 302b⟩

⟨*function* scaledbnconv 132b⟩

I.8.2 kernel/network/ip/arp.c

⟨*enum* _anon_ (kernel/network/ip/arp.c) 304⟩≡ (313a)

```
/*
 *  address resolution tables
 */

enum
{
    NHASH       = (1<<6),
    NCACHE      = 256,

    AOK     = 1,
    AWAIT       = 2,
};
```

⟨*global* arpstate 305a⟩≡ (313a)

```
char *arpstate[] =
{
    "UNUSED",
    "OK",
    "WAIT",
};
```

⟨*global* Ebadarp 305b⟩≡ (313a)

```
char *Ebadarp = "bad arp";
```

⟨*macro* haship 305c⟩≡ (313a)

```
#define haship(s) ((s)[IPaddrlen-1]%NHASH)
```

⟨*global* ReTransTimer 305d⟩≡ (313a)

```
int     ReTransTimer = RETRANS_TIMER;
```

⟨*function* newarp6 305e⟩≡ (313a)

```
/*
 *  create a new arp entry for an ip address.
 */
static Arpent*
newarp6(Arp *arp, uchar *ip, Ipifc *ifc, int addrxt)
{
    uint t;
    Block *next, *xp;
    Arpent *a, *e, *f, **l;
    Medium *m = ifc->m;
    int empty;

    /* find oldest entry */
    e = &arp->cache[NCACHE];
    a = arp->cache;
    t = a->utime;
    for(f = a; f < e; f++){
        if(f->utime < t){
            t = f->utime;
            a = f;
        }
    }

    /* dump waiting packets */
    xp = a->hold;
    a->hold = nil;

    if(isv4(a->ip)){
        while(xp){
            next = xp->list;
            freeblist(xp);
            xp = next;
        }
    }
    else { /* queue icmp unreachable for rxmitproc later on, w/o arp lock */
        if(xp){
            if(arp->dropl == nil)
                arp->dropf = xp;
            else
                arp->dropl->list = xp;

            for(next = xp->list; next; next = next->list)
```

```
                xp = next;
            arp->dropl = xp;
            wakeup(&arp->rxmtq);
        }
    }

    /* take out of current chain */
    l = &arp->hash[haship(a->ip)];
    for(f = *l; f; f = f->hash){
        if(f == a){
            *l = a->hash;
            break;
        }
        l = &f->hash;
    }

    /* insert into new chain */
    l = &arp->hash[haship(ip)];
    a->hash = *l;
    *l = a;

    memmove(a->ip, ip, sizeof(a->ip));
    a->utime = NOW;
    a->ctime = 0;
    a->type = m;

    a->rtime = NOW + ReTransTimer;
    a->rxtsrem = MAX_MULTICAST_SOLICIT;
    a->ifc = ifc;
    a->ifcid = ifc->ifcid;

    /* put to the end of re-transmit chain; addrxt is 0 when isv4(a->ip) */
    if(!ipismulticast(a->ip) && addrxt){
        l = &arp->rxmt;
        empty = (*l==nil);

        for(f = *l; f; f = f->nextrxt){
            if(f == a){
                *l = a->nextrxt;
                break;
            }
            l = &f->nextrxt;
        }
        for(f = *l; f; f = f->nextrxt){
            l = &f->nextrxt;
        }
        *l = a;
        if(empty)
            wakeup(&arp->rxmtq);
    }

    a->nextrxt = nil;

    return a;
}
```

⟨*function* cleanarpent 306⟩≡ (313a)

```
/* called with arp qlocked */

void
```

```
cleanarpent(Arp *arp, Arpent *a)
{
    Arpent *f, **l;

    a->utime = 0;
    a->ctime = 0;
    a->type = 0;
    a->state = 0;

    /* take out of current chain */
    l = &arp->hash[haship(a->ip)];
    for(f = *l; f; f = f->hash){
        if(f == a){
            *l = a->hash;
            break;
        }
        l = &f->hash;
    }

    /* take out of re-transmit chain */
    l = &arp->rxmt;
    for(f = *l; f; f = f->nextrxt){
        if(f == a){
            *l = a->nextrxt;
            break;
        }
        l = &f->nextrxt;
    }
    a->nextrxt = nil;
    a->hash = nil;
    a->hold = nil;
    a->last = nil;
    a->ifc = nil;
}
```

⟨*function* arpget 307⟩≡ (313a)

```
/*
 *  fill in the media address if we have it.  Otherwise return an
 *  Arpent that represents the state of the address resolution FSM
 *  for ip.  Add the packet to be sent onto the list of packets
 *  waiting for ip->mac to be resolved.
 */
Arpent*
arpget(Arp *arp, Block *bp, int version, Ipifc *ifc, uchar *ip, uchar *mac)
{
    int hash;
    Arpent *a;
    Medium *type = ifc->m;
    ipaddr v6ip;

    if(version == V4){
        v4tov6(v6ip, ip);
        ip = v6ip;
    }

    qlock(arp);
    hash = haship(ip);
    for(a = arp->hash[hash]; a; a = a->hash){
        if(memcmp(ip, a->ip, sizeof(a->ip)) == 0)
        if(type == a->type)
```

```
            break;
        }

        if(a == nil){
            a = newarp6(arp, ip, ifc, (version != V4));
            a->state = AWAIT;
        }
        a->utime = NOW;
        if(a->state == AWAIT){
            if(bp != nil){
                if(a->hold)
                    a->last->list = bp;
                else
                    a->hold = bp;
                a->last = bp;
                bp->list = nil;
            }
            return a;       /* return with arp qlocked */
        }

        memmove(mac, a->mac, a->type->maclen);

        /* remove old entries */
        if(NOW - a->ctime > 15*60*1000)
            cleanarpent(arp, a);

        qunlock(arp);
        return nil;
    }
```

⟨*function* arprelease 308a⟩≡ (313a)

```
    /*
     * called with arp locked
     */
    void
    arprelease(Arp *arp, Arpent*)
    {
        qunlock(arp);
    }
```

⟨*function* arpresolve 308b⟩≡ (313a)

```
    /*
     * Copy out the mac address from the Arpent.  Return the
     * block waiting to get sent to this mac address.
     *
     * called with arp locked
     */
    Block*
    arpresolve(Arp *arp, Arpent *a, Medium *type, uchar *mac)
    {
        Block *bp;
        Arpent *f, **l;

        if(!isv4(a->ip)){
            l = &arp->rxmt;
            for(f = *l; f; f = f->nextrxt){
                if(f == a){
                    *l = a->nextrxt;
                    break;
                }
```

```
            l = &f->nextrxt;
        }
    }

    memmove(a->mac, mac, type->maclen);
    a->type = type;
    a->state = AOK;
    a->utime = NOW;
    bp = a->hold;
    a->hold = nil;
    qunlock(arp);

    return bp;
}
```

⟨*function* arpenter 309⟩≡ (313a)

```
void
arpenter(Fs *fs, int version, uchar *ip, uchar *mac, int n, int refresh)
{
    Arp *arp;
    Route *r;
    Arpent *a, *f, **l;
    Ipifc *ifc;
    Medium *type;
    Block *bp, *next;
    ipaddr v6ip;

    arp = fs->arp;

    if(n != 6){
//      print("arp: len = %d\n", n);
        return;
    }

    switch(version){
    case V4:
        r = v4lookup(fs, ip, nil);
        v4tov6(v6ip, ip);
        ip = v6ip;
        break;
    case V6:
        r = v6lookup(fs, ip, nil);
        break;
    default:
        panic("arpenter: version %d", version);
        return; /* to supress warnings */
    }

    if(r == nil){
//      print("arp: no route for entry\n");
        return;
    }

    ifc = r->ifc;
    type = ifc->m;

    qlock(arp);
    for(a = arp->hash[haship(ip)]; a; a = a->hash){
        if(a->type != type || (a->state != AWAIT && a->state != AOK))
            continue;
```

```
        if(ipcmp(a->ip, ip) == 0){
            a->state = AOK;
            memmove(a->mac, mac, type->maclen);

            if(version == V6){
                /* take out of re-transmit chain */
                l = &arp->rxmt;
                for(f = *l; f; f = f->nextrxt){
                    if(f == a){
                        *l = a->nextrxt;
                        break;
                    }
                    l = &f->nextrxt;
                }
            }

            a->ifc = ifc;
            a->ifcid = ifc->ifcid;
            bp = a->hold;
            a->hold = nil;
            if(version == V4)
                ip += IPv4off;
            a->utime = NOW;
            a->ctime = a->utime;
            qunlock(arp);

            while(bp){
                next = bp->list;
                if(ifc != nil){
                    if(waserror()){
                        runlock(ifc);
                        nexterror();
                    }
                    rlock(ifc);
                    if(ifc->m != nil)
                        ifc->m->bwrite(ifc, bp, version, ip);
                    else
                        freeb(bp);
                    runlock(ifc);
                    poperror();
                } else
                    freeb(bp);
                bp = next;
            }
            return;
        }
    }

    if(refresh == 0){
        a = newarp6(arp, ip, ifc, 0);
        a->state = AOK;
        a->type = type;
        a->ctime = NOW;
        memmove(a->mac, mac, type->maclen);
    }

    qunlock(arp);
}
```

⟨*enum* _anon_ (kernel/network/ip/arp.c)2 311a⟩≡ (313a)

```
enum
{
    Alinelen=   90,
};
```

⟨*global* aformat 311b⟩≡ (313a)

```
char *aformat = "%-6.6s %-8.8s %-40.40I %-32.32s\n";
```

⟨*function* convmac 311c⟩≡ (313a)

```
static void
convmac(char *p, char *ep, uchar *mac, int n)
{
    while(n-- > 0)
        p = seprint(p, ep, "%2.2ux", *mac++);
}
```

⟨*function* rxmitsols 311d⟩≡ (313a)

```
extern int
rxmitsols(Arp *arp)
{
    uint sflag;
    Block *next, *xp;
    Arpent *a, *b, **l;
    Fs *f;
    ipaddr ipsrc;
    Ipifc *ifc = nil;
    long nrxt;

    qlock(arp);
    f = arp->f;

    a = arp->rxmt;
    if(a==nil){
        nrxt = 0;
        goto dodrops;       /* return nrxt; */
    }
    nrxt = a->rtime - NOW;
    if(nrxt > 3*ReTransTimer/4)
        goto dodrops;       /* return nrxt; */

    for(; a; a = a->nextrxt){
        ifc = a->ifc;
        assert(ifc != nil);
        if((a->rxtsrem <= 0) || !(canrlock(ifc)) || (a->ifcid != ifc->ifcid)){
            xp = a->hold;
            a->hold = nil;

            if(xp){
                if(arp->dropl == nil)
                    arp->dropf = xp;
                else
                    arp->dropl->list = xp;
            }

            cleanarpent(arp, a);
        }
        else
            break;
    }
```

```
    if(a == nil)
        goto dodrops;


    qunlock(arp);   /* for icmpns */
    if((sflag = ipv6anylocal(ifc, ipsrc)) != SRC_UNSPEC)
        icmpns(f, ipsrc, sflag, a->ip, TARG_MULTI, ifc->mac);

    runlock(ifc);
    qlock(arp);

    /* put to the end of re-transmit chain */
    l = &arp->rxmt;
    for(b = *l; b; b = b->nextrxt){
        if(b == a){
            *l = a->nextrxt;
            break;
        }
        l = &b->nextrxt;
    }
    for(b = *l; b; b = b->nextrxt){
        l = &b->nextrxt;
    }
    *l = a;
    a->rxtsrem--;
    a->nextrxt = nil;
    a->rtime = NOW + ReTransTimer;

    a = arp->rxmt;
    if(a==nil)
        nrxt = 0;
    else
        nrxt = a->rtime - NOW;

dodrops:
    xp = arp->dropf;
    arp->dropf = nil;
    arp->dropl = nil;
    qunlock(arp);

    for(; xp; xp = next){
        next = xp->list;
        icmphostunr(f, ifc, xp, Icmp6_adr_unreach, 1);
    }

    return nrxt;

}
```

⟨*function* rxready 312⟩≡ (313a)

```
static int
rxready(void *v)
{
    Arp *arp = (Arp *) v;
    int x;

    x = ((arp->rxmt != nil) || (arp->dropf != nil));

    return x;
}
```

⟨kernel/network/ip/arp.c 313a⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "../port/error.h"

#include "ip.h"
#include "ipv6.h"

⟨enum _anon_ (kernel/network/ip/arp.c) 304⟩

⟨global arpstate 305a⟩

⟨struct Arp 91b⟩

⟨global Ebadarp 305b⟩

⟨macro haship 305c⟩

⟨global ReTransTimer 305d⟩

static void     rxmitproc(void *v);

⟨function arpinit 92b⟩

⟨function newarp6 305e⟩

⟨function cleanarpent 306⟩

⟨function arpget 307⟩

⟨function arprelease 308a⟩

⟨function arpresolve 308b⟩

⟨function arpenter 309⟩

⟨function arpwrite 93e⟩

⟨enum _anon_ (kernel/network/ip/arp.c)2 311a⟩

⟨global aformat 311b⟩

⟨function convmac 311c⟩

⟨function arpread 93c⟩

⟨function rxmitsols 311d⟩

⟨function rxready 312⟩

⟨function rxmitproc 92c⟩
```

I.8.3 kernel/network/ip/icmp.c

⟨struct Icmp 313b⟩≡ (322b)

```
typedef struct Icmp {
    // ip header
    uchar   vihl;       /* Version and header length */
    uchar   tos;        /* Type of service */
    uchar   length[2];  /* packet length */
    uchar   id[2];      /* Identification */
    uchar   frag[2];    /* Fragment information */
    uchar   ttl;        /* Time to live */
    uchar   proto;      /* Protocol */
    uchar   ipcksum[2]; /* Header checksum */
    uchar   src[4];     /* Ip source */
    uchar   dst[4];     /* Ip destination */

    // ICMP specifics
    uchar   type;
    uchar   code;
    uchar   cksum[2];
    uchar   icmpid[2];
    uchar   seq[2];
    uchar   data[1];
} Icmp;
```

⟨*enum* _anon_ (kernel/network/ip/icmp.c) 314a⟩≡ (322b)

```
enum {          /* Packet Types */
    EchoReply   = 0,
    Unreachable = 3,
    SrcQuench   = 4,
    Redirect    = 5,
    EchoRequest = 8,
    TimeExceed  = 11,
    InParmProblem   = 12,
    Timestamp   = 13,
    TimestampReply  = 14,
    InfoRequest = 15,
    InfoReply   = 16,
    AddrMaskRequest = 17,
    AddrMaskReply   = 18,

    Maxtype     = 18,
};
```

⟨*enum* _anon_ (kernel/network/ip/icmp.c)2 314b⟩≡ (322b)

```
enum
{
    MinAdvise   = 24,   /* minimum needed for us to advise another protocol */
};
```

⟨*global* icmpnames 314c⟩≡ (322b)

```
char *icmpnames[Maxtype+1] =
{
[EchoReply]     "EchoReply",
[Unreachable]       "Unreachable",
[SrcQuench]     "SrcQuench",
[Redirect]      "Redirect",
[EchoRequest]       "EchoRequest",
[TimeExceed]        "TimeExceed",
[InParmProblem]     "InParmProblem",
[Timestamp]     "Timestamp",
[TimestampReply]    "TimestampReply",
[InfoRequest]       "InfoRequest",
```

```
[InfoReply]     "InfoReply",
[AddrMaskRequest]   "AddrMaskRequest",
[AddrMaskReply  ]   "AddrMaskReply  ",
};
```

⟨*enum* _anon_ (kernel/network/ip/icmp.c)3 315a⟩≡ (322b)

```
enum {
    IP_ICMPPROTO    = 1,
    ICMP_IPSIZE = 20,
    ICMP_HDRSIZE    = 8,
};
```

⟨*enum* _anon_ (kernel/network/ip/icmp.c)4 315b⟩≡ (322b)

```
enum
{
    InMsgs,
    InErrors,
    OutMsgs,
    CsumErrs,
    LenErrs,
    HlenErrs,

    Nstats,
};
```

⟨*global* statnames((kernel/network/ip/icmp.c)) 315c⟩≡ (322b)

```
static char *statnames[Nstats] =
{
[InMsgs]    "InMsgs",
[InErrors]  "InErrors",
[OutMsgs]   "OutMsgs",
[CsumErrs]  "CsumErrs",
[LenErrs]   "LenErrs",
[HlenErrs]  "HlenErrs",
};
```

⟨*struct* Icmppriv 315d⟩≡ (322b)

```
struct Icmppriv
{
    ulong   stats[Nstats];

    /* message counts */
    ulong   in[Maxtype+1];
    ulong   out[Maxtype+1];
};
```

⟨*function* icmpcreate 315e⟩≡ (322b)

```
static void
icmpcreate(Conv *c)
{
    c->rq = qopen(64*1024, Qmsg, 0, c);
    c->wq = qbypass(icmpkick, c);
}
```

⟨*function* icmpconnect 315f⟩≡ (322b)

```
extern char*
icmpconnect(Conv *c, char **argv, int argc)
{
    char *e;
```

```
    e = Fsstdconnect(c, argv, argc);
    if(e != nil)
        return e;
    Fsconnected(c, e);

    return nil;
}
```

⟨*function* icmpstate 316a⟩≡ (322b)

```
extern int
icmpstate(Conv *c, char *state, int n)
{
    USED(c);
    return snprint(state, n, "%s qin %d qout %d\n",
        "Datagram",
        c->rq ? qlen(c->rq) : 0,
        c->wq ? qlen(c->wq) : 0
    );
}
```

⟨*function* icmpannounce 316b⟩≡ (322b)

```
extern char*
icmpannounce(Conv *c, char **argv, int argc)
{
    char *e;

    e = Fsstdannounce(c, argv, argc);
    if(e != nil)
        return e;
    Fsconnected(c, nil);

    return nil;
}
```

⟨*function* icmpclose 316c⟩≡ (322b)

```
extern void
icmpclose(Conv *c)
{
    qclose(c->rq);
    qclose(c->wq);
    ipmove(c->laddr, IPnoaddr);
    ipmove(c->raddr, IPnoaddr);
    c->lport = 0;
}
```

⟨*function* icmpkick 316d⟩≡ (322b)

```
static void
icmpkick(void *x, Block *bp)
{
    Conv *c = x;
    Icmp *p;
    Icmppriv *ipriv;

    if(bp == nil)
        return;

    if(blocklen(bp) < ICMP_IPSIZE + ICMP_HDRSIZE){
        freeblist(bp);
        return;
    }
```

```
    p = (Icmp *)(bp->rp);
    p->vihl = IP_VER4;
    ipriv = c->p->priv;
    if(p->type <= Maxtype)
        ipriv->out[p->type]++;

    v6tov4(p->dst, c->raddr);
    v6tov4(p->src, c->laddr);
    p->proto = IP_ICMPPROTO;
    hnputs(p->icmpid, c->lport);
    memset(p->cksum, 0, sizeof(p->cksum));
    hnputs(p->cksum, ptclcsum(bp, ICMP_IPSIZE, blocklen(bp) - ICMP_IPSIZE));
    ipriv->stats[OutMsgs]++;
    ipoput4(c->p->f, bp, 0, c->ttl, c->tos, nil);
}
```

⟨*function* icmpttlexceeded 317a⟩≡ (322b)

```
extern void
icmpttlexceeded(Fs *f, uchar *ia, Block *bp)
{
    Block   *nbp;
    Icmp    *p, *np;

    p = (Icmp *)bp->rp;

    netlog(f, Logicmp, "sending icmpttlexceeded -> %V\n", p->src);
    nbp = allocb(ICMP_IPSIZE + ICMP_HDRSIZE + ICMP_IPSIZE + 8);
    nbp->wp += ICMP_IPSIZE + ICMP_HDRSIZE + ICMP_IPSIZE + 8;
    np = (Icmp *)nbp->rp;
    np->vihl = IP_VER4;
    memmove(np->dst, p->src, sizeof(np->dst));
    v6tov4(np->src, ia);
    memmove(np->data, bp->rp, ICMP_IPSIZE + 8);
    np->type = TimeExceed;
    np->code = 0;
    np->proto = IP_ICMPPROTO;
    hnputs(np->icmpid, 0);
    hnputs(np->seq, 0);
    memset(np->cksum, 0, sizeof(np->cksum));
    hnputs(np->cksum, ptclcsum(nbp, ICMP_IPSIZE, blocklen(nbp) - ICMP_IPSIZE));
    ipoput4(f, nbp, 0, MAXTTL, DFLTTOS, nil);

}
```

⟨*function* icmpunreachable 317b⟩≡ (322b)

```
static void
icmpunreachable(Fs *f, Block *bp, int code, int seq)
{
    Block   *nbp;
    Icmp    *p, *np;
    int i;
    ipaddr   addr;

    p = (Icmp *)bp->rp;

    /* only do this for unicast sources and destinations */
    v4tov6(addr, p->dst);
    i = ipforme(f, addr);
    if((i&Runi) == 0)
        return;
```

```
    v4tov6(addr, p->src);
    i = ipforme(f, addr);
    if(i != 0 && (i&Runi) == 0)
        return;

    netlog(f, Logicmp, "sending icmpnoconv -> %V\n", p->src);
    nbp = allocb(ICMP_IPSIZE + ICMP_HDRSIZE + ICMP_IPSIZE + 8);
    nbp->wp += ICMP_IPSIZE + ICMP_HDRSIZE + ICMP_IPSIZE + 8;
    np = (Icmp *)nbp->rp;
    np->vihl = IP_VER4;
    memmove(np->dst, p->src, sizeof(np->dst));
    memmove(np->src, p->dst, sizeof(np->src));
    memmove(np->data, bp->rp, ICMP_IPSIZE + 8);
    np->type = Unreachable;
    np->code = code;
    np->proto = IP_ICMPPROTO;
    hnputs(np->icmpid, 0);
    hnputs(np->seq, seq);
    memset(np->cksum, 0, sizeof(np->cksum));
    hnputs(np->cksum, ptclcsum(nbp, ICMP_IPSIZE, blocklen(nbp) - ICMP_IPSIZE));
    ipoput4(f, nbp, 0, MAXTTL, DFLTTOS, nil);
}
```

⟨*function* icmpnoconv 318a⟩≡ (322b)

```
extern void
icmpnoconv(Fs *f, Block *bp)
{
    icmpunreachable(f, bp, 3, 0);
}
```

⟨*function* icmpcantfrag 318b⟩≡ (322b)

```
extern void
icmpcantfrag(Fs *f, Block *bp, int mtu)
{
    icmpunreachable(f, bp, 4, mtu);
}
```

⟨*function* goticmpkt 318c⟩≡ (322b)

```
static void
goticmpkt(Proto *icmp, Block *bp)
{
    Conv    **c, *s;
    Icmp    *p;
    ipaddr  dst;
    ushort  recid;

    p = (Icmp *) bp->rp;
    v4tov6(dst, p->src);
    recid = nhgets(p->icmpid);

    for(c = icmp->conv; *c; c++) {
        s = *c;
        if(s->lport == recid)
        if(ipcmp(s->raddr, dst) == 0){
            bp = concatblock(bp);
            if(bp != nil)
                qpass(s->rq, bp);
            return;
        }
    }
```

```
    freeblist(bp);
}
```

⟨*function* mkechoreply 319a⟩≡ (322b)

```
static Block *
mkechoreply(Block *bp)
{
    Icmp    *q;
    uchar   ip[4];

    q = (Icmp *)bp->rp;
    q->vihl = IP_VER4;
    memmove(ip, q->src, sizeof(q->dst));
    memmove(q->src, q->dst, sizeof(q->src));
    memmove(q->dst, ip,  sizeof(q->dst));
    q->type = EchoReply;
    memset(q->cksum, 0, sizeof(q->cksum));
    hnputs(q->cksum, ptclcsum(bp, ICMP_IPSIZE, blocklen(bp) - ICMP_IPSIZE));

    return bp;
}
```

⟨*global* unreachcode 319b⟩≡ (322b)

```
static char *unreachcode[] =
{
[0] "net unreachable",
[1] "host unreachable",
[2] "protocol unreachable",
[3] "port unreachable",
[4] "fragmentation needed and DF set",
[5] "source route failed",
};
```

⟨*function* icmpiput 319c⟩≡ (322b)

```
static void
icmpiput(Proto *icmp, Ipifc*, Block *bp)
{
    int n, iplen;
    Icmp    *p;
    Block   *r;
    Proto   *pr;
    char    *msg;
    char    m2[128];
    Icmppriv *ipriv;

    ipriv = icmp->priv;

    ipriv->stats[InMsgs]++;

    p = (Icmp *)bp->rp;
    netlog(icmp->f, Logicmp, "icmpiput %s (%d) %d\n",
        (p->type < nelem(icmpnames)? icmpnames[p->type]: ""),
        p->type, p->code);
    n = blocklen(bp);
    if(n < ICMP_IPSIZE+ICMP_HDRSIZE){
        ipriv->stats[InErrors]++;
        ipriv->stats[HlenErrs]++;
        netlog(icmp->f, Logicmp, "icmp hlen %d\n", n);
        goto raise;
    }
```

```
iplen = nhgets(p->length);
if(iplen > n){
    ipriv->stats[LenErrs]++;
    ipriv->stats[InErrors]++;
    netlog(icmp->f, Logicmp, "icmp length %d\n", iplen);
    goto raise;
}
if(ptclcsum(bp, ICMP_IPSIZE, iplen - ICMP_IPSIZE)){
    ipriv->stats[InErrors]++;
    ipriv->stats[CsumErrs]++;
    netlog(icmp->f, Logicmp, "icmp checksum error\n");
    goto raise;
}
if(p->type <= Maxtype)
    ipriv->in[p->type]++;

switch(p->type) {
case EchoRequest:
    if (iplen < n)
        bp = trimblock(bp, 0, iplen);
    r = mkechoreply(bp);
    ipriv->out[EchoReply]++;
    ipoput4(icmp->f, r, 0, MAXTTL, DFLTTOS, nil);
    break;
case Unreachable:
    if(p->code > 5)
        msg = unreachcode[1];
    else
        msg = unreachcode[p->code];

    bp->rp += ICMP_IPSIZE+ICMP_HDRSIZE;
    if(blocklen(bp) < MinAdvise){
        ipriv->stats[LenErrs]++;
        goto raise;
    }
    p = (Icmp *)bp->rp;
    pr = Fsrcvpcolx(icmp->f, p->proto);
    if(pr != nil && pr->advise != nil) {
        // Protocol dispatch
        (*pr->advise)(pr, bp, msg);
        return;
    }

    bp->rp -= ICMP_IPSIZE+ICMP_HDRSIZE;
    goticmpkt(icmp, bp);
    break;
case TimeExceed:
    if(p->code == 0){
        snprint(m2, sizeof m2, "ttl exceeded at %V", p->src);

        bp->rp += ICMP_IPSIZE+ICMP_HDRSIZE;
        if(blocklen(bp) < MinAdvise){
            ipriv->stats[LenErrs]++;
            goto raise;
        }
        p = (Icmp *)bp->rp;
        pr = Fsrcvpcolx(icmp->f, p->proto);
        if(pr != nil && pr->advise != nil) {
            // Protocol dispatch
            (*pr->advise)(pr, bp, m2);
```

```
                return;
            }
            bp->rp -= ICMP_IPSIZE+ICMP_HDRSIZE;
        }

        goticmpkt(icmp, bp);
        break;
    default:
        goticmpkt(icmp, bp);
        break;
    }
    return;

raise:
    freeblist(bp);
}
```

⟨*function* icmpadvise 321a⟩≡ (322b)

```
void
icmpadvise(Proto *icmp, Block *bp, char *msg)
{
    Conv    **c, *s;
    Icmp    *p;
    ipaddr   dst;
    ushort  recid;

    p = (Icmp *) bp->rp;
    v4tov6(dst, p->dst);
    recid = nhgets(p->icmpid);

    for(c = icmp->conv; *c; c++) {
        s = *c;
        if(s->lport == recid)
        if(ipcmp(s->raddr, dst) == 0){
            qhangup(s->rq, msg);
            qhangup(s->wq, msg);
            break;
        }
    }
    freeblist(bp);
}
```

⟨*function* icmpstats 321b⟩≡ (322b)

```
int
icmpstats(Proto *icmp, char *buf, int len)
{
    Icmppriv *priv;
    char *p, *e;
    int i;

    priv = icmp->priv;
    p = buf;
    e = p+len;
    for(i = 0; i < Nstats; i++)
        p = seprint(p, e, "%s: %lud\n", statnames[i], priv->stats[i]);
    for(i = 0; i <= Maxtype; i++){
        if(icmpnames[i])
            p = seprint(p, e, "%s: %lud %lud\n", icmpnames[i], priv->in[i], priv->out[i]);
        else
            p = seprint(p, e, "%d: %lud %lud\n", i, priv->in[i], priv->out[i]);
```

```
    }
    return p - buf;
}
```

⟨*function* icmpinit 322a⟩≡ (322b)

```
void
icmpinit(Fs *fs)
{
    Proto *icmp;

    icmp = smalloc(sizeof(Proto));
    icmp->priv = smalloc(sizeof(Icmppriv));
    icmp->name = "icmp";
    icmp->connect = icmpconnect;
    icmp->announce = icmpannounce;
    icmp->state = icmpstate;
    icmp->create = icmpcreate;
    icmp->close = icmpclose;
    icmp->rcv = icmpiput;
    icmp->stats = icmpstats;
    icmp->ctl = nil;
    icmp->advise = icmpadvise;
    icmp->gc = nil;
    icmp->ipproto = IP_ICMPPROTO;
    icmp->nc = 128;
    icmp->ptclsize = 0;

    Fsproto(fs, icmp);
}
```

⟨kernel/network/ip/icmp.c 322b⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "../port/error.h"

#include "ip.h"

typedef struct Icmppriv Icmppriv;

⟨struct Icmp 313b⟩

⟨enum _anon_ (kernel/network/ip/icmp.c) 314a⟩

⟨enum _anon_ (kernel/network/ip/icmp.c)2 314b⟩

⟨global icmpnames 314c⟩

⟨enum _anon_ (kernel/network/ip/icmp.c)3 315a⟩

⟨enum _anon_ (kernel/network/ip/icmp.c)4 315b⟩

⟨global statnames((kernel/network/ip/icmp.c)) 315c⟩

⟨struct Icmppriv 315d⟩

static void icmpkick(void *x, Block*);
```

⟨*function* icmpcreate 315e⟩

⟨*function* icmpconnect 315f⟩

⟨*function* icmpstate 316a⟩

⟨*function* icmpannounce 316b⟩

⟨*function* icmpclose 316c⟩

⟨*function* icmpkick 316d⟩

⟨*function* icmpttlexceeded 317a⟩

⟨*function* icmpunreachable 317b⟩

⟨*function* icmpnoconv 318a⟩

⟨*function* icmpcantfrag 318b⟩

⟨*function* goticmpkt 318c⟩

⟨*function* mkechoreply 319a⟩

⟨*global* unreachcode 319b⟩

⟨*function* icmpiput 319c⟩

⟨*function* icmpadvise 321a⟩

⟨*function* icmpstats 321b⟩

⟨*function* icmpinit 322a⟩

I.8.4 kernel/network/ip/chandial.c

⟨*enum* _anon_ (kernel/network/ip/chandial.c) 323a⟩≡ (325b)

```
enum
{
    Maxstring=  128,
};
```

⟨*struct* DS*(*(kernel/network/ip/chandial.c)*)* 323b⟩≡ (325b)

```
struct DS
{
    char    buf[Maxstring];         /* dist string */
    char    *netdir;
    char    *proto;
    char    *rem;
    char    *local;             /* other args */
    char    *dir;
    Chan    **ctlp;
};
```

⟨*function* chandial 323c⟩≡ (325b)

```
/*
 *  the dialstring is of the form '[/net/]proto!dest'
 */
Chan*
```

```
chandial(char *dest, char *local, char *dir, Chan **ctlp)
{
    DS ds;
    char clone[Maxpath];

    ds.local = local;
    ds.dir = dir;
    ds.ctlp = ctlp;

    _dial_string_parse(dest, &ds);
    if(ds.netdir == 0)
        ds.netdir = "/net";

    /* no connection server, don't translate */
    snprint(clone, sizeof(clone), "%s/%s/clone", ds.netdir, ds.proto);
    return call(clone, ds.rem, &ds);
}
```

⟨*function* call*(*(kernel/network/ip/chandial.c)*)* 324⟩≡ (325b)

```
static Chan*
call(char *clone, char *dest, DS *ds)
{
    int n;
    Chan *dchan, *cchan;
    char name[Maxpath], data[Maxpath], *p;

    cchan = namec(clone, Aopen, ORDWR, 0);

    /* get directory name */
    if(waserror()){
        cclose(cchan);
        nexterror();
    }
    n = devtab[cchan->type]->read(cchan, name, sizeof(name)-1, 0);
    name[n] = 0;
    for(p = name; *p == ' '; p++)
        ;
    snprint(name, sizeof name, "%lud", strtoul(p, 0, 0));
    p = strrchr(clone, '/');
    *p = 0;
    if(ds->dir)
        snprint(ds->dir, Maxpath, "%s/%s", clone, name);
    snprint(data, sizeof(data), "%s/%s/data", clone, name);

    /* connect */
    if(ds->local)
        snprint(name, sizeof(name), "connect %s %s", dest, ds->local);
    else
        snprint(name, sizeof(name), "connect %s", dest);
    devtab[cchan->type]->write(cchan, name, strlen(name), 0);

    /* open data connection */
    dchan = namec(data, Aopen, ORDWR, 0);
    if(ds->ctlp)
        *ds->ctlp = cchan;
    else
        cclose(cchan);
    poperror();
    return dchan;
```

```
}
```

⟨*function* _dial_string_parse*(*(kernel/network/ip/chandial.c)*)* 325a⟩≡ (325b)

```
/*
 *  parse a dial string
 */
static void
_dial_string_parse(char *str, DS *ds)
{
    char *p, *p2;

    strncpy(ds->buf, str, Maxstring);
    ds->buf[Maxstring-1] = 0;

    p = strchr(ds->buf, '!');
    if(p == 0) {
        ds->netdir = 0;
        ds->proto = "net";
        ds->rem = ds->buf;
    } else {
        if(*ds->buf != '/' && *ds->buf != '#'){
            ds->netdir = 0;
            ds->proto = ds->buf;
        } else {
            for(p2 = p; *p2 != '/'; p2--)
                ;
            *p2++ = 0;
            ds->netdir = ds->buf;
            ds->proto = p2;
        }
        *p = 0;
        ds->rem = p + 1;
    }
}
```

⟨kernel/network/ip/chandial.c 325b⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
#include    "../port/error.h"
#include    "../ip/ip.h"

typedef struct DS DS;
static Chan*    call(char*, char*, DS*);
static void _dial_string_parse(char*, DS*);

⟨enum _anon_ (kernel/network/ip/chandial.c) 323a⟩

⟨struct DS*(*(kernel/network/ip/chandial.c)*)* 323b⟩

⟨function chandial 323c⟩

⟨function call*(*(kernel/network/ip/chandial.c)*)* 324⟩

⟨function _dial_string_parse*(*(kernel/network/ip/chandial.c)*)* 325a⟩
```

I.8.5 kernel/network/ip/nullmedium.c

⟨kernel/network/ip/nullmedium.c 326a⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "../port/error.h"

#include "ip.h"

⟨function nullbind 190b⟩

⟨function nullunbind 190c⟩

⟨function nullbwrite 190d⟩

⟨global nullmedium 190a⟩

⟨function nullmediumlink 190e⟩
```

I.8.6 kernel/network/ip/loopbackmedium.c

⟨kernel/network/ip/loopbackmedium.c 326b⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "../port/error.h"

#include "ip.h"

typedef struct LB LB;

⟨enum _anon_ (kernel/network/ip/loopbackmedium.c) 191b⟩

⟨struct LB 191d⟩

static void loopbackread(void *a);

⟨function loopbackbind 191e⟩

⟨function loopbackunbind 191f⟩

⟨function loopbackbwrite 192b⟩

⟨function loopbackread 192a⟩

⟨global loopbackmedium 191a⟩

⟨function loopbackmediumlink 191c⟩
```

I.8.7 kernel/network/ip/ethermedium.c

⟨global ipbroadcast 326c⟩≡ (336c)

```
static ipaddr ipbroadcast = {
```

```
    0xff,0xff,0xff,0xff,
    0xff,0xff,0xff,0xff,
    0xff,0xff,0xff,0xff,
    0xff,0xff,0xff,0xff,
};
```

⟨*global* gbemedium 327a⟩≡ (336c)

```
Medium gbemedium =
{
.name=      "gbe",
.hsize=     14,
.mintu=     60,
.maxtu=     9014,
.maclen=    6,
.bind=      etherbind,
.unbind=    etherunbind,
.bwrite=    etherbwrite,
.addmulti=  etheraddmulti,
.remmulti=  etherremmulti,
.ares=      arpenter,
.areg=      sendgarp,
.pref2addr= etherpref2addr,
};
```

⟨*struct* Etherrock 327b⟩≡ (336c)

```
struct Etherrock
{
    Fs  *f;     /* file system we belong to */
    Proc    *arpp;      /* arp process */
    Proc    *read4p;    /* reading process (v4)*/
    Proc    *read6p;    /* reading process (v6)*/
    Chan    *mchan4;    /* Data channel for v4 */
    Chan    *achan;     /* Arp channel */
    Chan    *cchan4;    /* Control channel for v4 */
    Chan    *mchan6;    /* Data channel for v6 */
    Chan    *cchan6;    /* Control channel for v6 */
};
```

⟨*enum* _anon_ (kernel/network/ip/ethermedium.c) 327c⟩≡ (336c)

```
/*
 *  ethernet arp request
 */
enum
{
    ARPREQUEST  = 1,
    ARPREPLY    = 2,
};
```

⟨*struct* Etherarp 327d⟩≡ (336c)

```
struct Etherarp
{
    uchar   d[6];
    uchar   s[6];
    uchar   type[2];
    uchar   hrd[2];
    uchar   pro[2];
    uchar   hln;
    uchar   pln;
    uchar   op[2];
    uchar   sha[6];
```

```
    uchar   spa[4];
    uchar   tha[6];
    uchar   tpa[4];
};
```

⟨*global* nbmsg 328a⟩≡ (336c)

```
static char *nbmsg = "nonblocking";
```

⟨*function* etherunbind 328b⟩≡ (336c)

```
/*
 *  called with ifc wlock'd
 */
static void
etherunbind(Ipifc *ifc)
{
    Etherrock *er = ifc->arg;

    if(er->read4p)
        postnote(er->read4p, 1, "unbind", 0);
    if(er->read6p)
        postnote(er->read6p, 1, "unbind", 0);
    if(er->arpp)
        postnote(er->arpp, 1, "unbind", 0);

    /* wait for readers to die */
    while(er->arpp != 0 || er->read4p != 0 || er->read6p != 0)
        tsleep(&up->sleepr, returnfalse, 0, 300);

    if(er->mchan4 != nil)
        cclose(er->mchan4);
    if(er->achan != nil)
        cclose(er->achan);
    if(er->cchan4 != nil)
        cclose(er->cchan4);
    if(er->mchan6 != nil)
        cclose(er->mchan6);
    if(er->cchan6 != nil)
        cclose(er->cchan6);

    free(er);
}
```

⟨*function* etherbwrite*(*(kernel/network/ip/ethermedium.c)*)* 328c⟩≡ (336c)

```
/*
 *  called by ipoput with a single block to write with ifc rlock'd
 */
static void
etherbwrite(Ipifc *ifc, Block *bp, int version, uchar *ip)
{
    Etherhdr *eh;
    Arpent *a;
    uchar mac[6];
    Etherrock *er = ifc->arg;

    /* get mac address of destination */
    a = arpget(er->f->arp, bp, version, ifc, ip, mac);
    if(a){
        /* check for broadcast or multicast */
        bp = multicastarp(er->f, a, ifc->m, mac);
        if(bp==nil){
```

```
            switch(version){
            case V4:
                sendarp(ifc, a);
                break;
            case V6:
                resolveaddr6(ifc, a);
                break;
            default:
                panic("etherbwrite: version %d", version);
            }
            return;
        }
    }

    /* make it a single block with space for the ether header */
    bp = padblock(bp, ifc->m->hsize);
    if(bp->next)
        bp = concatblock(bp);
    if(BLEN(bp) < ifc->mintu)
        bp = adjustblock(bp, ifc->mintu);
    eh = (Etherhdr*)bp->rp;

    /* copy in mac addresses and ether type */
    memmove(eh->s, ifc->mac, sizeof(eh->s));
    memmove(eh->d, mac, sizeof(eh->d));

    switch(version){
    case V4:
        eh->t[0] = 0x08;
        eh->t[1] = 0x00;
        devtab[er->mchan4->type]->bwrite(er->mchan4, bp, 0);
        break;
    case V6:
        eh->t[0] = 0x86;
        eh->t[1] = 0xDD;
        devtab[er->mchan6->type]->bwrite(er->mchan6, bp, 0);
        break;
    default:
        panic("etherbwrite2: version %d", version);
    }
    ifc->out++;
}
```

⟨*function* etherread4 329⟩≡ (336c)

```
/*
 *  process to read from the ethernet
 */
static void
etherread4(void *a)
{
    Ipifc *ifc;
    Block *bp;
    Etherrock *er;

    ifc = a;
    er = ifc->arg;
    er->read4p = up;    /* hide identity under a rock for unbind */
    if(waserror()){
        er->read4p = 0;
        pexit("hangup", 1);
```

```
    }
    for(;;){
        bp = devtab[er->mchan4->type]->bread(er->mchan4, ifc->maxtu, 0);
        if(!canrlock(ifc)){
            freeb(bp);
            continue;
        }
        if(waserror()){
            runlock(ifc);
            nexterror();
        }
        ifc->in++;
        bp->rp += ifc->m->hsize;
        if(ifc->lifc == nil)
            freeb(bp);
        else
            ipiput4(er->f, ifc, bp);
        runlock(ifc);
        poperror();
    }
}
```

⟨*function* etheraddmulti 330a⟩≡ (336c)

```
static void
etheraddmulti(Ipifc *ifc, uchar *a, uchar *)
{
    uchar mac[6];
    char buf[64];
    Etherrock *er = ifc->arg;
    int version;

    version = multicastea(mac, a);
    snprint(buf, sizeof buf, "addmulti %E", mac);
    switch(version){
    case V4:
        devtab[er->cchan4->type]->write(er->cchan4, buf, strlen(buf), 0);
        break;
    case V6:
        devtab[er->cchan6->type]->write(er->cchan6, buf, strlen(buf), 0);
        break;
    default:
        panic("etheraddmulti: version %d", version);
    }
}
```

⟨*function* etherremmulti 330b⟩≡ (336c)

```
static void
etherremmulti(Ipifc *ifc, uchar *a, uchar *)
{
    uchar mac[6];
    char buf[64];
    Etherrock *er = ifc->arg;
    int version;

    version = multicastea(mac, a);
    snprint(buf, sizeof buf, "remmulti %E", mac);
    switch(version){
    case V4:
        devtab[er->cchan4->type]->write(er->cchan4, buf, strlen(buf), 0);
        break;
```

```
    case V6:
        devtab[er->cchan6->type]->write(er->cchan6, buf, strlen(buf), 0);
        break;
    default:
        panic("etherremmulti: version %d", version);
    }
}
```

⟨*function* sendarp 331⟩≡ (336c)

```
/*
 *  send an ethernet arp
 *  (only v4, v6 uses the neighbor discovery, rfc1970)
 */
static void
sendarp(Ipifc *ifc, Arpent *a)
{
    int n;
    Block *bp;
    Etherarp *e;
    Etherrock *er = ifc->arg;

    /* don't do anything if it's been less than a second since the last */
    if(NOW - a->ctime < 1000){
        arprelease(er->f->arp, a);
        return;
    }

    /* remove all but the last message */
    while((bp = a->hold) != nil){
        if(bp == a->last)
            break;
        a->hold = bp->list;
        freeblist(bp);
    }

    /* try to keep it around for a second more */
    a->ctime = NOW;
    arprelease(er->f->arp, a);

    n = sizeof(Etherarp);
    if(n < a->type->mintu)
        n = a->type->mintu;
    bp = allocb(n);
    memset(bp->rp, 0, n);
    e = (Etherarp*)bp->rp;
    memmove(e->tpa, a->ip+IPv4off, sizeof(e->tpa));
    ipv4local(ifc, e->spa);
    memmove(e->sha, ifc->mac, sizeof(e->sha));
    memset(e->d, 0xff, sizeof(e->d));       /* ethernet broadcast */
    memmove(e->s, ifc->mac, sizeof(e->s));

    hnputs(e->type, ETARP);
    hnputs(e->hrd, 1);
    hnputs(e->pro, ETIP4);
    e->hln = sizeof(e->sha);
    e->pln = sizeof(e->spa);
    hnputs(e->op, ARPREQUEST);
    bp->wp += n;

    devtab[er->achan->type]->bwrite(er->achan, bp, 0);
```

```
}
```

⟨*function* resolveaddr6 332a⟩≡ (336c)

```
static void
resolveaddr6(Ipifc *ifc, Arpent *a)
{
    int sflag;
    Block *bp;
    Etherrock *er = ifc->arg;
    ipaddr ipsrc;

    /* don't do anything if it's been less than a second since the last */
    if(NOW - a->ctime < ReTransTimer){
        arprelease(er->f->arp, a);
        return;
    }

    /* remove all but the last message */
    while((bp = a->hold) != nil){
        if(bp == a->last)
            break;
        a->hold = bp->list;
        freeblist(bp);
    }

    /* try to keep it around for a second more */
    a->ctime = NOW;
    a->rtime = NOW + ReTransTimer;
    if(a->rxtsrem <= 0) {
        arprelease(er->f->arp, a);
        return;
    }

    a->rxtsrem--;
    arprelease(er->f->arp, a);

    if(sflag = ipv6anylocal(ifc, ipsrc))
        icmpns(er->f, ipsrc, sflag, a->ip, TARG_MULTI, ifc->mac);
}
```

⟨*function* sendgarp 332b⟩≡ (336c)

```
/*
 *  send a gratuitous arp to refresh arp caches
 */
static void
sendgarp(Ipifc *ifc, uchar *ip)
{
    int n;
    Block *bp;
    Etherarp *e;
    Etherrock *er = ifc->arg;

    /* don't arp for our initial non address */
    if(ipcmp(ip, IPnoaddr) == 0)
        return;

    n = sizeof(Etherarp);
    if(n < ifc->m->mintu)
        n = ifc->m->mintu;
    bp = allocb(n);
```

```
    memset(bp->rp, 0, n);
    e = (Etherarp*)bp->rp;
    memmove(e->tpa, ip+IPv4off, sizeof(e->tpa));
    memmove(e->spa, ip+IPv4off, sizeof(e->spa));
    memmove(e->sha, ifc->mac, sizeof(e->sha));
    memset(e->d, 0xff, sizeof(e->d));       /* ethernet broadcast */
    memmove(e->s, ifc->mac, sizeof(e->s));

    hnputs(e->type, ETARP);
    hnputs(e->hrd, 1);
    hnputs(e->pro, ETIP4);
    e->hln = sizeof(e->sha);
    e->pln = sizeof(e->spa);
    hnputs(e->op, ARPREQUEST);
    bp->wp += n;

    devtab[er->achan->type]->bwrite(er->achan, bp, 0);
}
```

⟨*function* recvarp 333⟩≡ (336c)

```
static void
recvarp(Ipifc *ifc)
{
    int n;
    Block *ebp, *rbp;
    Etherarp *e, *r;
    ipaddr ip;
    static uchar eprinted[4];
    Etherrock *er = ifc->arg;

    ebp = devtab[er->achan->type]->bread(er->achan, ifc->maxtu, 0);
    if(ebp == nil)
        return;

    e = (Etherarp*)ebp->rp;
    switch(nhgets(e->op)) {
    default:
        break;

    case ARPREPLY:
        /* check for machine using my ip address */
        v4tov6(ip, e->spa);
        if(iplocalonifc(ifc, ip) || ipproxyifc(er->f, ifc, ip)){
            if(memcmp(e->sha, ifc->mac, sizeof(e->sha)) != 0){
                print("arprep: 0x%E/0x%E also has ip addr %V\n",
                    e->s, e->sha, e->spa);
                break;
            }
        }

        /* make sure we're not entering broadcast addresses */
        if(ipcmp(ip, ipbroadcast) == 0 ||
            !memcmp(e->sha, etherbroadcast, sizeof(e->sha))){
            print("arprep: 0x%E/0x%E cannot register broadcast address %I\n",
                e->s, e->sha, e->spa);
            break;
        }

        arpenter(er->f, V4, e->spa, e->sha, sizeof(e->sha), 0);
        break;
```

```
    case ARPREQUEST:
        /* don't answer arps till we know who we are */
        if(ifc->lifc == 0)
            break;

        /* check for machine using my ip or ether address */
        v4tov6(ip, e->spa);
        if(iplocalonifc(ifc, ip) || ipproxyifc(er->f, ifc, ip)){
            if(memcmp(e->sha, ifc->mac, sizeof(e->sha)) != 0){
                if (memcmp(eprinted, e->spa, sizeof(e->spa))){
                    /* print only once */
                    print("arpreq: 0x%E also has ip addr %V\n", e->sha, e->spa);
                    memmove(eprinted, e->spa, sizeof(e->spa));
                }
            }
        } else {
            if(memcmp(e->sha, ifc->mac, sizeof(e->sha)) == 0){
                print("arpreq: %V also has ether addr %E\n", e->spa, e->sha);
                break;
            }
        }

        /* refresh what we know about sender */
        arpenter(er->f, V4, e->spa, e->sha, sizeof(e->sha), 1);

        /* answer only requests for our address or systems we're proxying for */
        v4tov6(ip, e->tpa);
        if(!iplocalonifc(ifc, ip))
        if(!ipproxyifc(er->f, ifc, ip))
            break;

        n = sizeof(Etherarp);
        if(n < ifc->mintu)
            n = ifc->mintu;
        rbp = allocb(n);
        r = (Etherarp*)rbp->rp;
        memset(r, 0, sizeof(Etherarp));
        hnputs(r->type, ETARP);
        hnputs(r->hrd, 1);
        hnputs(r->pro, ETIP4);
        r->hln = sizeof(r->sha);
        r->pln = sizeof(r->spa);
        hnputs(r->op, ARPREPLY);
        memmove(r->tha, e->sha, sizeof(r->tha));
        memmove(r->tpa, e->spa, sizeof(r->tpa));
        memmove(r->sha, ifc->mac, sizeof(r->sha));
        memmove(r->spa, e->tpa, sizeof(r->spa));
        memmove(r->d, e->sha, sizeof(r->d));
        memmove(r->s, ifc->mac, sizeof(r->s));
        rbp->wp += n;

        devtab[er->achan->type]->bwrite(er->achan, rbp, 0);
    }
    freeb(ebp);
}
```

⟨*function* recvarpproc 334⟩≡ (336c)

```
static void
recvarpproc(void *v)
```

```
{
    Ipifc *ifc = v;
    Etherrock *er = ifc->arg;

    er->arpp = up;
    if(waserror()){
        er->arpp = 0;
        pexit("hangup", 1);
    }
    for(;;)
        recvarp(ifc);
}
```

⟨*function* multicastea 335a⟩≡ (336c)

```
static int
multicastea(uchar *ea, uchar *ip)
{
    int x;

    switch(x = ipismulticast(ip)){
    case V4:
        ea[0] = 0x01;
        ea[1] = 0x00;
        ea[2] = 0x5e;
        ea[3] = ip[13] & 0x7f;
        ea[4] = ip[14];
        ea[5] = ip[15];
        break;
    case V6:
        ea[0] = 0x33;
        ea[1] = 0x33;
        ea[2] = ip[12];
        ea[3] = ip[13];
        ea[4] = ip[14];
        ea[5] = ip[15];
        break;
    }
    return x;
}
```

⟨*function* multicastarp 335b⟩≡ (336c)

```
/*
 *  fill in an arp entry for broadcast or multicast
 *  addresses.  Return the first queued packet for the
 *  IP address.
 */
static Block*
multicastarp(Fs *f, Arpent *a, Medium *medium, uchar *mac)
{
    /* is it broadcast? */
    switch(ipforme(f, a->ip)){
    case Runi:
        return nil;
    case Rbcast:
        memset(mac, 0xff, 6);
        return arpresolve(f->arp, a, medium, mac);
    default:
        break;
    }
```

```
    /* if multicast, fill in mac */
    switch(multicastea(mac, a->ip)){
    case V4:
    case V6:
        return arpresolve(f->arp, a, medium, mac);
    }

    /* let arp take care of it */
    return nil;
}
```

⟨*function* ethermediumlink 336a⟩≡ (336c)

```
void
ethermediumlink(void)
{
    addipmedium(&ethermedium);
    addipmedium(&gbemedium);
}
```

⟨*function* etherpref2addr 336b⟩≡ (336c)

```
static void
etherpref2addr(uchar *pref, uchar *ea)
{
    pref[8] = ea[0] | 0x2;
    pref[9] = ea[1];
    pref[10] = ea[2];
    pref[11] = 0xFF;
    pref[12] = 0xFE;
    pref[13] = ea[3];
    pref[14] = ea[4];
    pref[15] = ea[5];
}
```

⟨kernel/network/ip/ethermedium.c 336c⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "../port/error.h"

#include "../port/netif.h"
#include "ip.h"
#include "ipv6.h"

typedef struct Etherhdr Etherhdr;
typedef struct  Etherrock Etherrock;
typedef struct Etherarp Etherarp;
```

⟨*struct* Etherhdr 74a⟩

⟨*global* ipbroadcast 326c⟩

⟨*global* etherbroadcast 20a⟩

```
static void etherread4(void *a);
static void etherread6(void *a);
static void etherbind(Ipifc *ifc, int argc, char **argv);
static void etherunbind(Ipifc *ifc);
static void etherbwrite(Ipifc *ifc, Block *bp, int version, uchar *ip);
```

```
static void etheraddmulti(Ipifc *ifc, uchar *a, uchar *ia);
static void etherremmulti(Ipifc *ifc, uchar *a, uchar *ia);
static Block*   multicastarp(Fs *f, Arpent *a, Medium*, uchar *mac);
static void sendarp(Ipifc *ifc, Arpent *a);
static void sendgarp(Ipifc *ifc, uchar*);
static int  multicastea(uchar *ea, uchar *ip);
static void recvarpproc(void*);
static void resolveaddr6(Ipifc *ifc, Arpent *a);
static void etherpref2addr(uchar *pref, uchar *ea);
```

⟨*global ethermedium (kernel)* 19g⟩

⟨*global* gbemedium 327a⟩

⟨*struct* Etherrock 327b⟩

⟨*enum* _anon_ (kernel/network/ip/ethermedium.c) 327c⟩

⟨*struct* Etherarp 327d⟩

⟨*global* nbmsg 328a⟩

⟨*function* etherbind 65c⟩

⟨*function* etherunbind 328b⟩

⟨*function* etherbwrite*(*(kernel/network/ip/ethermedium.c)*)* 328c⟩

⟨*function* etherread4 329⟩

⟨*function* etherread6 472c⟩

⟨*function* etheraddmulti 330a⟩

⟨*function* etherremmulti 330b⟩

⟨*function* sendarp 331⟩

⟨*function* resolveaddr6 332a⟩

⟨*function* sendgarp 332b⟩

⟨*function* recvarp 333⟩

⟨*function* recvarpproc 334⟩

⟨*function* multicastea 335a⟩

⟨*function* multicastarp 335b⟩

⟨*function* ethermediumlink 336a⟩

⟨*function* etherpref2addr 336b⟩

I.8.8 kernel/network/ip/pktmedium.c

⟨*global* pktmedium 338a⟩≡ (339b)

```
Medium pktmedium =
{
    .name=      "pkt",
    .hsize=     14,
    .mintu=     40,
    .maxtu=     4*1024,
    .maclen=    6,
    .bind=      pktbind,
    .unbind=    pktunbind,
    .bwrite=    pktbwrite,
    .pktin=     pktin,
};
```

⟨*function* pktbind 338b⟩≡ (339b)

```
/*
 *  called to bind an IP ifc to an ethernet device
 *  called with ifc wlock'd
 */
static void
pktbind(Ipifc*, int argc, char **argv)
{
    USED(argc, argv);
}
```

⟨*function* pktunbind 338c⟩≡ (339b)

```
/*
 *  called with ifc wlock'd
 */
static void
pktunbind(Ipifc*)
{
}
```

⟨*function* pktbwrite 338d⟩≡ (339b)

```
/*
 *  called by ipoput with a single packet to write
 */
static void
pktbwrite(Ipifc *ifc, Block *bp, int, uchar*)
{
    /* enqueue onto the conversation's rq */
    bp = concatblock(bp);
    if(ifc->conv->snoopers.ref > 0)
        qpass(ifc->conv->sq, copyblock(bp, BLEN(bp)));
    qpass(ifc->conv->rq, bp);
}
```

⟨*function* pktin 338e⟩≡ (339b)

```
/*
 *  called with ifc rlocked when someone write's to 'data'
 */
static void
pktin(Fs *f, Ipifc *ifc, Block *bp)
{
    if(ifc->lifc == nil)
        freeb(bp);
    else {
        if(ifc->conv->snoopers.ref > 0)
```

```
            qpass(ifc->conv->sq, copyblock(bp, BLEN(bp)));
        ipiput4(f, ifc, bp);
    }
}
```

⟨*function* pktmediumlink 339a⟩≡ (339b)

```
void
pktmediumlink(void)
{
    addipmedium(&pktmedium);
}
```

⟨kernel/network/ip/pktmedium.c 339b⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "../port/error.h"

#include "ip.h"


static void pktbind(Ipifc*, int, char**);
static void pktunbind(Ipifc*);
static void pktbwrite(Ipifc*, Block*, int, uchar*);
static void pktin(Fs*, Ipifc*, Block*);

⟨global pktmedium 338a⟩

⟨function pktbind 338b⟩

⟨function pktunbind 338c⟩

⟨function pktbwrite 338d⟩

⟨function pktin 338e⟩

⟨function pktmediumlink 339a⟩
```

I.8.9 kernel/network/ip/iproute.c

⟨kernel/network/ip/iproute.c 339c⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
#include    "../port/error.h"

#include    "ip.h"

static void walkadd(Fs*, Route**, Route*);
static void addnode(Fs*, Route**, Route*);
static void calcd(Route*);

⟨global v4freelist 98c⟩
⟨global v4routegeneration 29a⟩
⟨global routelock 98b⟩
```

⟨*global* `v6freelist` 472d⟩
⟨*global* `v6routegeneration` 484b⟩

⟨*function* `freeroute` 99a⟩

⟨*function* `allocroute` 98d⟩

⟨*function* `addqueue` 101e⟩

⟨*function* `lcmp` 484a⟩

⟨*enum* `_anon_ (kernel/network/ip/iproute.c)` 99b⟩

⟨*function* `rangecompare` 99c⟩

⟨*function* `copygate` 101a⟩

⟨*function* `walkadd` 101d⟩

⟨*function* `calcd` 102c⟩

⟨*function* `balancetree` 102b⟩

⟨*function* `addnode` 100⟩

⟨*macro* `V4H` 96c⟩

⟨*function* `v4addroute` 98a⟩

⟨*macro* `V6H` 473a⟩
⟨*function* `v6addroute` 473b⟩

⟨*function* `looknode` 113b⟩

⟨*function* `v4delroute` 113a⟩

⟨*function* `v6delroute` 473c⟩

⟨*function* `v4lookup` 96a⟩

⟨*function* `v6lookup` 474⟩

⟨*function* `routetype` 104c⟩

⟨*global* `rformat` 103e⟩

⟨*function* `convroute` 104b⟩

⟨*function* `sprintroute` 104a⟩

⟨*function* `rr` 105b⟩

⟨*function* `ipwalkroutes` 105a⟩

⟨*function* `routeread` 103d⟩

⟨*function* `delroute` 114b⟩

⟨*function* `routeflush` 114a⟩

⟨*function* iproute 107a⟩

⟨*function* printroute 107b⟩

⟨*function* routewrite 106⟩

I.8.10 kernel/network/ip/ipaux.c

⟨*global* v6Unspecified 341a⟩≡ (343)

```
/*
 *  well known IPv6 addresses
 */
ipaddr v6Unspecified = {
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0
};
```

⟨*global* v6loopback*(*(kernel/network/ip/ipaux.c)*)* 341b⟩≡ (343)

```
ipaddr v6loopback = {
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0x01
};
```

⟨*global* v6linklocal*(*(kernel/network/ip/ipaux.c)*)* 341c⟩≡ (343)

```
ipaddr v6linklocal = {
    0xfe, 0x80, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0
};
```

⟨*global* v6llpreflen*(*(kernel/network/ip/ipaux.c)*)* 341d⟩≡ (343)

```
//};
int v6llpreflen = 8;    /* link-local prefix length in bytes */
```

⟨*global* v6allnodesN 341e⟩≡ (343)

```
//int v6mcpreflen = 1;  /* multicast prefix length */

ipaddr v6allnodesN = {
    0xff, 0x01, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0x01
};
```

⟨*global* v6allnodesNmask 341f⟩≡ (343)

```
//};
ipaddr v6allnodesNmask = {
    0xff, 0xff, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0
};
```

⟨*global* v6allnodesL 342a⟩≡ (343)

```
//int v6aNpreflen = 2;  /* all nodes (N) prefix */

ipaddr v6allnodesL = {
    0xff, 0x02, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0x01
};
```

⟨*global* v6allnodesLmask 342b⟩≡ (343)

```
//};
ipaddr v6allnodesLmask = {
    0xff, 0xff, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0
};
```

⟨*global* v6solicitednode*(*(kernel/network/ip/ipaux.c)*)* 342c⟩≡ (343)

```
//int v6aLpreflen = 2;  /* all nodes (L) prefix */

ipaddr v6solicitednode = {
    0xff, 0x02, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0x01,
    0xff, 0, 0, 0
};
```

⟨*enum* _anon_ (kernel/network/ip/ipaux.c) 342d⟩≡ (343)

```
enum
{
    Isprefix= 16,
};
```

⟨*function* ipv62smcast 342e⟩≡ (343)

```
//#define CLASS(p) ((*(uchar*)(p))>>6)

void
ipv62smcast(uchar *smcast, uchar *a)
{
    assert(IPaddrlen == 16);
    memmove(smcast, v6solicitednode, IPaddrlen);
    smcast[13] = a[13];
    smcast[14] = a[14];
    smcast[15] = a[15];
}
```

⟨*function* parsemac 342f⟩≡ (343)

```
/*
 *  parse a hex mac address
 */
int
parsemac(uchar *to, char *from, int len)
{
    char nip[4];
    char *p;
    int i;

    p = from;
```

```
    memset(to, 0, len);
    for(i = 0; i < len; i++){
        if(p[0] == '\0' || p[1] == '\0')
            break;

        nip[0] = p[0];
        nip[1] = p[1];
        nip[2] = '\0';
        p += 2;

        to[i] = strtoul(nip, 0, 16);
        if(*p == ':')
            p++;
    }
    return i;
}
```

⟨kernel/network/ip/ipaux.c 343⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
#include    "../port/error.h"
#include    "ip.h"
#include    "ipv6.h"

//char *v6hdrtypes[Maxhdrtype] =
//{
//  [HBH]       "HopbyHop",
//  [ICMP]      "ICMP",
//  [IGMP]      "IGMP",
//  [GGP]       "GGP",
//  [IPINIP]    "IP",
//  [ST]        "ST",
//  [TCP]       "TCP",
//  [UDP]       "UDP",
//  [ISO_TP4]   "ISO_TP4",
//  [RH]        "Routinghdr",
//  [FH]        "Fraghdr",
//  [IDRP]      "IDRP",
//  [RSVP]      "RSVP",
//  [AH]        "Authhdr",
//  [ESP]       "ESP",
//  [ICMPv6]    "ICMPv6",
//  [NNH]       "Nonexthdr",
//  [ISO_IP]    "ISO_IP",
//  [IGRP]      "IGRP",
//  [OSPF]      "OSPF",
//};

⟨global v6Unspecified 341a⟩
⟨global v6loopback((kernel/network/ip/ipaux.c)) 341b⟩

⟨global v6linklocal((kernel/network/ip/ipaux.c)) 341c⟩
//ipaddr v6linklocalmask = {
//  0xff, 0xff, 0xff, 0xff,
//  0xff, 0xff, 0xff, 0xff,
//  0, 0, 0, 0,
//  0, 0, 0, 0
```

⟨*global* v6llpreflen((kernel/network/ip/ipaux.c)) 341d⟩

```
//ipaddr v6multicast = {
//  0xff, 0, 0, 0,
//  0, 0, 0, 0,
//  0, 0, 0, 0,
//  0, 0, 0, 0
//};
//ipaddr v6multicastmask = {
//  0xff, 0, 0, 0,
//  0, 0, 0, 0,
//  0, 0, 0, 0,
//  0, 0, 0, 0
//};
```
⟨*global* v6allnodesN 341e⟩
```
//ipaddr v6allroutersN = {
//  0xff, 0x01, 0, 0,
//  0, 0, 0, 0,
//  0, 0, 0, 0,
//  0, 0, 0, 0x02
```
⟨*global* v6allnodesNmask 341f⟩
⟨*global* v6allnodesL 342a⟩
```
//ipaddr v6allroutersL = {
//  0xff, 0x02, 0, 0,
//  0, 0, 0, 0,
//  0, 0, 0, 0,
//  0, 0, 0, 0x02
```
⟨*global* v6allnodesLmask 342b⟩
⟨*global* v6solicitednode((kernel/network/ip/ipaux.c)) 342c⟩
```
//ipaddr v6solicitednodemask = {
//  0xff, 0xff, 0xff, 0xff,
//  0xff, 0xff, 0xff, 0xff,
//  0xff, 0xff, 0xff, 0xff,
//  0xff, 0x0, 0x0, 0x0
//};
//int v6snpreflen = 13;
```

⟨*function* ptclcsum 77d⟩

⟨*enum* _anon_ (kernel/network/ip/ipaux.c) 342d⟩

⟨*function* ipv62smcast 342e⟩

⟨*function* parsemac 342f⟩

⟨*function* iphash 117f⟩

⟨*function* iphtadd 118a⟩

⟨*function* iphtrem 118b⟩

⟨*function* iphtlook 124⟩

I.8.11 kernel/network/ip/ipifc.c

⟨*enum* _anon_ (kernel/network/ip/ipifc.c) 344⟩≡ (354b)
```
enum {
    ⟨constant Maxmedia 19c⟩
```

```
    Nself       = Maxmedia*5,
    NHASH       = 1<<6,
    NCACHE      = 256,
    QMAX        = 192*1024-1,
};
```

⟨*global* sfixedformat 345a⟩≡ (354b)

```
char sfixedformat[] = "device %s maxtu %d sendra %d recvra %d mflag %d oflag"
" %d maxraint %d minraint %d linkmtu %d reachtime %d rxmitra %d ttl %d routerlt"
" %d pktin %lud pktout %lud errin %lud errout %lud\n";
```

⟨*global* slineformat 345b⟩≡ (354b)

```
char slineformat[] = "  %-40I %-10M %-40I %-12lud %-12lud\n";
```

⟨*function* ipifcstate 345c⟩≡ (354b)

```
static int
ipifcstate(Conv *c, char *state, int n)
{
    Ipifc *ifc;
    Iplifc *lifc;
    int m;

    ifc = (Ipifc*)c->ptcl;
    m = snprint(state, n, sfixedformat,
        ifc->dev, ifc->maxtu, ifc->sendra6, ifc->recvra6,
        ifc->rp.mflag, ifc->rp.oflag, ifc->rp.maxraint,
        ifc->rp.minraint, ifc->rp.linkmtu, ifc->rp.reachtime,
        ifc->rp.rxmitra, ifc->rp.ttl, ifc->rp.routerlt,
        ifc->in, ifc->out, ifc->inerr, ifc->outerr);

    rlock(ifc);
    for(lifc = ifc->lifc; lifc && n > m; lifc = lifc->next)
        m += snprint(state+m, n - m, slineformat, lifc->local,
            lifc->mask, lifc->remote, lifc->validlt, lifc->preflt);
    if(ifc->lifc == nil)
        m += snprint(state+m, n - m, "\n");
    runlock(ifc);
    return m;
}
```

⟨*function* ipifclocal 345d⟩≡ (354b)

```
static int
ipifclocal(Conv *c, char *state, int n)
{
    Ipifc *ifc;
    Iplifc *lifc;
    Iplink *link;
    int m;

    ifc = (Ipifc*)c->ptcl;
    m = 0;

    rlock(ifc);
    for(lifc = ifc->lifc; lifc; lifc = lifc->next){
        m += snprint(state+m, n - m, "%-40.40I ->", lifc->local);
        for(link = lifc->link; link; link = link->lifclink)
            m += snprint(state+m, n - m, " %-40.40I", link->self->a);
        m += snprint(state+m, n - m, "\n");
    }
    runlock(ifc);
    return m;
}
```

⟨*function* ipifcinuse 346a⟩≡ (354b)

```
static int
ipifcinuse(Conv *c)
{
    Ipifc *ifc;

    ifc = (Ipifc*)c->ptcl;
    return ifc->m != nil;
}
```

⟨*function* ipifckick 346b⟩≡ (354b)

```
/*
 *  called when a process writes to an interface's 'data'
 */
static void
ipifckick(void *x)
{
    Conv *c = x;
    Block *bp;
    Ipifc *ifc;

    bp = qget(c->wq);
    if(bp == nil)
        return;

    ifc = (Ipifc*)c->ptcl;
    if(!canrlock(ifc)){
        freeb(bp);
        return;
    }
    if(waserror()){
        runlock(ifc);
        nexterror();
    }
    if(ifc->m == nil || ifc->m->pktin == nil)
        freeb(bp);
    else
        (*ifc->m->pktin)(c->p->f, ifc, bp);
    runlock(ifc);
    poperror();
}
```

⟨*function* ipifcconnect 346c⟩≡ (354b)

```
/*
 *  associate an address with the interface.  This wipes out any previous
 *  addresses.  This is a macro that means, remove all the old interfaces
 *  and add a new one.
 */
static char*
ipifcconnect(Conv* c, char **argv, int argc)
{
    char *err;
    Ipifc *ifc;

    ifc = (Ipifc*)c->ptcl;

    if(ifc->m == nil)
         return "ipifc not yet bound to device";

    if(waserror()){
```

```
        wunlock(ifc);
        nexterror();
    }
    wlock(ifc);
    while(ifc->lifc){
        err = ipifcremlifc(ifc, ifc->lifc);
        if(err)
            error(err);
    }
    wunlock(ifc);
    poperror();

    err = ipifcadd(ifc, argv, argc, 0, nil);
    if(err)
        return err;

    Fsconnected(c, nil);
    return nil;
}
```

⟨*global* freeiplink 347a⟩≡ (354b)

```
/*
 *  These structures are unlinked from their chains while
 *  other threads may be using them.  To avoid excessive locking,
 *  just put them aside for a while before freeing them.
 *  called with f->self locked
 */
static Iplink *freeiplink;
```

⟨*global* freeipself 347b⟩≡ (354b)

```
static Ipself *freeipself;
```

⟨*function* iplinkfree 347c⟩≡ (354b)

```
static void
iplinkfree(Iplink *p)
{
    Iplink **l, *np;
    ulong now = NOW;

    l = &freeiplink;
    for(np = *l; np; np = *l){
        if(np->expire > now){
            *l = np->next;
            free(np);
            continue;
        }
        l = &np->next;
    }
    p->expire = now + 5000; /* give other threads 5 secs to get out */
    p->next = nil;
    *l = p;
}
```

⟨*function* ipselffree 347d⟩≡ (354b)

```
static void
ipselffree(Ipself *p)
{
    Ipself **l, *np;
    ulong now = NOW;
```

```
    l = &freeipself;
    for(np = *l; np; np = *l){
        if(np->expire > now){
            *l = np->next;
            free(np);
            continue;
        }
        l = &np->next;
    }
    p->expire = now + 5000; /* give other threads 5 secs to get out */
    p->next = nil;
    *l = p;
}
```

⟨*global* stformat 348a⟩≡ (354b)

```
static char *stformat = "%-44.44I %2.2d %4.4s\n";
```

⟨*enum* _anon_ (kernel/network/ip/ipifc.c)2 348b⟩≡ (354b)

```
enum
{
    Nstformat= 41,
};
```

⟨*function* iptentative 348c⟩≡ (354b)

```
int
iptentative(Fs *f, uchar *addr)
{
    Ipself *p;

    p = f->self->hash[hashipa(addr)];
    for(; p; p = p->next){
        if(ipcmp(addr, p->a) == 0)
            return p->link->lifc->tentative;
    }
    return 0;
}
```

⟨*function* findprimaryipv4 348d⟩≡ (354b)

```
/*
 *  returns first ip address configured
 */
static void
findprimaryipv4(Fs *f, uchar *local)
{
    Conv **cp, **e;
    Ipifc *ifc;
    Iplifc *lifc;

    /* find first ifc local address */
    e = &f->ipifc->conv[f->ipifc->nc];
    for(cp = f->ipifc->conv; cp < e; cp++){
        if(*cp == 0)
            continue;
        ifc = (Ipifc*)(*cp)->ptcl;
        if((lifc = ifc->lifc) != nil){
            ipmove(local, lifc->local);
            return;
        }
    }
}
```

⟨*function* findlocalip 349⟩≡ (354b)

```
/*
 *  find the local address 'closest' to the remote system, copy it to
 *  local and return the ifc for that address
 */
void
findlocalip(Fs *f, uchar *local, uchar *remote)
{
    int version, atype = unspecifiedv6, atypel = unknownv6;
    int atyper, deprecated;
    ipaddr gate, gnet;
    Ipifc *ifc;
    Iplifc *lifc;
    Route *r;

    USED(atype);
    USED(atypel);
    qlock(f->ipifc);
    r = v6lookup(f, remote, nil);
    version = (memcmp(remote, v4prefix, IPv4off) == 0)? V4: V6;

    if(r != nil){
        ifc = r->ifc;
        if(r->type & Rv4)
            v4tov6(gate, r->v4.gate);
        else {
            ipmove(gate, r->v6.gate);
            ipmove(local, v6Unspecified);
        }

        switch(version) {
        case V4:
            /* find ifc address closest to the gateway to use */
            for(lifc = ifc->lifc; lifc; lifc = lifc->next){
                maskip(gate, lifc->mask, gnet);
                if(ipcmp(gnet, lifc->net) == 0){
                    ipmove(local, lifc->local);
                    goto out;
                }
            }
            break;
        case V6:
            /* find ifc address with scope matching the destination */
            atyper = v6addrtype(remote);
            deprecated = 0;
            for(lifc = ifc->lifc; lifc; lifc = lifc->next){
                atypel = v6addrtype(lifc->local);
                /* prefer appropriate scope */
                if(atypel > atype && atype < atyper ||
                   atypel < atype && atype > atyper){
                    ipmove(local, lifc->local);
                    deprecated = !v6addrcurr(lifc);
                    atype = atypel;
                } else if(atypel == atype){
                    /* avoid deprecated addresses */
                    if(deprecated && v6addrcurr(lifc)){
                        ipmove(local, lifc->local);
                        atype = atypel;
                        deprecated = 0;
                    }
```

```
                }
                if(atype == atyper && !deprecated)
                    goto out;
            }
            if(atype >= atyper)
                goto out;
            break;
        default:
            panic("findlocalip: version %d", version);
        }
    }

    switch(version){
    case V4:
        findprimaryipv4(f, local);
        break;
    case V6:
        findprimaryipv6(f, local);
        break;
    default:
        panic("findlocalip2: version %d", version);
    }

out:
    qunlock(f->ipifc);
}
```

⟨*function* ipv4local 350a⟩≡ (354b)

```
/*
 *  return first v4 address associated with an interface
 */
int
ipv4local(Ipifc *ifc, uchar *addr)
{
    Iplifc *lifc;

    for(lifc = ifc->lifc; lifc; lifc = lifc->next){
        if(isv4(lifc->local)){
            memmove(addr, lifc->local+IPv4off, IPv4addrlen);
            return 1;
        }
    }
    return 0;
}
```

⟨*function* iplocalonifc 350b⟩≡ (354b)

```
/*
 *  see if this address is bound to the interface
 */
Iplifc*
iplocalonifc(Ipifc *ifc, uchar *ip)
{
    Iplifc *lifc;

    for(lifc = ifc->lifc; lifc; lifc = lifc->next)
        if(ipcmp(ip, lifc->local) == 0)
            return lifc;
    return nil;
}
```

⟨*function* ipproxyifc 351a⟩≡ (354b)

```
/*
 *  See if we're proxying for this address on this interface
 */
int
ipproxyifc(Fs *f, Ipifc *ifc, uchar *ip)
{
    Route *r;
    ipaddr net;
    Iplifc *lifc;

    /* see if this is a direct connected pt to pt address */
    r = v6lookup(f, ip, nil);
    if(r == nil || (r->type & (Rifc|Rproxy)) != (Rifc|Rproxy))
        return 0;

    /* see if this is on the right interface */
    for(lifc = ifc->lifc; lifc; lifc = lifc->next){
        maskip(ip, lifc->mask, net);
        if(ipcmp(net, lifc->remote) == 0)
            return 1;
    }
    return 0;
}
```

⟨*function* ipismulticast 351b⟩≡ (354b)

```
/*
 *  return multicast version if any
 */
int
ipismulticast(uchar *ip)
{
    if(isv4(ip)){
        if(ip[IPv4off] >= 0xe0 && ip[IPv4off] < 0xf0)
            return V4;
    }
    else if(ip[0] == 0xff)
        return V6;
    return 0;
}
```

⟨*function* ipifcaddmulti 351c⟩≡ (354b)

```
/*
 *  add a multicast address to an interface, called with c->car locked
 */
void
ipifcaddmulti(Conv *c, uchar *ma, uchar *ia)
{
    Ipifc *ifc;
    Iplifc *lifc;
    Conv **p;
    Ipmulti *multi, **l;
    Fs *f;

    f = c->p->f;

    for(l = &c->multi; *l; l = &(*l)->next)
        if(ipcmp(ma, (*l)->ma) == 0 && ipcmp(ia, (*l)->ia) == 0)
            return;     /* it's already there */
```

```
    multi = *l = smalloc(sizeof(*multi));
    ipmove(multi->ma, ma);
    ipmove(multi->ia, ia);
    multi->next = nil;

    for(p = f->ipifc->conv; *p; p++){
        if((*p)->inuse == 0)
            continue;
        ifc = (Ipifc*)(*p)->ptcl;
        if(waserror()){
            wunlock(ifc);
            nexterror();
        }
        wlock(ifc);
        for(lifc = ifc->lifc; lifc; lifc = lifc->next)
            if(ipcmp(ia, lifc->local) == 0)
                addselfcache(f, ifc, lifc, ma, Rmulti);
        wunlock(ifc);
        poperror();
    }
}
```

⟨*function* ipifcremmulti 352⟩≡ (354b)

```
/*
 *  remove a multicast address from an interface, called with c->car locked
 */
void
ipifcremmulti(Conv *c, uchar *ma, uchar *ia)
{
    Ipmulti *multi, **l;
    Iplifc *lifc;
    Conv **p;
    Ipifc *ifc;
    Fs *f;

    f = c->p->f;

    for(l = &c->multi; *l; l = &(*l)->next)
        if(ipcmp(ma, (*l)->ma) == 0 && ipcmp(ia, (*l)->ia) == 0)
            break;

    multi = *l;
    if(multi == nil)
        return;     /* we don't have it open */

    *l = multi->next;

    for(p = f->ipifc->conv; *p; p++){
        if((*p)->inuse == 0)
            continue;

        ifc = (Ipifc*)(*p)->ptcl;
        if(waserror()){
            wunlock(ifc);
            nexterror();
        }
        wlock(ifc);
        for(lifc = ifc->lifc; lifc; lifc = lifc->next)
            if(ipcmp(ia, lifc->local) == 0)
                remselfcache(f, ifc, lifc, ma);
```

```
        wunlock(ifc);
        poperror();
    }

    free(multi);
}
```

⟨*function* ipifcjoinmulti 353a⟩≡ (354b)

```
/*
 *  make lifc's join and leave multicast groups
 */
static char*
ipifcjoinmulti(Ipifc *ifc, char **argv, int argc)
{
    USED(ifc, argv, argc);
    return nil;
}
```

⟨*function* ipifcleavemulti 353b⟩≡ (354b)

```
static char*
ipifcleavemulti(Ipifc *ifc, char **argv, int argc)
{
    USED(ifc, argv, argc);
    return nil;
}
```

⟨*function* ipifcregisterproxy 353c⟩≡ (354b)

```
static void
ipifcregisterproxy(Fs *f, Ipifc *ifc, uchar *ip)
{
    Conv **cp, **e;
    Ipifc *nifc;
    Iplifc *lifc;
    Medium *m;
    ipaddr net;

    /* register the address on any network that will proxy for us */
    e = &f->ipifc->conv[f->ipifc->nc];

    if(!isv4(ip)) {             /* V6 */
        for(cp = f->ipifc->conv; cp < e; cp++){
            if(*cp == nil || (nifc = (Ipifc*)(*cp)->ptcl) == ifc)
                continue;
            rlock(nifc);
            m = nifc->m;
            if(m == nil || m->addmulti == nil) {
                runlock(nifc);
                continue;
            }
            for(lifc = nifc->lifc; lifc; lifc = lifc->next){
                maskip(ip, lifc->mask, net);
                if(ipcmp(net, lifc->remote) == 0) {
                    /* add solicited-node multicast addr */
                    ipv62smcast(net, ip);
                    addselfcache(f, nifc, lifc, net, Rmulti);
                    arpenter(f, V6, ip, nifc->mac, 6, 0);
                    // (*m->addmulti)(nifc, net, ip);
                    break;
                }
            }
```

```
            runlock(nifc);
        }
    }
    else {                  /* V4 */
        for(cp = f->ipifc->conv; cp < e; cp++){
            if(*cp == nil || (nifc = (Ipifc*)(*cp)->ptcl) == ifc)
                continue;
            rlock(nifc);
            m = nifc->m;
            if(m == nil || m->areg == nil){
                runlock(nifc);
                continue;
            }
            for(lifc = nifc->lifc; lifc; lifc = lifc->next){
                maskip(ip, lifc->mask, net);
                if(ipcmp(net, lifc->remote) == 0){
                    (*m->areg)(nifc, ip);
                    break;
                }
            }
            runlock(nifc);
        }
    }
}
```

⟨*enum* _anon_ (kernel/network/ip/ipifc.c)4 354a⟩≡ (354b)

```
//}

enum {
    Ngates = 3,
};
```

⟨kernel/network/ip/ipifc.c 354b⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "../port/error.h"

#include "ip.h"
#include "ipv6.h"

//#define DPRINT if(0)print

extern char*    ipifcadd6(Ipifc *ifc, char**argv, int argc);
extern char*    ipifcadd(Ipifc *ifc, char **argv, int argc, int tentative, Iplifc *lifcp);
extern char*    ipifcrem(Ipifc *ifc, char **argv, int argc);
```

⟨*enum* _anon_ (kernel/network/ip/ipifc.c) 344⟩

⟨*global* media 19b⟩

⟨*struct* Ipself 109a⟩

⟨*struct* Ipselftab 108d⟩

```
/*
 *  Multicast addresses are chained onto a Chan so that
 *  we can remove them when the Chan is closed.
```

```
 */
typedef struct Ipmcast Ipmcast;
```
⟨*struct* Ipmcast 158c⟩

⟨*macro* hashipa 110a⟩

⟨*global* tifc 61b⟩

```
static void addselfcache(Fs *f, Ipifc *ifc, Iplifc *lifc, uchar *a, int type);
static void remselfcache(Fs *f, Ipifc *ifc, Iplifc *lifc, uchar *a);
static char*    ipifcjoinmulti(Ipifc *ifc, char **argv, int argc);
static char*    ipifcleavemulti(Ipifc *ifc, char **argv, int argc);
static void ipifcregisterproxy(Fs*, Ipifc*, uchar*);
static char*    ipifcremlifc(Ipifc*, Iplifc*);
```

⟨*function* addipmedium 19d⟩

⟨*function* ipfindmedium 59c⟩

⟨*function* ipifcbind 57d⟩

⟨*function* ipifcunbind 71b⟩

⟨*global* sfixedformat 345a⟩

⟨*global* slineformat 345b⟩

⟨*function* ipifcstate 345c⟩

⟨*function* ipifclocal 345d⟩

⟨*function* ipifcinuse 346a⟩

⟨*function* ipifckick 346b⟩

⟨*function* ipifccreate 57c⟩

⟨*function* ipifcclose 72⟩

⟨*function* ipifcsetmtu 68d⟩

⟨*function* ipifcadd 60⟩

⟨*function* ipifcremlifc 70⟩

⟨*function* ipifcrem 69g⟩

⟨*function* ipifcaddroute 115b⟩

⟨*function* ipifcremroute 115c⟩

⟨*function* ipifcconnect 346c⟩

⟨*function* ipifcra6 475⟩

⟨*function* ipifcctl 59d⟩

⟨*function* ipifcstats 65a⟩

⟨*function* ipifcinit 56a⟩

⟨*function* addselfcache 109c⟩

⟨*global* freeiplink 347a⟩
⟨*global* freeipself 347b⟩

⟨*function* iplinkfree 347c⟩

⟨*function* ipselffree 347d⟩

⟨*function* remselfcache 110b⟩

⟨*global* stformat 348a⟩
⟨*enum* _anon_ (kernel/network/ip/ipifc.c)2 348b⟩

⟨*function* ipselftabread 112d⟩

⟨*function* iptentative 348c⟩

⟨*function* ipforme 108c⟩

⟨*function* findipifc 97c⟩

⟨*enum* _anon_ (kernel/network/ip/ipifc.c)3 476a⟩

⟨*function* v6addrtype 476b⟩

⟨*macro* v6addrcurr 477a⟩

⟨*function* findprimaryipv6 477b⟩

⟨*function* findprimaryipv4 348d⟩

⟨*function* findlocalip 349⟩

⟨*function* ipv4local 350a⟩

⟨*function* ipv6local 477c⟩

⟨*function* ipv6anylocal 477d⟩

⟨*function* iplocalonifc 350b⟩

⟨*function* ipproxyifc 351a⟩

⟨*function* ipismulticast 351b⟩

```
//int
//ipisbm(uchar *ip)
//{
//  if(isv4(ip)){
//      if(ip[IPv4off] >= 0xe0 && ip[IPv4off] < 0xf0)
//          return V4;
//      else if(ipcmp(ip, IPv4bcast) == 0)
//          return V4;
//  }
//  else if(ip[0] == 0xff)
//      return V6;
//  return 0;
```

```
//}
```

⟨*function* ipifcaddmulti 351c⟩

⟨*function* ipifcremmulti 352⟩

⟨*function* ipifcjoinmulti 353a⟩

⟨*function* ipifcleavemulti 353b⟩

⟨*function* ipifcregisterproxy 353c⟩

```
/* added for new v6 mesg types */
//static void
//adddefroute6(Fs *f, uchar *gate, int force)
//{
//  Route *r;
//
//  r = v6lookup(f, v6Unspecified, nil);
//  /*
//   * route entries generated by all other means take precedence
//   * over router announcements.
//   */
//  if (r && !force && strcmp(r->tag, "ra") != 0)
//      return;
//
//  v6delroute(f, v6Unspecified, v6Unspecified, 1);
//  v6addroute(f, "ra", v6Unspecified, v6Unspecified, gate, 0);
```
⟨*enum* _anon_ (kernel/network/ip/ipifc.c)4 354a⟩

⟨*function* ipifcadd6 478⟩

I.8.12 kernel/network/ip/netlog.c

⟨*struct* Netlogflag 357a⟩≡ (359a)

```
typedef struct Netlogflag {
    char*   name;
    int mask;
} Netlogflag;
```

⟨*global* flags 357b⟩≡ (359a)

```
static Netlogflag flags[] =
{
    { "ppp",    Logppp, },
    { "ip",     Logip, },
    { "fs",     Logfs, },
    { "tcp",    Logtcp, },
    { "icmp",   Logicmp, },
    { "udp",    Logudp, },
    { "compress",   Logcompress, },
    { "gre",    Loggre, },
    { "tcpwin", Logtcp|Logtcpwin, },
    { "tcprxmt",    Logtcp|Logtcprxmt, },
    { "udpmsg", Logudp|Logudpmsg, },
    { "ipmsg",  Logip|Logipmsg, },
    { "esp",    Logesp, },
```

```
    { nil,      0, },
};
```

⟨*global* Ebadnetctl 358a⟩≡ (359a)

```
char Ebadnetctl[] = "too few arguments for netlog control message";
```

⟨*function* netlogclose 358b⟩≡ (359a)

```
void
netlogclose(Fs *f)
{
    lock(f->alog);
    if(waserror()){
        unlock(f->alog);
        nexterror();
    }
    f->alog->opens--;
    if(f->alog->opens == 0){
        free(f->alog->buf);
        f->alog->buf = nil;
    }
    unlock(f->alog);
    poperror();
}
```

⟨*function* netlog 358c⟩≡ (359a)

```
void
netlog(Fs *f, int mask, char *fmt, ...)
{
    char buf[256], *t, *fp;
    int i, n;
    va_list arg;

    if(!(f->alog->logmask & mask))
        return;

    if(f->alog->opens == 0)
        return;

    va_start(arg, fmt);
    n = vseprint(buf, buf+sizeof(buf), fmt, arg) - buf;
    va_end(arg);

    lock(f->alog);
    i = f->alog->len + n - Nlog;
    if(i > 0){
        f->alog->len -= i;
        f->alog->rptr += i;
        if(f->alog->rptr >= f->alog->end)
            f->alog->rptr = f->alog->buf + (f->alog->rptr - f->alog->end);
    }
    t = f->alog->rptr + f->alog->len;
    fp = buf;
    f->alog->len += n;
    while(n-- > 0){
        if(t >= f->alog->end)
            t = f->alog->buf + (t - f->alog->end);
        *t++ = *fp++;
    }
    unlock(f->alog);
```

```
    wakeup(f->alog);
}
```

⟨kernel/network/ip/netlog.c 359a⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
#include    "../port/error.h"
#include    "../ip/ip.h"

⟨enum _anon_ (kernel/network/ip/netlog.c) 165b⟩

⟨struct Netlog 165a⟩

⟨struct Netlogflag 357a⟩

⟨global flags 357b⟩

⟨global Ebadnetctl 358a⟩

⟨enum _anon_ (kernel/network/ip/netlog.c)2 167c⟩

⟨global routecmd 168a⟩

⟨function netloginit 165d⟩

⟨function netlogopen 166g⟩

⟨function netlogclose 358b⟩

⟨function netlogready 167b⟩

⟨function netlogread 167a⟩

⟨function netlogctl 168b⟩

⟨function netlog 358c⟩
```

I.8.13 kernel/network/ip/ptclbsum.c

⟨global endian((kernel/network/ip/ptclbsum.c)) 359b⟩≡ (360)

```
static  short   endian  = 1;
```

⟨global aendian((kernel/network/ip/ptclbsum.c)) 359c⟩≡ (360)

```
static  uchar*  aendian = (uchar*)&endian;
```

⟨constant LITTLE((kernel/network/ip/ptclbsum.c)) 359d⟩≡ (360)

```
#define LITTLE  *aendian
```

⟨function ptclbsum((kernel/network/ip/ptclbsum.c)) 359e⟩≡ (360)

```
ushort
ptclbsum(uchar *addr, int len)
{
    ulong losum, hisum, mdsum, x;
    ulong t1, t2;

    losum = 0;
```

```
    hisum = 0;
    mdsum = 0;

    x = 0;
    if((ulong)addr & 1) {
        if(len) {
            hisum += addr[0];
            len--;
            addr++;
        }
        x = 1;
    }
    while(len >= 16) {
        t1 = *(ushort*)(addr+0);
        t2 = *(ushort*)(addr+2);    mdsum += t1;
        t1 = *(ushort*)(addr+4);    mdsum += t2;
        t2 = *(ushort*)(addr+6);    mdsum += t1;
        t1 = *(ushort*)(addr+8);    mdsum += t2;
        t2 = *(ushort*)(addr+10);   mdsum += t1;
        t1 = *(ushort*)(addr+12);   mdsum += t2;
        t2 = *(ushort*)(addr+14);   mdsum += t1;
        mdsum += t2;
        len -= 16;
        addr += 16;
    }
    while(len >= 2) {
        mdsum += *(ushort*)addr;
        len -= 2;
        addr += 2;
    }
    if(x) {
        if(len)
            losum += addr[0];
        if(LITTLE)
            losum += mdsum;
        else
            hisum += mdsum;
    } else {
        if(len)
            hisum += addr[0];
        if(LITTLE)
            hisum += mdsum;
        else
            losum += mdsum;
    }

    losum += hisum >> 8;
    losum += (hisum & 0xff) << 8;
    while(hisum = losum>>16)
        losum = hisum + (losum & 0xffff);

    return losum & 0xffff;
}
```

⟨kernel/network/ip/ptclbsum.c 360⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
```

```
#include    "../port/error.h"
#include    "ip.h"

⟨global endian((kernel/network/ip/ptclbsum.c)) 359b⟩
⟨global aendian((kernel/network/ip/ptclbsum.c)) 359c⟩
⟨constant LITTLE((kernel/network/ip/ptclbsum.c)) 359d⟩

⟨function ptclbsum((kernel/network/ip/ptclbsum.c)) 359e⟩
```

I.8.14 kernel/network/ip/il.c

```
⟨enum _anon_ (kernel/network/ip/il.c)3 361a⟩≡                    (370b)
  enum
  {
      Seconds  = 1000,
      Iltickms  = 50,  /* time base */
      AckDelay = 2*Iltickms, /* max time twixt message rcvd & ack sent */
      MaxTimeout  = 30*Seconds, /* max time between rexmit */
      QueryTime = 10*Seconds, /* time between subsequent queries */
      DeathTime = 30*QueryTime,

      MaxRexmit  = 16,  /* max retransmissions before hangup */
      Defaultwin = 20,

      LogAGain = 3,
      AGain  = 1<<LogAGain,
      LogDGain = 2,
      DGain  = 1<<LogDGain,

      DefByteRate = 100,  /* assume a megabit link */
      DefRtt  = 50,  /* cross country on a great day */

      ⟨constant Maxrq(IL) 136c⟩
  };

⟨enum _anon_ (kernel/network/ip/il.c)4 361b⟩≡                    (370b)
  enum
  {
      Nqt= 8,
  };

⟨enum _anon_ (kernel/network/ip/il.c)5 361c⟩≡                    (370b)
  enum
  {
      ⟨constant IL_xxxSIZE 136a⟩
      ⟨constant IP_ILPROTO 131b⟩

  };

⟨global etime 361d⟩≡                                             (370b)
  static char *etime = "connection timed out";

⟨function ilackq 361e⟩≡                                          (370b)
  void
  ilackq(Ilcb *ic, Block *bp)
  {
      Block *np;
      int n;
```

```
    n = blocklen(bp);

    /* Enqueue a copy on the unacked queue in case this one gets lost */
    np = copyblock(bp, n);
    if(ic->unacked)
        ic->unackedtail->list = np;
    else
        ic->unacked = np;
    ic->unackedtail = np;
    np->list = nil;
    ic->unackedbytes += n;
}
```

⟨*function* ilrttcalc 362a⟩≡ (370b)

```
static
void
ilrttcalc(Ilcb *ic, Block *bp)
{
    int rtt, tt, pt, delay, rate;

    rtt = arch_fastticks(nil) - ic->rttstart;
    rtt = (rtt*scalemul)/scalediv;
    delay = ic->delay;
    rate = ic->rate;

    /* Guard against zero wrap */
    if(rtt > 120000 || rtt < 0)
        return;

    /* this block had to be transmitted after the one acked so count its size */
    ic->rttlen += blocklen(bp)  + IL_IPSIZE + IL_HDRSIZE;

    if(ic->rttlen < 256){
        /* guess fixed delay as rtt of small packets */
        delay += rtt - (delay>>LogAGain);
        if(delay < AGain)
            delay = AGain;
        ic->delay = delay;
    } else {
        /* if packet took longer than avg rtt delay, recalc rate */
        tt = rtt - (delay>>LogAGain);
        if(tt > 0){
            rate += ic->rttlen/tt - (rate>>LogAGain);
            if(rate < AGain)
                rate = AGain;
            ic->rate = rate;
        }
    }

    /* mdev */
    pt = ic->rttlen/(rate>>LogAGain) + (delay>>LogAGain);
    ic->mdev += abs(rtt-pt) - (ic->mdev>>LogDGain);

    if(rtt > ic->maxrtt)
        ic->maxrtt = rtt;
}
```

⟨*function* ilackto 362b⟩≡ (370b)

```
void
ilackto(Ilcb *ic, ulong ackto, Block *bp)
```

```
{
    Ilhdr *h;
    ulong id;

    if(ic->rttack == ackto)
        ilrttcalc(ic, bp);

    /* Cancel if we've passed the packet we were interested in */
    if(ic->rttack <= ackto)
        ic->rttack = 0;

    qlock(&ic->ackq);
    while(ic->unacked) {
        h = (Ilhdr *)ic->unacked->rp;
        id = nhgetl(h->ilid);
        if(ackto < id)
            break;

        bp = ic->unacked;
        ic->unacked = bp->list;
        bp->list = nil;
        ic->unackedbytes -= blocklen(bp);
        freeblist(bp);
        ic->rexmit = 0;
        ilsettimeout(ic);
    }
    qunlock(&ic->ackq);
}
```

⟨*function* ilrexmit 363⟩≡ (370b)

```
void
ilrexmit(Ilcb *ic)
{
    Ilhdr *h;
    Block *nb;
    Conv *c;
//    ulong id; BUG?
    Ilpriv *priv;

    nb = nil;
    qlock(&ic->ackq);
    if(ic->unacked)
        nb = copyblock(ic->unacked, blocklen(ic->unacked));
    qunlock(&ic->ackq);

    if(nb == nil)
        return;

    h = (Ilhdr*)nb->rp;
    h->vihl = IP_VER4;

    h->iltype = Ildataquery;
    hnputl(h->ilack, ic->recvd);
    h->ilspec = ilnextqt(ic);
    h->ilsum[0] = 0;
    h->ilsum[1] = 0;
    hnputs(h->ilsum, ptclcsum(nb, IL_IPSIZE, nhgets(h->illen)));

    c = ic->conv;
    //id = nhgetl(h->ilid);
```

```
    //netlog(c->p->f, Logil, "il: rexmit %lud %lud: %d %lud: %I %d/%d\n", id, ic->recvd,
//  ic->rexmit, ic->timeout,
//  c->raddr, c->lport, c->rport);

    ilbackoff(ic);

    ipoput4(c->p->f, nb, 0, c->ttl, c->tos, c);

    /* statistics */
    ic->rxtot++;
    priv = c->p->priv;
    priv->rexmit++;
}
```

⟨*function* ilhangup 364a⟩≡ (370b)

```
void
ilhangup(Conv *s, char *msg)
{
    Ilcb *ic;
    int callout;

    //netlog(s->p->f, Logil, "il: hangup! %I %d/%d: %s\n", s->raddr,
//  s->lport, s->rport, msg?msg:"no reason");

    ic = (Ilcb*)s->ptcl;
    callout = ic->state == Ilsyncer;
    illocalclose(s);

    qhangup(s->rq, msg);
    qhangup(s->wq, msg);

    if(callout)
        Fsconnected(s, msg);
}
```

⟨*function* ilpullup 364b⟩≡ (370b)

```
void
ilpullup(Conv *s)
{
    Ilcb *ic;
    Ilhdr *oh;
    Block *bp;
    ulong oid, dlen;
    Ilpriv *ipriv;

    ic = (Ilcb*)s->ptcl;
    if(ic->state != Ilestablished)
        return;

    qlock(&ic->outo);
    while(ic->outoforder) {
        bp = ic->outoforder;
        oh = (Ilhdr*)bp->rp;
        oid = nhgetl(oh->ilid);
        if(oid <= ic->recvd) {
            ic->outoforder = bp->list;
            freeblist(bp);
            continue;
        }
        if(oid != ic->recvd+1){
```

```
            ipriv = s->p->priv;
            ipriv->stats[OutOfOrder]++;
            break;
        }

        ic->recvd = oid;
        ic->outoforder = bp->list;

        bp->list = nil;
        dlen = nhgets(oh->illen)-IL_HDRSIZE;
        bp = trimblock(bp, IL_IPSIZE+IL_HDRSIZE, dlen);
        /*
         * Upper levels don't know about multiple-block
         * messages so copy all into one (yick).
         */
        bp = concatblock(bp);
        if(bp == 0)
            panic("ilpullup");
        bp = packblock(bp);
        if(bp == 0)
            panic("ilpullup2");
        qpass(s->rq, bp);
    }
    qunlock(&ic->outo);
}
```

⟨*function* iloutoforder 365⟩≡ (370b)

```
void
iloutoforder(Conv *s, Ilhdr *h, Block *bp)
{
    Ilcb *ic;
    uchar *lid;
    Block *f, **l;
    ulong id, newid;
    Ilpriv *ipriv;

    ipriv = s->p->priv;
    ic = (Ilcb*)s->ptcl;
    bp->list = nil;

    id = nhgetl(h->ilid);
    /* Window checks */
    if(id <= ic->recvd || id > ic->recvd+ic->window) {
        //netlog(s->p->f, Logil, "il: message outside window %lud <%lud-%lud>: %I %d/%d\n",
//   id, ic->recvd, ic->recvd+ic->window, s->raddr, s->lport, s->rport);
        freeblist(bp);
        return;
    }

    /* Packet is acceptable so sort onto receive queue for pullup */
    qlock(&ic->outo);
    if(ic->outoforder == nil)
        ic->outoforder = bp;
    else {
        l = &ic->outoforder;
        for(f = *l; f; f = f->list) {
            lid = ((Ilhdr*)(f->rp))->ilid;
            newid = nhgetl(lid);
            if(id <= newid) {
                if(id == newid) {
```

```
                    ipriv->stats[DupMsg]++;
                    ipriv->stats[DupBytes] += blocklen(bp);
                    qunlock(&ic->outo);
                    freeblist(bp);
                    return;
                }
                bp->list = f;
                *l = bp;
                qunlock(&ic->outo);
                return;
            }
            l = &f->list;
        }
        *l = bp;
    }
    qunlock(&ic->outo);
}
```

⟨*function* ilreject 366a⟩≡ (370b)

```
void
ilreject(Fs *f, Ilhdr *inih)
{
    Ilhdr *ih;
    Block *bp;

    bp = allocb(IL_IPSIZE+IL_HDRSIZE);
    bp->wp += IL_IPSIZE+IL_HDRSIZE;

    ih = (Ilhdr *)(bp->rp);
    ih->vihl = IP_VER4;

    /* Ip fields */
    ih->proto = IP_ILPROTO;
    hnputs(ih->illen, IL_HDRSIZE);
    ih->frag[0] = 0;
    ih->frag[1] = 0;
    hnputl(ih->dst, nhgetl(inih->src));
    hnputl(ih->src, nhgetl(inih->dst));
    hnputs(ih->ilsrc, nhgets(inih->ildst));
    hnputs(ih->ildst, nhgets(inih->ilsrc));
    hnputl(ih->ilid, nhgetl(inih->ilack));
    hnputl(ih->ilack, nhgetl(inih->ilid));
    ih->iltype = Ilclose;
    ih->ilspec = 0;
    ih->ilsum[0] = 0;
    ih->ilsum[1] = 0;

    if(ilcksum)
        hnputs(ih->ilsum, ptclcsum(bp, IL_IPSIZE, IL_HDRSIZE));

    ipoput4(f, bp, 0, MAXTTL, DFLTTOS, nil);
}
```

⟨*function* ilsettimeout 366b⟩≡ (370b)

```
void
ilsettimeout(Ilcb *ic)
{
    ulong pt;

    pt = (ic->delay>>LogAGain)
```

```
        + ic->unackedbytes/(ic->rate>>LogAGain)
        + (ic->mdev>>(LogDGain-1))
        + AckDelay;
    if(pt > MaxTimeout)
        pt = MaxTimeout;
    ic->timeout = NOW + pt;
}
```

⟨*function* ilbackoff 367a⟩≡ (370b)

```
void
ilbackoff(Ilcb *ic)
{
    ulong pt;
    int i;

    pt = (ic->delay>>LogAGain)
        + ic->unackedbytes/(ic->rate>>LogAGain)
        + (ic->mdev>>(LogDGain-1))
        + AckDelay;
    for(i = 0; i < ic->rexmit; i++)
        pt = pt + (pt>>1);
    if(pt > MaxTimeout)
        pt = MaxTimeout;
    ic->timeout = NOW + pt;

    if(ic->fasttimeout)
        ic->timeout = NOW+Iltickms;

    ic->rexmit++;
}
```

⟨*constant* Tfuture 367b⟩≡ (370b)

```
// complain if two numbers not within an hour of each other
#define Tfuture (1000*60*60)
```

⟨*function* later 367c⟩≡ (370b)

```
int
later(ulong t1, ulong t2, char *x)
{
    int dt;

    dt = t1 - t2;
    if(dt > 0) {
        if(x != nil && dt > Tfuture)
            print("%s: way future %d\n", x, dt);
        return 1;
    }
    if(dt < -Tfuture) {
        if(x != nil)
            print("%s: way past %d\n", x, -dt);
        return 1;
    }
    return 0;
}
```

⟨*function* ilackproc 367d⟩≡ (370b)

```
void
ilackproc(void *x)
{
    Ilcb *ic;
```

```
    Conv **s, *p;
    Proto *il;

    il = x;

    while(waserror())
        ;
loop:
    tsleep(&up->sleepr, returnfalse, 0, Iltickms);
    for(s = il->conv; s && *s; s++) {
        p = *s;
        ic = (Ilcb*)p->ptcl;

        switch(ic->state) {
        case Ilclosed:
        case Illistening:
            break;
        case Ilclosing:
            if(later(NOW, ic->timeout, "timeout0")) {
                if(ic->rexmit > MaxRexmit){
                    ilhangup(p, nil);
                    break;
                }
                ilsendctl(p, nil, Ilclose, ic->next, ic->recvd, 0);
                ilbackoff(ic);
            }
            break;

        case Ilsyncee:
        case Ilsyncer:
            if(later(NOW, ic->timeout, "timeout1")) {
                if(ic->rexmit > MaxRexmit){
                    ilhangup(p, etime);
                    break;
                }
                ilsendctl(p, nil, Ilsync, ic->start, ic->recvd, 0);
                ilbackoff(ic);
            }
            break;

        case Ilestablished:
            if(ic->recvd != ic->acksent)
            if(later(NOW, ic->acktime, "acktime"))
                ilsendctl(p, nil, Ilack, ic->next, ic->recvd, 0);

            if(later(NOW, ic->querytime, "querytime")){
                if(later(NOW, ic->lastrecv+DeathTime, "deathtime")){
                    //netlog(il->f, Logil, "il: hangup: deathtime\n");
                    ilhangup(p, etime);
                    break;
                }
                ilsendctl(p, nil, Ilquery, ic->next, ic->recvd, ilnextqt(ic));
                ic->querytime = NOW + QueryTime;
            }

            if(ic->unacked != nil)
            if(later(NOW, ic->timeout, "timeout2")) {
                if(ic->rexmit > MaxRexmit){
                    //netlog(il->f, Logil, "il: hangup: too many rexmits\n");
                    ilhangup(p, etime);
```

```
                    break;
                }
                ilsendctl(p, nil, Ilquery, ic->next, ic->recvd, ilnextqt(ic));
                ic->rxquery++;
                ilbackoff(ic);
            }
            break;
        }
    }
    goto loop;
}
```

⟨*function* ilfreeq 369a⟩≡ (370b)

```
void
ilfreeq(Ilcb *ic)
{
    Block *bp, *next;

    qlock(&ic->ackq);
    for(bp = ic->unacked; bp; bp = next) {
        next = bp->list;
        freeblist(bp);
    }
    ic->unacked = nil;
    qunlock(&ic->ackq);

    qlock(&ic->outo);
    for(bp = ic->outoforder; bp; bp = next) {
        next = bp->list;
        freeblist(bp);
    }
    ic->outoforder = nil;
    qunlock(&ic->outo);
}
```

⟨*function* iladvise 369b⟩≡ (370b)

```
void
iladvise(Proto *il, Block *bp, char *msg)
{
    Ilhdr *h;
    Ilcb *ic;
    ipaddr source, dest;
    ushort psource;
    Conv *s, **p;

    h = (Ilhdr*)(bp->rp);

    v4tov6(dest, h->dst);
    v4tov6(source, h->src);
    psource = nhgets(h->ilsrc);


    /* Look for a connection, unfortunately the destination port is missing */
    qlock(il);
    for(p = il->conv; *p; p++) {
        s = *p;
        if(s->lport == psource)
        if(ipcmp(s->laddr, source) == 0)
        if(ipcmp(s->raddr, dest) == 0){
            qunlock(il);
```

```
            ic = (Ilcb*)s->ptcl;
            switch(ic->state){
            case Ilsyncer:
                ilhangup(s, msg);
                break;
            }
            freeblist(bp);
            return;
        }
    }
    qunlock(il);
    freeblist(bp);
}
```

⟨*function* ilnextqt 370a⟩≡ (370b)

```
int
ilnextqt(Ilcb *ic)
{
    int x;

    qlock(&ic->ackq);
    x = ic->qtx;
    if(++x > Nqt)
        x = 1;
    ic->qtx = x;
    ic->qt[x] = ic->next-1; /* highest xmitted packet */
    ic->qt[0] = ic->qt[x]; /* compatibility with old implementations */
    qunlock(&ic->ackq);

    return x;
}
```

⟨kernel/network/ip/il.c 370b⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "../port/error.h"

#include "ip.h"

// forward decl
typedef struct Ilcb Ilcb;
typedef struct Ilhdr Ilhdr;
typedef struct Ilpriv Ilpriv;
```

⟨*enum* _anon_ (kernel/network/ip/il.c) 132c⟩

⟨*global* ilstates 132d⟩

⟨*enum* _anon_ (kernel/network/ip/il.c)2 133b⟩

⟨*global* iltype 133c⟩

⟨*enum* _anon_ (kernel/network/ip/il.c)3 361a⟩

⟨*enum* _anon_ (kernel/network/ip/il.c)4 361b⟩

⟨*struct* Ilcb 133d⟩

⟨*enum* _anon_ (kernel/network/ip/il.c)5 361c⟩

⟨*enum* mode((kernel/network/ip/il.c)) 136d⟩

⟨*struct* Ilhdr 135c⟩

⟨*enum* _anon_ (kernel/network/ip/il.c)6 135a⟩

⟨*global* statnames((kernel/network/ip/il.c)) 135b⟩

⟨*struct* Ilpriv 134a⟩

```
/* state for query/dataquery messages */

void ilsendctl(Conv*, Ilhdr*, int, ulong, ulong, int);
void ilackq(Ilcb*, Block*);
void ilprocess(Conv*, Ilhdr*, Block*);
void ilpullup(Conv*);
void ilhangup(Conv*, char*);
void ilfreeq(Ilcb*);
void ilrexmit(Ilcb*);
void ilbackoff(Ilcb*);
void ilsettimeout(Ilcb*);
char* ilstart(Conv*, int, int);
void ilackproc(void*);
void iloutoforder(Conv*, Ilhdr*, Block*);
void iliput(Proto*, Ipifc*, Block*);
void iladvise(Proto*, Block*, char*);
int ilnextqt(Ilcb*);
void ilcbinit(Ilcb*);
int later(ulong, ulong, char*);
void ilreject(Fs*, Ilhdr*);
void illocalclose(Conv *c);
```

⟨*global* ilcksum 147a⟩
⟨*global* scalexxx 132a⟩
⟨*global* etime 361d⟩

⟨*function* ilconnect 136e⟩

⟨*function* ilstate 147c⟩

⟨*function* ilinuse 133a⟩

⟨*function* ilannounce 136f⟩

⟨*function* illocalclose 146b⟩

⟨*function* ilclose 146a⟩

⟨*function* ilkick 137⟩

⟨*function* ilcreate 136b⟩

⟨*function* ilxstats 147b⟩

⟨*function* ilackq 361e⟩

⟨*function* ilrttcalc 362a⟩

⟨*function* ilackto 362b⟩

⟨*function* iliput 138⟩

⟨*function* _ilprocess 142⟩

⟨*function* ilrexmit 363⟩

⟨*function* ilprocess 141⟩

⟨*function* ilhangup 364a⟩

⟨*function* ilpullup 364b⟩

⟨*function* iloutoforder 365⟩

⟨*function* ilsendctl 140⟩

⟨*function* ilreject 366a⟩

⟨*function* ilsettimeout 366b⟩

⟨*function* ilbackoff 367a⟩

⟨*constant* Tfuture 367b⟩
⟨*function* later 367c⟩

⟨*function* ilackproc 367d⟩

⟨*function* ilcbinit 145⟩

⟨*function* ilstart 144⟩

⟨*function* ilfreeq 369a⟩

⟨*function* iladvise 369b⟩

⟨*function* ilnextqt 370a⟩

⟨*function* inittimescale 131c⟩

⟨*function* ilinit 131a⟩

I.8.15 kernel/network/ip/udp.c

⟨*constant* DPRINT 372a⟩≡ (374)

```
#define DPRINT if(0)print
```

⟨*enum* _anon_ (kernel/network/ip/udp.c) 372b⟩≡ (374)

```
enum
{
    ⟨constant UDP_UDPHDR_SZ 119d⟩

    ⟨constant UDP4_PHDR_OFF 119e⟩
    ⟨constant UDP4_PHDR_SZ 119f⟩
    ⟨constant UDP4_IPHDR_SZ 119c⟩
```

```
    ⟨constant UDP6_xxx 480b⟩
    ⟨constant IP_UDPPROTO 116b⟩

    ⟨constant UDP_USEAD7 129d⟩

    Udprxms     = 200,
    Udptickms   = 100,
    Udpmaxxmit  = 10,
};
```

⟨*function* udpadvise 373⟩≡ (374)

```
void
udpadvise(Proto *udp, Block *bp, char *msg)
{
    Udp4hdr *h4;
    Udp6hdr *h6;
    ipaddr source, dest;
    ushort psource, pdest;
    Conv *s, **p;
    int version;

    h4 = (Udp4hdr*)(bp->rp);
    version = ((h4->vihl&0xF0)==IP_VER6) ? 6 : 4;

    switch(version) {
    case V4:
        v4tov6(dest, h4->udpdst);
        v4tov6(source, h4->udpsrc);
        psource = nhgets(h4->udpsport);
        pdest = nhgets(h4->udpdport);
        break;
    case V6:
        h6 = (Udp6hdr*)(bp->rp);
        ipmove(dest, h6->udpdst);
        ipmove(source, h6->udpsrc);
        psource = nhgets(h6->udpsport);
        pdest = nhgets(h6->udpdport);
        break;
    default:
        panic("udpadvise: version %d", version);
        return;  /* to avoid a warning */
    }

    /* Look for a connection */
    qlock(udp);
    for(p = udp->conv; *p; p++) {
        s = *p;
        if(s->rport == pdest)
        if(s->lport == psource)
        if(ipcmp(s->raddr, dest) == 0)
        if(ipcmp(s->laddr, source) == 0){
            if(s->ignoreadvice)
                break;
            qlock(s);
            qunlock(udp);
            qhangup(s->rq, msg);
            qhangup(s->wq, msg);
            qunlock(s);
            freeblist(bp);
            return;
```

```
        }
    }
    qunlock(udp);
    freeblist(bp);
}
```

⟨kernel/network/ip/udp.c 374⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
#include    "../port/error.h"

#include    "ip.h"
#include    "ipv6.h"


⟨constant DPRINT 372a⟩

⟨enum _anon_ (kernel/network/ip/udp.c) 372b⟩

// forward decl
typedef struct Udp4hdr Udp4hdr;
typedef struct Udp6hdr Udp6hdr;
typedef struct Udpstats Udpstats;
typedef struct Udppriv Udppriv;
typedef struct Udpcb Udpcb;

//void (*etherprofiler)(char *name, int qlen);
void udpkick(void *x, Block *bp);


⟨struct Udp4hdr 119b⟩

⟨struct Udp6hdr 479b⟩

⟨struct Udpstats 119a⟩

⟨struct Udppriv 117a⟩

/*
 *  protocol specific part of Conv
 */
⟨struct Udpcb 129a⟩

⟨function udpconnect 120d⟩


⟨function udpstate 128a⟩

⟨function udpannounce 121a⟩

⟨function udpcreate 120b⟩

⟨function udpclose 120c⟩

⟨function udpkick 121b⟩

⟨function udpiput 122⟩
```

⟨*function* udpctl 129c⟩

⟨*function* udpadvise 373⟩

⟨*function* udpstats 127c⟩

⟨*function* udpinit 116a⟩

I.8.16 kernel/network/ip/tcp.c

⟨*enum* _anon_ (kernel/network/ip/tcp.c) 375⟩≡ (431b)

```
enum
{
    QMAX        = 64*1024-1,
    IP_TCPPROTO = 6,

    TCP4_IPLEN  = 8,
    TCP4_PHDRSIZE   = 12,
    TCP4_HDRSIZE    = 20,
    TCP4_TCBPHDRSZ  = 40,
    TCP4_PKT    = TCP4_IPLEN+TCP4_PHDRSIZE,

    TCP6_IPLEN  = 0,
    TCP6_PHDRSIZE   = 40,
    TCP6_HDRSIZE    = 20,
    TCP6_TCBPHDRSZ  = 60,
    TCP6_PKT    = TCP6_IPLEN+TCP6_PHDRSIZE,

    TcptimerOFF = 0,
    TcptimerON  = 1,
    TcptimerDONE    = 2,
    MAX_TIME    = (1<<20),  /* Forever */
    TCP_ACK     = 50,       /* Timed ack sequence in ms */
    MAXBACKMS   = 9*60*1000,    /* longest backoff time (ms) before hangup */

    URG     = 0x20,     /* Data marked urgent */
    ACK     = 0x10,     /* Acknowledge is valid */
    PSH     = 0x08,     /* Whole data pipe is pushed */
    RST     = 0x04,     /* Reset connection */
    SYN     = 0x02,     /* Pkt. is synchronise */
    FIN     = 0x01,     /* Start close down */

    EOLOPT      = 0,
    NOOPOPT     = 1,
    MSSOPT      = 2,
    MSS_LENGTH  = 4,        /* Maximum segment size */
    WSOPT       = 3,
    WS_LENGTH   = 3,        /* Bits to scale window size by */
    MSL2        = 10,
    MSPTICK     = 50,       /* Milliseconds per timer tick */
    DEF_MSS     = 1460,     /* Default maximum segment */
    DEF_MSS6    = 1280,     /* Default maximum segment (min) for v6 */
    DEF_RTT     = 500,      /* Default round trip */
    DEF_KAT     = 120000,   /* Default time (ms) between keep alives */
    TCP_LISTEN  = 0,        /* Listen connection */
    TCP_CONNECT = 1,        /* Outgoing connection */
    SYNACK_RXTIMER  = 250,      /* ms between SYNACK retransmits */
```

```
    TCPREXMTTHRESH  = 3,        /* dupack threshhold for rxt */

    FORCE       = 1,
    CLONE       = 2,
    RETRAN      = 4,
    ACTIVE      = 8,
    SYNACK      = 16,

    LOGAGAIN    = 3,
    LOGDGAIN    = 2,

    Closed      = 0,        /* Connection states */
    Listen,
    Syn_sent,
    Syn_received,
    Established,
    Finwait1,
    Finwait2,
    Close_wait,
    Closing,
    Last_ack,
    Time_wait,

    Maxlimbo    = 1000,     /* maximum procs waiting for response to SYN ACK */
    NLHT        = 256,      /* hash table size, must be a power of 2 */
    LHTMASK     = NLHT-1,

    /*
     * window is 64kb * 2^n
     * these factors determine the ultimate bandwidth-delay product.
     * 64kb * 2^5 = 2mb, or 2× overkill for 100mbps * 70ms.
     */
    Maxqscale   = 4,        /* maximum queuing scale */
    Defadvscale = 4,        /* default advertisement */
};
```

⟨*global* tcpstates 376a⟩≡ (431b)

```
/* Must correspond to the enumeration above */
char *tcpstates[] =
{
    "Closed",   "Listen",   "Syn_sent", "Syn_received",
    "Established",  "Finwait1", "Finwait2", "Close_wait",
    "Closing",  "Last_ack",     "Time_wait"
};
```

⟨*struct* Tcptimer 376b⟩≡ (431b)

```
struct Tcptimer
{
    Tcptimer    *next;
    Tcptimer    *prev;
    Tcptimer    *readynext;
    int state;
    int start;
    int count;
    void    (*func)(void*);
    void    *arg;
};
```

⟨*struct* Tcp4hdr 376c⟩≡ (431b)

```
struct Tcp4hdr
```

```
{
    uchar   vihl;       /* Version and header length */
    uchar   tos;        /* Type of service */
    uchar   length[2];  /* packet length */
    uchar   id[2];      /* Identification */
    uchar   frag[2];    /* Fragment information */
    uchar   Unused;
    uchar   proto;
    uchar   tcplen[2];
    uchar   tcpsrc[4];
    uchar   tcpdst[4];
    uchar   tcpsport[2];
    uchar   tcpdport[2];
    uchar   tcpseq[4];
    uchar   tcpack[4];
    uchar   tcpflag[2];
    uchar   tcpwin[2];
    uchar   tcpcksum[2];
    uchar   tcpurg[2];
    /* Options segment */
    uchar   tcpopt[1];
};
```

⟨*struct* Tcp6hdr 377a⟩≡ (431b)

```
struct Tcp6hdr
{
    uchar   vcf[4];
    uchar   ploadlen[2];
    uchar   proto;
    uchar   ttl;
    ipaddr   tcpsrc;
    ipaddr   tcpdst;
    uchar   tcpsport[2];
    uchar   tcpdport[2];
    uchar   tcpseq[4];
    uchar   tcpack[4];
    uchar   tcpflag[2];
    uchar   tcpwin[2];
    uchar   tcpcksum[2];
    uchar   tcpurg[2];
    /* Options segment */
    uchar   tcpopt[1];
};
```

⟨*struct* Tcp 377b⟩≡ (431b)

```
struct  Tcp
{
    ushort  source;
    ushort  dest;
    ulong   seq;
    ulong   ack;
    uchar   flags;
    uchar   update;
    ushort  ws; /* window scale option */
    ulong   wnd;    /* prescaled window*/
    ushort  urg;
    ushort  mss;    /* max segment size option (if not zero) */
    ushort  len;    /* size of data */
};
```

⟨*struct* Reseq 378a⟩≡ (431b)

```
struct Reseq
{
    Reseq   *next;
    Tcp seg;
    Block   *bp;
    ushort  length;
};
```

⟨*struct* Tcpctl 378b⟩≡ (431b)

```
struct Tcpctl
{
    uchar   state;          /* Connection state */
    uchar   type;           /* Listening or active connection */
    uchar   code;           /* Icmp code */
    struct {
        ulong   una;        /* Unacked data pointer */
        ulong   nxt;        /* Next sequence expected */
        ulong   ptr;        /* Data pointer */
        ulong   wnd;        /* Tcp send window */
        ulong   urg;        /* Urgent data pointer */
        ulong   wl2;
        uint    scale;      /* how much to right shift window */
                    /* in xmitted packets */
        /* to implement tahoe and reno TCP */
        ulong   dupacks;    /* number of duplicate acks rcvd */
        ulong   partialack;
        int recovery;   /* loss recovery flag */
        int retransmit; /* retransmit 1 packet @ una flag */
        int rto;
        ulong   rxt;        /* right window marker for recovery */
                    /* "recover" rfc3782 */
    } snd;
    struct {
        ulong   nxt;        /* Receive pointer to next uchar slot */
        ulong   wnd;        /* Receive window incoming */
        ulong   wsnt;       /* Last wptr sent.  important to */
                    /* track for large bdp */
        ulong   wptr;
        ulong   urg;        /* Urgent pointer */
        ulong   ackptr;     /* last acked sequence */
        int blocked;
        uint    scale;      /* how much to left shift window in */
                    /* rcv'd packets */
    } rcv;
    ulong   iss;            /* Initial sequence number */
    ulong   cwind;          /* Congestion window */
    ulong   abcbytes;       /* appropriate byte counting rfc 3465 */
    uint    scale;          /* desired snd.scale */
    ulong   ssthresh;       /* Slow start threshold */
    int resent;         /* Bytes just resent */
    int irs;            /* Initial received squence */
    ushort  mss;            /* Maximum segment size */
    int rerecv;         /* Overlap of data rerecevived */
    ulong   window;         /* Our receive window (queue) */
    uint    qscale;         /* Log2 of our receive window (queue) */
    uchar   backoff;        /* Exponential backoff counter */
    int backedoff;      /* ms we've backed off for rexmits */
    uchar   flags;          /* State flags */
    Reseq   *reseq;         /* Resequencing queue */
```

```
    int nreseq;
    int reseqlen;
    Tcptimer    timer;          /* Activity timer */
    Tcptimer    acktimer;       /* Acknowledge timer */
    Tcptimer    rtt_timer;      /* Round trip timer */
    Tcptimer    katimer;        /* keep alive timer */
    ulong   rttseq;         /* Round trip sequence */
    int srtt;           /* Smoothed round trip */
    int mdev;           /* Mean deviation of round trip */
    int kacounter;      /* count down for keep alive */
    uint    sndsyntime;     /* time syn sent */
    ulong   time;           /* time Finwait2 or Syn_received was sent */
    ulong   timeuna;        /* snd.una when time was set */
    int nochecksum;     /* non-zero means don't send checksums */
    int flgcnt;         /* number of flags in the sequence (FIN,SEQ) */

    union {
        Tcp4hdr tcp4hdr;
        Tcp6hdr tcp6hdr;
    } protohdr;     /* prototype header */
};
```

⟨*struct* Limbo 379a⟩≡ (431b)

```
struct Limbo
{
    Limbo   *next;

    ipaddr   laddr;
    ipaddr   raddr;
    ushort  lport;
    ushort  rport;
    ulong   irs;        /* initial received sequence */
    ulong   iss;        /* initial sent sequence */
    ushort  mss;        /* mss from the other end */
    ushort  rcvscale;   /* how much to scale rcvd windows */
    ushort  sndscale;   /* how much to scale sent windows */
    ulong   lastsend;   /* last time we sent a synack */
    uchar   version;    /* v4 or v6 */
    uchar   rexmits;    /* number of retransmissions */
};
```

⟨*global* tcp_irtt 379b⟩≡ (431b)

```
int tcp_irtt = DEF_RTT; /* Initial guess at round trip time */
```

⟨*enum* _anon_ (kernel/network/ip/tcp.c)2 379c⟩≡ (431b)

```
enum {
    /* MIB stats */
    MaxConn,
    Mss,
    ActiveOpens,
    PassiveOpens,
    EstabResets,
    CurrEstab,
    InSegs,
    OutSegs,
    RetransSegs,
    RetransSegsSent,
    RetransTimeouts,
    InErrs,
    OutRsts,
```

```
    /* non-MIB stats */
    CsumErrs,
    HlenErrs,
    LenErrs,
    Resequenced,
    OutOfOrder,
    ReseqBytelim,
    ReseqPktlim,
    Delayack,
    Wopenack,

    Recovery,
    RecoveryDone,
    RecoveryRTO,
    RecoveryNoSeq,
    RecoveryCwind,
    RecoveryPA,

    Nstats
};
```

⟨*global* statnames*(*(kernel/network/ip/tcp.c)*)* 380a⟩≡ (431b)

```
static char *statnames[Nstats] =
{
[MaxConn]   "MaxConn",
[Mss]       "MaxSegment",
[ActiveOpens]   "ActiveOpens",
[PassiveOpens]  "PassiveOpens",
[EstabResets]   "EstabResets",
[CurrEstab] "CurrEstab",
[InSegs]    "InSegs",
[OutSegs]   "OutSegs",
[RetransSegs]   "RetransSegs",
[RetransSegsSent]   "RetransSegsSent",
[RetransTimeouts]   "RetransTimeouts",
[InErrs]    "InErrs",
[OutRsts]   "OutRsts",
[CsumErrs]  "CsumErrs",
[HlenErrs]  "HlenErrs",
[LenErrs]   "LenErrs",
[OutOfOrder]    "OutOfOrder",
[Resequenced]   "Resequenced",
[ReseqBytelim]  "ReseqBytelim",
[ReseqPktlim]   "ReseqPktlim",
[Delayack]  "Delayack",
[Wopenack]  "Wopenack",

[Recovery]  "Recovery",
[RecoveryDone]  "RecoveryDone",
[RecoveryRTO]   "RecoveryRTO",

[RecoveryNoSeq] "RecoveryNoSeq",
[RecoveryCwind] "RecoveryCwind",
[RecoveryPA]    "RecoveryPA",
};
```

⟨*struct* Tcppriv 380b⟩≡ (431b)

```
struct Tcppriv
{
```

```
    /* List of active timers */
    QLock   tl;
    Tcptimer *timers;

    /* hash table for matching conversations */
    Ipht    ht;

    /* calls in limbo waiting for an ACK to our SYN ACK */
    int nlimbo;
    Limbo   *lht[NLHT];

    /* for keeping track of tcpackproc */
    QLock   apl;
    int ackprocstarted;

    uvlong  stats[Nstats];
};
```

⟨*global* tcpporthogdefense 381a⟩≡ (431b)

```
/*
 *  Setting tcpporthogdefense to non-zero enables Dong Lin's
 *  solution to hijacked systems staking out port's as a form
 *  of DoS attack.
 *
 *  To avoid stateless Conv hogs, we pick a sequence number at random.  If
 *  that number gets acked by the other end, we shut down the connection.
 *  Look for tcpporthogdefense in the code.
 */
int tcpporthogdefense = 0;
```

⟨*function* tcpsetstate 381b⟩≡ (431b)

```
static void
tcpsetstate(Conv *s, uchar newstate)
{
    Tcpctl *tcb;
    uchar oldstate;
    Tcppriv *tpriv;

    tpriv = s->p->priv;

    tcb = (Tcpctl*)s->ptcl;

    oldstate = tcb->state;
    if(oldstate == newstate)
        return;

    if(oldstate == Established)
        tpriv->stats[CurrEstab]--;
    if(newstate == Established)
        tpriv->stats[CurrEstab]++;

    switch(newstate) {
    case Closed:
        qclose(s->rq);
        qclose(s->wq);
        qclose(s->eq);
        break;

    case Close_wait:        /* Remote closes */
        qhangup(s->rq, nil);
```

```
        break;
    }

    tcb->state = newstate;

    if(oldstate == Syn_sent && newstate != Closed)
        Fsconnected(s, nil);
}
```

⟨*function* tcpconnect 382a⟩≡ (431b)

```
static char*
tcpconnect(Conv *c, char **argv, int argc)
{
    char *e;
    Tcpctl *tcb;

    tcb = (Tcpctl*)(c->ptcl);
    if(tcb->state != Closed)
        return Econinuse;

    e = Fsstdconnect(c, argv, argc);
    if(e != nil)
        return e;
    tcpstart(c, TCP_CONNECT);

    return nil;
}
```

⟨*function* tcpstate 382b⟩≡ (431b)

```
static int
tcpstate(Conv *c, char *state, int n)
{
    Tcpctl *s;

    s = (Tcpctl*)(c->ptcl);

    return snprint(state, n,
        "%s qin %d qout %d rq %d.%d srtt %d mdev %d sst %lud cwin %lud "
        "swin %lud>>%d rwin %lud>>%d qscale %d timer.start %d "
        "timer.count %d rerecv %d katimer.start %d katimer.count %d\n",
        tcpstates[s->state],
        c->rq ? qlen(c->rq) : 0,
        c->wq ? qlen(c->wq) : 0,
        s->nreseq, s->reseqlen,
        s->srtt, s->mdev, s->ssthresh,
        s->cwind, s->snd.wnd, s->rcv.scale, s->rcv.wnd, s->snd.scale,
        s->qscale,
        s->timer.start, s->timer.count, s->rerecv,
        s->katimer.start, s->katimer.count);
}
```

⟨*function* tcpinuse 382c⟩≡ (431b)

```
static int
tcpinuse(Conv *c)
{
    Tcpctl *s;

    s = (Tcpctl*)(c->ptcl);
    return s->state != Closed;
}
```

⟨*function* tcpannounce 383a⟩≡ (431b)

```
static char*
tcpannounce(Conv *c, char **argv, int argc)
{
    char *e;
    Tcpctl *tcb;

    tcb = (Tcpctl*)(c->ptcl);
    if(tcb->state != Closed)
        return Econinuse;

    e = Fsstdannounce(c, argv, argc);
    if(e != nil)
        return e;
    tcpstart(c, TCP_LISTEN);
    Fsconnected(c, nil);

    return nil;
}
```

⟨*function* tcpclose 383b⟩≡ (431b)

```
/*
 *  tcpclose is always called with the q locked
 */
static void
tcpclose(Conv *c)
{
    Tcpctl *tcb;

    tcb = (Tcpctl*)c->ptcl;

    qhangup(c->rq, nil);
    qhangup(c->wq, nil);
    qhangup(c->eq, nil);
    qflush(c->rq);

    switch(tcb->state) {
    case Listen:
        /*
         *  reset any incoming calls to this listener
         */
        Fsconnected(c, "Hangup");

        localclose(c, nil);
        break;
    case Closed:
    case Syn_sent:
        localclose(c, nil);
        break;
    case Syn_received:
    case Established:
        tcb->flgcnt++;
        tcb->snd.nxt++;
        tcpsetstate(c, Finwait1);
        tcpoutput(c);
        break;
    case Close_wait:
        tcb->flgcnt++;
        tcb->snd.nxt++;
        tcpsetstate(c, Last_ack);
```

```
        tcpoutput(c);
        break;
    }
}
```

⟨*function* tcpkick 384a⟩≡ (431b)

```
static void
tcpkick(void *x)
{
    Conv *s = x;
    Tcpctl *tcb;

    tcb = (Tcpctl*)s->ptcl;

    if(waserror()){
        qunlock(s);
        nexterror();
    }
    qlock(s);

    switch(tcb->state) {
    case Syn_sent:
    case Syn_received:
    case Established:
    case Close_wait:
        /*
         * Push data
         */
        tcpoutput(s);
        break;
    default:
        localclose(s, "Hangup");
        break;
    }

    qunlock(s);
    poperror();
}
```

⟨*function* tcprcvwin 384b⟩≡ (431b)

```
static void
tcprcvwin(Conv *s)              /* Call with tcb locked */
{
    int w;
    Tcpctl *tcb;

    tcb = (Tcpctl*)s->ptcl;
    w = tcb->window - qlen(s->rq);
    if(w < 0)
        w = 0;
    /* RFC 1122 § 4.2.2.17 do not move right edge of window left */
    if(seq_lt(tcb->rcv.nxt + w, tcb->rcv.wptr))
        w = tcb->rcv.wptr - tcb->rcv.nxt;
    if(w != tcb->rcv.wnd)
    if(w>>tcb->rcv.scale == 0 || tcb->window > 4*tcb->mss && w < tcb->mss/4){
        tcb->rcv.blocked = 1;
        netlog(s->p->f, Logtcp, "tcprcvwin: window %lud qlen %d ws %ud lport %d\n",
            tcb->window, qlen(s->rq), tcb->rcv.scale, s->lport);
    }
    tcb->rcv.wnd = w;
```

```
    tcb->rcv.wptr = tcb->rcv.nxt + w;
}
```

⟨*function* tcpacktimer 385a⟩≡ (431b)

```
static void
tcpacktimer(void *v)
{
    Tcpctl *tcb;
    Conv *s;

    s = v;
    tcb = (Tcpctl*)s->ptcl;

    if(waserror()){
        qunlock(s);
        nexterror();
    }
    qlock(s);
    if(tcb->state != Closed){
        tcb->flags |= FORCE;
        tcpoutput(s);
    }
    qunlock(s);
    poperror();
}
```

⟨*function* tcpcongestion 385b⟩≡ (431b)

```
static void
tcpcongestion(Tcpctl *tcb)
{
    ulong inflight;

    inflight = tcb->snd.nxt - tcb->snd.una;
    if(inflight > tcb->cwind)
        inflight = tcb->cwind;
    tcb->ssthresh = inflight / 2;
    if(tcb->ssthresh < 2*tcb->mss)
        tcb->ssthresh = 2*tcb->mss;
}
```

⟨*enum* _anon_ (kernel/network/ip/tcp.c)3 385c⟩≡ (431b)

```
enum {
    L   = 2,    /* aggressive slow start; legal values ∈ (1.0, 2.0) */
};
```

⟨*function* tcpabcincr 385d⟩≡ (431b)

```
static void
tcpabcincr(Tcpctl *tcb, uint acked)
{
    uint limit;

    tcb->abcbytes += acked;
    if(tcb->cwind < tcb->ssthresh){
        /* slow start */
        if(tcb->snd.rto)
            limit = tcb->mss;
        else
            limit = L*tcb->mss;
        tcb->cwind += MIN(tcb->abcbytes, limit);
        tcb->abcbytes = 0;
```

```
    } else {
        tcb->snd.rto = 0;
        /* avoidance */
        if(tcb->abcbytes >= tcb->cwind){
            tcb->abcbytes -= tcb->cwind;
            tcb->cwind += tcb->mss;
        }
    }
}
```

⟨*function* tcpcreate 386a⟩≡ (431b)

```
static void
tcpcreate(Conv *c)
{
    c->rq = qopen(QMAX, Qcoalesce, tcpacktimer, c);
    c->wq = qopen(QMAX, Qkick, tcpkick, c);
}
```

⟨*function* timerstate 386b⟩≡ (431b)

```
static void
timerstate(Tcppriv *priv, Tcptimer *t, int newstate)
{
    if(newstate != TcptimerON){
        if(t->state == TcptimerON){
            /* unchain */
            if(priv->timers == t){
                priv->timers = t->next;
                if(t->prev != nil)
                    panic("timerstate1");
            }
            if(t->next)
                t->next->prev = t->prev;
            if(t->prev)
                t->prev->next = t->next;
            t->next = t->prev = nil;
        }
    } else {
        if(t->state != TcptimerON){
            /* chain */
            if(t->prev != nil || t->next != nil)
                panic("timerstate2");
            t->prev = nil;
            t->next = priv->timers;
            if(t->next)
                t->next->prev = t;
            priv->timers = t;
        }
    }
    t->state = newstate;
}
```

⟨*function* tcpackproc 386c⟩≡ (431b)

```
static void
tcpackproc(void *a)
{
    Tcptimer *t, *tp, *timeo;
    Proto *tcp;
    Tcppriv *priv;
    int loop;
```

```
    tcp = a;
    priv = tcp->priv;

    for(;;) {
        tsleep(&up->sleepr, returnfalse, 0, MSPTICK);

        qlock(&priv->tl);
        timeo = nil;
        loop = 0;
        for(t = priv->timers; t != nil; t = tp) {
            if(loop++ > 10000)
                panic("tcpackproc1");
            tp = t->next;
            if(t->state == TcptimerON) {
                t->count--;
                if(t->count == 0) {
                    timerstate(priv, t, TcptimerDONE);
                    t->readynext = timeo;
                    timeo = t;
                }
            }
        }
        qunlock(&priv->tl);

        loop = 0;
        for(t = timeo; t != nil; t = t->readynext) {
            if(loop++ > 10000)
                panic("tcpackproc2");
            if(t->state == TcptimerDONE && t->func != nil && !waserror()){
                (*t->func)(t->arg);
                poperror();
            }
        }

        limborexmit(tcp);
    }
}
```

⟨*function* tcpgo 387a⟩≡ (431b)

```
static void
tcpgo(Tcppriv *priv, Tcptimer *t)
{
    if(t == nil || t->start == 0)
        return;

    qlock(&priv->tl);
    t->count = t->start;
    timerstate(priv, t, TcptimerON);
    qunlock(&priv->tl);
}
```

⟨*function* tcphalt 387b⟩≡ (431b)

```
static void
tcphalt(Tcppriv *priv, Tcptimer *t)
{
    if(t == nil)
        return;

    qlock(&priv->tl);
    timerstate(priv, t, TcptimerOFF);
```

```
    qunlock(&priv->tl);
}
```

⟨*function* backoff 388a⟩≡ (431b)

```
static int
backoff(int n)
{
    return 1 << n;
}
```

⟨*function* localclose 388b⟩≡ (431b)

```
static void
localclose(Conv *s, char *reason)   /* called with tcb locked */
{
    Tcpctl *tcb;
    Tcppriv *tpriv;

    tpriv = s->p->priv;
    tcb = (Tcpctl*)s->ptcl;

    iphtrem(&tpriv->ht, s);

    tcphalt(tpriv, &tcb->timer);
    tcphalt(tpriv, &tcb->rtt_timer);
    tcphalt(tpriv, &tcb->acktimer);
    tcphalt(tpriv, &tcb->katimer);

    /* Flush reassembly queue; nothing more can arrive */
    dumpreseq(tcb);

    if(tcb->state == Syn_sent)
        Fsconnected(s, reason);
    if(s->state == Announced)
        wakeup(&s->listenr);

    qhangup(s->rq, reason);
    qhangup(s->wq, reason);

    tcpsetstate(s, Closed);
}
```

⟨*function* tcpmtu 388c⟩≡ (431b)

```
/* mtu (- TCP + IP hdr len) of 1st hop */
static int
tcpmtu(Proto *tcp, uchar *addr, int version, uint *scale)
{
    Ipifc *ifc;
    int mtu;

    ifc = findipifc(tcp->f, addr, 0);
    switch(version){
    default:
    case V4:
        mtu = DEF_MSS;
        if(ifc != nil)
            mtu = ifc->maxtu - ifc->m->hsize - (TCP4_PKT + TCP4_HDRSIZE);
        break;
    case V6:
        mtu = DEF_MSS6;
        if(ifc != nil)
```

```
            mtu = ifc->maxtu - ifc->m->hsize - (TCP6_PKT + TCP6_HDRSIZE);
        break;
    }
    /*
     * set the ws.  it doesn't commit us to anything.
     * ws is the ultimate limit to the bandwidth-delay product.
     */
    *scale = Defadvscale;

    return mtu;
}
```

⟨*function* inittcpctl 389⟩≡ (431b)

```
static void
inittcpctl(Conv *s, int mode)
{
    Tcpctl *tcb;
    Tcp4hdr* h4;
    Tcp6hdr* h6;
    Tcppriv *tpriv;
    int mss;

    tcb = (Tcpctl*)s->ptcl;

    memset(tcb, 0, sizeof(Tcpctl));

    tcb->ssthresh = QMAX;           /* reset by tcpsetscale() */
    tcb->srtt = tcp_irtt<<LOGAGAIN;
    tcb->mdev = 0;

    /* setup timers */
    tcb->timer.start = tcp_irtt / MSPTICK;
    tcb->timer.func = tcptimeout;
    tcb->timer.arg = s;
    tcb->rtt_timer.start = MAX_TIME;
    tcb->acktimer.start = TCP_ACK / MSPTICK;
    tcb->acktimer.func = tcpacktimer;
    tcb->acktimer.arg = s;
    tcb->katimer.start = DEF_KAT / MSPTICK;
    tcb->katimer.func = tcpkeepalive;
    tcb->katimer.arg = s;

    mss = DEF_MSS;

    /* create a prototype(pseudo) header */
    if(mode != TCP_LISTEN){
        if(ipcmp(s->laddr, IPnoaddr) == 0)
            findlocalip(s->p->f, s->laddr, s->raddr);

        switch(s->ipversion){
        case V4:
            h4 = &tcb->protohdr.tcp4hdr;
            memset(h4, 0, sizeof(*h4));
            h4->proto = IP_TCPPROTO;
            hnputs(h4->tcpsport, s->lport);
            hnputs(h4->tcpdport, s->rport);
            v6tov4(h4->tcpsrc, s->laddr);
            v6tov4(h4->tcpdst, s->raddr);
            break;
        case V6:
```

```
            h6 = &tcb->protohdr.tcp6hdr;
            memset(h6, 0, sizeof(*h6));
            h6->proto = IP_TCPPROTO;
            hnputs(h6->tcpsport, s->lport);
            hnputs(h6->tcpdport, s->rport);
            ipmove(h6->tcpsrc, s->laddr);
            ipmove(h6->tcpdst, s->raddr);
            mss = DEF_MSS6;
            break;
        default:
            panic("inittcpctl: version %d", s->ipversion);
        }
    }

    tcb->mss = tcb->cwind = mss;
    tcb->abcbytes = 0;
    tpriv = s->p->priv;
    tpriv->stats[Mss] = tcb->mss;

    /* default is no window scaling */
    tcpsetscale(s, tcb, 0, 0);
}
```

⟨*function* tcpstart 390⟩≡ (431b)

```
/*
 *  called with s qlocked
 */
static void
tcpstart(Conv *s, int mode)
{
    Tcpctl *tcb;
    Tcppriv *tpriv;
    char kpname[KNAMELEN];

    tpriv = s->p->priv;

    if(tpriv->ackprocstarted == 0){
        qlock(&tpriv->apl);
        if(tpriv->ackprocstarted == 0){
            snprint(kpname, sizeof kpname, "#I%dtcpack", s->p->f->dev);
            kproc(kpname, tcpackproc, s->p);
            tpriv->ackprocstarted = 1;
        }
        qunlock(&tpriv->apl);
    }

    tcb = (Tcpctl*)s->ptcl;

    inittcpctl(s, mode);

    iphtadd(&tpriv->ht, s);
    switch(mode) {
    case TCP_LISTEN:
        tpriv->stats[PassiveOpens]++;
        tcb->flags |= CLONE;
        tcpsetstate(s, Listen);
        break;

    case TCP_CONNECT:
        tpriv->stats[ActiveOpens]++;
```

```
        tcb->flags |= ACTIVE;
        tcpsndsyn(s, tcb);
        tcpsetstate(s, Syn_sent);
        tcpoutput(s);
        break;
    }
}
```

⟨*function* htontcp6 391⟩≡ (431b)

```
//}

static Block*
htontcp6(Tcp *tcph, Block *data, Tcp6hdr *ph, Tcpctl *tcb)
{
    int dlen;
    Tcp6hdr *h;
    ushort csum;
    ushort hdrlen, optpad = 0;
    uchar *opt;

    hdrlen = TCP6_HDRSIZE;
    if(tcph->flags & SYN){
        if(tcph->mss)
            hdrlen += MSS_LENGTH;
        if(tcph->ws)
            hdrlen += WS_LENGTH;
        optpad = hdrlen & 3;
        if(optpad)
            optpad = 4 - optpad;
        hdrlen += optpad;
    }

    if(data) {
        dlen = blocklen(data);
        data = padblock(data, hdrlen + TCP6_PKT);
        if(data == nil)
            return nil;
    }
    else {
        dlen = 0;
        data = allocb(hdrlen + TCP6_PKT + 64);  /* the 64 pad is to meet mintu's */
        if(data == nil)
            return nil;
        data->wp += hdrlen + TCP6_PKT;
    }

    /* copy in pseudo ip header plus port numbers */
    h = (Tcp6hdr *)(data->rp);
    memmove(h, ph, TCP6_TCBPHDRSZ);

    /* compose pseudo tcp header, do cksum calculation */
    hnputl(h->vcf, hdrlen + dlen);
    h->ploadlen[0] = h->ploadlen[1] = h->proto = 0;
    h->ttl = ph->proto;

    /* copy in variable bits */
    hnputl(h->tcpseq, tcph->seq);
    hnputl(h->tcpack, tcph->ack);
    hnputs(h->tcpflag, (hdrlen<<10) | tcph->flags);
    hnputs(h->tcpwin, tcph->wnd>>(tcb != nil ? tcb->snd.scale : 0));
```

```
    hnputs(h->tcpurg, tcph->urg);

    if(tcph->flags & SYN){
        opt = h->tcpopt;
        if(tcph->mss != 0){
            *opt++ = MSSOPT;
            *opt++ = MSS_LENGTH;
            hnputs(opt, tcph->mss);
            opt += 2;
        }
        if(tcph->ws != 0){
            *opt++ = WSOPT;
            *opt++ = WS_LENGTH;
            *opt++ = tcph->ws;
        }
        while(optpad-- > 0)
            *opt++ = NOOPOPT;
    }

    if(tcb != nil && tcb->nochecksum){
        h->tcpcksum[0] = h->tcpcksum[1] = 0;
    } else {
        csum = ptclcsum(data, TCP6_IPLEN, hdrlen+dlen+TCP6_PHDRSIZE);
        hnputs(h->tcpcksum, csum);
    }

    /* move from pseudo header back to normal ip header */
    memset(h->vcf, 0, 4);
    h->vcf[0] = IP_VER6;
    hnputs(h->ploadlen, hdrlen+dlen);
    h->proto = ph->proto;

    return data;
}
```

⟨*function* htontcp4 392⟩≡ (431b)

```
static Block*
htontcp4(Tcp *tcph, Block *data, Tcp4hdr *ph, Tcpctl *tcb)
{
    int dlen;
    Tcp4hdr *h;
    ushort csum;
    ushort hdrlen, optpad = 0;
    uchar *opt;

    hdrlen = TCP4_HDRSIZE;
    if(tcph->flags & SYN){
        if(tcph->mss)
            hdrlen += MSS_LENGTH;
        if(1)
            hdrlen += WS_LENGTH;
        optpad = hdrlen & 3;
        if(optpad)
            optpad = 4 - optpad;
        hdrlen += optpad;
    }

    if(data) {
        dlen = blocklen(data);
        data = padblock(data, hdrlen + TCP4_PKT);
```

```
        if(data == nil)
            return nil;
    }
    else {
        dlen = 0;
        data = allocb(hdrlen + TCP4_PKT + 64);  /* the 64 pad is to meet mintu's */
        if(data == nil)
            return nil;
        data->wp += hdrlen + TCP4_PKT;
    }

    /* copy in pseudo ip header plus port numbers */
    h = (Tcp4hdr *)(data->rp);
    memmove(h, ph, TCP4_TCBPHDRSZ);

    /* copy in variable bits */
    hnputs(h->tcplen, hdrlen + dlen);
    hnputl(h->tcpseq, tcph->seq);
    hnputl(h->tcpack, tcph->ack);
    hnputs(h->tcpflag, (hdrlen<<10) | tcph->flags);
    hnputs(h->tcpwin, tcph->wnd>>(tcb != nil ? tcb->snd.scale : 0));
    hnputs(h->tcpurg, tcph->urg);

    if(tcph->flags & SYN){
        opt = h->tcpopt;
        if(tcph->mss != 0){
            *opt++ = MSSOPT;
            *opt++ = MSS_LENGTH;
            hnputs(opt, tcph->mss);
            opt += 2;
        }
        /* always offer.  rfc1323 §2.2 */
        if(1){
            *opt++ = WSOPT;
            *opt++ = WS_LENGTH;
            *opt++ = tcph->ws;
        }
        while(optpad-- > 0)
            *opt++ = NOOPOPT;
    }

    if(tcb != nil && tcb->nochecksum){
        h->tcpcksum[0] = h->tcpcksum[1] = 0;
    } else {
        csum = ptclcsum(data, TCP4_IPLEN, hdrlen+dlen+TCP4_PHDRSIZE);
        hnputs(h->tcpcksum, csum);
    }

    return data;
}
```

⟨*function* ntohtcp6 393⟩≡ (431b)

```
static int
ntohtcp6(Tcp *tcph, Block **bpp)
{
    Tcp6hdr *h;
    uchar *optr;
    ushort hdrlen;
    ushort optlen;
    int n;
```

```
    *bpp = pullupblock(*bpp, TCP6_PKT+TCP6_HDRSIZE);
    if(*bpp == nil)
        return -1;

    h = (Tcp6hdr *)((*bpp)->rp);
    tcph->source = nhgets(h->tcpsport);
    tcph->dest = nhgets(h->tcpdport);
    tcph->seq = nhgetl(h->tcpseq);
    tcph->ack = nhgetl(h->tcpack);
    hdrlen = (h->tcpflag[0]>>2) & ~3;
    if(hdrlen < TCP6_HDRSIZE) {
        freeblist(*bpp);
        return -1;
    }

    tcph->flags = h->tcpflag[1];
    tcph->wnd = nhgets(h->tcpwin);
    tcph->urg = nhgets(h->tcpurg);
    tcph->mss = 0;
    tcph->ws = 0;
    tcph->update = 0;
    tcph->len = nhgets(h->ploadlen) - hdrlen;

    *bpp = pullupblock(*bpp, hdrlen+TCP6_PKT);
    if(*bpp == nil)
        return -1;

    optr = h->tcpopt;
    n = hdrlen - TCP6_HDRSIZE;
    while(n > 0 && *optr != EOLOPT) {
        if(*optr == NOOPOPT) {
            n--;
            optr++;
            continue;
        }
        optlen = optr[1];
        if(optlen < 2 || optlen > n)
            break;
        switch(*optr) {
        case MSSOPT:
            if(optlen == MSS_LENGTH)
                tcph->mss = nhgets(optr+2);
            break;
        case WSOPT:
            if(optlen == WS_LENGTH && *(optr+2) <= 14)
                tcph->ws = *(optr+2);
            break;
        }
        n -= optlen;
        optr += optlen;
    }
    return hdrlen;
}
```

⟨*function* ntohtcp4 394⟩≡ (431b)

```
static int
ntohtcp4(Tcp *tcph, Block **bpp)
{
    Tcp4hdr *h;
```

```
    uchar *optr;
    ushort hdrlen;
    ushort optlen;
    int n;

    *bpp = pullupblock(*bpp, TCP4_PKT+TCP4_HDRSIZE);
    if(*bpp == nil)
        return -1;

    h = (Tcp4hdr *)((*bpp)->rp);
    tcph->source = nhgets(h->tcpsport);
    tcph->dest = nhgets(h->tcpdport);
    tcph->seq = nhgetl(h->tcpseq);
    tcph->ack = nhgetl(h->tcpack);

    hdrlen = (h->tcpflag[0]>>2) & ~3;
    if(hdrlen < TCP4_HDRSIZE) {
        freeblist(*bpp);
        return -1;
    }

    tcph->flags = h->tcpflag[1];
    tcph->wnd = nhgets(h->tcpwin);
    tcph->urg = nhgets(h->tcpurg);
    tcph->mss = 0;
    tcph->ws = 0;
    tcph->update = 0;
    tcph->len = nhgets(h->length) - (hdrlen + TCP4_PKT);

    *bpp = pullupblock(*bpp, hdrlen+TCP4_PKT);
    if(*bpp == nil)
        return -1;

    optr = h->tcpopt;
    n = hdrlen - TCP4_HDRSIZE;
    while(n > 0 && *optr != EOLOPT) {
        if(*optr == NOOPOPT) {
            n--;
            optr++;
            continue;
        }
        optlen = optr[1];
        if(optlen < 2 || optlen > n)
            break;
        switch(*optr) {
        case MSSOPT:
            if(optlen == MSS_LENGTH)
                tcph->mss = nhgets(optr+2);
            break;
        case WSOPT:
            if(optlen == WS_LENGTH && *(optr+2) <= 14)
                tcph->ws = *(optr+2);
            break;
        }
        n -= optlen;
        optr += optlen;
    }
    return hdrlen;
}
```

⟨*function* tcpsndsyn 396a⟩≡ (431b)

```
/*
 *  For outgoing calls, generate an initial sequence
 *  number and put a SYN on the send queue
 */
static void
tcpsndsyn(Conv *s, Tcpctl *tcb)
{
    Tcppriv *tpriv;

    tcb->iss = (nrand(1<<16)<<16)|nrand(1<<16);
    tcb->rttseq = tcb->iss;
    tcb->snd.wl2 = tcb->iss;
    tcb->snd.una = tcb->iss;
    tcb->snd.rxt = tcb->iss;
    tcb->snd.ptr = tcb->rttseq;
    tcb->snd.nxt = tcb->rttseq;
    tcb->flgcnt++;
    tcb->flags |= FORCE;
    tcb->sndsyntime = NOW;

    /* set desired mss and scale */
    tcb->mss = tcpmtu(s->p, s->laddr, s->ipversion, &tcb->scale);
    tpriv = s->p->priv;
    tpriv->stats[Mss] = tcb->mss;
}
```

⟨*function* sndrst 396b⟩≡ (431b)

```
void
sndrst(Proto *tcp, uchar *source, uchar *dest, ushort length, Tcp *seg, uchar version, char *reason)
{
    Block *hbp;
    uchar rflags;
    Tcppriv *tpriv;
    Tcp4hdr ph4;
    Tcp6hdr ph6;

    netlog(tcp->f, Logtcp, "sndrst: %s\n", reason);

    tpriv = tcp->priv;

    if(seg->flags & RST)
        return;

    /* make pseudo header */
    switch(version) {
    case V4:
        memset(&ph4, 0, sizeof(ph4));
        ph4.vihl = IP_VER4;
        v6tov4(ph4.tcpsrc, dest);
        v6tov4(ph4.tcpdst, source);
        ph4.proto = IP_TCPPROTO;
        hnputs(ph4.tcplen, TCP4_HDRSIZE);
        hnputs(ph4.tcpsport, seg->dest);
        hnputs(ph4.tcpdport, seg->source);
        break;
    case V6:
        memset(&ph6, 0, sizeof(ph6));
        ph6.vcf[0] = IP_VER6;
        ipmove(ph6.tcpsrc, dest);
```

```
        ipmove(ph6.tcpdst, source);
        ph6.proto = IP_TCPPROTO;
        hnputs(ph6.ploadlen, TCP6_HDRSIZE);
        hnputs(ph6.tcpsport, seg->dest);
        hnputs(ph6.tcpdport, seg->source);
        break;
    default:
        panic("sndrst: version %d", version);
    }

    tpriv->stats[OutRsts]++;
    rflags = RST;

    /* convince the other end that this reset is in band */
    if(seg->flags & ACK) {
        seg->seq = seg->ack;
        seg->ack = 0;
    }
    else {
        rflags |= ACK;
        seg->ack = seg->seq;
        seg->seq = 0;
        if(seg->flags & SYN)
            seg->ack++;
        seg->ack += length;
        if(seg->flags & FIN)
            seg->ack++;
    }
    seg->flags = rflags;
    seg->wnd = 0;
    seg->urg = 0;
    seg->mss = 0;
    seg->ws = 0;
    switch(version) {
    case V4:
        hbp = htontcp4(seg, nil, &ph4, nil);
        if(hbp == nil)
            return;
        ipoput4(tcp->f, hbp, 0, MAXTTL, DFLTTOS, nil);
        break;
    case V6:
        hbp = htontcp6(seg, nil, &ph6, nil);
        if(hbp == nil)
            return;
        ipoput6(tcp->f, hbp, 0, MAXTTL, DFLTTOS, nil);
        break;
    default:
        panic("sndrst2: version %d", version);
    }
}
```

⟨*function* tcphangup 397⟩≡ (431b)

```
/*
 *  send a reset to the remote side and close the conversation
 *  called with s qlocked
 */
static char*
tcphangup(Conv *s)
{
    Tcp seg;
```

```
    Tcpctl *tcb;
    Block *hbp;

    tcb = (Tcpctl*)s->ptcl;
    if(waserror())
        return up->errstr;
    if(ipcmp(s->raddr, IPnoaddr) != 0) {
        if(!waserror(){
            memset(&seg, 0, sizeof seg);
            seg.flags = RST | ACK;
            seg.ack = tcb->rcv.nxt;
            tcb->rcv.ackptr = seg.ack;
            seg.seq = tcb->snd.ptr;
            seg.wnd = 0;
            seg.urg = 0;
            seg.mss = 0;
            seg.ws = 0;
            switch(s->ipversion) {
            case V4:
                tcb->protohdr.tcp4hdr.vihl = IP_VER4;
                hbp = htontcp4(&seg, nil, &tcb->protohdr.tcp4hdr, tcb);
                ipoput4(s->p->f, hbp, 0, s->ttl, s->tos, s);
                break;
            case V6:
                tcb->protohdr.tcp6hdr.vcf[0] = IP_VER6;
                hbp = htontcp6(&seg, nil, &tcb->protohdr.tcp6hdr, tcb);
                ipoput6(s->p->f, hbp, 0, s->ttl, s->tos, s);
                break;
            default:
                panic("tcphangup: version %d", s->ipversion);
            }
            poperror();
        }
    }
    localclose(s, nil);
    poperror();
    return nil;
}
```

⟨*function* sndsynack 398⟩≡ (431b)

```
/*
 *  (re)send a SYN ACK
 */
static int
sndsynack(Proto *tcp, Limbo *lp)
{
    Block *hbp;
    Tcp4hdr ph4;
    Tcp6hdr ph6;
    Tcp seg;
    uint scale;

    /* make pseudo header */
    switch(lp->version) {
    case V4:
        memset(&ph4, 0, sizeof(ph4));
        ph4.vihl = IP_VER4;
        v6tov4(ph4.tcpsrc, lp->laddr);
        v6tov4(ph4.tcpdst, lp->raddr);
        ph4.proto = IP_TCPPROTO;
```

```
        hnputs(ph4.tcplen, TCP4_HDRSIZE);
        hnputs(ph4.tcpsport, lp->lport);
        hnputs(ph4.tcpdport, lp->rport);
        break;
    case V6:
        memset(&ph6, 0, sizeof(ph6));
        ph6.vcf[0] = IP_VER6;
        ipmove(ph6.tcpsrc, lp->laddr);
        ipmove(ph6.tcpdst, lp->raddr);
        ph6.proto = IP_TCPPROTO;
        hnputs(ph6.ploadlen, TCP6_HDRSIZE);
        hnputs(ph6.tcpsport, lp->lport);
        hnputs(ph6.tcpdport, lp->rport);
        break;
    default:
        panic("sndrst: version %d", lp->version);
    }

    memset(&seg, 0, sizeof seg);
    seg.seq = lp->iss;
    seg.ack = lp->irs+1;
    seg.flags = SYN|ACK;
    seg.urg = 0;
    seg.mss = tcpmtu(tcp, lp->laddr, lp->version, &scale);
    seg.wnd = QMAX;

    /* if the other side set scale, we should too */
    if(lp->rcvscale){
        seg.ws = scale;
        lp->sndscale = scale;
    } else {
        seg.ws = 0;
        lp->sndscale = 0;
    }

    switch(lp->version) {
    case V4:
        hbp = htontcp4(&seg, nil, &ph4, nil);
        if(hbp == nil)
            return -1;
        ipoput4(tcp->f, hbp, 0, MAXTTL, DFLTTOS, nil);
        break;
    case V6:
        hbp = htontcp6(&seg, nil, &ph6, nil);
        if(hbp == nil)
            return -1;
        ipoput6(tcp->f, hbp, 0, MAXTTL, DFLTTOS, nil);
        break;
    default:
        panic("sndsnack: version %d", lp->version);
    }
    lp->lastsend = NOW;
    return 0;
}
```

⟨*macro* hashipa(kernel/network/ip/tcp.c)⟩ 399a⟩≡ (431b)

```
#define hashipa(a, p) ( ( (a)[IPaddrlen-2] + (a)[IPaddrlen-1] + p )&LHTMASK )
```

⟨*function* limbo 399b⟩≡ (431b)

```
/*
```

```
 *  put a call into limbo and respond with a SYN ACK
 *
 *  called with proto locked
 */
static void
limbo(Conv *s, uchar *source, uchar *dest, Tcp *seg, int version)
{
    Limbo *lp, **l;
    Tcppriv *tpriv;
    int h;

    tpriv = s->p->priv;
    h = hashipa(source, seg->source);

    for(l = &tpriv->lht[h]; *l != nil; l = &lp->next){
        lp = *l;
        if(lp->lport != seg->dest || lp->rport != seg->source || lp->version != version)
            continue;
        if(ipcmp(lp->raddr, source) != 0)
            continue;
        if(ipcmp(lp->laddr, dest) != 0)
            continue;

        /* each new SYN restarts the retransmits */
        lp->irs = seg->seq;
        break;
    }
    lp = *l;
    if(lp == nil){
        if(tpriv->nlimbo >= Maxlimbo && tpriv->lht[h]){
            lp = tpriv->lht[h];
            tpriv->lht[h] = lp->next;
            lp->next = nil;
        } else {
            lp = malloc(sizeof(*lp));
            if(lp == nil)
                return;
            tpriv->nlimbo++;
        }
        *l = lp;
        lp->version = version;
        ipmove(lp->laddr, dest);
        ipmove(lp->raddr, source);
        lp->lport = seg->dest;
        lp->rport = seg->source;
        lp->mss = seg->mss;
        lp->rcvscale = seg->ws;
        lp->irs = seg->seq;
        lp->iss = (nrand(1<<16)<<16)|nrand(1<<16);
    }

    if(sndsynack(s->p, lp) < 0){
        *l = lp->next;
        tpriv->nlimbo--;
        free(lp);
    }
}
```

⟨*function* limborexmit 400⟩≡ (431b)

```
/*
```

```
 *  resend SYN ACK's once every SYNACK_RXTIMER ms.
 */
static void
limborexmit(Proto *tcp)
{
    Tcppriv *tpriv;
    Limbo **l, *lp;
    int h;
    int seen;
    ulong now;

    tpriv = tcp->priv;

    if(!canqlock(tcp))
        return;
    seen = 0;
    now = NOW;
    for(h = 0; h < NLHT && seen < tpriv->nlimbo; h++){
        for(l = &tpriv->lht[h]; *l != nil && seen < tpriv->nlimbo; ){
            lp = *l;
            seen++;
            if(now - lp->lastsend < (lp->rexmits+1)*SYNACK_RXTIMER)
                continue;

            /* time it out after 1 second */
            if(++(lp->rexmits) > 5){
                tpriv->nlimbo--;
                *l = lp->next;
                free(lp);
                continue;
            }

            /* if we're being attacked, don't bother resending SYN ACK's */
            if(tpriv->nlimbo > 100)
                continue;

            if(sndsynack(tcp, lp) < 0){
                tpriv->nlimbo--;
                *l = lp->next;
                free(lp);
                continue;
            }

            l = &lp->next;
        }
    }
    qunlock(tcp);
}
```

⟨*function* limborst 401⟩≡ (431b)

```
/*
 *  lookup call in limbo.  if found, throw it out.
 *
 *  called with proto locked
 */
static void
limborst(Conv *s, Tcp *segp, uchar *src, uchar *dst, uchar version)
{
    Limbo *lp, **l;
    int h;
```

```
    Tcppriv *tpriv;

    tpriv = s->p->priv;

    /* find a call in limbo */
    h = hashipa(src, segp->source);
    for(l = &tpriv->lht[h]; *l != nil; l = &lp->next){
        lp = *l;
        if(lp->lport != segp->dest || lp->rport != segp->source || lp->version != version)
            continue;
        if(ipcmp(lp->laddr, dst) != 0)
            continue;
        if(ipcmp(lp->raddr, src) != 0)
            continue;

        /* RST can only follow the SYN */
        if(segp->seq == lp->irs+1){
            tpriv->nlimbo--;
            *l = lp->next;
            free(lp);
        }
        break;
    }
}
```

⟨*function* initialwindow 402a⟩≡ (431b)

```
static void
initialwindow(Tcpctl *tcb)
{
    /* RFC 3390 initial window */
    if(tcb->mss < 1095)
        tcb->cwind = 4*tcb->mss;
    else if(tcb->mss < 2190)
        tcb->cwind = 2*2190;
    else
        tcb->cwind = 2*tcb->mss;
}
```

⟨*function* tcpincoming 402b⟩≡ (431b)

```
/*
 *  come here when we finally get an ACK to our SYN-ACK.
 *  lookup call in limbo.  if found, create a new conversation
 *
 *  called with proto locked
 */
static Conv*
tcpincoming(Conv *s, Tcp *segp, uchar *src, uchar *dst, uchar version)
{
    Conv *new;
    Tcpctl *tcb;
    Tcppriv *tpriv;
    Tcp4hdr *h4;
    Tcp6hdr *h6;
    Limbo *lp, **l;
    int h;

    /* unless it's just an ack, it can't be someone coming out of limbo */
    if((segp->flags & SYN) || (segp->flags & ACK) == 0)
        return nil;
```

```
tpriv = s->p->priv;

/* find a call in limbo */
h = hashipa(src, segp->source);
for(l = &tpriv->lht[h]; (lp = *l) != nil; l = &lp->next){
    netlog(s->p->f, Logtcp, "tcpincoming s %I!%ud/%I!%ud d %I!%ud/%I!%ud v %d/%d\n",
        src, segp->source, lp->raddr, lp->rport,
        dst, segp->dest, lp->laddr, lp->lport,
        version, lp->version
    );

    if(lp->lport != segp->dest || lp->rport != segp->source || lp->version != version)
        continue;
    if(ipcmp(lp->laddr, dst) != 0)
        continue;
    if(ipcmp(lp->raddr, src) != 0)
        continue;

    /* we're assuming no data with the initial SYN */
    if(segp->seq != lp->irs+1 || segp->ack != lp->iss+1){
        netlog(s->p->f, Logtcp, "tcpincoming s %lux/%lux a %lux %lux\n",
            segp->seq, lp->irs+1, segp->ack, lp->iss+1);
        lp = nil;
    } else {
        tpriv->nlimbo--;
        *l = lp->next;
    }
    break;
}
if(lp == nil)
    return nil;

new = Fsnewcall(s, src, segp->source, dst, segp->dest, version);
if(new == nil)
    return nil;

memmove(new->ptcl, s->ptcl, sizeof(Tcpctl));
tcb = (Tcpctl*)new->ptcl;
tcb->flags &= ~CLONE;
tcb->timer.arg = new;
tcb->timer.state = TcptimerOFF;
tcb->acktimer.arg = new;
tcb->acktimer.state = TcptimerOFF;
tcb->katimer.arg = new;
tcb->katimer.state = TcptimerOFF;
tcb->rtt_timer.arg = new;
tcb->rtt_timer.state = TcptimerOFF;

tcb->irs = lp->irs;
tcb->rcv.nxt = tcb->irs+1;
tcb->rcv.wptr = tcb->rcv.nxt;
tcb->rcv.wsnt = 0;
tcb->rcv.urg = tcb->rcv.nxt;

tcb->iss = lp->iss;
tcb->rttseq = tcb->iss;
tcb->snd.wl2 = tcb->iss;
tcb->snd.una = tcb->iss+1;
tcb->snd.ptr = tcb->iss+1;
tcb->snd.nxt = tcb->iss+1;
```

```
    tcb->snd.rxt = tcb->iss+1;
    tcb->flgcnt = 0;
    tcb->flags |= SYNACK;

    /* our sending max segment size cannot be bigger than what he asked for */
    if(lp->mss != 0 && lp->mss < tcb->mss) {
        tcb->mss = lp->mss;
        tpriv->stats[Mss] = tcb->mss;
    }

    /* window scaling */
    tcpsetscale(new, tcb, lp->rcvscale, lp->sndscale);

    /* congestion window */
    tcb->snd.wnd = segp->wnd;
    initialwindow(tcb);

    /* set initial round trip time */
    tcb->sndsyntime = lp->lastsend+lp->rexmits*SYNACK_RXTIMER;
    tcpsynackrtt(new);

    free(lp);

    /* set up proto header */
    switch(version){
    case V4:
        h4 = &tcb->protohdr.tcp4hdr;
        memset(h4, 0, sizeof(*h4));
        h4->proto = IP_TCPPROTO;
        hnputs(h4->tcpsport, new->lport);
        hnputs(h4->tcpdport, new->rport);
        v6tov4(h4->tcpsrc, dst);
        v6tov4(h4->tcpdst, src);
        break;
    case V6:
        h6 = &tcb->protohdr.tcp6hdr;
        memset(h6, 0, sizeof(*h6));
        h6->proto = IP_TCPPROTO;
        hnputs(h6->tcpsport, new->lport);
        hnputs(h6->tcpdport, new->rport);
        ipmove(h6->tcpsrc, dst);
        ipmove(h6->tcpdst, src);
        break;
    default:
        panic("tcpincoming: version %d", new->ipversion);
    }

    tcpsetstate(new, Established);

    iphtadd(&tpriv->ht, new);

    return new;
}
```

⟨*function* seq_within 404⟩≡ (431b)

```
static int
seq_within(ulong x, ulong low, ulong high)
{
    if(low <= high){
        if(low <= x && x <= high)
```

```
            return 1;
    }
    else {
        if(x >= low || x <= high)
            return 1;
    }
    return 0;
}
```

⟨*function* seq_lt 405a⟩≡ (431b)

```
static int
seq_lt(ulong x, ulong y)
{
    return (int)(x-y) < 0;
}
```

⟨*function* seq_le 405b⟩≡ (431b)

```
static int
seq_le(ulong x, ulong y)
{
    return (int)(x-y) <= 0;
}
```

⟨*function* seq_gt 405c⟩≡ (431b)

```
static int
seq_gt(ulong x, ulong y)
{
    return (int)(x-y) > 0;
}
```

⟨*function* seq_ge 405d⟩≡ (431b)

```
static int
seq_ge(ulong x, ulong y)
{
    return (int)(x-y) >= 0;
}
```

⟨*function* tcpsynackrtt 405e⟩≡ (431b)

```
/*
 *  use the time between the first SYN and it's ack as the
 *  initial round trip time
 */
static void
tcpsynackrtt(Conv *s)
{
    Tcpctl *tcb;
    int delta;
    Tcppriv *tpriv;

    tcb = (Tcpctl*)s->ptcl;
    tpriv = s->p->priv;

    delta = NOW - tcb->sndsyntime;
    tcb->srtt = delta<<LOGAGAIN;
    tcb->mdev = delta<<LOGDGAIN;

    /* halt round trip timer */
    tcphalt(tpriv, &tcb->rtt_timer);
}
```

⟨*function* update 406⟩≡ (431b)

```
static void
update(Conv *s, Tcp *seg)
{
    int rtt, delta;
    Tcpctl *tcb;
    ulong acked;
    Tcppriv *tpriv;

    if(seg->update)
        return;
    seg->update = 1;

    tpriv = s->p->priv;
    tcb = (Tcpctl*)s->ptcl;

    /* catch zero-window updates, update window & recover */
    if(tcb->snd.wnd == 0 && seg->wnd > 0 &&
        seq_lt(seg->ack, tcb->snd.ptr)){
        netlog(s->p->f, Logtcp, "tcp: zwu ack %lud una %lud ptr %lud win %lud\n",
            seg->ack,  tcb->snd.una, tcb->snd.ptr, seg->wnd);
        tcb->snd.wnd = seg->wnd;
        goto recovery;
    }

    /* newreno fast retransmit */
    if(seg->ack == tcb->snd.una && tcb->snd.una != tcb->snd.nxt &&
        ++tcb->snd.dupacks == 3){       /* was TCPREXMTTHRESH */
recovery:
        if(tcb->snd.recovery){
            tpriv->stats[RecoveryCwind]++;
            tcb->cwind += tcb->mss;
        }else if(seq_le(tcb->snd.rxt, seg->ack)){
            tpriv->stats[Recovery]++;
            tcb->abcbytes = 0;
            tcb->snd.recovery = 1;
            tcb->snd.partialack = 0;
            tcb->snd.rxt = tcb->snd.nxt;
            tcpcongestion(tcb);
            tcb->cwind = tcb->ssthresh + 3*tcb->mss;
            netlog(s->p->f, Logtcpwin, "recovery inflate %ld ss %ld @%lud\n",
                tcb->cwind, tcb->ssthresh, tcb->snd.rxt);
            tcprxmit(s);
        }else{
            tpriv->stats[RecoveryNoSeq]++;
            netlog(s->p->f, Logtcpwin, "!recov %lud not <= %lud %ld\n",
                tcb->snd.rxt, seg->ack, tcb->snd.rxt - seg->ack);
            /* don't enter fast retransmit, don't change ssthresh */
        }
    }else if(tcb->snd.recovery){
        tpriv->stats[RecoveryCwind]++;
        tcb->cwind += tcb->mss;
    }

    /*
     *  update window
     */
    if(seq_gt(seg->ack, tcb->snd.wl2)
    || (tcb->snd.wl2 == seg->ack && seg->wnd > tcb->snd.wnd)){
        /* clear dupack if we advance wl2 */
```

```
    if(tcb->snd.wl2 != seg->ack)
        tcb->snd.dupacks = 0;
    tcb->snd.wnd = seg->wnd;
    tcb->snd.wl2 = seg->ack;
}

if(!seq_gt(seg->ack, tcb->snd.una)){
    /*
     *  don't let us hangup if sending into a closed window and
     *  we're still getting acks
     */
    if((tcb->flags&RETRAN) && tcb->snd.wnd == 0)
        tcb->backedoff = MAXBACKMS/4;
    return;
}

/* Compute the new send window size */
acked = seg->ack - tcb->snd.una;

/* avoid slow start and timers for SYN acks */
if((tcb->flags & SYNACK) == 0) {
    tcb->flags |= SYNACK;
    acked--;
    tcb->flgcnt--;
    goto done;
}

/*
 * congestion control
 */
if(tcb->snd.recovery){
    if(seq_ge(seg->ack, tcb->snd.rxt)){
        /* recovery finished; deflate window */
        tpriv->stats[RecoveryDone]++;
        tcb->snd.dupacks = 0;
        tcb->snd.recovery = 0;
        tcb->cwind = (tcb->snd.nxt - tcb->snd.una) + tcb->mss;
        if(tcb->ssthresh < tcb->cwind)
            tcb->cwind = tcb->ssthresh;
        netlog(s->p->f, Logtcpwin, "recovery deflate %ld %ld\n",
            tcb->cwind, tcb->ssthresh);
    } else {
        /* partial ack; we lost more than one segment */
        tpriv->stats[RecoveryPA]++;
        if(tcb->cwind > acked)
            tcb->cwind -= acked;
        else{
            netlog(s->p->f, Logtcpwin, "partial ack neg\n");
            tcb->cwind = tcb->mss;
        }
        netlog(s->p->f, Logtcpwin, "partial ack %ld left %ld cwind %ld\n",
            acked, tcb->snd.rxt - seg->ack, tcb->cwind);

        if(acked >= tcb->mss)
            tcb->cwind += tcb->mss;
        tcb->snd.partialack++;
    }
} else
    tcpabcincr(tcb, acked);
```

```
    /* Adjust the timers according to the round trip time */
    /* TODO: fix sloppy treatment of overflow cases here. */
    if(tcb->rtt_timer.state == TcptimerON && seq_ge(seg->ack, tcb->rttseq)) {
        tcphalt(tpriv, &tcb->rtt_timer);
        if((tcb->flags&RETRAN) == 0) {
            tcb->backoff = 0;
            tcb->backedoff = 0;
            rtt = tcb->rtt_timer.start - tcb->rtt_timer.count;
            if(rtt == 0)
                rtt = 1; /* else all close sys's will rexmit in 0 time */
            rtt *= MSPTICK;
            if(tcb->srtt == 0) {
                tcb->srtt = rtt << LOGAGAIN;
                tcb->mdev = rtt << LOGDGAIN;
            } else {
                delta = rtt - (tcb->srtt>>LOGAGAIN);
                tcb->srtt += delta;
                if(tcb->srtt <= 0)
                    tcb->srtt = 1;

                delta = abs(delta) - (tcb->mdev>>LOGDGAIN);
                tcb->mdev += delta;
                if(tcb->mdev <= 0)
                    tcb->mdev = 1;
            }
            tcpsettimer(tcb);
        }
    }

done:
    if(qdiscard(s->wq, acked) < acked)
        tcb->flgcnt--;
    tcb->snd.una = seg->ack;

    /* newreno fast recovery */
    if(tcb->snd.recovery)
        tcprxmit(s);

    if(seq_gt(seg->ack, tcb->snd.urg))
        tcb->snd.urg = seg->ack;

    if(tcb->snd.una != tcb->snd.nxt){
        /* 'impatient' variant */
        if(!tcb->snd.recovery || tcb->snd.partialack == 1){
            tcb->time = NOW;
            tcb->timeuna = tcb->snd.una;
            tcpgo(tpriv, &tcb->timer);
        }
    } else
        tcphalt(tpriv, &tcb->timer);

    if(seq_lt(tcb->snd.ptr, tcb->snd.una))
        tcb->snd.ptr = tcb->snd.una;

    if(!tcb->snd.recovery)
        tcb->flags &= ~RETRAN;
    tcb->backoff = 0;
    tcb->backedoff = 0;
}
```

⟨*function* tcpiput 409⟩≡ (431b)

```
static void
tcpiput(Proto *tcp, Ipifc*, Block *bp)
{
    Tcp seg;
    Tcp4hdr *h4;
    Tcp6hdr *h6;
    int hdrlen;
    Tcpctl *tcb;
    ushort length, csum;
    ipaddr source, dest;
    Conv *s;
    Fs *f;
    Tcppriv *tpriv;
    uchar version;

    f = tcp->f;
    tpriv = tcp->priv;

    tpriv->stats[InSegs]++;

    h4 = (Tcp4hdr*)(bp->rp);
    h6 = (Tcp6hdr*)(bp->rp);

    if((h4->vihl&0xF0)==IP_VER4) {
        version = V4;
        length = nhgets(h4->length);
        v4tov6(dest, h4->tcpdst);
        v4tov6(source, h4->tcpsrc);

        h4->Unused = 0;
        hnputs(h4->tcplen, length-TCP4_PKT);
        if(!(bp->flag & Btcpck) && (h4->tcpcksum[0] || h4->tcpcksum[1]) &&
            ptclcsum(bp, TCP4_IPLEN, length-TCP4_IPLEN)) {
            tpriv->stats[CsumErrs]++;
            tpriv->stats[InErrs]++;
            netlog(f, Logtcp, "bad tcp proto cksum\n");
            freeblist(bp);
            return;
        }

        hdrlen = ntohtcp4(&seg, &bp);
        if(hdrlen < 0){
            tpriv->stats[HlenErrs]++;
            tpriv->stats[InErrs]++;
            netlog(f, Logtcp, "bad tcp hdr len\n");
            return;
        }

        /* trim the packet to the size claimed by the datagram */
        length -= hdrlen+TCP4_PKT;
        bp = trimblock(bp, hdrlen+TCP4_PKT, length);
        if(bp == nil){
            tpriv->stats[LenErrs]++;
            tpriv->stats[InErrs]++;
            netlog(f, Logtcp, "tcp len < 0 after trim\n");
            return;
        }
    }
    else {
```

```
        int ttl = h6->ttl;
        int proto = h6->proto;

        version = V6;
        length = nhgets(h6->ploadlen);
        ipmove(dest, h6->tcpdst);
        ipmove(source, h6->tcpsrc);

        h6->ploadlen[0] = h6->ploadlen[1] = h6->proto = 0;
        h6->ttl = proto;
        hnputl(h6->vcf, length);
        if((h6->tcpcksum[0] || h6->tcpcksum[1]) &&
            (csum = ptclcsum(bp, TCP6_IPLEN, length+TCP6_PHDRSIZE)) != 0) {
            tpriv->stats[CsumErrs]++;
            tpriv->stats[InErrs]++;
            netlog(f, Logtcp,
                "bad tcpv6 proto cksum: got %#ux, computed %#ux\n",
                h6->tcpcksum[0]<<8 | h6->tcpcksum[1], csum);
            freeblist(bp);
            return;
        }
        h6->ttl = ttl;
        h6->proto = proto;
        hnputs(h6->ploadlen, length);

        hdrlen = ntohtcp6(&seg, &bp);
        if(hdrlen < 0){
            tpriv->stats[HlenErrs]++;
            tpriv->stats[InErrs]++;
            netlog(f, Logtcp, "bad tcpv6 hdr len\n");
            return;
        }

        /* trim the packet to the size claimed by the datagram */
        length -= hdrlen;
        bp = trimblock(bp, hdrlen+TCP6_PKT, length);
        if(bp == nil){
            tpriv->stats[LenErrs]++;
            tpriv->stats[InErrs]++;
            netlog(f, Logtcp, "tcpv6 len < 0 after trim\n");
            return;
        }
    }

    /* lock protocol while searching for a conversation */
    qlock(tcp);

    /* Look for a matching conversation */
    s = iphtlook(&tpriv->ht, source, seg.source, dest, seg.dest);
    if(s == nil){
        netlog(f, Logtcp, "iphtlook(src %I!%d, dst %I!%d) failed\n",
            source, seg.source, dest, seg.dest);
reset:
        qunlock(tcp);
        sndrst(tcp, source, dest, length, &seg, version, "no conversation");
        freeblist(bp);
        return;
    }

    /* if it's a listener, look for the right flags and get a new conv */
```

```
tcb = (Tcpctl*)s->ptcl;
if(tcb->state == Listen){
    if(seg.flags & RST){
        limborst(s, &seg, source, dest, version);
        qunlock(tcp);
        freeblist(bp);
        return;
    }

    /* if this is a new SYN, put the call into limbo */
    if((seg.flags & SYN) && (seg.flags & ACK) == 0){
        limbo(s, source, dest, &seg, version);
        qunlock(tcp);
        freeblist(bp);
        return;
    }

    /*
     *  if there's a matching call in limbo, tcpincoming will
     *  return it in state Syn_received
     */
    s = tcpincoming(s, &seg, source, dest, version);
    if(s == nil)
        goto reset;
}

/* The rest of the input state machine is run with the control block
 * locked and implements the state machine directly out of the RFC.
 * Out-of-band data is ignored - it was always a bad idea.
 */
tcb = (Tcpctl*)s->ptcl;
if(waserror()){
    qunlock(s);
    nexterror();
}
qlock(s);
qunlock(tcp);

/* fix up window */
seg.wnd <<= tcb->rcv.scale;

/* every input packet in puts off the keep alive time out */
tcpsetkacounter(tcb);

switch(tcb->state) {
case Closed:
    sndrst(tcp, source, dest, length, &seg, version, "sending to Closed");
    goto raise;
case Syn_sent:
    if(seg.flags & ACK) {
        if(!seq_within(seg.ack, tcb->iss+1, tcb->snd.nxt)) {
            sndrst(tcp, source, dest, length, &seg, version,
                 "bad seq in Syn_sent");
            goto raise;
        }
    }
    if(seg.flags & RST) {
        if(seg.flags & ACK)
            localclose(s, Econrefused);
        goto raise;
```

```
    }

    if(seg.flags & SYN) {
        procsyn(s, &seg);
        if(seg.flags & ACK){
            update(s, &seg);
            tcpsynackrtt(s);
            tcpsetstate(s, Established);
            tcpsetscale(s, tcb, seg.ws, tcb->scale);
        }
        else {
            tcb->time = NOW;
            tcpsetstate(s, Syn_received);   /* DLP - shouldn't this be a reset? */
        }

        if(length != 0 || (seg.flags & FIN))
            break;

        freeblist(bp);
        goto output;
    }
    else
        freeblist(bp);

    qunlock(s);
    poperror();
    return;
case Syn_received:
    /* doesn't matter if it's the correct ack, we're just trying to set timing */
    if(seg.flags & ACK)
        tcpsynackrtt(s);
    break;
}

/*
 *  One DOS attack is to open connections to us and then forget about them,
 *  thereby tying up a conv at no long term cost to the attacker.
 *  This is an attempt to defeat these stateless DOS attacks.  See
 *  corresponding code in tcpsendka().
 */
if(tcb->state != Syn_received && (seg.flags & RST) == 0){
    if(tcpporthogdefense
    && seq_within(seg.ack, tcb->snd.una-(1<<31), tcb->snd.una-(1<<29))){
        print("stateless hog %I.%d->%I.%d f %ux %lux - %lux - %lux\n",
            source, seg.source, dest, seg.dest, seg.flags,
            tcb->snd.una-(1<<31), seg.ack, tcb->snd.una-(1<<29));
        localclose(s, "stateless hog");
    }
}

/* Cut the data to fit the receive window */
tcprcvwin(s);
if(tcptrim(tcb, &seg, &bp, &length) == -1) {
    if(seg.seq+1 != tcb->rcv.nxt || length != 1)
    netlog(f, Logtcp, "tcp: trim: !inwind: seq %lud-%lud win "
        "%lud-%lud l %d from %I\n", seg.seq,
        seg.seq + length - 1, tcb->rcv.nxt,
        tcb->rcv.nxt + tcb->rcv.wnd-1, length, s->raddr);
    update(s, &seg);
    if(qlen(s->wq)+tcb->flgcnt == 0 && tcb->state == Closing) {
```

```
        tcphalt(tpriv, &tcb->rtt_timer);
        tcphalt(tpriv, &tcb->acktimer);
        tcphalt(tpriv, &tcb->katimer);
        tcpsetstate(s, Time_wait);
        tcb->timer.start = MSL2*(1000 / MSPTICK);
        tcpgo(tpriv, &tcb->timer);
    }
    if(!(seg.flags & RST)) {
        tcb->flags |= FORCE;
        goto output;
    }
    qunlock(s);
    poperror();
    return;
}

/* Cannot accept so answer with a rst */
if(length && tcb->state == Closed) {
    sndrst(tcp, source, dest, length, &seg, version, "sending to Closed");
    goto raise;
}

/* The segment is beyond the current receive pointer so
 * queue the data in the resequence queue
 */
if(seg.seq != tcb->rcv.nxt)
if(length != 0 || (seg.flags & (SYN|FIN))) {
    update(s, &seg);
    if(addreseq(f, tcb, tpriv, &seg, bp, length) < 0)
        print("reseq %I.%d -> %I.%d\n", s->raddr, s->rport,
            s->laddr, s->lport);
    tcb->flags |= FORCE;    /* force duplicate ack; RFC 5681 §3.2 */
    goto output;
}

if(tcb->nreseq > 0)
    tcb->flags |= FORCE; /* filled hole in seq. space; RFC 5681 §3.2 */

/*
 *  keep looping till we've processed this packet plus any
 *  adjacent packets in the resequence queue
 */
for(;;) {
    if(seg.flags & RST) {
        if(tcb->state == Established) {
            tpriv->stats[EstabResets]++;
            if(tcb->rcv.nxt != seg.seq)
                print("out of order RST rcvd: %I.%d -> "
                    "%I.%d, rcv.nxt %lux seq %lux\n",
                    s->raddr, s->rport, s->laddr,
                    s->lport, tcb->rcv.nxt, seg.seq);
        }
        localclose(s, Econrefused);
        goto raise;
    }

    if((seg.flags&ACK) == 0)
        goto raise;

    switch(tcb->state) {
```

```
case Syn_received:
    if(!seq_within(seg.ack, tcb->snd.una+1, tcb->snd.nxt)){
        sndrst(tcp, source, dest, length, &seg, version,
            "bad seq in Syn_received");
        goto raise;
    }
    update(s, &seg);
    tcpsetstate(s, Established);
case Established:
case Close_wait:
    update(s, &seg);
    break;
case Finwait1:
    update(s, &seg);
    if(qlen(s->wq)+tcb->flgcnt == 0){
        tcphalt(tpriv, &tcb->rtt_timer);
        tcphalt(tpriv, &tcb->acktimer);
        tcpsetkacounter(tcb);
        tcb->time = NOW;
        tcpsetstate(s, Finwait2);
        tcb->katimer.start = MSL2 * (1000 / MSPTICK);
        tcpgo(tpriv, &tcb->katimer);
    }
    break;
case Finwait2:
    update(s, &seg);
    break;
case Closing:
    update(s, &seg);
    if(qlen(s->wq)+tcb->flgcnt == 0) {
        tcphalt(tpriv, &tcb->rtt_timer);
        tcphalt(tpriv, &tcb->acktimer);
        tcphalt(tpriv, &tcb->katimer);
        tcpsetstate(s, Time_wait);
        tcb->timer.start = MSL2*(1000 / MSPTICK);
        tcpgo(tpriv, &tcb->timer);
    }
    break;
case Last_ack:
    update(s, &seg);
    if(qlen(s->wq)+tcb->flgcnt == 0) {
        localclose(s, nil);
        goto raise;
    }
case Time_wait:
    tcb->flags |= FORCE;
    if(tcb->timer.state != TcptimerON)
        tcpgo(tpriv, &tcb->timer);
}

if((seg.flags&URG) && seg.urg) {
    if(seq_gt(seg.urg + seg.seq, tcb->rcv.urg)) {
        tcb->rcv.urg = seg.urg + seg.seq;
        pullblock(&bp, seg.urg);
    }
}
else
if(seq_gt(tcb->rcv.nxt, tcb->rcv.urg))
    tcb->rcv.urg = tcb->rcv.nxt;
```

```
if(length == 0) {
    if(bp != nil)
        freeblist(bp);
}
else {
    switch(tcb->state){
    default:
        /* Ignore segment text */
        if(bp != nil)
            freeblist(bp);
        break;

    case Syn_received:
    case Established:
    case Finwait1:
        /* If we still have some data place on
         * receive queue
         */
        if(bp) {
            bp = packblock(bp);
            if(bp == nil)
                panic("tcp packblock");
            qpassnolim(s->rq, bp);
            bp = nil;
        }
        tcb->rcv.nxt += length;

        /*
         *  turn on the acktimer if there's something
         *  to ack
         */
        if(tcb->acktimer.state != TcptimerON)
            tcpgo(tpriv, &tcb->acktimer);

        break;
    case Finwait2:
        /* no process to read the data, send a reset */
        if(bp != nil)
            freeblist(bp);
        sndrst(tcp, source, dest, length, &seg, version,
            "send to Finwait2");
        qunlock(s);
        poperror();
        return;
    }
}

if(seg.flags & FIN) {
    tcb->flags |= FORCE;

    switch(tcb->state) {
    case Syn_received:
    case Established:
        tcb->rcv.nxt++;
        tcpsetstate(s, Close_wait);
        break;
    case Finwait1:
        tcb->rcv.nxt++;
        if(qlen(s->wq)+tcb->flgcnt == 0) {
            tcphalt(tpriv, &tcb->rtt_timer);
```

```
                    tcphalt(tpriv, &tcb->acktimer);
                    tcphalt(tpriv, &tcb->katimer);
                    tcpsetstate(s, Time_wait);
                    tcb->timer.start = MSL2*(1000/MSPTICK);
                    tcpgo(tpriv, &tcb->timer);
                }
                else
                    tcpsetstate(s, Closing);
                break;
            case Finwait2:
                tcb->rcv.nxt++;
                tcphalt(tpriv, &tcb->rtt_timer);
                tcphalt(tpriv, &tcb->acktimer);
                tcphalt(tpriv, &tcb->katimer);
                tcpsetstate(s, Time_wait);
                tcb->timer.start = MSL2 * (1000/MSPTICK);
                tcpgo(tpriv, &tcb->timer);
                break;
            case Close_wait:
            case Closing:
            case Last_ack:
                break;
            case Time_wait:
                tcpgo(tpriv, &tcb->timer);
                break;
            }
        }

        /*
         *  get next adjacent segment from the resequence queue.
         *  dump/trim any overlapping segments
         */
        for(;;) {
            if(tcb->reseq == nil)
                goto output;

            if(seq_ge(tcb->rcv.nxt, tcb->reseq->seg.seq) == 0)
                goto output;

            getreseq(tcb, &seg, &bp, &length);

            tcprcvwin(s);
            if(tcptrim(tcb, &seg, &bp, &length) == 0){
                tcb->flags |= FORCE;
                break;
            }
        }
    }
output:
    tcpoutput(s);
    qunlock(s);
    poperror();
    return;
raise:
    qunlock(s);
    poperror();
    freeblist(bp);
    tcpkick(s);
}
```

⟨*function* tcpoutput 417⟩≡ (431b)

```
/*
 *  always enters and exits with the s locked.  We drop
 *  the lock to ipoput the packet so some care has to be
 *  taken by callers.
 */
static void
tcpoutput(Conv *s)
{
    Tcp seg;
    uint msgs;
    Tcpctl *tcb;
    Block *hbp, *bp;
    int sndcnt;
    ulong ssize, dsize, sent;
    Fs *f;
    Tcppriv *tpriv;
    uchar version;

    f = s->p->f;
    tpriv = s->p->priv;
    version = s->ipversion;

    tcb = (Tcpctl*)s->ptcl;

    /* force ack every 2*mss */
    if((tcb->flags & FORCE) == 0 &&
        tcb->rcv.nxt - tcb->rcv.ackptr >= 2*tcb->mss){
        tpriv->stats[Delayack]++;
        tcb->flags |= FORCE;
    }

    /* force ack if window opening */
    if((tcb->flags & FORCE) == 0){
        tcprcvwin(s);
        if((int)(tcb->rcv.wptr - tcb->rcv.wsnt) >= 2*tcb->mss){
            tpriv->stats[Wopenack]++;
            tcb->flags |= FORCE;
        }
    }

    for(msgs = 0; msgs < 100; msgs++) {
        switch(tcb->state) {
        case Listen:
        case Closed:
        case Finwait2:
            return;
        }

        /* Don't send anything else until our SYN has been acked */
        if(tcb->snd.ptr != tcb->iss && (tcb->flags & SYNACK) == 0)
            break;

        /* force an ack when a window has opened up */
        tcprcvwin(s);
        if(tcb->rcv.blocked && tcb->rcv.wnd > 0){
            tcb->rcv.blocked = 0;
            tcb->flags |= FORCE;
        }
```

```
sndcnt = qlen(s->wq)+tcb->flgcnt;
sent = tcb->snd.ptr - tcb->snd.una;
ssize = sndcnt;
if(tcb->snd.wnd == 0){
    /* zero window probe */
    if(sent > 0 && !(tcb->flags & FORCE))
        break;  /* already probing, rto re-probes */
    if(ssize < sent)
        ssize = 0;
    else{
        ssize -= sent;
        if(ssize > 0)
            ssize = 1;
    }
} else {
    /* calculate usable segment size */
    if(ssize > tcb->cwind)
        ssize = tcb->cwind;
    if(ssize > tcb->snd.wnd)
        ssize = tcb->snd.wnd;

    if(ssize < sent)
        ssize = 0;
    else {
        ssize -= sent;
        if(ssize > tcb->mss)
            ssize = tcb->mss;
    }
}

dsize = ssize;
seg.urg = 0;

if(!(tcb->flags & FORCE))
    if(ssize == 0 ||
        ssize < tcb->mss && tcb->snd.nxt == tcb->snd.ptr &&
        sent > TCPREXMTTHRESH * tcb->mss)
        break;

tcb->flags &= ~FORCE;

/* By default we will generate an ack */
tcphalt(tpriv, &tcb->acktimer);
seg.source = s->lport;
seg.dest = s->rport;
seg.flags = ACK;
seg.mss = 0;
seg.ws = 0;
seg.update = 0;
switch(tcb->state){
case Syn_sent:
    seg.flags = 0;
    if(tcb->snd.ptr == tcb->iss){
        seg.flags |= SYN;
        dsize--;
        seg.mss = tcb->mss;
        seg.ws = tcb->scale;
    }
    break;
case Syn_received:
```

```
    /*
     *  don't send any data with a SYN/ACK packet
     *  because Linux rejects the packet in its
     *  attempt to solve the SYN attack problem
     */
    if(tcb->snd.ptr == tcb->iss){
        seg.flags |= SYN;
        dsize = 0;
        ssize = 1;
        seg.mss = tcb->mss;
        seg.ws = tcb->scale;
    }
    break;
}
seg.seq = tcb->snd.ptr;
seg.ack = tcb->rcv.nxt;
seg.wnd = tcb->rcv.wnd;

/* Pull out data to send */
bp = nil;
if(dsize != 0) {
    bp = qcopy(s->wq, dsize, sent);
    if(BLEN(bp) != dsize) {
        seg.flags |= FIN;
        dsize--;
    }
}

if(sent+dsize == sndcnt && dsize)
    seg.flags |= PSH;

tcb->snd.ptr += ssize;

/* Pull up the send pointer so we can accept acks
 * for this window
 */
if(seq_gt(tcb->snd.ptr,tcb->snd.nxt))
    tcb->snd.nxt = tcb->snd.ptr;

/* Build header, link data and compute cksum */
switch(version){
case V4:
    tcb->protohdr.tcp4hdr.vihl = IP_VER4;
    hbp = htontcp4(&seg, bp, &tcb->protohdr.tcp4hdr, tcb);
    if(hbp == nil) {
        freeblist(bp);
        return;
    }
    break;
case V6:
    tcb->protohdr.tcp6hdr.vcf[0] = IP_VER6;
    hbp = htontcp6(&seg, bp, &tcb->protohdr.tcp6hdr, tcb);
    if(hbp == nil) {
        freeblist(bp);
        return;
    }
    break;
default:
    hbp = nil;  /* to suppress a warning */
    panic("tcpoutput: version %d", version);
```

```
        }

        /* Start the transmission timers if there is new data and we
         * expect acknowledges
         */
        if(ssize != 0){
            if(tcb->timer.state != TcptimerON){
                tcb->time = NOW;
                tcb->timeuna = tcb->snd.una;
                tcpgo(tpriv, &tcb->timer);
            }

            /*  If round trip timer isn't running, start it.
             *  measure the longest packet only in case the
             *  transmission time dominates RTT
             */
            if(tcb->snd.retransmit == 0)
            if(tcb->rtt_timer.state != TcptimerON)
            if(ssize == tcb->mss) {
                tcpgo(tpriv, &tcb->rtt_timer);
                tcb->rttseq = tcb->snd.ptr;
            }
        }

        tpriv->stats[OutSegs]++;
        if(tcb->snd.retransmit)
            tpriv->stats[RetransSegsSent]++;
        tcb->rcv.ackptr = seg.ack;
        tcb->rcv.wsnt = tcb->rcv.wptr;

        /* put off the next keep alive */
        tcpgo(tpriv, &tcb->katimer);

        switch(version){
        case V4:
            if(ipoput4(f, hbp, 0, s->ttl, s->tos, s) < 0){
                /* a negative return means no route */
                localclose(s, "no route");
            }
            break;
        case V6:
            if(ipoput6(f, hbp, 0, s->ttl, s->tos, s) < 0){
                /* a negative return means no route */
                localclose(s, "no route");
            }
            break;
        default:
            panic("tcpoutput2: version %d", version);
        }
        if((msgs%4) == 3){
            qunlock(s);
            qlock(s);
        }
    }
}
```

⟨*function* tcpsendka 420⟩≡ (431b)

```
/*
 *  the BSD convention (hack?) for keep alives.  resend last uchar acked.
 */
```

```
static void
tcpsendka(Conv *s)
{
    Tcp seg;
    Tcpctl *tcb;
    Block *hbp,*dbp;

    tcb = (Tcpctl*)s->ptcl;

    dbp = nil;
    memset(&seg, 0, sizeof seg);
    seg.urg = 0;
    seg.source = s->lport;
    seg.dest = s->rport;
    seg.flags = ACK|PSH;
    seg.mss = 0;
    seg.ws = 0;
    if(tcpporthogdefense)
        seg.seq = tcb->snd.una-(1<<30)-nrand(1<<20);
    else
        seg.seq = tcb->snd.una-1;
    seg.ack = tcb->rcv.nxt;
    tcb->rcv.ackptr = seg.ack;
    tcprcvwin(s);
    seg.wnd = tcb->rcv.wnd;
    if(tcb->state == Finwait2){
        seg.flags |= FIN;
    } else {
        dbp = allocb(1);
        dbp->wp++;
    }

    if(isv4(s->raddr)) {
        /* Build header, link data and compute cksum */
        tcb->protohdr.tcp4hdr.vihl = IP_VER4;
        hbp = htontcp4(&seg, dbp, &tcb->protohdr.tcp4hdr, tcb);
        if(hbp == nil) {
            freeblist(dbp);
            return;
        }
        ipoput4(s->p->f, hbp, 0, s->ttl, s->tos, s);
    }
    else {
        /* Build header, link data and compute cksum */
        tcb->protohdr.tcp6hdr.vcf[0] = IP_VER6;
        hbp = htontcp6(&seg, dbp, &tcb->protohdr.tcp6hdr, tcb);
        if(hbp == nil) {
            freeblist(dbp);
            return;
        }
        ipoput6(s->p->f, hbp, 0, s->ttl, s->tos, s);
    }
}
```

⟨*function* tcpsetkacounter 421⟩≡ (431b)

```
/*
 *  set connection to time out after 12 minutes
 */
static void
tcpsetkacounter(Tcpctl *tcb)
```

```
{
    tcb->kacounter = (12 * 60 * 1000) / (tcb->katimer.start*MSPTICK);
    if(tcb->kacounter < 3)
        tcb->kacounter = 3;
}
```

⟨*function* tcpkeepalive 422a⟩≡ (431b)

```
/*
 *  if we've timed out, close the connection
 *  otherwise, send a keepalive and restart the timer
 */
static void
tcpkeepalive(void *v)
{
    Tcpctl *tcb;
    Conv *s;

    s = v;
    tcb = (Tcpctl*)s->ptcl;
    if(waserror()){
        qunlock(s);
        nexterror();
    }
    qlock(s);
    if(tcb->state != Closed){
        if(--(tcb->kacounter) <= 0) {
            localclose(s, Etimedout);
        } else {
            tcpsendka(s);
            tcpgo(s->p->priv, &tcb->katimer);
        }
    }
    qunlock(s);
    poperror();
}
```

⟨*function* tcpstartka 422b⟩≡ (431b)

```
/*
 *  start keepalive timer
 */
static char*
tcpstartka(Conv *s, char **f, int n)
{
    Tcpctl *tcb;
    int x;

    tcb = (Tcpctl*)s->ptcl;
    if(tcb->state != Established)
        return "connection must be in Establised state";
    if(n > 1){
        x = atoi(f[1]);
        if(x >= MSPTICK)
            tcb->katimer.start = x/MSPTICK;
    }
    tcpsetkacounter(tcb);
    tcpgo(s->p->priv, &tcb->katimer);

    return nil;
}
```

⟨*function* tcpsetchecksum 423a⟩≡ (431b)

```
/*
 *  turn checksums on/off
 */
static char*
tcpsetchecksum(Conv *s, char **f, int)
{
    Tcpctl *tcb;

    tcb = (Tcpctl*)s->ptcl;
    tcb->nochecksum = !atoi(f[1]);

    return nil;
}
```

⟨*function* tcprxmit 423b⟩≡ (431b)

```
/*
 *  retransmit (at most) one segment at snd.una.
 *  preserve cwind & snd.ptr
 */
static void
tcprxmit(Conv *s)
{
    Tcpctl *tcb;
    Tcppriv *tpriv;
    ulong tcwind, tptr;

    tcb = (Tcpctl*)s->ptcl;
    tcb->flags |= RETRAN|FORCE;

    tptr = tcb->snd.ptr;
    tcwind = tcb->cwind;
    tcb->snd.ptr = tcb->snd.una;
    tcb->cwind = tcb->mss;
    tcb->snd.retransmit = 1;
    tcpoutput(s);
    tcb->snd.retransmit = 0;
    tcb->cwind = tcwind;
    tcb->snd.ptr = tptr;

    tpriv = s->p->priv;
    tpriv->stats[RetransSegs]++;
}
```

⟨*function* tcptimeout 423c⟩≡ (431b)

```
/*
 *  TODO: RFC 4138 F-RTO
 */
static void
tcptimeout(void *arg)
{
    Conv *s;
    Tcpctl *tcb;
    int maxback;
    Tcppriv *tpriv;

    s = (Conv*)arg;
    tpriv = s->p->priv;
    tcb = (Tcpctl*)s->ptcl;
```

```
    if(waserror()){
        qunlock(s);
        nexterror();
    }
    qlock(s);
    switch(tcb->state){
    default:
        tcb->backoff++;
        if(tcb->state == Syn_sent)
            maxback = MAXBACKMS/2;
        else
            maxback = MAXBACKMS;
        tcb->backedoff += tcb->timer.start * MSPTICK;
        if(tcb->backedoff >= maxback) {
            localclose(s, Etimedout);
            break;
        }
        netlog(s->p->f, Logtcprxmt, "rxm %d/%d %ldms %lud rto %d %lud %s\n",
            tcb->srtt, tcb->mdev, NOW - tcb->time,
            tcb->snd.una - tcb->timeuna, tcb->snd.rto, tcb->snd.ptr,
            tcpstates[s->state]);
        tcpsettimer(tcb);
        if(tcb->snd.rto == 0)
            tcpcongestion(tcb);
        tcprxmit(s);
        tcb->snd.ptr = tcb->snd.una;
        tcb->cwind = tcb->mss;
        tcb->snd.rto = 1;
        tpriv->stats[RetransTimeouts]++;

        if(tcb->snd.recovery){
            tcb->snd.dupacks = 0;       /* reno rto */
            tcb->snd.recovery = 0;
            tpriv->stats[RecoveryRTO]++;
            tcb->snd.rxt = tcb->snd.nxt;
            netlog(s->p->f, Logtcpwin,
                "rto recovery rxt @%lud\n", tcb->snd.nxt);
        }

        tcb->abcbytes = 0;
        break;
    case Time_wait:
        localclose(s, nil);
        break;
    case Closed:
        break;
    }
    qunlock(s);
    poperror();
}
```

⟨*function* inwindow 424a⟩≡ (431b)

```
static int
inwindow(Tcpctl *tcb, int seq)
{
    return seq_within(seq, tcb->rcv.nxt, tcb->rcv.nxt+tcb->rcv.wnd-1);
}
```

⟨*function* procsyn 424b⟩≡ (431b)

```
/*
```

```
 *  set up state for a received SYN (or SYN ACK) packet
 */
static void
procsyn(Conv *s, Tcp *seg)
{
    Tcpctl *tcb;
    Tcppriv *tpriv;

    tcb = (Tcpctl*)s->ptcl;
    tcb->flags |= FORCE;

    tcb->rcv.nxt = seg->seq + 1;
    tcb->rcv.wptr = tcb->rcv.nxt;
    tcb->rcv.wsnt = 0;
    tcb->rcv.urg = tcb->rcv.nxt;
    tcb->irs = seg->seq;

    /* our sending max segment size cannot be bigger than what he asked for */
    if(seg->mss != 0 && seg->mss < tcb->mss) {
        tcb->mss = seg->mss;
        tpriv = s->p->priv;
        tpriv->stats[Mss] = tcb->mss;
    }

    tcb->snd.wnd = seg->wnd;
    initialwindow(tcb);
}
```

⟨*function* dumpreseq 425a⟩≡ (431b)

```
static int
dumpreseq(Tcpctl *tcb)
{
    Reseq *r, *next;

    for(r = tcb->reseq; r != nil; r = next){
        next = r->next;
        freeblist(r->bp);
        free(r);
    }
    tcb->reseq = nil;
    tcb->nreseq = 0;
    tcb->reseqlen = 0;
    return -1;
}
```

⟨*function* logreseq 425b⟩≡ (431b)

```
static void
logreseq(Fs *f, Reseq *r, ulong n)
{
    char *s;

    for(; r != nil; r = r->next){
        s = nil;
        if(r->next == nil && r->seg.seq != n)
            s = "hole/end";
        else if(r->next == nil)
            s = "end";
        else if(r->seg.seq != n)
            s = "hole";
        if(s != nil)
```

```
            netlog(f, Logtcp, "%s %lud-%lud (%ld) %#ux\n", s,
                n, r->seg.seq, r->seg.seq - n, r->seg.flags);
        n = r->seg.seq + r->seg.len;
    }
}
```

⟨*function* addreseq 426a⟩≡ (431b)

```
static int
addreseq(Fs *f, Tcpctl *tcb, Tcppriv *tpriv, Tcp *seg, Block *bp, ushort length)
{
    Reseq *rp, **rr;
    int qmax;

    rp = malloc(sizeof *rp);
    if(rp == nil){
        freeblist(bp);      /* bp always consumed by addreseq */
        return 0;
    }

    rp->seg = *seg;
    rp->bp = bp;
    rp->length = length;

    tcb->reseqlen += length;
    tcb->nreseq++;

    /* Place on reassembly list sorting by starting seq number */
    for(rr = &tcb->reseq; ; rr = &(*rr)->next)
        if(*rr == nil || seq_lt(seg->seq, (*rr)->seg.seq)){
            rp->next = *rr;
            *rr = rp;
            tpriv->stats[Resequenced]++;
            if(rp->next != nil)
                tpriv->stats[OutOfOrder]++;
            break;
        }

    qmax = tcb->window;
    if(tcb->reseqlen > qmax){
        netlog(f, Logtcp, "tcp: reseq: queue > window: %d > %d; %d packets\n",
            tcb->reseqlen, qmax, tcb->nreseq);
        logreseq(f, tcb->reseq, tcb->rcv.nxt);
        tpriv->stats[ReseqBytelim]++;
        return dumpreseq(tcb);
    }
    qmax = tcb->window / tcb->mss; /* ~190 for qscale=2, 390 for qscale=3 */
    if(tcb->nreseq > qmax){
        netlog(f, Logtcp, "resequence queue > packets: %d %d; %d bytes\n",
            tcb->nreseq, qmax, tcb->reseqlen);
        logreseq(f, tcb->reseq, tcb->rcv.nxt);
        tpriv->stats[ReseqPktlim]++;
        return dumpreseq(tcb);
    }
    return 0;
}
```

⟨*function* getreseq 426b⟩≡ (431b)

```
static void
getreseq(Tcpctl *tcb, Tcp *seg, Block **bp, ushort *length)
{
```

```
    Reseq *rp;

    rp = tcb->reseq;
    if(rp == nil)
        return;

    tcb->reseq = rp->next;

    *seg = rp->seg;
    *bp = rp->bp;
    *length = rp->length;

    tcb->nreseq--;
    tcb->reseqlen -= rp->length;

    free(rp);
}
```

⟨*function* tcptrim 427⟩≡ (431b)

```
static int
tcptrim(Tcpctl *tcb, Tcp *seg, Block **bp, ushort *length)
{
    ushort len;
    uchar accept;
    int dupcnt, excess;

    accept = 0;
    len = *length;
    if(seg->flags & SYN)
        len++;
    if(seg->flags & FIN)
        len++;

    if(tcb->rcv.wnd == 0) {
        if(len == 0 && seg->seq == tcb->rcv.nxt)
            return 0;
    }
    else {
        /* Some part of the segment should be in the window */
        if(inwindow(tcb,seg->seq))
            accept++;
        else
        if(len != 0) {
            if(inwindow(tcb, seg->seq+len-1) ||
            seq_within(tcb->rcv.nxt, seg->seq,seg->seq+len-1))
                accept++;
        }
    }
    if(!accept) {
        freeblist(*bp);
        return -1;
    }
    dupcnt = tcb->rcv.nxt - seg->seq;
    if(dupcnt > 0){
        tcb->rerecv += dupcnt;
        if(seg->flags & SYN){
            seg->flags &= ~SYN;
            seg->seq++;

            if(seg->urg > 1)
```

```
                seg->urg--;
            else
                seg->flags &= ~URG;
            dupcnt--;
        }
        if(dupcnt > 0){
            pullblock(bp, (ushort)dupcnt);
            seg->seq += dupcnt;
            *length -= dupcnt;

            if(seg->urg > dupcnt)
                seg->urg -= dupcnt;
            else {
                seg->flags &= ~URG;
                seg->urg = 0;
            }
        }
    }
    excess = seg->seq + *length - (tcb->rcv.nxt + tcb->rcv.wnd);
    if(excess > 0) {
        tcb->rerecv += excess;
        *length -= excess;
        *bp = trimblock(*bp, 0, *length);
        if(*bp == nil)
            panic("presotto is a boofhead");
        seg->flags &= ~FIN;
    }
    return 0;
}
```

⟨*function* tcpadvise 428⟩≡ (431b)

```
static void
tcpadvise(Proto *tcp, Block *bp, char *msg)
{
    Tcp4hdr *h4;
    Tcp6hdr *h6;
    Tcpctl *tcb;
    ipaddr source, dest;
    ushort psource, pdest;
    Conv *s, **p;

    h4 = (Tcp4hdr*)(bp->rp);
    h6 = (Tcp6hdr*)(bp->rp);

    if((h4->vihl&0xF0)==IP_VER4) {
        v4tov6(dest, h4->tcpdst);
        v4tov6(source, h4->tcpsrc);
        psource = nhgets(h4->tcpsport);
        pdest = nhgets(h4->tcpdport);
    }
    else {
        ipmove(dest, h6->tcpdst);
        ipmove(source, h6->tcpsrc);
        psource = nhgets(h6->tcpsport);
        pdest = nhgets(h6->tcpdport);
    }

    /* Look for a connection */
    qlock(tcp);
    for(p = tcp->conv; *p; p++) {
```

```
        s = *p;
        tcb = (Tcpctl*)s->ptcl;
        if(s->rport == pdest)
        if(s->lport == psource)
        if(tcb->state != Closed)
        if(ipcmp(s->raddr, dest) == 0)
        if(ipcmp(s->laddr, source) == 0){
            qlock(s);
            qunlock(tcp);
            switch(tcb->state){
            case Syn_sent:
                localclose(s, msg);
                break;
            }
            qunlock(s);
            freeblist(bp);
            return;
        }
    }
    qunlock(tcp);
    freeblist(bp);
}
```

⟨*function* tcpporthogdefensectl 429a⟩≡ (431b)

```
static char*
tcpporthogdefensectl(char *val)
{
    if(strcmp(val, "on") == 0)
        tcpporthogdefense = 1;
    else if(strcmp(val, "off") == 0)
        tcpporthogdefense = 0;
    else
        return "unknown value for tcpporthogdefense";
    return nil;
}
```

⟨*function* tcpctl 429b⟩≡ (431b)

```
/* called with c qlocked */
static char*
tcpctl(Conv* c, char** f, int n)
{
    if(n == 1 && strcmp(f[0], "hangup") == 0)
        return tcphangup(c);
    if(n >= 1 && strcmp(f[0], "keepalive") == 0)
        return tcpstartka(c, f, n);
    if(n >= 1 && strcmp(f[0], "checksum") == 0)
        return tcpsetchecksum(c, f, n);
    if(n >= 1 && strcmp(f[0], "tcpporthogdefense") == 0)
        return tcpporthogdefensectl(f[1]);
    return "unknown control request";
}
```

⟨*function* tcpstats 429c⟩≡ (431b)

```
static int
tcpstats(Proto *tcp, char *buf, int len)
{
    Tcppriv *priv;
    char *p, *e;
    int i;
```

```
    priv = tcp->priv;
    p = buf;
    e = p+len;
    for(i = 0; i < Nstats; i++)
        p = seprint(p, e, "%s: %llud\n", statnames[i], priv->stats[i]);
    return p - buf;
}
```

⟨*function* tcpgc 430a⟩≡ (431b)

```
/*
 *  garbage collect any stale conversations:
 *  - SYN received but no SYN-ACK after 5 seconds (could be the SYN attack)
 *  - Finwait2 after 5 minutes
 *
 *  this is called whenever we run out of channels.  Both checks are
 *  of questionable validity so we try to use them only when we're
 *  up against the wall.
 */
static int
tcpgc(Proto *tcp)
{
    Conv *c, **pp, **ep;
    int n;
    Tcpctl *tcb;


    n = 0;
    ep = &tcp->conv[tcp->nc];
    for(pp = tcp->conv; pp < ep; pp++) {
        c = *pp;
        if(c == nil)
            break;
        if(!canqlock(c))
            continue;
        tcb = (Tcpctl*)c->ptcl;
        switch(tcb->state){
        case Syn_received:
            if(NOW - tcb->time > 5000){
                localclose(c, Etimedout);
                n++;
            }
            break;
        case Finwait2:
            if(NOW - tcb->time > 5*60*1000){
                localclose(c, Etimedout);
                n++;
            }
            break;
        }
        qunlock(c);
    }
    return n;
}
```

⟨*function* tcpsettimer 430b⟩≡ (431b)

```
static void
tcpsettimer(Tcpctl *tcb)
{
    int x;
```

```
    /* round trip dependency */
    x = backoff(tcb->backoff) *
        (tcb->mdev + (tcb->srtt>>LOGAGAIN) + MSPTICK) / MSPTICK;

    /* bounded twixt 0.3 and 64 seconds */
    if(x < 300/MSPTICK)
        x = 300/MSPTICK;
    else if(x > (64000/MSPTICK))
        x = 64000/MSPTICK;
    tcb->timer.start = x;
}
```

⟨*function* tcpsetscale 431a⟩≡ (431b)

```
static void
tcpsetscale(Conv *s, Tcpctl *tcb, ushort rcvscale, ushort sndscale)
{
    /*
     * guess at reasonable queue sizes.  there's no current way
     * to know how many nic receive buffers we can safely tie up in the
     * tcp stack, and we don't adjust our queues to maximize throughput
     * and minimize bufferbloat.  n.b. the offer (rcvscale) needs to be
     * respected, but we still control our own buffer commitment by
     * keeping a seperate qscale.
     */
    tcb->rcv.scale = rcvscale & 0xff;
    tcb->snd.scale = sndscale & 0xff;
    tcb->qscale = rcvscale & 0xff;
    if(rcvscale > Maxqscale)
        tcb->qscale = Maxqscale;

    if(rcvscale != tcb->rcv.scale)
        netlog(s->p->f, Logtcp, "tcpsetscale: window %lud "
            "qlen %d >> window %ud lport %d\n",
            tcb->window, qlen(s->rq), QMAX<<tcb->qscale, s->lport);
    tcb->window = QMAX << tcb->qscale;
    tcb->ssthresh = tcb->window;

    /*
     * it's important to set wq large enough to cover the full
     * bandwidth-delay product.  it's possible to be in loss
     * recovery with a big window, and we need to keep sending
     * into the inflated window.  the difference can be huge
     * for even modest (70ms) ping times.
     */
    qsetlimit(s->rq, tcb->window);
    qsetlimit(s->wq, tcb->window);
    tcprcvwin(s);
}
```

⟨kernel/network/ip/tcp.c 431b⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    "ip.h"

typedef struct Tcptimer Tcptimer;
```

```
typedef struct Tcp4hdr Tcp4hdr;
typedef struct Tcp6hdr Tcp6hdr;
typedef struct Tcp Tcp;
typedef struct Reseq Reseq;
typedef struct Tcpctl Tcpctl;
typedef struct Limbo Limbo;
typedef struct Tcppriv Tcppriv;
```

⟨*enum* _anon_ (kernel/network/ip/tcp.c) 375⟩

⟨*global* tcpstates 376a⟩

⟨*struct* Tcptimer 376b⟩

```
/*
 *  v4 and v6 pseudo headers used for
 *  checksuming tcp
 */
```
⟨*struct* Tcp4hdr 376c⟩

⟨*struct* Tcp6hdr 377a⟩

```
/*
 *  this represents the control info
 *  for a single packet.  It is derived from
 *  a packet in ntohtcp{4,6}() and stuck into
 *  a packet in htontcp{4,6}().
 */
```
⟨*struct* Tcp 377b⟩

```
/*
 *  this header is malloc'd to thread together fragments
 *  waiting to be coalesced
 */
```
⟨*struct* Reseq 378a⟩

```
/*
 *  the qlock in the Conv locks this structure
 */
```
⟨*struct* Tcpctl 378b⟩

```
/*
 *  New calls are put in limbo rather than having a conversation structure
 *  allocated.  Thus, a SYN attack results in lots of limbo'd calls but not
 *  any real Conv structures mucking things up.  Calls in limbo rexmit their
 *  SYN ACK every SYNACK_RXTIMER ms up to 4 times, i.e., they disappear after 1 second.
 *
 *  In particular they aren't on a listener's queue so that they don't figure
 *  in the input queue limit.
 *
 *  If 1/2 of a T3 was attacking SYN packets, we'ld have a permanent queue
 *  of 70000 limbo'd calls.  Not great for a linear list but doable.  Therefore
 *  there is no hashing of this list.
 */
```
⟨*struct* Limbo 379a⟩

⟨*global* tcp_irtt 379b⟩

⟨*enum* _anon_ (kernel/network/ip/tcp.c)2 379c⟩

⟨*global* statnames*(*(kernel/network/ip/tcp.c)*)* 380a⟩

⟨*struct* Tcppriv 380b⟩

⟨*global* tcpporthogdefense 381a⟩

```
static  int addreseq(Fs*, Tcpctl*, Tcppriv*, Tcp*, Block*, ushort);
static  int dumpreseq(Tcpctl*);
static  void    getreseq(Tcpctl*, Tcp*, Block**, ushort*);
static  void    limbo(Conv*, uchar*, uchar*, Tcp*, int);
static  void    limborexmit(Proto*);
static  void    localclose(Conv*, char*);
static  void    procsyn(Conv*, Tcp*);
static  void    tcpacktimer(void*);
static  void    tcpiput(Proto*, Ipifc*, Block*);
static  void    tcpkeepalive(void*);
static  void    tcpoutput(Conv*);
static  void    tcprcvwin(Conv*);
static  void    tcprxmit(Conv*);
static  void    tcpsetkacounter(Tcpctl*);
static  void    tcpsetscale(Conv*, Tcpctl*, ushort, ushort);
static  void    tcpsettimer(Tcpctl*);
static  void    tcpsndsyn(Conv*, Tcpctl*);
static  void    tcpstart(Conv*, int);
static  void    tcpsynackrtt(Conv*);
static  void    tcptimeout(void*);
static  int tcptrim(Tcpctl*, Tcp*, Block**, ushort*);
```

⟨*function* tcpsetstate 381b⟩

⟨*function* tcpconnect 382a⟩

⟨*function* tcpstate 382b⟩

⟨*function* tcpinuse 382c⟩

⟨*function* tcpannounce 383a⟩

⟨*function* tcpclose 383b⟩

⟨*function* tcpkick 384a⟩

```
static int seq_lt(ulong, ulong);
```

⟨*function* tcprcvwin 384b⟩

⟨*function* tcpacktimer 385a⟩

⟨*function* tcpcongestion 385b⟩

⟨*enum* _anon_ (kernel/network/ip/tcp.c)3 385c⟩

⟨*function* tcpabcincr 385d⟩

⟨*function* tcpcreate 386a⟩

⟨*function* timerstate 386b⟩

⟨*function* tcpackproc 386c⟩

⟨*function* tcpgo 387a⟩

⟨*function* tcphalt 387b⟩

⟨*function* backoff 388a⟩

⟨*function* localclose 388b⟩

⟨*function* tcpmtu 388c⟩

⟨*function* inittcpctl 389⟩

⟨*function* tcpstart 390⟩

```
//static char*
//tcpflag(char *buf, char *e, ushort flag)
//{
//  char *p;
//
//  p = seprint(buf, e, "%d", flag>>10);    /* Head len */
//  if(flag & URG)
//      p = seprint(p, e, " URG");
//  if(flag & ACK)
//      p = seprint(p, e, " ACK");
//  if(flag & PSH)
//      p = seprint(p, e, " PSH");
//  if(flag & RST)
//      p = seprint(p, e, " RST");
//  if(flag & SYN)
//      p = seprint(p, e, " SYN");
//  if(flag & FIN)
//      p = seprint(p, e, " FIN");
//  USED(p);
//  return buf;
```

⟨*function* htontcp6 391⟩

⟨*function* htontcp4 392⟩

⟨*function* ntohtcp6 393⟩

⟨*function* ntohtcp4 394⟩

⟨*function* tcpsndsyn 396a⟩

⟨*function* sndrst 396b⟩

⟨*function* tcphangup 397⟩

⟨*function* sndsynack 398⟩

⟨*macro* hashipa*(*(kernel/network/ip/tcp.c)*)* 399a⟩

⟨*function* limbo 399b⟩

⟨*function* limborexmit 400⟩

⟨*function* limborst 401⟩

⟨*function* initialwindow 402a⟩

⟨*function* tcpincoming 402b⟩

⟨*function* seq_within 404⟩

⟨*function* seq_lt 405a⟩

⟨*function* seq_le 405b⟩

⟨*function* seq_gt 405c⟩

⟨*function* seq_ge 405d⟩

⟨*function* tcpsynackrtt 405e⟩

⟨*function* update 406⟩

⟨*function* tcpiput 409⟩

⟨*function* tcpoutput 417⟩

⟨*function* tcpsendka 420⟩

⟨*function* tcpsetkacounter 421⟩

⟨*function* tcpkeepalive 422a⟩

⟨*function* tcpstartka 422b⟩

⟨*function* tcpsetchecksum 423a⟩

⟨*function* tcprxmit 423b⟩

⟨*function* tcptimeout 423c⟩

⟨*function* inwindow 424a⟩

⟨*function* procsyn 424b⟩

⟨*function* dumpreseq 425a⟩

⟨*function* logreseq 425b⟩

⟨*function* addreseq 426a⟩

⟨*function* getreseq 426b⟩

⟨*function* tcptrim 427⟩

⟨*function* tcpadvise 428⟩

⟨*function* tcpporthogdefensectl 429a⟩

⟨*function* tcpctl 429b⟩

⟨*function* tcpstats 429c⟩

⟨*function* tcpgc 430a⟩

⟨*function* tcpsettimer 430b⟩

⟨*function* tcpinit 149⟩

⟨*function* tcpsetscale 431a⟩

I.9 kernel/network/ip/

kernel/network/ip/icmp6.c

⟨*enum* _anon_ (kernel/network/ip/icmp6.c) 436a⟩≡ (451)

```
enum
{
    InMsgs6,
    InErrors6,
    OutMsgs6,
    CsumErrs6,
    LenErrs6,
    HlenErrs6,
    HoplimErrs6,
    IcmpCodeErrs6,
    TargetErrs6,
    OptlenErrs6,
    AddrmxpErrs6,
    RouterAddrErrs6,

    Nstats6,
};
```

⟨*enum* _anon_ (kernel/network/ip/icmp6.c)2 436b⟩≡ (451)

```
enum {
    ICMP_USEAD6 = 40,
};
```

⟨*enum* _anon_ (kernel/network/ip/icmp6.c)3 436c⟩≡ (451)

```
enum {
    Oflag   = 1<<5,
    Sflag   = 1<<6,
    Rflag   = 1<<7,
};
```

⟨*enum* _anon_ (kernel/network/ip/icmp6.c)4 436d⟩≡ (451)

```
enum {
    /* ICMPv6 types */
    EchoReply   = 0,
    UnreachableV6   = 1,
    PacketTooBigV6  = 2,
    TimeExceedV6    = 3,
    SrcQuench   = 4,
    ParamProblemV6  = 4,
    Redirect    = 5,
    EchoRequest = 8,
    TimeExceed  = 11,
    InParmProblem   = 12,
    Timestamp   = 13,
    TimestampReply  = 14,
    InfoRequest = 15,
    InfoReply   = 16,
    AddrMaskRequest = 17,
    AddrMaskReply   = 18,
```

```
    EchoRequestV6   = 128,
    EchoReplyV6 = 129,
    RouterSolicit   = 133,
    RouterAdvert    = 134,
    NbrSolicit  = 135,
    NbrAdvert   = 136,
    RedirectV6  = 137,

    Maxtype6    = 137,
};
```

⟨*struct* IPICMP 437a⟩≡ (451)

```
struct IPICMP {
    ICMPHDR;
    uchar   payload[];
};
```

⟨*constant* IPICMPSZ 437b⟩≡ (451)

```
#define IPICMPSZ offsetof(IPICMP, payload[0])
```

⟨*struct* NdiscC 437c⟩≡ (451)

```
// used though NDISCSZ macro below
struct NdiscC {
    ICMPHDR;
    uchar   target[IPaddrlen];
    uchar   payload[];
};
```

⟨*constant* NDISCSZ 437d⟩≡ (451)

```
#define NDISCSZ offsetof(NdiscC, payload[0])
```

⟨*struct* Ndpkt 437e⟩≡ (451)

```
struct Ndpkt {
    ICMPHDR;
    uchar   target[IPaddrlen];
    uchar   otype;
    uchar   olen;       /* length in units of 8 octets(incl type, code),
                 * 1 for IEEE 802 addresses */
    uchar   lnaddr[6];  /* link-layer address */
    uchar   payload[];
};
```

⟨*constant* NDPKTSZ 437f⟩≡ (451)

```
#define NDPKTSZ offsetof(Ndpkt, payload[0])
```

⟨*struct* Icmppriv6 437g⟩≡ (451)

```
typedef struct Icmppriv6
{
    ulong   stats[Nstats6];

    /* message counts */
    ulong   in[Maxtype6+1];
    ulong   out[Maxtype6+1];
} Icmppriv6;
```

⟨*struct* Icmpcb6 437h⟩≡ (451)

```
typedef struct Icmpcb6
{
    QLock;
    uchar   headers;
} Icmpcb6;
```

⟨*global* icmpnames6 438a⟩≡ (451)

```
char *icmpnames6[Maxtype6+1] =
{
[EchoReply]     "EchoReply",
[UnreachableV6]     "UnreachableV6",
[PacketTooBigV6]    "PacketTooBigV6",
[TimeExceedV6]      "TimeExceedV6",
[SrcQuench]     "SrcQuench",
[Redirect]      "Redirect",
[EchoRequest]       "EchoRequest",
[TimeExceed]        "TimeExceed",
[InParmProblem]     "InParmProblem",
[Timestamp]     "Timestamp",
[TimestampReply]    "TimestampReply",
[InfoRequest]       "InfoRequest",
[InfoReply]     "InfoReply",
[AddrMaskRequest]   "AddrMaskRequest",
[AddrMaskReply]     "AddrMaskReply",
[EchoRequestV6]     "EchoRequestV6",
[EchoReplyV6]       "EchoReplyV6",
[RouterSolicit]     "RouterSolicit",
[RouterAdvert]      "RouterAdvert",
[NbrSolicit]        "NbrSolicit",
[NbrAdvert]     "NbrAdvert",
[RedirectV6]        "RedirectV6",
};
```

⟨*global* statnames6 438b⟩≡ (451)

```
static char *statnames6[Nstats6] =
{
[InMsgs6]   "InMsgs",
[InErrors6] "InErrors",
[OutMsgs6]  "OutMsgs",
[CsumErrs6] "CsumErrs",
[LenErrs6]  "LenErrs",
[HlenErrs6] "HlenErrs",
[HoplimErrs6]   "HoplimErrs",
[IcmpCodeErrs6] "IcmpCodeErrs",
[TargetErrs6]   "TargetErrs",
[OptlenErrs6]   "OptlenErrs",
[AddrmxpErrs6]  "AddrmxpErrs",
[RouterAddrErrs6]   "RouterAddrErrs",
};
```

⟨*global* unreachcode*(*(kernel/network/ip/icmp6.c)*)* 438c⟩≡ (451)

```
static char *unreachcode[] =
{
[Icmp6_no_route]    "no route to destination",
[Icmp6_ad_prohib]   "comm with destination administratively prohibited",
[Icmp6_out_src_scope]   "beyond scope of source address",
[Icmp6_adr_unreach] "address unreachable",
[Icmp6_port_unreach]    "port unreachable",
[Icmp6_gress_src_fail]  "source address failed ingress/egress policy",
[Icmp6_rej_route]   "reject route to destination",
[Icmp6_unknown]     "icmp unreachable: unknown code",
};
```

⟨*function* icmpcreate6 438d⟩≡ (451)

```
static void
icmpcreate6(Conv *c)
```

```
{
    c->rq = qopen(64*1024, Qmsg, 0, c);
    c->wq = qbypass(icmpkick6, c);
}
```

⟨*function* set_cksum 439a⟩≡ (451)

```
static void
set_cksum(Block *bp)
{
    IPICMP *p = (IPICMP *)(bp->rp);

    hnputl(p->vcf, 0);      /* borrow IP header as pseudoheader */
    hnputs(p->ploadlen, blocklen(bp) - IP6HDR);
    p->proto = 0;
    p->ttl = ICMPv6;    /* ttl gets set later */
    hnputs(p->cksum, 0);
    hnputs(p->cksum, ptclcsum(bp, 0, blocklen(bp)));
    p->proto = ICMPv6;
}
```

⟨*function* newIPICMP 439b⟩≡ (451)

```
static Block *
newIPICMP(int packetlen)
{
    Block *nbp;

    nbp = allocb(packetlen);
    nbp->wp += packetlen;
    memset(nbp->rp, 0, packetlen);
    return nbp;
}
```

⟨*function* icmpadvise6 439c⟩≡ (451)

```
void
icmpadvise6(Proto *icmp, Block *bp, char *msg)
{
    ushort recid;
    Conv **c, *s;
    IPICMP *p;

    p = (IPICMP *)bp->rp;
    recid = nhgets(p->icmpid);

    for(c = icmp->conv; *c; c++) {
        s = *c;
        if(s->lport == recid && ipcmp(s->raddr, p->dst) == 0){
            qhangup(s->rq, msg);
            qhangup(s->wq, msg);
            break;
        }
    }
    freeblist(bp);
}
```

⟨*function* icmpkick6 439d⟩≡ (451)

```
static void
icmpkick6(void *x, Block *bp)
{
    uchar laddr[IPaddrlen], raddr[IPaddrlen];
    Conv *c = x;
```

```
    IPICMP *p;
    Icmppriv6 *ipriv = c->p->priv;
    Icmpcb6 *icb = (Icmpcb6*)c->ptcl;

    if(bp == nil)
        return;

    if(icb->headers==6) {
        /* get user specified addresses */
        bp = pullupblock(bp, ICMP_USEAD6);
        if(bp == nil)
            return;
        bp->rp += 8;
        ipmove(laddr, bp->rp);
        bp->rp += IPaddrlen;
        ipmove(raddr, bp->rp);
        bp->rp += IPaddrlen;
        bp = padblock(bp, IP6HDR);
    }

    if(blocklen(bp) < IPICMPSZ){
        freeblist(bp);
        return;
    }
    p = (IPICMP *)(bp->rp);
    if(icb->headers == 6) {
        ipmove(p->dst, raddr);
        ipmove(p->src, laddr);
    } else {
        ipmove(p->dst, c->raddr);
        ipmove(p->src, c->laddr);
        hnputs(p->icmpid, c->lport);
    }

    set_cksum(bp);
    p->vcf[0] = 0x06 << 4;
    if(p->type <= Maxtype6)
        ipriv->out[p->type]++;
    ipoput6(c->p->f, bp, 0, c->ttl, c->tos, nil);
}
```

⟨*function* icmpctl6 440a⟩≡ (451)

```
char*
icmpctl6(Conv *c, char **argv, int argc)
{
    Icmpcb6 *icb;

    icb = (Icmpcb6*) c->ptcl;
    if(argc==1 && strcmp(argv[0], "headers")==0) {
        icb->headers = 6;
        return nil;
    }
    return "unknown control request";
}
```

⟨*function* goticmpkt6 440b⟩≡ (451)

```
static void
goticmpkt6(Proto *icmp, Block *bp, int muxkey)
{
    ushort recid;
```

```
    uchar *addr;
    Conv **c, *s;
    IPICMP *p = (IPICMP *)bp->rp;

    if(muxkey == 0) {
        recid = nhgets(p->icmpid);
        addr = p->src;
    } else {
        recid = muxkey;
        addr = p->dst;
    }

    for(c = icmp->conv; *c; c++){
        s = *c;
        if(s->lport == recid && ipcmp(s->raddr, addr) == 0){
            bp = concatblock(bp);
            if(bp != nil)
                qpass(s->rq, bp);
            return;
        }
    }

    freeblist(bp);
}
```

⟨*function* mkechoreply6 441a⟩≡ (451)

```
static Block *
mkechoreply6(Block *bp, Ipifc *ifc)
{
    uchar addr[IPaddrlen];
    IPICMP *p = (IPICMP *)(bp->rp);

    ipmove(addr, p->src);
    if(!isv6mcast(p->dst))
        ipmove(p->src, p->dst);
    else if (!ipv6anylocal(ifc, p->src))
        return nil;
    ipmove(p->dst, addr);
    p->type = EchoReplyV6;
    set_cksum(bp);
    return bp;
}
```

⟨*function* icmpns 441b⟩≡ (451)

```
/*
 * sends out an ICMPv6 neighbor solicitation
 *  suni == SRC_UNSPEC or SRC_UNI,
 *  tuni == TARG_MULTI => multicast for address resolution,
 *  and tuni == TARG_UNI => neighbor reachability.
 */
extern void
icmpns(Fs *f, uchar* src, int suni, uchar* targ, int tuni, uchar* mac)
{
    Block *nbp;
    Ndpkt *np;
    Proto *icmp = f->t2p[ICMPv6];
    Icmppriv6 *ipriv = icmp->priv;

    nbp = newIPICMP(NDPKTSZ);
    np = (Ndpkt*) nbp->rp;
```

```
    if(suni == SRC_UNSPEC)
        memmove(np->src, v6Unspecified, IPaddrlen);
    else
        memmove(np->src, src, IPaddrlen);

    if(tuni == TARG_UNI)
        memmove(np->dst, targ, IPaddrlen);
    else
        ipv62smcast(np->dst, targ);

    np->type = NbrSolicit;
    np->code = 0;
    memmove(np->target, targ, IPaddrlen);
    if(suni != SRC_UNSPEC) {
        np->otype = SRC_LLADDR;
        np->olen = 1;       /* 1+1+6 = 8 = 1 8-octet */
        memmove(np->lnaddr, mac, sizeof(np->lnaddr));
    } else
        nbp->wp -= NDPKTSZ - NDISCSZ;

    set_cksum(nbp);
    np = (Ndpkt*)nbp->rp;
    np->ttl = HOP_LIMIT;
    np->vcf[0] = 0x06 << 4;
    ipriv->out[NbrSolicit]++;
    netlog(f, Logicmp, "sending neighbor solicitation %I\n", targ);
    ipoput6(f, nbp, 0, MAXTTL, DFLTTOS, nil);
}
```

⟨*function* icmpna 442⟩≡ (451)

```
/*
 * sends out an ICMPv6 neighbor advertisement. pktflags == RSO flags.
 */
extern void
icmpna(Fs *f, uchar* src, uchar* dst, uchar* targ, uchar* mac, uchar flags)
{
    Block *nbp;
    Ndpkt *np;
    Proto *icmp = f->t2p[ICMPv6];
    Icmppriv6 *ipriv = icmp->priv;

    nbp = newIPICMP(NDPKTSZ);
    np = (Ndpkt*)nbp->rp;

    memmove(np->src, src, IPaddrlen);
    memmove(np->dst, dst, IPaddrlen);

    np->type = NbrAdvert;
    np->code = 0;
    np->icmpid[0] = flags;
    memmove(np->target, targ, IPaddrlen);

    np->otype = TARGET_LLADDR;
    np->olen = 1;
    memmove(np->lnaddr, mac, sizeof(np->lnaddr));

    set_cksum(nbp);
    np = (Ndpkt*) nbp->rp;
    np->ttl = HOP_LIMIT;
```

```
    np->vcf[0] = 0x06 << 4;
    ipriv->out[NbrAdvert]++;
    netlog(f, Logicmp, "sending neighbor advertisement %I\n", src);
    ipoput6(f, nbp, 0, MAXTTL, DFLTTOS, nil);
}
```

⟨*function* icmphostunr 443⟩≡ (451)

```
extern void
icmphostunr(Fs *f, Ipifc *ifc, Block *bp, int code, int free)
{
    int osz = BLEN(bp);
    int sz = MIN(IPICMPSZ + osz, v6MINTU);
    Block *nbp;
    IPICMP *np;
    Ip6hdr *p;
    Proto *icmp = f->t2p[ICMPv6];
    Icmppriv6 *ipriv = icmp->priv;

    p = (Ip6hdr *)bp->rp;

    if(isv6mcast(p->src))
        goto clean;

    nbp = newIPICMP(sz);
    np = (IPICMP *)nbp->rp;

    rlock(ifc);
    if(ipv6anylocal(ifc, np->src))
        netlog(f, Logicmp, "send icmphostunr -> src %I dst %I\n",
            p->src, p->dst);
    else {
        netlog(f, Logicmp, "icmphostunr fail -> src %I dst %I\n",
            p->src, p->dst);
        freeblist(nbp);
        if(free)
            goto clean;
        else
            return;
    }

    memmove(np->dst, p->src, IPaddrlen);
    np->type = UnreachableV6;
    np->code = code;
    memmove(nbp->rp + IPICMPSZ, bp->rp, sz - IPICMPSZ);
    set_cksum(nbp);
    np->ttl = HOP_LIMIT;
    np->vcf[0] = 0x06 << 4;
    ipriv->out[UnreachableV6]++;

    if(free)
        ipiput6(f, ifc, nbp);
    else {
        ipoput6(f, nbp, 0, MAXTTL, DFLTTOS, nil);
        return;
    }

clean:
    runlock(ifc);
    freeblist(bp);
}
```

⟨*function* icmpttlexceeded6 444a⟩≡ (451)

```
extern void
icmpttlexceeded6(Fs *f, Ipifc *ifc, Block *bp)
{
    int osz = BLEN(bp);
    int sz = MIN(IPICMPSZ + osz, v6MINTU);
    Block *nbp;
    IPICMP *np;
    Ip6hdr *p;
    Proto *icmp = f->t2p[ICMPv6];
    Icmppriv6 *ipriv = icmp->priv;

    p = (Ip6hdr *)bp->rp;

    if(isv6mcast(p->src))
        return;

    nbp = newIPICMP(sz);
    np = (IPICMP *) nbp->rp;

    if(ipv6anylocal(ifc, np->src))
        netlog(f, Logicmp, "send icmpttlexceeded6 -> src %I dst %I\n",
            p->src, p->dst);
    else {
        netlog(f, Logicmp, "icmpttlexceeded6 fail -> src %I dst %I\n",
            p->src, p->dst);
        return;
    }

    memmove(np->dst, p->src, IPaddrlen);
    np->type = TimeExceedV6;
    np->code = 0;
    memmove(nbp->rp + IPICMPSZ, bp->rp, sz - IPICMPSZ);
    set_cksum(nbp);
    np->ttl = HOP_LIMIT;
    np->vcf[0] = 0x06 << 4;
    ipriv->out[TimeExceedV6]++;
    ipoput6(f, nbp, 0, MAXTTL, DFLTTOS, nil);
}
```

⟨*function* icmppkttoobig6 444b⟩≡ (451)

```
extern void
icmppkttoobig6(Fs *f, Ipifc *ifc, Block *bp)
{
    int osz = BLEN(bp);
    int sz = MIN(IPICMPSZ + osz, v6MINTU);
    Block *nbp;
    IPICMP *np;
    Ip6hdr *p;
    Proto *icmp = f->t2p[ICMPv6];
    Icmppriv6 *ipriv = icmp->priv;

    p = (Ip6hdr *)bp->rp;

    if(isv6mcast(p->src))
        return;

    nbp = newIPICMP(sz);
    np = (IPICMP *)nbp->rp;
```

```
    if(ipv6anylocal(ifc, np->src))
        netlog(f, Logicmp, "send icmppkttoobig6 -> src %I dst %I\n",
            p->src, p->dst);
    else {
        netlog(f, Logicmp, "icmppkttoobig6 fail -> src %I dst %I\n",
            p->src, p->dst);
        return;
    }

    memmove(np->dst, p->src, IPaddrlen);
    np->type = PacketTooBigV6;
    np->code = 0;
    hnputl(np->icmpid, ifc->maxtu - ifc->m->hsize);
    memmove(nbp->rp + IPICMPSZ, bp->rp, sz - IPICMPSZ);
    set_cksum(nbp);
    np->ttl = HOP_LIMIT;
    np->vcf[0] = 0x06 << 4;
    ipriv->out[PacketTooBigV6]++;
    ipoput6(f, nbp, 0, MAXTTL, DFLTTOS, nil);
}
```

⟨*function* valid 445⟩≡ (451)

```
/*
 * RFC 2461, pages 39-40, pages 57-58.
 */
static int
valid(Proto *icmp, Ipifc *ifc, Block *bp, Icmppriv6 *ipriv)
{
    int sz, osz, unsp, n, ttl, iplen;
    int pktsz = BLEN(bp);
    uchar *packet = bp->rp;
    IPICMP *p = (IPICMP *) packet;
    Ndpkt *np;

    USED(ifc);
    n = blocklen(bp);
    if(n < IPICMPSZ) {
        ipriv->stats[HlenErrs6]++;
        netlog(icmp->f, Logicmp, "icmp hlen %d\n", n);
        goto err;
    }

    iplen = nhgets(p->ploadlen);
    if(iplen > n - IP6HDR) {
        ipriv->stats[LenErrs6]++;
        netlog(icmp->f, Logicmp, "icmp length %d\n", iplen);
        goto err;
    }

    /* Rather than construct explicit pseudoheader, overwrite IPv6 header */
    if(p->proto != ICMPv6) {
        /* This code assumes no extension headers!!! */
        netlog(icmp->f, Logicmp, "icmp error: extension header\n");
        goto err;
    }
    memset(packet, 0, 4);
    ttl = p->ttl;
    p->ttl = p->proto;
    p->proto = 0;
    if(ptclcsum(bp, 0, iplen + IP6HDR)) {
```

```
    ipriv->stats[CsumErrs6]++;
    netlog(icmp->f, Logicmp, "icmp checksum error\n");
    goto err;
}
p->proto = p->ttl;
p->ttl = ttl;

/* additional tests for some pkt types */
if (p->type == NbrSolicit   || p->type == NbrAdvert ||
    p->type == RouterAdvert || p->type == RouterSolicit ||
    p->type == RedirectV6) {
    if(p->ttl != HOP_LIMIT) {
        ipriv->stats[HoplimErrs6]++;
        goto err;
    }
    if(p->code != 0) {
        ipriv->stats[IcmpCodeErrs6]++;
        goto err;
    }

    switch (p->type) {
    case NbrSolicit:
    case NbrAdvert:
        np = (Ndpkt*) p;
        if(isv6mcast(np->target)) {
            ipriv->stats[TargetErrs6]++;
            goto err;
        }
        if(optexsts(np) && np->olen == 0) {
            ipriv->stats[OptlenErrs6]++;
            goto err;
        }

        if (p->type == NbrSolicit &&
            ipcmp(np->src, v6Unspecified) == 0)
            if(!issmcast(np->dst) || optexsts(np)) {
                ipriv->stats[AddrmxpErrs6]++;
                goto err;
            }

        if(p->type == NbrAdvert)
            if(isv6mcast(np->dst) &&
                (nhgets(np->icmpid) & Sflag)){
                ipriv->stats[AddrmxpErrs6]++;
                goto err;
            }
        break;

    case RouterAdvert:
        if(pktsz - IP6HDR < 16) {
            ipriv->stats[HlenErrs6]++;
            goto err;
        }
        if(!islinklocal(p->src)) {
            ipriv->stats[RouterAddrErrs6]++;
            goto err;
        }
        sz = IPICMPSZ + 8;
        while (sz+1 < pktsz) {
            osz = packet[sz+1];
```

```
                if(osz <= 0) {
                    ipriv->stats[OptlenErrs6]++;
                    goto err;
                }
                sz += 8*osz;
            }
            break;

        case RouterSolicit:
            if(pktsz - IP6HDR < 8) {
                ipriv->stats[HlenErrs6]++;
                goto err;
            }
            unsp = (ipcmp(p->src, v6Unspecified) == 0);
            sz = IPICMPSZ + 8;
            while (sz+1 < pktsz) {
                osz = packet[sz+1];
                if(osz <= 0 ||
                    (unsp && packet[sz] == SRC_LLADDR)) {
                    ipriv->stats[OptlenErrs6]++;
                    goto err;
                }
                sz += 8*osz;
            }
            break;

        case RedirectV6:
            /* to be filled in */
            break;

        default:
            goto err;
        }
    }
    return 1;
err:
    ipriv->stats[InErrors6]++;
    return 0;
}
```

⟨*function* targettype 447⟩≡ (451)

```
static int
targettype(Fs *f, Ipifc *ifc, uchar *target)
{
    Iplifc *lifc;
    int t;

    rlock(ifc);
    if(ipproxyifc(f, ifc, target)) {
        runlock(ifc);
        return Tuniproxy;
    }

    for(lifc = ifc->lifc; lifc; lifc = lifc->next)
        if(ipcmp(lifc->local, target) == 0) {
            t = (lifc->tentative)? Tunitent: Tunirany;
            runlock(ifc);
            return t;
        }
```

```
    runlock(ifc);
    return 0;
}
```

⟨*function* icmpiput6 448⟩≡ (451)

```
static void
icmpiput6(Proto *icmp, Ipifc *ipifc, Block *bp)
{
    int refresh = 1;
    char *msg, m2[128];
    uchar pktflags;
    uchar *packet = bp->rp;
    uchar lsrc[IPaddrlen];
    Block *r;
    IPICMP *p = (IPICMP *)packet;
    Icmppriv6 *ipriv = icmp->priv;
    Iplifc *lifc;
    Ndpkt* np;
    Proto *pr;

    if(!valid(icmp, ipifc, bp, ipriv) || p->type > Maxtype6)
        goto raise;

    ipriv->in[p->type]++;

    switch(p->type) {
    case EchoRequestV6:
        r = mkechoreply6(bp, ipifc);
        if(r == nil)
            goto raise;
        ipriv->out[EchoReply]++;
        ipoput6(icmp->f, r, 0, MAXTTL, DFLTTOS, nil);
        break;

    case UnreachableV6:
        if(p->code >= nelem(unreachcode))
            msg = unreachcode[Icmp6_unknown];
        else
            msg = unreachcode[p->code];

        bp->rp += IPICMPSZ;
        if(blocklen(bp) < 8){
            ipriv->stats[LenErrs6]++;
            goto raise;
        }
        p = (IPICMP *)bp->rp;
        pr = Fsrcvpcolx(icmp->f, p->proto);
        if(pr != nil && pr->advise != nil) {
            (*pr->advise)(pr, bp, msg);
            return;
        }

        bp->rp -= IPICMPSZ;
        goticmpkt6(icmp, bp, 0);
        break;

    case TimeExceedV6:
        if(p->code == 0){
            snprint(m2, sizeof m2, "ttl exceeded at %I", p->src);
```

```
        bp->rp += IPICMPSZ;
        if(blocklen(bp) < 8){
            ipriv->stats[LenErrs6]++;
            goto raise;
        }
        p = (IPICMP *)bp->rp;
        pr = Fsrcvpcolx(icmp->f, p->proto);
        if(pr && pr->advise) {
            (*pr->advise)(pr, bp, m2);
            return;
        }
        bp->rp -= IPICMPSZ;
    }

    goticmpkt6(icmp, bp, 0);
    break;

case RouterAdvert:
case RouterSolicit:
    /* using lsrc as a temp, munge hdr for goticmp6 */
    if (0) {
        memmove(lsrc, p->src, IPaddrlen);
        memmove(p->src, p->dst, IPaddrlen);
        memmove(p->dst, lsrc, IPaddrlen);
    }
    goticmpkt6(icmp, bp, p->type);
    break;

case NbrSolicit:
    np = (Ndpkt*) p;
    pktflags = 0;
    switch (targettype(icmp->f, ipifc, np->target)) {
    case Tunirany:
        pktflags |= Oflag;
        /* fall through */

    case Tuniproxy:
        if(ipcmp(np->src, v6Unspecified) != 0) {
            arpenter(icmp->f, V6, np->src, np->lnaddr,
                8*np->olen-2, 0);
            pktflags |= Sflag;
        }
        if(ipv6local(ipifc, lsrc))
            icmpna(icmp->f, lsrc,
                (ipcmp(np->src, v6Unspecified) == 0?
                    v6allnodesL: np->src),
                np->target, ipifc->mac, pktflags);
        else
            freeblist(bp);
        break;

    case Tunitent:
        /* not clear what needs to be done. send up
         * an icmp mesg saying don't use this address? */
    default:
        freeblist(bp);
    }
    break;

case NbrAdvert:
```

```
        np = (Ndpkt*) p;

        /*
         * if the target address matches one of the local interface
         * addresses and the local interface address has tentative bit
         * set, insert into ARP table. this is so the duplicate address
         * detection part of ipconfig can discover duplication through
         * the arp table.
         */
        lifc = iplocalonifc(ipifc, np->target);
        if(lifc && lifc->tentative)
            refresh = 0;
        arpenter(icmp->f, V6, np->target, np->lnaddr, 8*np->olen-2,
            refresh);
        freeblist(bp);
        break;

    case PacketTooBigV6:
    default:
        goticmpkt6(icmp, bp, 0);
        break;
    }
    return;
raise:
    freeblist(bp);
}
```

⟨*function* icmpstats6 450a⟩≡ (451)

```
int
icmpstats6(Proto *icmp6, char *buf, int len)
{
    Icmppriv6 *priv;
    char *p, *e;
    int i;

    priv = icmp6->priv;
    p = buf;
    e = p+len;
    for(i = 0; i < Nstats6; i++)
        p = seprint(p, e, "%s: %lud\n", statnames6[i], priv->stats[i]);
    for(i = 0; i <= Maxtype6; i++)
        if(icmpnames6[i])
            p = seprint(p, e, "%s: %lud %lud\n", icmpnames6[i],
                priv->in[i], priv->out[i]);
/*      else
            p = seprint(p, e, "%d: %lud %lud\n", i, priv->in[i],
                priv->out[i]);
 */
    return p - buf;
}
```

⟨*function* icmp6init 450b⟩≡ (451)

```
void
icmp6init(Fs *fs)
{
    Proto *icmp6 = smalloc(sizeof(Proto));

    icmp6->priv = smalloc(sizeof(Icmppriv6));
    icmp6->name = "icmpv6";
    icmp6->connect = icmpconnect;
```

```
    icmp6->announce = icmpannounce;
    icmp6->state = icmpstate;
    icmp6->create = icmpcreate6;
    icmp6->close = icmpclose;
    icmp6->rcv = icmpiput6;
    icmp6->stats = icmpstats6;
    icmp6->ctl = icmpctl6;
    icmp6->advise = icmpadvise6;
    icmp6->gc = nil;
    icmp6->ipproto = ICMPv6;
    icmp6->nc = 16;
    icmp6->ptclsize = sizeof(Icmpcb6);

    Fsproto(fs, icmp6);
}
```

⟨kernel/network/ip/icmp6.c 451⟩≡

```
/*
 * Internet Control Message Protocol for IPv6
 */
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "../port/error.h"
#include "ip.h"
#include "ipv6.h"

⟨enum _anon_ (kernel/network/ip/icmp6.c) 436a⟩

⟨enum _anon_ (kernel/network/ip/icmp6.c)2 436b⟩

⟨enum _anon_ (kernel/network/ip/icmp6.c)3 436c⟩

⟨enum _anon_ (kernel/network/ip/icmp6.c)4 436d⟩

/* on-the-wire packet formats */
typedef struct IPICMP IPICMP;
typedef struct Ndpkt Ndpkt;
typedef struct NdiscC NdiscC;

/* we do this to avoid possible struct padding  */
#define ICMPHDR \
    IPV6HDR; \
    uchar   type; \
    uchar   code; \
    uchar   cksum[2]; \
    uchar   icmpid[2]; \
    uchar   seq[2]

⟨struct IPICMP 437a⟩

⟨constant IPICMPSZ 437b⟩

⟨struct NdiscC 437c⟩

⟨constant NDISCSZ 437d⟩

⟨struct Ndpkt 437e⟩
```

⟨*constant* NDPKTSZ 437f⟩

⟨*struct* Icmppriv6 437g⟩

⟨*struct* Icmpcb6 437h⟩

⟨*global* icmpnames6 438a⟩

⟨*global* statnames6 438b⟩

⟨*global* unreachcode*(*(kernel/network/ip/icmp6.c)*)* 438c⟩

```
static void icmpkick6(void *x, Block *bp);
```

⟨*function* icmpcreate6 438d⟩

⟨*function* set_cksum 439a⟩

⟨*function* newIPICMP 439b⟩

⟨*function* icmpadvise6 439c⟩

⟨*function* icmpkick6 439d⟩

⟨*function* icmpctl6 440a⟩

⟨*function* goticmpkt6 440b⟩

⟨*function* mkechoreply6 441a⟩

⟨*function* icmpns 441b⟩

⟨*function* icmpna 442⟩

⟨*function* icmphostunr 443⟩

⟨*function* icmpttlexceeded6 444a⟩

⟨*function* icmppkttoobig6 444b⟩

⟨*function* valid 445⟩

⟨*function* targettype 447⟩

⟨*function* icmpiput6 448⟩

⟨*function* icmpstats6 450a⟩

```
/* import from icmp.c */
extern int  icmpstate(Conv *c, char *state, int n);
extern char*    icmpannounce(Conv *c, char **argv, int argc);
extern char*    icmpconnect(Conv *c, char **argv, int argc);
extern void icmpclose(Conv *c);
```

⟨*function* icmp6init 450b⟩

kernel/network/ip/ipv6.c

⟨*enum* _anon_ (kernel/network/ip/ipv6.c) 453a⟩≡ (463)

```
enum
{
    IP6FHDR     = 8,        /* sizeof(Fraghdr6) */
};
```

⟨*macro* IPV6CLASS 453b⟩≡ (463)

```
#define IPV6CLASS(hdr)  (((hdr)->vcf[0]&0x0F)<<2 | ((hdr)->vcf[1]&0xF0)>>2)
```

⟨*macro* BLKIPVER(kernel/network/ip/ipv6.c) 453c⟩≡ (463)

```
#define BLKIPVER(xp)    (((Ip6hdr*)((xp)->rp))->vcf[0] & 0xF0)
```

⟨*macro* BKFG(kernel/network/ip/ipv6.c) 453d⟩≡ (463)

```
/*
 * This sleazy macro is stolen shamelessly from ip.c, see comment there.
 */
#define BKFG(xp)    ((Ipfrag*)((xp)->base))
```

⟨*function* ipoput6 453e⟩≡ (463)

```
int
ipoput6(Fs *f, Block *bp, int gating, int ttl, int tos, Conv *c)
{
    int medialen, len, chunk, uflen, flen, seglen, lid, offset, fragoff;
    int morefrags, blklen, rv = 0, tentative;
    uchar *gate, nexthdr;
    Block *xp, *nb;
    Fraghdr6 fraghdr;
    IP *ip;
    Ip6hdr *eh;
    Ipifc *ifc;
    Route *r, *sr;

    ip = f->ip;

    /* Fill out the ip header */
    eh = (Ip6hdr*)(bp->rp);

    ip->stats[OutRequests]++;

    /* Number of uchars in data and ip header to write */
    len = blocklen(bp);

    tentative = iptentative(f, eh->src);
    if(tentative){
        netlog(f, Logip, "reject tx of packet with tentative src address %I\n",
            eh->src);
        goto free;
    }

    if(gating){
        chunk = nhgets(eh->ploadlen);
        if(chunk > len){
            ip->stats[OutDiscards]++;
            netlog(f, Logip, "short gated packet\n");
            goto free;
        }
        if(chunk + IP6HDR < len)
            len = chunk + IP6HDR;
    }
```

```
    if(len >= IP_MAX){
        ip->stats[OutDiscards]++;
        netlog(f, Logip, "exceeded ip max size %I\n", eh->dst);
        goto free;
    }

    r = v6lookup(f, eh->dst, c);
    if(r == nil){
//      print("no route for %I, src %I free\n", eh->dst, eh->src);
        ip->stats[OutNoRoutes]++;
        netlog(f, Logip, "no interface %I\n", eh->dst);
        rv = -1;
        goto free;
    }

    ifc = r->ifc;
    if(r->type & (Rifc|Runi))
        gate = eh->dst;
    else if(r->type & (Rbcast|Rmulti)) {
        gate = eh->dst;
        sr = v6lookup(f, eh->src, nil);
        if(sr && (sr->type & Runi))
            ifc = sr->ifc;
    }
    else
        gate = r->v6.gate;

    if(!gating)
        eh->vcf[0] = IP_VER6;
    eh->ttl = ttl;
    if(!gating) {
        eh->vcf[0] |= tos >> 4;
        eh->vcf[1]  = tos << 4;
    }

    if(!canrlock(ifc))
        goto free;

    if(waserror()){
        runlock(ifc);
        nexterror();
    }

    if(ifc->m == nil)
        goto raise;

    /* If we dont need to fragment just send it */
    medialen = ifc->maxtu - ifc->m->hsize;
    if(len <= medialen) {
        hnputs(eh->ploadlen, len - IP6HDR);
        ifc->m->bwrite(ifc, bp, V6, gate);
        runlock(ifc);
        poperror();
        return 0;
    }

    if(gating && ifc->reassemble <= 0) {
        /*
         * v6 intermediate nodes are not supposed to fragment pkts;
```

```
     * we fragment if ifc->reassemble is turned on; an exception
     * needed for nat.
     */
    ip->stats[OutDiscards]++;
    icmppkttoobig6(f, ifc, bp);
    netlog(f, Logip, "%I: gated pkts not fragmented\n", eh->dst);
    goto raise;
}

/* start v6 fragmentation */
uflen = unfraglen(bp, &nexthdr, 1);
if(uflen > medialen) {
    ip->stats[FragFails]++;
    ip->stats[OutDiscards]++;
    netlog(f, Logip, "%I: unfragmentable part too big\n", eh->dst);
    goto raise;
}

flen = len - uflen;
seglen = (medialen - (uflen + IP6FHDR)) & ~7;
if(seglen < 8) {
    ip->stats[FragFails]++;
    ip->stats[OutDiscards]++;
    netlog(f, Logip, "%I: seglen < 8\n", eh->dst);
    goto raise;
}

lid = incref(&ip->id6);
fraghdr.nexthdr = nexthdr;
fraghdr.res = 0;
hnputl(fraghdr.id, lid);

xp = bp;
offset = uflen;
while (xp && offset && offset >= BLEN(xp)) {
    offset -= BLEN(xp);
    xp = xp->next;
}
xp->rp += offset;

fragoff = 0;
morefrags = 1;

for(; fragoff < flen; fragoff += seglen) {
    nb = allocb(uflen + IP6FHDR + seglen);

    if(fragoff + seglen >= flen) {
        seglen = flen - fragoff;
        morefrags = 0;
    }

    hnputs(eh->ploadlen, seglen+IP6FHDR);
    memmove(nb->wp, eh, uflen);
    nb->wp += uflen;

    hnputs(fraghdr.offsetRM, fragoff); /* last 3 bits must be 0 */
    fraghdr.offsetRM[1] |= morefrags;
    memmove(nb->wp, &fraghdr, IP6FHDR);
    nb->wp += IP6FHDR;
```

```
        /* Copy data */
        chunk = seglen;
        while (chunk) {
            if(!xp) {
                ip->stats[OutDiscards]++;
                ip->stats[FragFails]++;
                freeblist(nb);
                netlog(f, Logip, "!xp: chunk in v6%d\n", chunk);
                goto raise;
            }
            blklen = chunk;
            if(BLEN(xp) < chunk)
                blklen = BLEN(xp);
            memmove(nb->wp, xp->rp, blklen);

            nb->wp += blklen;
            xp->rp += blklen;
            chunk -= blklen;
            if(xp->rp == xp->wp)
                xp = xp->next;
        }

        ifc->m->bwrite(ifc, nb, V6, gate);
        ip->stats[FragCreates]++;
    }
    ip->stats[FragOKs]++;

raise:
    runlock(ifc);
    poperror();
free:
    freeblist(bp);
    return rv;
}
```

⟨*function* ipiput6 456⟩≡ (463)

```
void
ipiput6(Fs *f, Ipifc *ifc, Block *bp)
{
    int hl, hop, tos, notforme, tentative;
    uchar proto;
    uchar v6dst[IPaddrlen];
    IP *ip;
    Ip6hdr *h;
    Proto *p;
    Route *r, *sr;

    ip = f->ip;
    ip->stats[InReceives]++;

    /*
     *  Ensure we have all the header info in the first
     *  block.  Make life easier for other protocols by
     *  collecting up to the first 64 bytes in the first block.
     */
    if(BLEN(bp) < 64) {
        hl = blocklen(bp);
        if(hl < IP6HDR)
            hl = IP6HDR;
        if(hl > 64)
```

```
        hl = 64;
    bp = pullupblock(bp, hl);
    if(bp == nil)
        return;
}

h = (Ip6hdr *)bp->rp;

memmove(&v6dst[0], &h->dst[0], IPaddrlen);
notforme = ipforme(f, v6dst) == 0;
tentative = iptentative(f, v6dst);

if(tentative && h->proto != ICMPv6) {
    print("ipv6 tentative addr %I, drop\n", v6dst);
    freeblist(bp);
    return;
}

/* Check header version */
if(BLKIPVER(bp) != IP_VER6) {
    ip->stats[InHdrErrors]++;
    netlog(f, Logip, "ip: bad version %ux\n", (h->vcf[0]&0xF0)>>2);
    freeblist(bp);
    return;
}

/* route */
if(notforme) {
    if(!ip->iprouting){
        freeblist(bp);
        return;
    }

    /* don't forward to link-local destinations */
    if(islinklocal(h->dst) ||
       (isv6mcast(h->dst) && (h->dst[1]&0xF) <= Link_local_scop)){
        ip->stats[OutDiscards]++;
        freeblist(bp);
        return;
    }

    /* don't forward to source's network */
    sr = v6lookup(f, h->src, nil);
    r  = v6lookup(f, h->dst, nil);

    if(r == nil || sr == r){
        ip->stats[OutDiscards]++;
        freeblist(bp);
        return;
    }

    /* don't forward if packet has timed out */
    hop = h->ttl;
    if(hop < 1) {
        ip->stats[InHdrErrors]++;
        icmpttlexceeded6(f, ifc, bp);
        freeblist(bp);
        return;
    }
```

```
        /* process headers & reassemble if the interface expects it */
        bp = procxtns(ip, bp, r->ifc->reassemble);
        if(bp == nil)
            return;

        ip->stats[ForwDatagrams]++;
        h = (Ip6hdr *)bp->rp;
        tos = IPV6CLASS(h);
        hop = h->ttl;
        ipoput6(f, bp, 1, hop-1, tos, nil);
        return;
    }

    /* reassemble & process headers if needed */
    bp = procxtns(ip, bp, 1);
    if(bp == nil)
        return;

    h = (Ip6hdr *) (bp->rp);
    proto = h->proto;
    p = Fsrcvpcol(f, proto);
    if(p && p->rcv) {
        ip->stats[InDelivers]++;
        (*p->rcv)(p, ifc, bp);
        return;
    }

    ip->stats[InDiscards]++;
    ip->stats[InUnknownProtos]++;
    freeblist(bp);
}
```

⟨*function* ipfragfree6 458⟩≡ (463)

```
/*
 * ipfragfree6 - copied from ipfragfree4 - assume hold fraglock6
 */
void
ipfragfree6(IP *ip, Fragment6 *frag)
{
    Fragment6 *fl, **l;

    if(frag->blist)
        freeblist(frag->blist);

    memset(frag->src, 0, IPaddrlen);
    frag->id = 0;
    frag->blist = nil;

    l = &ip->flisthead6;
    for(fl = *l; fl; fl = fl->next) {
        if(fl == frag) {
            *l = frag->next;
            break;
        }
        l = &fl->next;
    }

    frag->next = ip->fragfree6;
    ip->fragfree6 = frag;
}
```

⟨*function* ipfragallo6 459a⟩≡ (463)

```
/*
 * ipfragallo6 - copied from ipfragalloc4
 */
Fragment6*
ipfragallo6(IP *ip)
{
    Fragment6 *f;

    while(ip->fragfree6 == nil) {
        /* free last entry on fraglist */
        for(f = ip->flisthead6; f->next; f = f->next)
            ;
        ipfragfree6(ip, f);
    }
    f = ip->fragfree6;
    ip->fragfree6 = f->next;
    f->next = ip->flisthead6;
    ip->flisthead6 = f;
    f->age = NOW + 30000;

    return f;
}
```

⟨*function* procxtns 459b⟩≡ (463)

```
static Block*
procxtns(IP *ip, Block *bp, int doreasm)
{
    int offset;
    uchar proto;
    Ip6hdr *h;

    h = (Ip6hdr *)bp->rp;
    offset = unfraglen(bp, &proto, 0);

    if(proto == FH && doreasm != 0) {
        bp = ip6reassemble(ip, offset, bp, h);
        if(bp == nil)
            return nil;
        offset = unfraglen(bp, &proto, 0);
    }

    if(proto == DOH || offset > IP6HDR)
        bp = procopts(bp);
    return bp;
}
```

⟨*function* unfraglen 459c⟩≡ (463)

```
/*
 * returns length of "Unfragmentable part", i.e., sum of lengths of ipv6 hdr,
 * hop-by-hop & routing headers if present; *nexthdr is set to nexthdr value
 * of the last header in the "Unfragmentable part"; if setfh != 0, nexthdr
 * field of the last header in the "Unfragmentable part" is set to FH.
 */
int
unfraglen(Block *bp, uchar *nexthdr, int setfh)
{
    uchar *p, *q;
    int ufl, hs;
```

```
    p = bp->rp;
    q = p+6;   /* proto, = p+sizeof(Ip6hdr.vcf)+sizeof(Ip6hdr.ploadlen) */
    *nexthdr = *q;
    ufl = IP6HDR;
    p += ufl;

    while (*nexthdr == HBH || *nexthdr == RH) {
        *nexthdr = *p;
        hs = ((int)*(p+1) + 1) * 8;
        ufl += hs;
        q = p;
        p += hs;
    }

    if(*nexthdr == FH)
        *q = *p;
    if(setfh)
        *q = FH;
    return ufl;
}
```

⟨*function* procopts 460a⟩≡ (463)

```
Block*
procopts(Block *bp)
{
    return bp;
}
```

⟨*function* ip6reassemble 460b⟩≡ (463)

```
Block*
ip6reassemble(IP* ip, int uflen, Block* bp, Ip6hdr* ih)
{
    int fend, offset, ovlap, len, fragsize, pktposn;
    uint id;
    uchar src[IPaddrlen], dst[IPaddrlen];
    Block *bl, **l, *last, *prev;
    Fraghdr6 *fraghdr;
    Fragment6 *f, *fnext;

    fraghdr = (Fraghdr6 *)(bp->rp + uflen);
    memmove(src, ih->src, IPaddrlen);
    memmove(dst, ih->dst, IPaddrlen);
    id = nhgetl(fraghdr->id);
    offset = nhgets(fraghdr->offsetRM) & ~7;

    /*
     *  block lists are too hard, pullupblock into a single block
     */
    if(bp->next){
        bp = pullupblock(bp, blocklen(bp));
        ih = (Ip6hdr *)bp->rp;
    }

    qlock(&ip->fraglock6);

    /*
     *  find a reassembly queue for this fragment
     */
    for(f = ip->flisthead6; f; f = fnext){
        fnext = f->next;
```

```
    if(ipcmp(f->src, src)==0 && ipcmp(f->dst, dst)==0 && f->id == id)
        break;
    if(f->age < NOW){
        ip->stats[ReasmTimeout]++;
        ipfragfree6(ip, f);
    }
}

/*
 *  if this isn't a fragmented packet, accept it
 *  and get rid of any fragments that might go
 *  with it.
 */
if(nhgets(fraghdr->offsetRM) == 0) {    /* 1st frag is also last */
    if(f) {
        ipfragfree6(ip, f);
        ip->stats[ReasmFails]++;
    }
    qunlock(&ip->fraglock6);
    return bp;
}

if(bp->base+IPFRAGSZ >= bp->rp){
    bp = padblock(bp, IPFRAGSZ);
    bp->rp += IPFRAGSZ;
}

BKFG(bp)->foff = offset;
BKFG(bp)->flen = nhgets(ih->ploadlen) + IP6HDR - uflen - IP6FHDR;

/* First fragment allocates a reassembly queue */
if(f == nil) {
    f = ipfragallo6(ip);
    f->id = id;
    memmove(f->src, src, IPaddrlen);
    memmove(f->dst, dst, IPaddrlen);

    f->blist = bp;

    qunlock(&ip->fraglock6);
    ip->stats[ReasmReqds]++;
    return nil;
}

/*
 *  find the new fragment's position in the queue
 */
prev = nil;
l = &f->blist;
bl = f->blist;
while(bl != nil && BKFG(bp)->foff > BKFG(bl)->foff) {
    prev = bl;
    l = &bl->next;
    bl = bl->next;
}

/* Check overlap of a previous fragment - trim away as necessary */
if(prev) {
    ovlap = BKFG(prev)->foff + BKFG(prev)->flen - BKFG(bp)->foff;
    if(ovlap > 0) {
```

```
        if(ovlap >= BKFG(bp)->flen) {
            freeblist(bp);
            qunlock(&ip->fraglock6);
            return nil;
        }
        BKFG(prev)->flen -= ovlap;
    }
}

/* Link onto assembly queue */
bp->next = *l;
*l = bp;

/* Check to see if succeeding segments overlap */
if(bp->next) {
    l = &bp->next;
    fend = BKFG(bp)->foff + BKFG(bp)->flen;

    /* Take completely covered segments out */
    while(*l) {
        ovlap = fend - BKFG(*l)->foff;
        if(ovlap <= 0)
            break;
        if(ovlap < BKFG(*l)->flen) {
            BKFG(*l)->flen -= ovlap;
            BKFG(*l)->foff += ovlap;
            /* move up ih hdrs */
            memmove((*l)->rp + ovlap, (*l)->rp, uflen);
            (*l)->rp += ovlap;
            break;
        }
        last = (*l)->next;
        (*l)->next = nil;
        freeblist(*l);
        *l = last;
    }
}

/*
 *  look for a complete packet.  if we get to a fragment
 *  with the trailing bit of fraghdr->offsetRM[1] set, we're done.
 */
pktposn = 0;
for(bl = f->blist; bl && BKFG(bl)->foff == pktposn; bl = bl->next) {
    fraghdr = (Fraghdr6 *)(bl->rp + uflen);
    if((fraghdr->offsetRM[1] & 1) == 0) {
        bl = f->blist;

        /* get rid of frag header in first fragment */
        memmove(bl->rp + IP6FHDR, bl->rp, uflen);
        bl->rp += IP6FHDR;
        len = nhgets(((Ip6hdr*)bl->rp)->ploadlen) - IP6FHDR;
        bl->wp = bl->rp + len + IP6HDR;
        /*
         * Pullup all the fragment headers and
         * return a complete packet
         */
        for(bl = bl->next; bl; bl = bl->next) {
            fragsize = BKFG(bl)->flen;
            len += fragsize;
```

```
                bl->rp += uflen + IP6FHDR;
                bl->wp = bl->rp + fragsize;
            }

            bl = f->blist;
            f->blist = nil;
            ipfragfree6(ip, f);
            ih = (Ip6hdr*)bl->rp;
            hnputs(ih->ploadlen, len);
            qunlock(&ip->fraglock6);
            ip->stats[ReasmOKs]++;
            return bl;
        }
        pktposn += BKFG(bl)->flen;
    }
    qunlock(&ip->fraglock6);
    return nil;
}
```

⟨kernel/network/ip/ipv6.c 463⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"
#include    "../port/error.h"

#include    "ip.h"
#include    "ipv6.h"

⟨enum _anon_ (kernel/network/ip/ipv6.c) 453a⟩

⟨macro IPV6CLASS 453b⟩
⟨macro BLKIPVER((kernel/network/ip/ipv6.c)) 453c⟩
⟨macro BKFG((kernel/network/ip/ipv6.c)) 453d⟩

Block*      ip6reassemble(IP*, int, Block*, Ip6hdr*);
Fragment6*  ipfragallo6(IP*);
void        ipfragfree6(IP*, Fragment6*);
Block*      procopts(Block *bp);
static Block*   procxtns(IP *ip, Block *bp, int doreasm);
int     unfraglen(Block *bp, uchar *nexthdr, int setfh);

⟨function ipoput6 453e⟩

⟨function ipiput6 456⟩

⟨function ipfragfree6 458⟩

⟨function ipfragallo6 459a⟩

⟨function procxtns 459b⟩

⟨function unfraglen 459c⟩

⟨function procopts 460a⟩

⟨function ip6reassemble 460b⟩
```

kernel/network/ip/ipv6.h

⟨*macro* isv6mcast 464a⟩≡ (466c)

```
/*
 * Internet Protocol Version 6
 *
 * rfc2460 defines the protocol, rfc2461 neighbour discovery, and
 * rfc2462 address autoconfiguration.  rfc4443 defines ICMP; was rfc2463.
 * rfc4291 defines the address architecture (including prefices), was rfc3513.
 * rfc4007 defines the scoped address architecture.
 *
 * global unicast is anything but unspecified (::), loopback (::1),
 * multicast (ff00::/8), and link-local unicast (fe80::/10).
 *
 * site-local (fec0::/10) is now deprecated, originally by rfc3879.
 *
 * Unique Local IPv6 Unicast Addresses are defined by rfc4193.
 * prefix is fc00::/7, scope is global, routing is limited to roughly a site.
 */
#define isv6mcast(addr)   ((addr)[0] == 0xff)
```

⟨*macro* islinklocal 464b⟩≡ (466c)

```
#define islinklocal(addr) ((addr)[0] == 0xfe && ((addr)[1] & 0xc0) == 0x80)
```

⟨*macro* optexsts 464c⟩≡ (466c)

```
#define optexsts(np)  (nhgets((np)->ploadlen) > 24)
```

⟨*macro* issmcast 464d⟩≡ (466c)

```
#define issmcast(addr)  (memcmp((addr), v6solicitednode, 13) == 0)
```

⟨*enum* _anon_ (kernel/network/ip/ipv6.h) 464e⟩≡ (466c)

```
enum {        /* Header Types */
  HBH   = 0,  /* hop-by-hop multicast routing protocol */
  ICMP    = 1,
  IGMP    = 2,
  GGP   = 3,
  IPINIP    = 4,
  ST    = 5,
  TCP   = 6,
  UDP   = 17,
  ISO_TP4   = 29,
  RH    = 43,
  FH    = 44,
  IDRP    = 45,
  RSVP    = 46,
  AH    = 51,
  ESP   = 52,
  ICMPv6    = 58,
  NNH   = 59,
  DOH   = 60,
  ISO_IP    = 80,
  IGRP    = 88,
  OSPF    = 89,

  Maxhdrtype  = 256,
};
```

⟨*enum* _anon_ (kernel/network/ip/ipv6.h)2 465⟩≡ (466c)

```
enum {
  /* multicast flags and scopes */

//  Well_known_flg  = 0,
//  Transient_flg = 1,

//  Interface_local_scop = 1,
  Link_local_scop = 2,
//  Site_local_scop = 5,
//  Org_local_scop  = 8,
  Global_scop = 14,

  /* various prefix lengths */
  SOLN_PREF_LEN = 13,

  /* icmpv6 unreachability codes */
  Icmp6_no_route    = 0,
  Icmp6_ad_prohib   = 1,
  Icmp6_out_src_scope = 2,
  Icmp6_adr_unreach = 3,
  Icmp6_port_unreach  = 4,
  Icmp6_gress_src_fail  = 5,
  Icmp6_rej_route   = 6,
  Icmp6_unknown   = 7,  /* our own invention for internal use */

  /* various flags & constants */
  v6MINTU   = 1280,
  HOP_LIMIT = 255,
  IP6HDR    = 40,   /* sizeof(Ip6hdr) = 8 + 2*16 */

  /* option types */

  /* neighbour discovery */
  SRC_LLADDR  = 1,
  TARGET_LLADDR = 2,
  PREFIX_INFO = 3,
  REDIR_HEADER  = 4,
  MTU_OPTION  = 5,
  /* new since rfc2461; see iana.org/assignments/icmpv6-parameters */
  V6nd_home = 8,
  V6nd_srcaddrs = 9,    /* rfc3122 */
  V6nd_ip   = 17,
  /* /lib/rfc/drafts/draft-jeong-dnsop-ipv6-dns-discovery-12.txt */
  V6nd_rdns = 25,
  /* plan 9 extensions */
  V6nd_9fs  = 250,
  V6nd_9auth  = 251,

  SRC_UNSPEC  = 0,
  SRC_UNI   = 1,
  TARG_UNI  = 2,
  TARG_MULTI  = 3,

  Tunitent  = 1,
  Tuniproxy = 2,
  Tunirany  = 3,

  /* Node constants */
  MAX_MULTICAST_SOLICIT = 3,
```

```
    RETRANS_TIMER   = 1000,
  };
```

⟨*struct* Ip6hdr*(*(kernel/network/ip/ipv6.h)*)* 466a⟩≡ (466c)

```
  struct  Ip6hdr {
    IPV6HDR;
    uchar payload[];
  };
```

⟨*struct* Fraghdr6 466b⟩≡ (466c)

```
  //};

  struct  Fraghdr6 {
    uchar nexthdr;
    uchar res;
    uchar offsetRM[2];  /* Offset, Res, M flag */
    uchar id[4];
  };
```

⟨kernel/network/ip/ipv6.h 466c⟩≡

```
  ⟨macro isv6mcast 464a⟩
  ⟨macro islinklocal 464b⟩

  ⟨macro optexsts 464c⟩
  ⟨macro issmcast 464d⟩

  ⟨enum _anon_ (kernel/network/ip/ipv6.h) 464e⟩

  ⟨enum _anon_ (kernel/network/ip/ipv6.h)2 465⟩

  typedef struct Ip6hdr Ip6hdr;
  typedef struct Opthdr Opthdr;
  typedef struct Routinghdr Routinghdr;
  typedef struct Fraghdr6 Fraghdr6;

  /* we do this in case there's padding at the end of Ip6hdr */
  #define IPV6HDR \
    uchar vcf[4];   /* version:4, traffic class:8, flow label:20 */\
    uchar ploadlen[2];  /* payload length: packet length - 40 */ \
    uchar proto;    /* next header type */ \
    uchar ttl;    /* hop limit */ \
    uchar src[IPaddrlen]; \
    uchar dst[IPaddrlen]

  ⟨struct Ip6hdr*(*(kernel/network/ip/ipv6.h)*)* 466a⟩

  //struct  Opthdr {    /* unused */
  //  uchar nexthdr;
  //  uchar len;
  //};

  /*
   * Beware routing header type 0 (loose source routing); see
   * http://www.secdev.org/conf/IPv6_RH_security-csw07.pdf.
   * Type 1 is unused.  Type 2 is for MIPv6 (mobile IPv6) filtering
   * against type 0 header.
   */
  //struct  Routinghdr {    /* unused */
  //  uchar nexthdr;
  //  uchar len;
```

```
//  uchar rtetype;
//  uchar segrem;
⟨struct Fraghdr6 466b⟩

extern uchar v6allnodesN[IPaddrlen];
extern uchar v6allnodesL[IPaddrlen];
//extern uchar v6allroutersN[IPaddrlen];
//extern uchar v6allroutersL[IPaddrlen];
extern uchar v6allnodesNmask[IPaddrlen];
extern uchar v6allnodesLmask[IPaddrlen];
extern uchar v6solicitednode[IPaddrlen];
//extern uchar v6solicitednodemask[IPaddrlen];
extern uchar v6Unspecified[IPaddrlen];
extern uchar v6loopback[IPaddrlen];
//extern uchar v6loopbackmask[IPaddrlen];
extern uchar v6linklocal[IPaddrlen];
//extern uchar v6linklocalmask[IPaddrlen];
//extern uchar v6multicast[IPaddrlen];
//extern uchar v6multicastmask[IPaddrlen];

extern int v6llpreflen;
//extern int v6mcpreflen;
//extern int v6snpreflen;
//extern int v6aNpreflen;
//extern int v6aLpreflen;

extern int ReTransTimer;

void ipv62smcast(uchar *, uchar *);
void icmpns(Fs *f, uchar* src, int suni, uchar* targ, int tuni, uchar* mac);
//void icmpna(Fs *f, uchar* src, uchar* dst, uchar* targ, uchar* mac, uchar flags);
void icmpttlexceeded6(Fs *f, Ipifc *ifc, Block *bp);
void icmppkttoobig6(Fs *f, Ipifc *ifc, Block *bp);
void icmphostunr(Fs *f, Ipifc *ifc, Block *bp, int code, int free);
```

⟨v6tov4() *else if ipv6 address* 467a⟩≡ (14b)

```
else {
       memset(v4, 0, 4);
       if(memcmp(v6, IPnoaddr, IPaddrlen) == 0)
           return OK_0;
       return ERROR_NEG1;
}
```

⟨defmask() *if ipv6* 467b⟩≡ (15a)

```
else {
    if(ipcmp(ip, v6loopback) == 0)
        return IPallbits;
    else if(memcmp(ip, v6linklocal, v6llpreflen) == 0)
        return v6linklocalmask;
    else if(memcmp(ip, v6solicitednode, v6snpreflen) == 0)
        return v6solicitednodemask;
    else if(memcmp(ip, v6multicast, v6mcpreflen) == 0)
        return v6multicastmask;
    return IPallbits;
}
```

⟨Ipifc(user) *ipv6 fields* 467c⟩≡ (16b) 467d ▷

```
Ipv6rp rp;
```

⟨Ipifc(user) *ipv6 fields* 467d⟩+≡ (16b) ◁ 467c

```
uchar sendra6;  /* on == send router adv */
uchar recvra6;  /* on == rcv router adv */
```

⟨Fs(kernel) *ipv6 fields* 468a⟩≡ (21c)

```
    v6params  *v6p;
    Route *v6root[1<<Lroot];  /* v6 routing forest */
```

⟨IP(kernel) *ipv6 fields* 468b⟩≡ (22a)

```
    QLock   fraglock6;
    Fragment6*  flisthead6;
    Fragment6*  fragfree6;
    Ref   id6;
```

⟨Ipifc(kernel) *ipv6 fields* 468c⟩≡ (17c)

```
  uchar sendra6;  /* flag: send router advs on this ifc */
  uchar recvra6;  /* flag: recv router advs on this ifc */
```

⟨*function* ip_init_6 468d⟩≡ (212b)

```
  void
  ip_init_6(Fs *f)
  {
      v6params *v6p;

      v6p = smalloc(sizeof(v6params));

      v6p->rp.mflag       = 0;        /* default not managed */
      v6p->rp.oflag       = 0;
      v6p->rp.maxraint    = 600000;   /* millisecs */
      v6p->rp.minraint    = 200000;
      v6p->rp.linkmtu     = 0;        /* no mtu sent */
      v6p->rp.reachtime   = 0;
      v6p->rp.rxmitra     = 0;
      v6p->rp.ttl     = MAXTTL;
      v6p->rp.routerlt    = 3 * v6p->rp.maxraint;

      v6p->hp.rxmithost   = 1000;     /* v6 RETRANS_TIMER */

      v6p->cdrouter       = -1;

      f->v6p          = v6p;
  }
```

⟨*struct* Ipv6rp 468e⟩≡ (196c)

```
  /* default values, one per stack */
  struct Ipv6rp
  {
      int mflag;
      int oflag;
      int maxraint;
      int minraint;
      int linkmtu;
      int reachtime;
      int rxmitra;
      int ttl;
      int routerlt;
  };
```

⟨*macro* ISIPV6MCAST 468f⟩≡ (196c)

```
  #define ISIPV6MCAST(addr) ((addr)[0] == 0xff)
```

⟨*macro* ISIPV6LINKLOCAL 468g⟩≡ (196c)

```
  #define ISIPV6LINKLOCAL(addr) ((addr)[0] == 0xfe && ((addr)[1] & 0xc0) == 0x80)
```

⟨*enum* _anon_ (include/net/ip.h) 469a⟩≡ (196c)

```
/*
 * ipv6 constants
 * ‘ra’ is ‘router advertisement’, ‘rs’ is ‘router solicitation’.
 * ‘na’ is ‘neighbour advertisement’.
 */
enum {
    IPV6HDR_LEN = 40,

    /* neighbour discovery option types */
    V6nd_srclladdr = 1,
    V6nd_targlladdr = 2,
    V6nd_pfxinfo = 3,
    V6nd_redirhdr = 4,
    V6nd_mtu = 5,
    /* new since rfc2461; see iana.org/assignments/icmpv6-parameters */
    V6nd_home = 8,
    V6nd_srcaddrs = 9,  /* rfc3122 */
    V6nd_ip  = 17,
    /* /lib/rfc/drafts/draft-jeong-dnsop-ipv6-dns-discovery-12.txt */
    V6nd_rdns = 25,
    /* plan 9 extensions */
    V6nd_9fs = 250,
    V6nd_9auth = 251,

    /* Router constants (all times in ms.) */
    Maxv6initraintvl= 16000,
    Maxv6initras = 3,
    Maxv6finalras = 3,
    Minv6interradelay= 3000,
    Maxv6radelay = 500,

    /* Host constants */
    Maxv6rsdelay = 1000,
    V6rsintvl = 4000,
    Maxv6rss = 3,

    /* Node constants */
    Maxv6mcastrss = 3,
    Maxv6unicastrss = 3,
    Maxv6anycastdelay= 1000,
    Maxv6na  = 3,
    V6reachabletime = 30000,
    V6retranstimer = 1000,
    V6initprobedelay= 5000,
};
```

⟨*struct* Ip6hdr 469b⟩≡ (196c)

```
/* V6 header on the wire */
struct Ip6hdr {
    uchar vcf[4];  /* version:4, traffic class:8, flow label:20 */
    uchar ploadlen[2]; /* payload length: packet length - 40 */
    uchar proto;  /* next header type */
    uchar ttl;  /* hop limit */
    ipaddr src; /* source address */
    ipaddr dst; /* destination address */
    uchar payload[];
};
```

⟨*struct* Icmp6hdr 469c⟩≡ (196c)

```
/*
 *  user-level icmpv6 with control message "headers"
 */
struct Icmp6hdr {
    uchar _0_[8];
    ipaddr laddr; /* local address */
    ipaddr raddr; /* remote address */
};
```

⟨*function* equivip6 470a⟩≡ (201a)

```
bool
equivip6(uchar *a, uchar *b)
{
    int i;

    for(i = 0; i < IPaddrlen; i++)
        if(a[i] != b[i])
            return false;
    return true;
}
```

⟨*global* v6loopback 470b⟩≡ (203d)

```
static uchar v6loopback[IPaddrlen] = {
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0x01
};
```

⟨*global* v6linklocal 470c⟩≡ (203d)

```
static uchar v6linklocal[IPaddrlen] = {
    0xfe, 0x80, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0
};
```

⟨*global* v6linklocalmask 470d⟩≡ (203d)

```
static uchar v6linklocalmask[IPaddrlen] = {
    0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff,
    0, 0, 0, 0,
    0, 0, 0, 0
};
```

⟨*global* v6llpreflen 470e⟩≡ (203d)

```
static int v6llpreflen = 8; /* link-local prefix length in bytes */
```

⟨*global* v6multicast 470f⟩≡ (203d)

```
static uchar v6multicast[IPaddrlen] = {
    0xff, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0
};
```

⟨*global* v6multicastmask 470g⟩≡ (203d)

```
static uchar v6multicastmask[IPaddrlen] = {
    0xff, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0
};
```

⟨*global* v6mcpreflen 471a⟩≡ (203d)
```
static int v6mcpreflen = 1; /* multicast prefix length */
```

⟨*global* v6solicitednode 471b⟩≡ (203d)
```
static uchar v6solicitednode[IPaddrlen] = {
    0xff, 0x02, 0, 0,
    0, 0, 0, 0,
    0, 0, 0, 0x01,
    0xff, 0, 0, 0
};
```

⟨*global* v6solicitednodemask 471c⟩≡ (203d)
```
static uchar v6solicitednodemask[IPaddrlen] = {
    0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff,
    0xff, 0x0, 0x0, 0x0
};
```

⟨*global* v6snpreflen 471d⟩≡ (203d)
```
static int v6snpreflen = 13;
```

⟨ip_init() *ipv6 init* 471e⟩≡ (32d)
```
ip_init_6(f);
```

⟨initfrag() *locals* 471f⟩≡ (33a)
```
    Fragment6 *fq6, *eq6;
```

⟨initfrag() *ipv6 init fragfree6* 471g⟩≡ (33a)
```
    ip->fragfree6 = (Fragment6*)malloc(sizeof(Fragment6) * size);
    if(ip->fragfree6 == nil)
        panic("initfrag");

    eq6 = &ip->fragfree6[size];
    for(fq6 = ip->fragfree6; fq6 < eq6; fq6++)
        fq6->next = fq6+1;

    ip->fragfree6[size-1].next = nil;
```

⟨*struct* Fragment6 471h⟩≡ (208)
```
struct Fragment6
{
  Block*  blist;
  Fragment6*  next;
  uchar   src[IPaddrlen];
  uchar   dst[IPaddrlen];
  uint  id;
  ulong   age;
};
```

⟨*struct* v6router 471i⟩≡ (208)
```
/* one per default router known to host */
struct v6router {
  uchar inuse;
  Ipifc *ifc;
  int ifcid;
  uchar routeraddr[IPaddrlen];
  long  ltorigin;
  Routerparams  rp;
};
```

⟨*struct* v6params 472a⟩≡ (208)

```
struct v6params
{
  Routerparams  rp;   /* v6 params, one copy per node now */
  Hostparams  hp;
  v6router  v6rlist[3]; /* max 3 default routers, currently */
  int   cdrouter; /* uses only v6rlist[cdrouter] if   */
          /* cdrouter >= 0. */
};
```

⟨*struct* V6route 472b⟩≡ (208)

```
struct V6route
{
  ulong address[IPllen];
  ulong endaddress[IPllen];
  uchar gate[IPaddrlen];
};
```

⟨*function* etherread6 472c⟩≡ (336c)

```
/*
 *  process to read from the ethernet, IPv6
 */
static void
etherread6(void *a)
{
    Ipifc *ifc;
    Block *bp;
    Etherrock *er;

    ifc = a;
    er = ifc->arg;
    er->read6p = up;    /* hide identity under a rock for unbind */
    if(waserror()){
        er->read6p = 0;
        pexit("hangup", 1);
    }
    for(;;){
        bp = devtab[er->mchan6->type]->bread(er->mchan6, ifc->maxtu, 0);
        if(!canrlock(ifc)){
            freeb(bp);
            continue;
        }
        if(waserror()){
            runlock(ifc);
            nexterror();
        }
        ifc->in++;
        bp->rp += ifc->m->hsize;
        if(ifc->lifc == nil)
            freeb(bp);
        else
            ipiput6(er->f, ifc, bp);
        runlock(ifc);
        poperror();
    }
}
```

⟨*global* v6freelist 472d⟩≡ (339c)

```
static Route*   v6freelist;
```

⟨*macro* V6H 473a⟩≡ (339c)

```
#define V6H(a)  (((a)[IPllen-1] & 0x07ffffff)>>(32-Lroot-5))
```

⟨*function* v6addroute 473b⟩≡ (339c)

```
//#define ISDFLT(a, mask, tag) ((ipcmp((a),v6Unspecified)==0) && (ipcmp((mask),v6Unspecified)==0) && (strcmp((t

void
v6addroute(Fs *f, char *tag, uchar *a, uchar *mask, uchar *gate, int type)
{
    Route *p;
    ulong sa[IPllen], ea[IPllen];
    ulong x, y;
    int h, eh;

    /*
    if(ISDFLT(a, mask, tag))
        f->v6p->cdrouter = -1;
    */


    for(h = 0; h < IPllen; h++){
        x = nhgetl(a+4*h);
        y = nhgetl(mask+4*h);
        sa[h] = x & y;
        ea[h] = x | ~y;
    }

    eh = V6H(ea);
    for(h = V6H(sa); h <= eh; h++) {
        p = allocroute(type);
        memmove(p->v6.address, sa, IPaddrlen);
        memmove(p->v6.endaddress, ea, IPaddrlen);
        memmove(p->v6.gate, gate, IPaddrlen);
        memmove(p->tag, tag, sizeof(p->tag));

        wlock(&routelock);
        addnode(f, &f->v6root[h], p);
        while(p = f->queue) {
            f->queue = p->mid;
            walkadd(f, &f->v6root[h], p->left);
            freeroute(p);
        }
        wunlock(&routelock);
    }
    v6routegeneration++;

    ipifcaddroute(f, 0, a, mask, gate, type);
}
```

⟨*function* v6delroute 473c⟩≡ (339c)

```
void
v6delroute(Fs *f, uchar *a, uchar *mask, int dolock)
{
    Route **r, *p;
    Route rt;
    int h, eh;
    ulong x, y;

    for(h = 0; h < IPllen; h++){
        x = nhgetl(a+4*h);
```

```
        y = nhgetl(mask+4*h);
        rt.v6.address[h] = x & y;
        rt.v6.endaddress[h] = x | ~y;
    }
    rt.type = 0;

    eh = V6H(rt.v6.endaddress);
    for(h=V6H(rt.v6.address); h<=eh; h++) {
        if(dolock)
            wlock(&routelock);
        r = looknode(&f->v6root[h], &rt);
        if(r) {
            p = *r;
            if(--(p->ref) == 0){
                *r = 0;
                addqueue(&f->queue, p->left);
                addqueue(&f->queue, p->mid);
                addqueue(&f->queue, p->right);
                freeroute(p);
                while(p = f->queue) {
                    f->queue = p->mid;
                    walkadd(f, &f->v6root[h], p->left);
                    freeroute(p);
                }
            }
        }
        if(dolock)
            wunlock(&routelock);
    }
    v6routegeneration++;

    ipifcremroute(f, 0, a, mask);
}
```

⟨*function* v6lookup 474⟩≡ (339c)

```
Route*
v6lookup(Fs *f, uchar *a, Conv *c)
{
    Route *p, *q;
    ulong la[IPllen];
    int h;
    ulong x, y;
    uchar gate[IPaddrlen];
    Ipifc *ifc;

    if(memcmp(a, v4prefix, IPv4off) == 0){
        q = v4lookup(f, a+IPv4off, c);
        if(q != nil)
            return q;
    }

    if(c != nil && c->r != nil && c->r->ifc != nil && c->rgen == v6routegeneration)
        return c->r;

    for(h = 0; h < IPllen; h++)
        la[h] = nhgetl(a+4*h);

    q = 0;
    for(p=f->v6root[V6H(la)]; p;){
        for(h = 0; h < IPllen; h++){
```

```
            x = la[h];
            y = p->v6.address[h];
            if(x == y)
                continue;
            if(x < y){
                p = p->left;
                goto next;
            }
            break;
        }
        for(h = 0; h < IPllen; h++){
            x = la[h];
            y = p->v6.endaddress[h];
            if(x == y)
                continue;
            if(x > y){
                p = p->right;
                goto next;
            }
            break;
        }
        q = p;
        p = p->mid;
next:       ;
    }

    if(q && (q->ifc == nil || q->ifcid != q->ifc->ifcid)){
        if(q->type & Rifc) {
            for(h = 0; h < IPllen; h++)
                hnputl(gate+4*h, q->v6.address[h]);
            ifc = findipifc(f, gate, q->type);
        } else
            ifc = findipifc(f, q->v6.gate, q->type);
        if(ifc == nil)
            return nil;
        q->ifc = ifc;
        q->ifcid = ifc->ifcid;
    }
    if(c != nil){
        c->r = q;
        c->rgen = v6routegeneration;
    }

    return q;
}
```

⟨*function* ipifcra6 475⟩≡ (354b)

```
char*
ipifcra6(Ipifc *ifc, char **argv, int argc)
{
    int i, argsleft, vmax = ifc->rp.maxraint, vmin = ifc->rp.minraint;

    argsleft = argc - 1;
    i = 1;

    if(argsleft % 2 != 0)
        return Ebadarg;

    while (argsleft > 1) {
        if(strcmp(argv[i], "recvra") == 0)
```

```
            ifc->recvra6 = (atoi(argv[i+1]) != 0);
        else if(strcmp(argv[i], "sendra") == 0)
            ifc->sendra6 = (atoi(argv[i+1]) != 0);
        else if(strcmp(argv[i], "mflag") == 0)
            ifc->rp.mflag = (atoi(argv[i+1]) != 0);
        else if(strcmp(argv[i], "oflag") == 0)
            ifc->rp.oflag = (atoi(argv[i+1]) != 0);
        else if(strcmp(argv[i], "maxraint") == 0)
            ifc->rp.maxraint = atoi(argv[i+1]);
        else if(strcmp(argv[i], "minraint") == 0)
            ifc->rp.minraint = atoi(argv[i+1]);
        else if(strcmp(argv[i], "linkmtu") == 0)
            ifc->rp.linkmtu = atoi(argv[i+1]);
        else if(strcmp(argv[i], "reachtime") == 0)
            ifc->rp.reachtime = atoi(argv[i+1]);
        else if(strcmp(argv[i], "rxmitra") == 0)
            ifc->rp.rxmitra = atoi(argv[i+1]);
        else if(strcmp(argv[i], "ttl") == 0)
            ifc->rp.ttl = atoi(argv[i+1]);
        else if(strcmp(argv[i], "routerlt") == 0)
            ifc->rp.routerlt = atoi(argv[i+1]);
        else
            return Ebadarg;

        argsleft -= 2;
        i += 2;
    }

    /* consistency check */
    if(ifc->rp.maxraint < ifc->rp.minraint) {
        ifc->rp.maxraint = vmax;
        ifc->rp.minraint = vmin;
        return Ebadarg;
    }
    return nil;
}
```

⟨*enum* _anon_ (kernel/network/ip/ipifc.c)3 476a⟩≡ (354b)

```
enum {
    unknownv6,      /* UGH */
//  multicastv6,
    unspecifiedv6,
    linklocalv6,
    globalv6,
};
```

⟨*function* v6addrtype 476b⟩≡ (354b)

```
int
v6addrtype(uchar *addr)
{
    if(isv4(addr) || ipcmp(addr, IPnoaddr) == 0)
        return unknownv6;
    else if(islinklocal(addr) ||
        isv6mcast(addr) && (addr[1] & 0xF) <= Link_local_scop)
        return linklocalv6;
    else
        return globalv6;
}
```

⟨*macro* v6addrcurr 477a⟩≡ (354b)

```
#define v6addrcurr(lifc) ((lifc)->preflt == ~0L || \
            (lifc)->origint + (lifc)->preflt >= NOW/1000)
```

⟨*function* findprimaryipv6 477b⟩≡ (354b)

```
static void
findprimaryipv6(Fs *f, uchar *local)
{
    int atype, atypel;
    Conv **cp, **e;
    Ipifc *ifc;
    Iplifc *lifc;

    ipmove(local, v6Unspecified);
    atype = unspecifiedv6;

    /*
     * find "best" (global > link local > unspecified)
     * local address; address must be current.
     */
    e = &f->ipifc->conv[f->ipifc->nc];
    for(cp = f->ipifc->conv; cp < e; cp++){
        if(*cp == 0)
            continue;
        ifc = (Ipifc*)(*cp)->ptcl;
        for(lifc = ifc->lifc; lifc; lifc = lifc->next){
            atypel = v6addrtype(lifc->local);
            if(atypel > atype && v6addrcurr(lifc)) {
                ipmove(local, lifc->local);
                atype = atypel;
                if(atype == globalv6)
                    return;
            }
        }
    }
}
```

⟨*function* ipv6local 477c⟩≡ (354b)

```
/*
 *  return first v6 address associated with an interface
 */
int
ipv6local(Ipifc *ifc, uchar *addr)
{
    Iplifc *lifc;

    for(lifc = ifc->lifc; lifc; lifc = lifc->next){
        if(!isv4(lifc->local) && !(lifc->tentative)){
            ipmove(addr, lifc->local);
            return 1;
        }
    }
    return 0;
}
```

⟨*function* ipv6anylocal 477d⟩≡ (354b)

```
int
ipv6anylocal(Ipifc *ifc, uchar *addr)
{
    Iplifc *lifc;
```

```
    for(lifc = ifc->lifc; lifc; lifc = lifc->next){
        if(!isv4(lifc->local)){
            ipmove(addr, lifc->local);
            return SRC_UNI;
        }
    }
    return SRC_UNSPEC;
}
```

⟨*function* ipifcadd6 478⟩≡ (354b)

```
char*
ipifcadd6(Ipifc *ifc, char**argv, int argc)
{
    int plen = 64;
    long origint = NOW / 1000, preflt = ~0L, validlt = ~0L;
    char addr[40], preflen[6];
    char *params[3];
    uchar autoflag = 1, onlink = 1;
    uchar prefix[IPaddrlen];
    Iplifc *lifc;

    switch(argc) {
    case 7:
        preflt = atoi(argv[6]);
        /* fall through */
    case 6:
        validlt = atoi(argv[5]);
        /* fall through */
    case 5:
        autoflag = atoi(argv[4]);
        /* fall through */
    case 4:
        onlink = atoi(argv[3]);
        /* fall through */
    case 3:
        plen = atoi(argv[2]);
        /* fall through */
    case 2:
        break;
    default:
        return Ebadarg;
    }

    if (parseip(prefix, argv[1]) != 6 || validlt < preflt || plen < 0 ||
        plen > 64 || islinklocal(prefix))
        return Ebadarg;

    lifc = smalloc(sizeof(Iplifc));
    lifc->onlink = (onlink != 0);
    lifc->autoflag = (autoflag != 0);
    lifc->validlt = validlt;
    lifc->preflt = preflt;
    lifc->origint = origint;

    /* issue "add" ctl msg for v6 link-local addr and prefix len */
    if(!ifc->m->pref2addr)
        return Ebadarg;
    ifc->m->pref2addr(prefix, ifc->mac);    /* mac → v6 link-local addr */
    snprint(addr, sizeof addr, "%I", prefix);
```

```
    snprint(preflen, sizeof preflen, "/%d", plen);
    params[0] = "add";
    params[1] = addr;
    params[2] = preflen;

    return ipifcadd(ifc, params, 3, 0, lifc);
}
```

⟨Medium(kernel) *ipv6 methods* 479a⟩≡ (18d)

```
/* v6 address generation */
void  (*pref2addr)(uchar *pref, uchar *ea);
```

⟨*struct* Udp6hdr 479b⟩≡ (374)

```
struct Udp6hdr {
    uchar viclfl[4];
    uchar len[2];
    uchar nextheader;
    uchar hoplimit;
    uchar udpsrc[IPaddrlen];
    uchar udpdst[IPaddrlen];

    /* udp header */
    uchar   udpsport[2];    /* Source port */
    uchar   udpdport[2];    /* Destination port */
    uchar   udplen[2];  /* data length */
    uchar   udpcksum[2];    /* Checksum */
};
```

⟨udpkick() *locals* 479c⟩+≡ (121b) ◁129e

```
Udp6hdr *uh6;
```

⟨udpkick() *switch version ipv6 case* 479d⟩≡ (121b)

```
case V6:
    bp = padblock(bp, UDP6_IPHDR_SZ+UDP_UDPHDR_SZ);
    if(bp == nil)
        return;

    /*
     * using the v6 ip header to create pseudo header
     * first then reset it to the normal ip header
     */
    uh6 = (Udp6hdr *)(bp->rp);
    memset(uh6, 0, 8);
    ptcllen = dlen + UDP_UDPHDR_SZ;
    hnputl(uh6->viclfl, ptcllen);
    uh6->hoplimit = IP_UDPPROTO;
    if(ucb->headers) {
        ipmove(uh6->udpdst, raddr);
        hnputs(uh6->udpdport, rport);
        ipmove(uh6->udpsrc, laddr);
        rc = nil;
    } else {
        ipmove(uh6->udpdst, c->raddr);
        hnputs(uh6->udpdport, c->rport);
        if(ipcmp(c->laddr, IPnoaddr) == 0)
            findlocalip(f, c->laddr, c->raddr);
        ipmove(uh6->udpsrc, c->laddr);
        rc = c;
    }
    hnputs(uh6->udpsport, c->lport);
```

```
    hnputs(uh6->udplen, ptcllen);
    uh6->udpcksum[0] = 0;
    uh6->udpcksum[1] = 0;
    hnputs(uh6->udpcksum,
           ptclcsum(bp, UDP6_PHDR_OFF, dlen+UDP_UDPHDR_SZ+UDP6_PHDR_SZ));
    memset(uh6, 0, 8);
    uh6->viclfl[0] = IP_VER6;
    hnputs(uh6->len, ptcllen);
    uh6->nextheader = IP_UDPPROTO;
    ipoput6(f, bp, 0, c->ttl, c->tos, rc);
    break;
```

⟨udpkick() *set version to V4 or V6* 480a⟩≡ (121b)

```
  ⟨udpkick() set version to V4 or V6 if special headers 481d⟩
  else {
      if( (memcmp(c->raddr, v4prefix, IPv4off) == 0 &&
          memcmp(c->laddr, v4prefix, IPv4off) == 0)
          || ipcmp(c->raddr, IPnoaddr) == 0)
          version = V4;
      else
          version = V6;
  }
```

⟨*constant* UDP6_xxx 480b⟩≡ (372b)

```
  UDP6_IPHDR_SZ = 40,
  UDP6_PHDR_SZ = 40,
  UDP6_PHDR_OFF = 0,
```

⟨ipiput4() *call ipiput6 if block is not ipv4* 480c⟩≡ (83f)

```
  if(BLKIPVER(bp) != IP_VER4) {
      ipiput6(f, ifc, bp);
      return;
  }
```

⟨udpiput() *locals* 480d⟩+≡ (122) ◁126a

```
  Udp6hdr *uh6;
  int oviclfl;
```

⟨udpiput() *checking checksum ipv6 case* 480e⟩≡ (126b)

```
  case V6:
      uh6 = (Udp6hdr*)(bp->rp);
      len = nhgets(uh6->udplen);
      oviclfl = nhgetl(uh6->viclfl);
      olen = nhgets(uh6->len);
      ottl = uh6->hoplimit;
      ipmove(raddr, uh6->udpsrc);
      ipmove(laddr, uh6->udpdst);
      lport = nhgets(uh6->udpdport);
      rport = nhgets(uh6->udpsport);
      memset(uh6, 0, 8);
      hnputl(uh6->viclfl, len);
      uh6->hoplimit = IP_UDPPROTO;
      if(ptclcsum(bp, UDP6_PHDR_OFF, len+UDP6_PHDR_SZ)) {
          upriv->ustats.udpInErrors++;
          netlog(f, Logudp, "udp: checksum error %I\n", raddr);
          DPRINT("udp: checksum error %I\n", raddr);
          freeblist(bp);
          return;
      }
```

```
    hnputl(uh6->viclfl, oviclfl);
    hnputs(uh6->len, olen);
    uh6->nextheader = IP_UDPPROTO;
    uh6->hoplimit = ottl;
    break;
```

⟨udpiput() *no conversation found, ipv6 case* 481a⟩≡ (127a)

```
  case V6:
    icmphostunr(f, ifc, bp, Icmp6_port_unreach, 0);
    break;
```

⟨udpiput() *trim the packet, ipv6 case* 481b⟩≡ (126c)

```
  case V6:
    bp = trimblock(bp, UDP6_IPHDR_SZ+UDP_UDPHDR_SZ, len);
    break;
```

⟨rr() *else if ipv6 address* 481c⟩≡ (105b)

```
  else
    h = V6H(r->v6.address);
```

⟨udpkick() *set version to V4 or V6 if special headers* 481d⟩≡ (480a)

```
  if(ucb->headers) {
    if(memcmp(laddr, v4prefix, IPv4off) == 0
    || ipcmp(laddr, IPnoaddr) == 0)
      version = V4;
    else
      version = V6;
  }
```

⟨udpiput() *new conv to create, ipv6 case* 481e⟩≡ (128d)

```
  case V6:
    ipmove(laddr, ifc->lifc->local);
    break;
```

⟨udpiput() *set version to V4 or V6* 481f⟩≡ (122)

```
  version = ((uh4->vihl&0xF0)==IP_VER6) ? V6 : V4;
```

⟨Route *ipv6 route union case* 481g⟩≡ (29b)

```
  V6route v6;
```

⟨Conv(kernel) *ipv6 fields* 481h⟩≡ (25a)

```
  // enum<v6_or_v4>
  uchar ipversion;
```

⟨Fsstdconnect() *set ipversion field to V4 or V6* 481i⟩≡ (46b)

```
  if( (memcmp(c->raddr, v4prefix, IPv4off) == 0 &&
       memcmp(c->laddr, v4prefix, IPv4off) == 0)
    || ipcmp(c->raddr, IPnoaddr) == 0)
    c->ipversion = V4;
  else
    c->ipversion = V6;
```

⟨ipifcadd() *add route if ipv6 case* 481j⟩≡ (60)

```
  else
    v6addroute(f, tifc, rem, mask, rem, type);
```

⟨Iplifc(user) *ipv6 fields* 481k⟩≡ (17a)

```
  ulong preflt;   /* preferred lifetime */
  ulong validlt;  /* valid lifetime */
```

⟨Iplifc(kernel) *ipv6 fields* 482a⟩≡ (18a) 482b ▷

```
long  preflt;   /* v6 preferred lifetime */
long  validlt;  /* v6 valid lifetime */
```

⟨Iplifc(kernel) *ipv6 fields* 482b⟩+≡ (18a) ◁482a

```
uchar onlink;   /* =1 => onlink, =0 offlink. */
uchar tentative;  /* =1 => v6 dup disc on, =0 => confirmed unique */
uchar autoflag; /* v6 autonomous flag */
long  origint;  /* time when addr was added */
```

⟨ipifcadd() *set tentative for ipv6* 482c⟩≡ (60)

```
    if(isv4(ip))
        tentative = false;
```

⟨ipifcctl() *else if other string* 482d⟩+≡ (59d) ◁159c 482e ▷

```
else if(strcmp(argv[0], "try") == 0)
    return ipifcadd(ifc, argv, argc, true, nil);
```

⟨ipifcctl() *else if other string* 482e⟩+≡ (59d) ◁482d 482f ▷

```
else if(strcmp(argv[0], "add6") == 0)
    return ipifcadd6(ifc, argv, argc);
```

⟨ipifcctl() *else if other string* 482f⟩+≡ (59d) ◁482e

```
else if(strcmp(argv[0], "ra6") == 0)
    return ipifcra6(ifc, argv, argc);
```

⟨ipifcadd() *if ipv6 tentative and broacast* 482g⟩≡ (60)

```
if(tentative && sendnbrdisc)
    icmpns(f, 0, SRC_UNSPEC, ip, TARG_MULTI, ifc->mac);
```

⟨ipifcadd() *locals* 482h⟩+≡ (60) ◁157c

```
bool sendnbrdisc = false;
```

⟨ipifcadd() *if ipv6 add multicast addresses to self cache* 482i⟩≡ (60)

```
else {
    if(ipcmp(ip, v6loopback) == 0) {
        /* add node-local mcast address */
        addselfcache(f, ifc, lifc, v6allnodesN, Rmulti);

        /* add route for all node multicast */
        v6addroute(f, tifc, v6allnodesN, v6allnodesNmask,
            v6allnodesN, Rmulti);
    }

    /* add all nodes multicast address */
    addselfcache(f, ifc, lifc, v6allnodesL, Rmulti);

    /* add route for all nodes multicast */
    v6addroute(f, tifc, v6allnodesL, v6allnodesLmask, v6allnodesL,
        Rmulti);

    /* add solicited-node multicast address */
    ipv62smcast(bcast, ip);
    addselfcache(f, ifc, lifc, bcast, Rmulti);

    sendnbrdisc = true;
}
```

⟨ipifcadd() *set ipv6 fields for lifc* 483a⟩≡ (60)

```
lifc->tentative = tentative;
if(lifcp) {
    lifc->onlink = lifcp->onlink;
    lifc->autoflag = lifcp->autoflag;

    lifc->validlt = lifcp->validlt;
    lifc->preflt = lifcp->preflt;
    lifc->origint = lifcp->origint;
} else {        /* default values */
    lifc->onlink = lifc->autoflag = 1;
    lifc->validlt = lifc->preflt = ~0L;
    lifc->origint = NOW / 1000;
}
```

⟨ipifcadd() *when already local address for ifc, copy ipv6 fields* 483b⟩≡ (61c)

```
if(lifc->tentative != tentative)
    lifc->tentative = tentative;
if(lifcp) {
    lifc->onlink = lifcp->onlink;
    lifc->autoflag = lifcp->autoflag;

    lifc->validlt = lifcp->validlt;
    lifc->preflt = lifcp->preflt;
    lifc->origint = lifcp->origint;
}
```

⟨ipifcremlifc() *if ipv6 local* 483c⟩≡ (70)

```
else {
    v6delroute(f, lifc->remote, lifc->mask, 1);
    if(ipcmp(lifc->local, v6loopback) == 0)
        /* remove route for all node multicast */
        v6delroute(f, v6allnodesN, v6allnodesNmask, 1);
    else if(memcmp(lifc->local, v6linklocal, v6llpreflen) == 0)
        /* remove route for all link multicast */
        v6delroute(f, v6allnodesL, v6allnodesLmask, 1);
}
```

⟨allocroute() *if ipv6 route* 483d⟩≡ (98d)

```
else {
    n = sizeof(RouteTree) + sizeof(V6route);
    l = &v6freelist;
}
```

⟨freeroute() *if ipv6 route* 483e⟩≡ (99a)

```
else
    l = &v6freelist;
```

⟨rangecompare() *if ipv6 routes* 483f⟩≡ (99c)

```
if(lcmp(a->v6.endaddress, b->v6.address) < 0)
    return Rpreceeds;

if(lcmp(a->v6.address, b->v6.endaddress) > 0)
    return Rfollows;

if(lcmp(a->v6.address, b->v6.address) <= 0
&& lcmp(a->v6.endaddress, b->v6.endaddress) >= 0){
    if(lcmp(a->v6.address, b->v6.address) == 0
    && lcmp(a->v6.endaddress, b->v6.endaddress) == 0)
            return Requals;
```

```
    return Rcontains;
}

return Rcontained;
```

⟨*function* lcmp 484a⟩≡ (339c)

```
/*
 *   compare 2 v6 addresses
 */
static int
lcmp(ulong *a, ulong *b)
{
    int i;

    for(i = 0; i < IPllen; i++){
        if(a[i] > b[i])
            return 1;
        if(a[i] < b[i])
            return -1;
    }
    return 0;
}
```

⟨*global* v6routegeneration 484b⟩≡ (339c)

```
static ulong v6routegeneration;
```

Glossary

MTU
TTL
TOS
IP
UDP
TCP
IL
IFC
ICMP

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 11

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 11